ACSL by Example

Towards a Formally Verified Standard Library

Version 32.0.3
for Frama-C 32.0 (Germanium)
April 2026

Jens Gerlach

Former Authors

Malte Brodmann, Jochen Burghardt,
Andreas Carben Robert Clausecker,
Denis Efremov, Liangliang Gu
Kerstin Hartig, Timon Lapawczyk
Hans Werner Pohl, Tim Sikatzki
Juan Soto, Kim Völlinger

Fraunhofer
FOKUS

Except where otherwise noted, this work is licensed under the Creative Commons Attribution–NonCommercial–ShareAlike 4.0 International License:
https://creativecommons.org/licenses/by-nc-sa/4.0

Acknowledgements and Publication Information

Many members of the Frama-C community provided valuable input and comments during the course of the development of this document. In particular, we wish to thank our project partners Patrick Baudin, Allan Blanchard, Loïc Correnson, Zaynah Dargaye, Florent Kirchner, Virgile Prevosto, and Armand Puccetti from CEA LIST[1] and Pascal Cuoq from TrustInSoft[2].

We also express our gratitude to Claude Marché (LRI/INRIA)[3] and Yannick Moy (AdaCore)[4] for their helpful comments and detailed suggestions for improvement. Finally, we thank Aaron Rocha for valuable improvement suggestions and error reports.

The research presented in this report was supported by the following projects:

- **VESSEDIA** — European Union Horizon 2020 research and innovation programme, Grant Agreement No 731453 (2017–2019), https://vessedia.eu.
- **STANCE** — European Union Seventh Framework Programme (FP7/2007–2013), Grant Agreement No 317753 (2012–2016), http://www.stance-project.eu.
- **DeviceSoft** — supported by the Programme Inter Carnot Fraunhofer from BMBF (Grant 01SF0804) and ANR (2009–2012).

Availability. This document is hosted at

https://github.com/fraunhoferfokus/acsl-by-example

The repository also contains the source code of all algorithms discussed in this report, their formal contracts, and the associated predicate definitions and lemmas. All examples are developed and proved using the Frama-C/WP [1] plugin.

Suggestions or error reports may be submitted via the GitHub issue tracker:

https://github.com/fraunhoferfokus/acsl-by-example/issues

Alternatively, correspondence may be addressed to:

jens.gerlach@fokus.fraunhofer.de

[1] http://www-list.cea.fr/en
[2] http://trust-in-soft.com
[3] https://www.lri.fr/index_en.php?lang=EN
[4] http://www.adacore.com

Contents

1 Introduction **9**
1.1 Frama-C . . . 9
1.2 Structure of this document . . . 10
1.3 Recent changes . . . 11
List of Open Issues . . . 12

I Deductive Verification Basics **13**

Introduction **15**

2 Modeling conventions **17**
2.1 Types, arrays, and ranges . . . 17

3 The Hoare calculus **19**
3.1 Introduction . . . 19
3.2 The assignment rule . . . 21
3.3 The sequence rule . . . 23
3.4 The implication rule . . . 23
3.5 The choice rule . . . 24
3.6 The loop rule . . . 25
3.7 Derived rules . . . 27

II Nonmutating Algorithms for Selection and Search **29**

Introduction **31**

4 Nonmutating algorithms **33**
4.1 The `find` algorithm . . . 34
4.2 The `find2` algorithm—reuse of specification elements . . . 37
4.3 The `find3` algorithm—using a logic function . . . 40
4.4 The `find_if_not` algorithm . . . 45
4.5 The `find_first_of` algorithm . . . 49
4.6 The `adjacent_find` algorithm . . . 52
4.7 The `equal` and `mismatch` algorithms . . . 55
4.8 The `search` algorithm . . . 59
4.9 The `search_n` algorithm . . . 62
4.10 The `find_end` algorithm . . . 65
4.11 The `count` algorithm . . . 68

5 Minimum and maximum algorithms **73**
5.1 A note on relational operators . . . 74

5.2 Predicates for bounds and extrema of arrays . . . 75
5.3 The auxiliary function `make_pair` . . . 76
5.4 The `clamp` algorithm . . . 77
5.5 The `max_element` algorithm . . . 79
5.6 The `max_element2` algorithm (predicate-based specification) . . . 81
5.7 The `max_seq` algorithm . . . 83
5.8 The `min_element` algorithm . . . 84
5.9 The `minmax_element` algorithm . . . 86

6 Binary search algorithms **89**
6.1 The predicate `Increasing` . . . 89
6.2 Global vs. local monotonicity . . . 90
6.3 The `lower_bound` algorithm . . . 92
6.4 The `upper_bound` algorithm . . . 94
6.5 The `equal_range` algorithm . . . 96
6.6 The `binary_search` algorithm . . . 101

III Mutating and Numeric Algorithms **103**

Introduction **105**

7 Mutating algorithms **107**
7.1 The predicate `Unchanged` . . . 108
7.2 Reasoning about local array updates . . . 111
7.3 The `fill` algorithm . . . 112
7.4 The `swap` algorithm . . . 114
7.5 The `swap_ranges` algorithm . . . 115
7.6 The `copy` algorithm . . . 117
7.7 The `copy_backward` algorithm . . . 119
7.8 The `reverse_copy` algorithm . . . 121
7.9 The `reverse` algorithm . . . 124
7.10 The `rotate_copy` algorithm . . . 126
7.11 The `rotate` algorithm . . . 129
7.12 The `replace_copy` algorithm . . . 133
7.13 The `replace` algorithm . . . 135
7.14 The `remove_copy` algorithm (basic contract) . . . 137
7.15 The `remove_copy2` algorithm (number of copied elements) . . . 140
7.16 The `remove_copy3` algorithm (final contract) . . . 143
7.17 The `remove` algorithm . . . 150
7.18 The `shuffle` algorithm . . . 152
7.19 Verifying a random number generator . . . 157

8 Numeric algorithms **159**
8.1 The `iota` algorithm . . . 160
8.2 The `accumulate` algorithm . . . 162
8.3 The `inner_product` algorithm . . . 166
8.4 The `partial_sum` algorithm . . . 169
8.5 The `adjacent_difference` algorithm . . . 172
8.6 Inverting `partial_sum` and `adjacent_difference` . . . 175

IV Heaps and Sorting Algorithms 179

Introduction 181

9 Heap Algorithms 183
9.1 Basic heap concepts . . . 184
9.2 Representation of heap concepts in ACSL . . . 187
9.3 The auxiliary functions `heap_parent` and `heap_child` . . . 189
9.4 The `is_heap_until` algorithm . . . 191
9.5 The `is_heap` algorithm . . . 193
9.6 Reorderings and temporary changes in multiplicity . . . 194
9.7 The `push_heap` algorithm . . . 198
9.8 The `pop_heap` algorithm . . . 204
9.9 The `make_heap` algorithm . . . 210
9.10 The `sort_heap` algorithm . . . 212

10 Sorting Algorithms 214
10.1 Common contract of sorting algorithms . . . 215
10.2 The `is_sorted` algorithm . . . 215
10.3 The `partial_sort` algorithm . . . 217
10.4 The `bubble_sort` algorithm . . . 223
10.5 The `selection_sort` algorithm . . . 225
10.6 The `insertion_sort` algorithm . . . 227
10.7 The `heap_sort` algorithm . . . 231
10.8 The `merge` algorithm . . . 232

V Verification of data structures 237

Introduction 239

11 The Stack data type 241
11.1 Methodology overview . . . 242
11.2 Axioms and invariants of finite-capacity stacks . . . 244
11.3 The structure `Stack` and its associated functions . . . 246
11.4 Logical interface and invariants of `Stack` . . . 247
11.5 Equality of stacks . . . 249
11.6 Specification and implementation of stack functions . . . 251
11.7 Witnesses of stack axioms . . . 258
11.8 Witnesses for well-definedness . . . 264

VI Appendices 269

Introduction 271

A Results of Formal Verification with Frama-C 273
A.1 Verification settings . . . 273
A.2 Verification results . . . 274

B Changes in previous releases 279
B.1 New in Version 32.0.2 (Germanium, January 2026) . . . 279

B.2 New in Version 32.0.1 (Germanium, January 2026) . . . 280
B.3 New in Version 32.0.0 (Germanium, December 2025) . . . 280
B.4 New in Version 22.0.0 (Titanium, November 2020) . . . 281
B.5 New in Version 21.1.1 (Scandium, September 2020) . . . 282
B.6 New in Version 21.1.0 (Scandium, July 2020) . . . 283
B.7 New in Version 20.0.2 (Calcium, April 2020) . . . 285
B.8 New in Version 20.0.1 (Calcium, March 2020) . . . 286
B.9 New in Version 20.0.0 (Calcium, December 2019) . . . 287
B.10 New in Version 19.1.0 (Potassium, October 2019) . . . 288
B.11 New in Version 19.0.0 (Potassium, June 2019) . . . 290
B.12 New in Version 18.0.0 (Argon, December 2018) . . . 291
B.13 New in Version 17.1.0 (Chlorine, July 2018) . . . 291
B.14 New in Version 16.1.1 (Sulfur, March 2018) . . . 291
B.15 New in Version 16.1.0 (Sulfur, December 2017) . . . 291
B.16 New in Version 15.1.2 (Phosphorus, October 2017) . . . 292
B.17 New in Version 15.1.1 (Phosphorus, September 2017) . . . 293
B.18 New in Version 15.1.0 (Phosphorus, June 2017) . . . 294
B.19 New in Version 14.1.1 (Silicon, April 2017) . . . 294
B.20 New in Version 14.1.0 (Silicon, January 2017) . . . 295
B.21 New in Version 13.1.1 (Aluminium, November 2016) . . . 295
B.22 New in Version 13.1.0 (Aluminium, August 2016) . . . 295
B.23 New in Version 12.1.0 (Magnesium, February 2016) . . . 296
B.24 New in Version 11.1.1 (Sodium, June 2015) . . . 298
B.25 New in Version 11.1.0 (Sodium, March 2015) . . . 298
B.26 New in Version 10.1.1 (Neon, January 2015) . . . 299
B.27 New in Version 10.1.0 (Neon, September 2014) . . . 300
B.28 New in Version 9.3.1 (Fluorine, not published) . . . 300
B.29 New in Version 9.3.0 (Fluorine, December 2013) . . . 300
B.30 New in Version 8.1.0 (Oxygen, not published) . . . 301
B.31 New in Version 7.1.1 (Nitrogen, August 2012) . . . 301
B.32 New in Version 7.1.0 (Nitrogen, December 2011) . . . 301
B.33 New in Version 6.1.0 (Carbon, not published) . . . 301
B.34 New in Version 5.1.1 (Boron, February 2011) . . . 301
B.35 New in Version 5.1.0 (Boron, May 2010) . . . 302
B.36 New in Version 4.2.2 (Beryllium, May 2010) . . . 302
B.37 New in Version 4.2.1 (Beryllium, April 2010) . . . 302
B.38 New in Version 4.2.0 (Beryllium, January 2010) . . . 303

Bibliography **305**

Index of ACSL definitions **307**

Index of examples **313**

1. Introduction

This report presents a collection of examples illustrating the formal specification, implementation, and deductive verification of C programs using the ANSI/ISO-C Specification Language (ACSL [2]) and the Frama-C/WP plug-in [1] of Frama-C [3] (Framework for Modular Analysis of C programs).

Our examples are drawn from the C++ Standard Library, whose initial version is still known as the *Standard Template Library* (STL). The C++ Standard Library comprises a broad collection of *generic* algorithms that operate not only on C arrays but also on more elaborate container data structures. For the purposes of this document, we select representative algorithms and translate their implementations from C++ function templates into C functions operating on arrays of type `int`.

We intend to continue extending and refining this report by describing additional STL algorithms and data structures. In this way, the document is meant to evolve gradually from an ACSL tutorial into a report on a formally specified and deductively verified Standard Library for ANSI/ISO-C. Moreover, as ACSL is extended towards a specification language for C++, our approach may naturally be carried over to a deductively verified C++ Standard Library.

We encourage the reader to examine critically whether the formal specifications presented here faithfully capture the intent of the corresponding informal descriptions of the STL algorithms. We welcome feedback[5] and hope that this document contributes to a broader adoption of deductive verification techniques.

1.1. Frama-C

The Framework for Modular Analysis of C, Frama-C [3], is a suite of software tools dedicated to the analysis of C source code. Its development is conducted and coordinated primarily at CEA LIST [4], a laboratory for applied research on software-intensive technologies, and at INRIA Saclay[5], the French National Institute for Research in Computer Science and Control, in collaboration with LRI [6], the Laboratory for Computer Science at Université Paris-Sud.

ACSL (ANSI/ISO-C Specification Language) [2] is a formal specification language for expressing behavioral properties of C programs. It allows programs to be annotated with precise contracts, including preconditions and postconditions, as well as with type invariants, global invariants, and logical specifications such as predicates, lemmas, axioms, and logic functions. In addition, ACSL supports statement-level annotations, including assertions and loop annotations, which play a central role in modular verification.

Within Frama-C, the Frama-C/WP plug-in [1] supports the deductive verification of C programs annotated with ACSL. It is based on Hoare-style weakest precondition reasoning and generates verification conditions that are discharged by external automatic theorem provers or, where necessary, by interactive proof assistants.

The Verification Group at Fraunhofer FOKUS[7] recognizes the considerable potential of deductive verification based on ACSL. At the same time, we acknowledge that the effective use of the Frama-C/WP plug-in poses non-trivial challenges, particularly for users who are new to formal methods. In order to lower this

[5] We suggest GitHub's issue tracker: `https://github.com/fraunhoferfokus/acsl-by-example/issues`

barrier to entry, this document focuses on concrete, worked examples that demonstrate how to annotate existing C programs and how to guide the deductive verification process in practice.

For a thorough introduction to ACSL and its underlying concepts, we strongly recommend that readers consult the official Frama-C introductory tutorial [8] before proceeding. The principles employed throughout this report are also documented in the ACSL reference manual [9].

1.2. Structure of this document

The functions presented in this document were selected from the C++ Standard Library. The original C++ implementation was stripped of its generic implementation and mapped to C arrays of type `value_type`.

The document is structured into several parts, each of which groups chapters with a common conceptual focus.

- Part I, *Deductive Verification Basics*, introduces the methodological foundations on which the remainder of the document is built. It first fixes basic modeling conventions for types, arrays, and memory ranges in C and ACSL, and then introduces the Hoare calculus as the central framework for reasoning about the functional correctness of imperative programs. Together, these chapters establish the conceptual and technical prerequisites required for all subsequent verification examples.
- Part II, *Nonmutating Algorithms for Selection and Search*, groups algorithms that operate on sequences without modifying their contents. The chapters in this part study traversal-based inspection algorithms, the determination of extremal elements, and binary search under suitable structural assumptions. Together, they introduce fundamental patterns of search and selection that rely solely on observation and comparison, and that serve as a conceptual bridge between the verification basics established earlier and the mutating and reordering algorithms studied in later parts.
- Part III, *Mutating and Numeric Algorithms*, groups algorithms that modify program state or compute numeric results during traversal of a sequence. The chapters in this part study in-place transformations of data and the accumulation of numeric values derived from sequence elements. Together, they introduce algorithmic patterns in which memory updates and numeric computation play a central role, and they extend the nonmutating inspection and selection schemes studied earlier to settings that involve state change and aggregation.
- Part IV, *Heaps and Sorting Algorithms*, groups algorithms whose primary purpose is the reorganization of sequence elements according to an ordering relation. The chapters in this part study heap-based algorithms and classical sorting algorithms, focusing on the use of structured ordering constraints and in-place permutation of data. Together, they address algorithmic patterns that combine mutation with global ordering properties, and they require reasoning about both structural invariants and the preservation of multisets of values.
- Part V, *Verification of Data Structures*, shifts the focus from individual algorithms to abstract data types and their associated operations. The chapters in this part study the specification and verification of data structures with internal state and representation invariants, using a stack as a first and representative example. Together, they introduce verification techniques that go beyond functional correctness of individual operations and address well-definedness with respect to user-defined equality and abstraction, thereby laying the groundwork for reasoning about the correctness of complete data structure interfaces.
- Part VI, *Appendices*, collects supplementary material that supports the main exposition but is not required for a linear reading of the document. The chapters in this part provide additional technical details, background information, and reference material that complement the core verification

examples. Together, they document aspects such as verification setup and tool configuration, as well as the historical evolution of the tutorial across previous releases, thereby preserving context and traceability without interrupting the conceptual flow of the main text.

1.3. Recent changes

For changes in previous versions we refer to Appendix B on Page 279.

New in Version 32.0.3 (April 2026, Germanium, 32.0)

This release is intended for Frama-C [3, p. v32.0] issued in December 2025. We are also using for this release the Why3 platform[10, p. v1.8.2] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.6.2	[11]
CVC5	automatic	1.3.3	[12]
Z3	automatic	4.15.2	[13]
Coq	interactive	8.20.1	[14]

Table 1.1.: Information on automatic and interactive theorem provers

New examples

None.

Improvements

- refactoring of example `Stack`. The new version shifts from a procedure-oriented verification workflow to a conceptually structured, multi-level methodology with a strong emphasis on observational equality, well-definedness, and modular reasoning.
  - Introduction of a clear three-level methodology separating abstract stack axioms, logical specifications, and implementation.
  - Stronger emphasis on observational equality of stacks, including its role as an equivalence relation and its use in reasoning.
  - Systematic treatment of well-definedness of stack functions via dedicated witnesses, with a clear separation from axiom verification.
  - Reorganization of the verification flow: operations, then axioms, then well-definedness.
  - Simplification and clarification of witness specifications, including omission of low-level clauses for readability.
  - Improved modularization of the logical interface of `Stack` and clearer separation between representation and specification.
- removed the axiomatic blocks in logic definitions.
- the third version of `Equal` is now an explicit expansion of the previously delegated definition, with the universal quantification written out directly.
- predicate `Reverse` now identifies the second range by its one-past-the-end index.

- fixed wrong argument in the first version of `Rotate`.
- simplified the function `make_pair`.
- added figures to aide the description of the predicate `Reverse` and the algorithms `reverse_copy` and `reverse`.
- improved the description of the implementation and verification of `rotate`.
- expanded description of the specification of `clamp`.
- fixed misplaced LaTeX labels for ACSL listings.

List of Open Issues

10.8.1 Completeness of the postcondition of merge 233

Part I.

Deductive Verification Basics

Introduction

This part introduces the basic principles and assumptions underlying the deductive verification of C programs in this document. Its purpose is to establish a common framework for modeling programs and for reasoning about their functional correctness.

We first fix a set of modeling conventions for types, arrays, and memory ranges, which are used implicitly throughout all specifications and proofs. Building on these conventions, we then introduce the Hoare calculus as the central reasoning framework employed in the remainder of this work.

2. Modeling conventions

This chapter introduces a number of basic modeling conventions that are used consistently throughout the remainder of this document. These conventions concern the representation of types, arrays, and memory ranges in C programs, as well as their specification in ACSL.

The purpose of this chapter is not to develop new verification techniques or reasoning principles. Rather, it serves to fix terminology, notation, and assumptions that underlie all subsequent specifications and proofs. Making these conventions explicit at an early stage helps avoid ambiguities and allows later chapters to focus on the essential aspects of deductive verification.

In particular, we clarify how abstract type names are used to separate conceptual roles such as values and indices, how arrays and ranges are modeled at function boundaries, and how validity of memory accesses is expressed in ACSL. These notions are prerequisites for the correct formulation of preconditions, postconditions, and invariants, and they will be used implicitly in all following chapters.

The next chapter builds on these conventions and introduces the Hoare calculus as the central framework for reasoning about the functional correctness of C programs.

2.1. Types, arrays, and ranges

Before introducing the Hoare calculus, we fix a number of basic modeling conventions that are used throughout this document. These concern the representation of types, arrays, and memory ranges in C and their specification in ACSL. Although largely technical in nature, these conventions are essential for understanding the specifications and proofs presented later.

2.1.1. Types

In order to keep algorithms and specifications as general as possible, we employ abstract type names on almost all occasions. At present, the following type definitions are used:

```
typedef int value_type;
typedef unsigned int size_type;
typedef int bool;
```

Readers familiar with the types associated with C++ Standard Library containers will recognize this approach. The type `value_type` denotes the type of elements stored in an array, whereas `size_type` is used for array indices.

This convention allows, for example, an algorithm operating on an **`int`** array to be adapted to a different element type by changing a single line of code, namely the definition of `value_type`. In addition, it improves readability by making the intended role of a variable apparent from its type: index variables and data values are clearly distinguished.

The same consideration motivates the explicit use of the type **`bool`**. Values of this type are represented by the identifiers **`false`** and **`true`**, which are defined to be `0` and `1`, respectively. While any non-zero value may

denote **true** in both C and ACSL, all algorithms in this document produce the value 1 to represent truth. Due to these definitions, the ACSL truth-value constants **false** and **true** may be used interchangeably with **false** and **true** in ACSL clauses, but not in C code.

2.1.2. Arrays and ranges

According to the C Standard, an array is a "contiguously allocated nonempty set of objects" [15, §6.2.5.20]. If n is a constant integer expression with a value greater than zero, then the declaration

```
int a[n];
```

defines an array of type **int**. In particular, for each index i satisfying 0 <= i < n, the pointer a+i may be safely dereferenced.

In this document, we frequently consider functions of the following form:

```
void example(value_type* a, size_type n);
```

Here, the pointer a denotes the address of the first element of a memory region, and n denotes the length of that region. Strictly speaking, such a pair describes a *range* rather than an array, since functions may be called with n == 0. Empty arrays, however, are not permitted by the C definition. For simplicity, we nevertheless use the terms *array* and *range* interchangeably where no confusion can arise.

2.1.3. Specification of valid ranges in ACSL

The following ACSL contract expresses the precondition that the function example may safely access all elements of the range a[0..n-1]:

```
/*@
  requires 0 <= n;
  requires \valid(a + (0.. n-1));
*/
void example(value_type* a, size_type n);
```

In this context, each index i satisfying 0 <= i < n is called a *valid index* of a.

The built-in ACSL predicates \valid(a + (0.. n)) and \valid_read(a + (0.. n)) refer to all addresses a+i with 0 <= i <= n. By contrast, the C array declaration **int** a[n] refers only to the elements a+i for which 0 <= i < n holds. Consequently, when specifying arrays of length n in ACSL, users must employ the range notation a+(0..n-1).

3. The Hoare calculus

The reasoning principles introduced in this chapter build on the modeling conventions established in the preceding chapter. In particular, the treatment of arrays, ranges, valid indices, and abstract types provides the semantic background against which assertions, preconditions, postconditions, and invariants are interpreted.

With these conventions fixed, we now introduce the Hoare calculus as the central framework for reasoning about the functional correctness of C programs. The calculus allows program fragments to be related systematically to logical assertions about program states and forms the basis for all subsequent verification examples in this document.

3.1. Introduction

In 1969, C. A. R. Hoare introduced a calculus for the formal reasoning about properties of imperative programs [16]. This calculus, now commonly referred to as the *Hoare calculus*, provides a systematic framework for relating program fragments to logical assertions about program states.

The central notion of the Hoare calculus may be illustrated as follows.

```
//@ assert P;
Q;
//@ assert R;
```

Here, `P` and `R` denote logical expressions describing properties of program states, while `Q` denotes a fragment of source code. Informally, this annotation expresses the following meaning:

If `P` holds before the execution of `Q`, then `R` holds after the execution of `Q`.

Usually, `P` and `R` are referred to as the *precondition* and *postcondition* of `Q`, respectively. The syntax of logical expressions in ACSL is described in full detail in [9, §2.2]. For the purposes of this tutorial, however, it suffices to consider the constructs listed in Table 3.1. Note that these constructs closely resemble the logical and relational operators provided by C.

The following examples illustrate the intended meaning of Hoare-style annotations by means of three simple source-code fragments.

`//@ assert x % 2 == 1;`
`++x;`
`//@ assert x % 2 == 0;`
	If `x` has an odd value before execution of the code `++x` then `x` has an even value thereafter.

ACSL syntax	Name	Reading
`!P`	negation	`P` is not true
`P && Q`	conjunction	`P` is true and `Q` is true
`P || Q`	disjunction	`P` is true or `Q` is true
`P ==> Q`	implication	if `P` is true, then `Q` is true
`P <==> Q`	equivalence	if, and only if, `P` is true, then `Q` is true
`x < y == z`	relation chain	`x` is less than `y` and `y` is equal to `z`
`\forall int x; P(x)`	universal quantifier	`P(x)` is true for every **`int`** value of `x`
`\exists int x; P(x)`	existential quantifier	`P(x)` is true for some **`int`** value of `x`

Table 3.1.: Some ACSL formula syntax

`//@ assert 0 <= x <= y;`
`++x;`
`//@ assert 0 <= x <= y + 1;`
	If the value of `x` is in the range $\{0, \ldots, y\}$ before execution of the same code, then `x`'s value is in the range $\{0, \ldots, y + 1\}$ after execution.

`//@ assert \true;`
`while (--x != 0)`
`  sum += a[x];`
`//@ assert x == 0;`
	Under any circumstances, the value of `x` is zero after execution of the loop code.

Any experienced C programmer will readily confirm that these properties are valid.[6] The examples were chosen not only for their simplicity, but also to illustrate the following general aspects of pre- and postconditions.

- For a given code fragment, there is no uniquely determined precondition or postcondition. Rather, the choice of suitable formulas depends on the particular property to be verified, which in turn arises from the application context. The first two examples illustrate this point by sharing the same code fragment while employing different pre- and postconditions.
- A postcondition need not be the strongest property that could be derived. In the second example, it is therefore not an error to state only `0 <= x`, even though the stronger property `1 <= x` would also hold.
- Pre- and postconditions are not required to mention all variables occurring in the corresponding code fragment. In particular, neither `sum` nor `a[]` appears in the formulas associated with the loop example.
- The logical constant **`\true`** of ACSL may be used to express the absence of a restrictive precondition, as demonstrated before the **`while`** loop.
- Termination properties cannot, in general, be expressed by pre- and postconditions alone. The loop example therefore states only that *if* the loop terminates, then `x == 0` will hold afterward. Indeed, if `x` has a negative value on entry, the loop will not terminate. Nevertheless, whenever the loop does terminate, the stated postcondition is satisfied, which is precisely the claim made by the example.

[6] For the time being, we deliberately ignore potential issues related to arithmetic overflow.

In practice, termination is usually treated separately from functional correctness. Termination proofs may, however, rely on properties that are stated and verified using the Hoare calculus.

Hoare introduced a collection of inference rules for reasoning about imperative programs. In Listings 3.2–3.14 we collect the rules and accompanying examples that will be introduced and discussed step by step in the following sections.

3.2. The assignment rule

We begin with the assignment rule, which is often regarded as the least intuitive rule of the Hoare calculus. Nevertheless, it is one of its most fundamental components. The rule is shown in Listing 3.2. Here,

$$\mathtt{P}\,\{\mathtt{x} \mapsto \mathtt{e}\}$$

denotes the predicate obtained by substituting every occurrence of the variable `x` in the predicate `P` by the expression `e`.

```
//@ assert P {x |--> e};
x = e;
//@ assert P;
```

Listing 3.2: The assignment rule

As a concrete example, let `P` be the predicate

```
x > 0 && a[2*x] == 0
```

Then the predicate $\mathtt{P}\,\{\mathtt{x} \mapsto \mathtt{y} + 1\}$ is given by

```
y+1 > 0 && a[2*(y+1)] == 0
```

Applying the assignment rule with this substitution yields Listing 3.3 as a concrete instance of the rule. Note that parentheses are required in the index expression in order to obtain the correct term `2*(y+1)` rather than the incorrect `2*y+1`.

```
//@ assert y+1 > 0 && a[2*(y+1)] == 0;
x = y+1;
//@ assert x > 0 && a[2*x] == 0;
```

Listing 3.3: An assignment rule example instance

It is important to observe that different predicates `P` may yield the same substituted predicate $\mathtt{P}\,\{\mathtt{x} \mapsto \mathtt{e}\}$. For example, under the substitution $\mathtt{P}\,\{\mathtt{x} \mapsto \mathtt{y} + 1\}$, each of the following predicates

```
  x > 0 && a[2*x]     == 0
  x > 0 && a[2*(y+1)] == 0
y+1 > 0 && a[2*x]     == 0
y+1 > 0 && a[2*(y+1)] == 0
```

is transformed into

```
y+1 > 0 && a[2*(y+1)] == 0
```

As a consequence, for a given assignment statement, the same precondition may give rise to several different postconditions. All four predicates listed above, for instance, are valid postconditions for the

assignment shown in Listing 3.3. Conversely, for a fixed statement and a given postcondition, there is exactly one corresponding precondition determined by the assignment rule.

When first encountering Hoare's assignment rule, many readers are tempted to consider a seemingly simpler and more intuitive alternative, shown in Listing 3.4.

```
//@ assert P;
x = e;
//@ assert P && x == e;
```

Listing 3.4: Simpler, but *faulty* assignment rule

Listings 3.5–3.7 illustrate several concrete instances of this alternative rule.

```
//@ assert y > 0;
x = y+1;
//@ assert y > 0 && x == y+1;
```

Listing 3.5: An example instance of the faulty rule from Listing 3.4

The instance shown in Listing 3.5 happens to be valid. However, the instances in Listings 3.6 and 3.7 give rise to postconditions that are clearly nonsensical.

```
//@ assert true;
x = x+1;
//@ assert x == x+1;
```

Listing 3.6: An example instance of the faulty rule from Listing 3.4

In Listing 3.6, the problem arises because the variable `x` occurring on the left-hand side of the assignment also appears in the right-hand side expression `e`. In contrast, the assignment in Listing 3.7 destroys the property stated in the precondition altogether.

```
//@ assert x < 0;
x = 5;
//@ assert x < 0 && x == 5;
```

Listing 3.7: An example instance of the faulty rule from Listing 3.4

Finally, observe that the valid instance shown in Listing 3.5 can also be derived using the correct assignment rule from Listing 3.2. Indeed, substituting `x` by `y+1` in the postcondition yields the precondition `y > 0 && y+1 == y+1`, which is logically equivalent to the simpler predicate `y > 0`.

3.3. The sequence rule

The sequence rule, shown in Listing 3.8, captures the effect of sequential composition. It allows two code fragments `Q` and `S` to be combined into a single fragment `Q ; S` and relates the corresponding pre- and postconditions.

```
//@ assert P;
Q;
//@ assert R;
```

and

```
//@ assert R;
S;
//@ assert T;
```

$\rightsquigarrow$

```
//@ assert P;
Q ; S;
//@ assert T;
```

Listing 3.8: The sequence rule

The key requirement of the sequence rule is that the postcondition of `Q` must coincide with the precondition of `S`. This intermediate assertion represents the program state after the execution of `Q` and before the execution of `S`. In this way, the rule reflects the intuitive reading of sequential execution—"first execute `Q`, then execute `S`"—in a precise logical form.

An important consequence of the sequence rule is that it allows programs to be annotated with assertions at intermediate program points. This style of annotation, in which assertions are interspersed with statements, is characteristic of Hoare-style reasoning and is directly supported by tools such as Frama-C.

As a simple example, consider the sequential composition of two assignments. After the first assignment, the value of `x` is known explicitly, which can then be used as a precondition for the second statement.

```
//@ assert true;
x = y + 1;
//@ assert x == y + 1;
z = 2 * x;
//@ assert z == 2 * (y + 1);
```

Listing 3.9: An example instance of the sequence rule

Here, the intermediate assertion `x == y + 1` serves as the postcondition of the first assignment and simultaneously as the precondition of the second. By applying the sequence rule, the two annotated fragments can be combined into a single correctness argument for the sequentially composed code.

3.4. The implication rule

The implication rule, shown in Listing 3.10, provides flexibility in the choice of pre- and postconditions. It allows a precondition to be strengthened and a postcondition to be weakened without invalidating the correctness of the annotated code fragment.

```
//@ assert P;
Q;
//@ assert R;
```

$\rightsquigarrow$

```
//@ assert P';
Q;
//@ assert R';
```

Listing 3.10: The implication rule

More precisely, suppose that `P'` implies `P` and that `R` implies `R'`. In this situation, any execution that satisfies the original contract also satisfies the modified one. Consequently, the contract on the left-hand side of Listing 3.10 may safely be replaced by the contract on the right-hand side.

The implication rule plays a crucial role in practice. It allows assertions to be adapted to the surrounding context and to be matched with the requirements of other rules, such as the sequence rule. In particular, it

is frequently used to align intermediate assertions so that sequentially composed code fragments can be combined.

As a simple example, consider the following two annotations of the same code fragment.

```
//@ assert x > 0;
x = x + 1;
//@ assert x > 1;
```

$\rightsquigarrow$

```
//@ assert x > 0;
x = x + 1;
//@ assert x >= 0;
```

Listing 3.11: An example instance of the implication rule

Here, the postcondition on the right-hand side is weaker than the one on the left-hand side. Since `x > 1 ==> x >= 0` holds, the implication rule allows the original contract to be replaced by the weaker one without affecting correctness.

Such a weakening is often required in practice in order to adapt a postcondition to the precondition of a subsequent code fragment.

3.5. The choice rule

The choice rule, shown in Listing 3.12, is used to reason about conditional statements of the form

```
if (C) {
   X;
}
else {
   Y;
}
```

The essential idea of the choice rule is that both the *then*-branch and the *else*-branch must establish the same postcondition, denoted by `S`. In each branch, the precondition is strengthened by the information provided by the condition `C`: in the then-branch, one may assume that `C` holds, whereas in the else-branch one may assume that `C` is false.

```
//@ assert P &&
    C;
X;
//@ assert S;
```

and

```
//@ assert P && !C;
Y;
//@ assert S;
```

$\rightsquigarrow$

```
//@ assert P;
if (C) X;
else   Y;
//@ assert S;
```

Listing 3.12: The choice rule

If the two branches naturally yield different postconditions, say `S1` and `S2`, the implication rule may be used to weaken them to a common postcondition. In particular, both `S1` and `S2` imply the disjunction `S1 || S2`, which can therefore serve as a unified postcondition for the conditional statement.

If the else-branch is omitted, it can be treated as an empty sequence. In this case, its postcondition coincides with its precondition, namely `P && !C`.

Listing 3.13 illustrates the application of the choice rule to a simple conditional statement. The annotations are written so that each assertion refers directly to the program state immediately before or after the corresponding statement.

```
//@ assert 0 <= i < n;          // given precondition
if (i < n-1) {
  //@ assert 0 <= i < n - 1;    // using that i < n-1 holds in this branch
  //@ assert 1 <= i+1 < n;      // by the implication rule
  i = i+1;
  //@ assert 1 <= i < n;        // by the assignment rule
  //@ assert 0 <= i < n;        // weakened by the implication rule
} else {
  //@ assert 0 <= i == n-1 < n; // using that !(i < n-1) holds in else part
  //@ assert 0 == 0 && 0 < n;   // weakened by the implication rule
  i = 0;
  //@ assert i == 0 && 0 < n;   // by the assignment rule
  //@ assert 0 <= i < n;        // weakened by the implication rule
}
//@ assert 0 <= i < n;          // by the choice rule from both branches
```

Listing 3.13: An example application of the choice rule

In this example, the variable `i` is intended to serve as an index into a ring buffer `int a[n]`. The code fragment advances the index `i` in a wrap-around fashion. The annotations show that `i` remains a valid index into the array `a[]` after execution of the conditional statement, provided that it was valid initially.

The example also illustrates the interplay between the choice rule, the assignment rule, and the implication rule. The latter is used both to establish suitable preconditions for the assignment statements and to align the postconditions of the two branches, as required by the choice rule.

3.6. The loop rule

The loop rule, shown in Listing 3.14, is used to verify `while` loops. Its central requirement is the identification of a suitable formula `P` that is preserved by every execution of the loop body. Such a formula is called a *loop invariant*.

```
//@ assert P && B;
S;
//@ assert P;
```

$\rightsquigarrow$

```
//@ assert P;
while (B) {
  S;
}
//@ assert !B && P;
```

Listing 3.14: The loop rule

Intuitively, the invariant `P` describes a property that holds before the first iteration of the loop and is re-established at the end of each execution of the loop body. Finding a suitable invariant often requires *insight* into the algorithm being verified and *intuition* and is therefore a major challenge in deductive verification. For this reason, the automatic discovery of loop invariants remains difficult for automated theorem provers.

As already noted, the loop rule does not guarantee termination. It only ensures that, if the loop terminates, the stated postcondition holds. For this reason, the properties established using the Hoare calculus are usually referred to as *partial correctness* properties. By contrast, properties that additionally guarantee termination are called *total correctness* properties.

As an example application, consider the abstract ring buffer shown in Listing 3.15. The goal of the verification is to show that the index variable `i` always remains within the valid range `[0..n-1]` during the execution of the loop and after its termination. The proof relies on the reasoning developed earlier in Listing 3.13 as a sub-derivation.

```
//@ assert  0 < n;                  // given precondition

int i = 0;
//@ assert  0 <= i < n;             // by the assignment rule

while (!done) {
  //@ assert 0 <= i < n && !done;   // may be assumed by the loop rule

  a[i] = getchar();
  //@ assert 0 <= i < n && !done;   // required property of getchar
  //@ assert 0 <= i < n;            // weakened by the implication rule

  i = (i < n-1) ? i+1 : 0;
  //@ assert 0 <= i < n;            // follows by the choice rule

  process(a, i, &done);
  //@ assert 0 <= i < n;            // required property of process
}
//@ assert 0 <= i < n;              // by the loop rule
```

Listing 3.15: An abstract ring buffer loop

In order to reuse the proof from Listing 3.13, the conjunct `!done` had to be dropped, since it was not considered in that proof. In general, one may *not* infer

```
//@ assert P && S;
Q;
//@ assert R && S;
```

from

```
//@ assert P;
Q;
//@ assert R;
```

since the code fragment `Q` may destroy the property `S`. This is already evident if `Q` is taken from Listing 3.13 and `S` is, for example, `i != 0`.

Suppose, for the sake of argument, that the function `process` were implemented in such a way that it refuses to set `done` to **`true`** unless it is **`false`** on entry. In that case, it would indeed be necessary to ensure that `!done` still holds after executing the code from Listing 3.13. This would require redoing the proof while threading the additional conjunct `!done` through all intermediate assertions.

Similar considerations apply when propagating properties such as `0 <= i < n && !done` or `0 <= i < n` across the statements `a[i] = getchar()` and `process(a, i, &done)`. To justify such reasoning, it must be specified that neither `getchar` nor `process` is allowed to modify the values of `i` or `n`. In ACSL, this is expressed using the `assigns` clause, which is introduced in §7.4 on Page 114.

In the present example, the loop invariant `0 <= i < n` happens to hold between all statements of the loop body. This need not be the case in general. The loop rule only requires the invariant to hold before the loop and at the end of each execution of the loop body. For instance, the function `process` could temporarily modify the values of `i`[7] or even `n`, as long as it re-establishes the invariant before returning.

[7] This would require changing the call to `process(a, &i, &done)` and adapting the implementation of `process` accordingly. In that case, the `assigns` clause discussed above could no longer be relied upon.

Finally, note that the loop invariant `0 <= i < n` is established by the proof in Listing 3.13 not only during the loop, but also after its termination. Consequently, a statement such as `a[i] = '\0'` placed after the loop would be guaranteed not to cause a bounds violation.

Even if a property like `0 <= i < n` were needed only immediately before a particular statement inside the loop body, it would still have to be established as a loop invariant. Within the Hoare calculus, the loop rule is the only mechanism by which properties can be obtained inside a loop body. Apart from this formal consideration, it is also intuitively clear that the property `0 <= i < n` would not hold in later iterations unless it were re-established at the end of each iteration, which is precisely what the loop rule enforces.

3.7. Derived rules

The rules presented so far do not directly cover all statement forms allowed in C. Nevertheless, statements that are not handled explicitly can be rewritten into semantically equivalent forms that fall within the scope of the basic Hoare rules.

As a first example, consider a **`switch`** statement. If the controlling expression `E` has no side effects, then a statement of the form

```
switch (E) {
    case E1: Q1; break; ...
    case En: Qn; break;
    default: Q0; break;
}
```

is semantically equivalent to a sequence of nested conditional statements,

```
if (E == E1) {
    Q1;
} else ... if (E == En) {
    Qn;
} else {
    Q0;
}
```

While the **`if-else`** form may be less efficient in terms of execution time on actual hardware, this distinction is irrelevant for verification purposes. Deductive verification is concerned with semantic correctness, not with performance characteristics.

Similarly, a loop of the form

```
for (P; Q; R) {
    S;
}
```

can be rewritten as

```
P;
while (Q) {
    S;
    R;
}
```

and verified using the sequence rule and the loop rule.

In this way, Hoare-style rules can be derived for many statement forms that were not introduced explicitly, simply by applying the basic rules to suitably transformed code. For this reason, we do not present such derived rules separately.

Procedures require a slightly different treatment. Non-recursive procedures can, in principle, be inlined, thereby reducing their verification to the application of the basic Hoare rules. Recursive procedures, however, cannot be handled in this way. Instead, their verification requires the formulation of suitable *procedure invariants*, which play a role analogous to loop invariants.

Finally, note that **`goto`** statements cannot be rewritten into the restricted statement forms considered above. As a consequence, programs containing arbitrary **`goto`** statements cannot be handled directly by the Hoare calculus. For calculi capable of reasoning about arbitrary control flow graphs, including unrestricted jumps, we refer to Floyd's work [17], which predates Hoare's calculus and strongly influenced its development. Later extensions and variants of Hoare-style reasoning have reintroduced support for such constructs.

In this tutorial, however, we restrict our attention to structured control flow and therefore do not consider examples involving **`goto`**.

Part II.

Nonmutating Algorithms for Selection and Search

Introduction

This part collects algorithms that operate on sequences without modifying their contents. In contrast to mutating, numeric, or sorting algorithms, the procedures studied here restrict themselves to inspection, comparison, and selection. Their purpose is not to transform a sequence, but to extract information from it.

Chapter 4 introduces the basic pattern of nonmutating traversal algorithms. These algorithms examine the elements of a sequence sequentially, typically maintaining auxiliary state such as counters, accumulators, or predicates, while leaving the underlying data unchanged. They form the conceptual foundation for all subsequent algorithms in this part.

Building on this pattern, Chapter 5 studies algorithms for determining extremal elements. Minimum and maximum selection illustrates how simple traversal schemes can be refined to solve well-defined selection problems efficiently and reliably.

Finally, Chapter 6 is devoted to binary search. Unlike the preceding algorithms, binary search relies on additional structural assumptions about the input, most notably ordering. It therefore serves as a first example of how stronger invariants enable asymptotically more efficient search procedures, while still adhering to the nonmutating paradigm.

Together, these chapters establish a class of algorithms whose defining characteristic is search and selection without modification, and they provide a point of reference for later parts of the document that introduce mutation, reordering, and structural verification.

4. Nonmutating algorithms

In this chapter, we study *non-mutating* algorithms of the C++ Standard Library [18, §28.5], that is, algorithms that do not modify their arguments nor any other memory locations. This property is uniformly expressed by the following *assigns clause*:

```
assigns \nothing;
```

This assigns clause is therefore used consistently in the specification of all algorithms considered in this chapter.

Although the functional behavior of non-mutating algorithms is often simpler than that of mutating ones, their specifications nevertheless illustrate a variety of important specification techniques. The examples are arranged in increasing order of complexity, and all algorithms operate on one-dimensional arrays.

- We begin with linear search algorithms. `find` in §4.1 returns the smallest index at which a given value occurs in a range. Two alternative specifications are then introduced: `find2` in §4.2, which relies on a user-defined predicate to factor out common specification elements, and `find3` in §4.3, which uses a user-defined logic function to express linear search directly at the logic level. We also show that these different specifications are equivalent.
- `find_if_not` in §4.4 is a simple variation of `find` that searches for the first position at which a given value does not occur.
- `find_first_of` in §4.5 generalizes linear search by searching for the first occurrence of an arbitrary element drawn from a given set.
- `adjacent_find` in §4.6 detects equal neighboring elements in an array.
- Range comparison algorithms are treated next. `equal` and `mismatch` in §4.7 compare two ranges element-by-element and, in the case of `merge`, identify the first position at which they differ.
- `search` and `search_n` in §4.8 and §4.9, respectively, search for a subsequence that matches a given sequence element-by-element and return the position of its first occurrence.
- `find_end` in §4.10 searches for a matching subsequence as well, but returns the *largest* index at which such a match starts, and can thus be seen as the counterpart of `search` that identifies the last occurrence rather than the first.
- The chapter concludes with `count` in §4.11, which returns the number of occurrences of a given value in a range. This example introduces an explicit logic function for counting elements and demonstrates how an implementation can be shown to comply with such a declarative specification.

4.1. The find algorithm

The `find` algorithm in the C++ Standard Library [18, §28.5.5] implements *sequential search* for general sequences. We have modified the generic implementation, which relies heavily on C++ templates, to that of a range of type `value_type`. The signature now reads:

```
size_type find(const value_type* a, size_type n, value_type v);
```

The function `find` returns the least *valid* index `i` of `a` where the condition `a[i] == v` holds. If no such index exists then `find` returns the length `n` of the array.

As an example, we consider in Figure 4.1 an array. The arrows indicate which indices will be returned by `find` for a given value. Note that the index 9 points *one past end* of the array. Values that are not contained in the array are colored in gray.

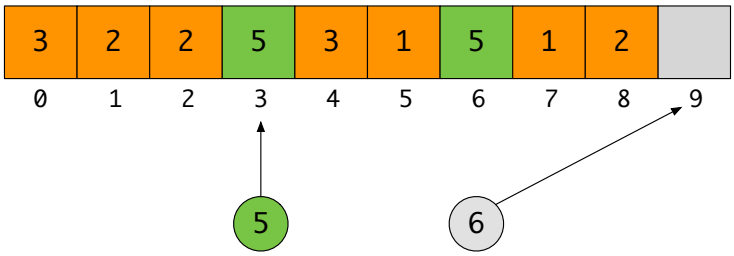


Figure 4.1.: Some simple examples for `find`

4.1.1. Formal specification of find

Listing 4.2 presents our first formal specification of `find`. It illustrates the basic structure of an ACSL function contract and introduces the most common specification clauses that will be used throughout this document.

The *preconditions* of the function are expressed by the `requires`-clauses. They state that the length `n` of the array is non-negative and that the pointer `a` denotes a contiguous block of `n` objects of type `value_type`. This pattern of specifying valid memory ranges occurs frequently and will be reused in many subsequent specifications (see §2).

The `assigns`-clause specifies the memory locations that may be modified by the function. Since `find` is a non-mutating algorithm, it does not modify any memory location visible to its caller; this is expressed by the clause `assigns \nothing` (see Page 33).

The clause `terminates \true;` states that the function is guaranteed to terminate for all inputs satisfying the preconditions. In particular, the implementation contains neither infinite loops nor unbounded recursion.

The clause `exits \false;` specifies that the function never terminates by an abnormal exit. That is, it cannot call `exit()`, trigger a runtime abort, or otherwise leave its execution context without returning normally to its caller.

```
/*@
  requires   \valid_read(a + (0..n-1));

  terminates \true;
  exits      \false;
  assigns    \nothing;

  ensures    0 <= \result <= n;

  behavior some:
    assumes  \exists integer i; 0 <= i < n && a[i] == v;
    assigns  \nothing;
    ensures  0 <= \result < n;
    ensures  a[\result] == v;
    ensures  \forall integer i; 0 <= i < \result ==> a[i] != v;

  behavior none:
    assumes  \forall integer i; 0 <= i < n ==> a[i] != v;
    assigns  \nothing;
    ensures  \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type find(const value_type* a, size_type n, value_type v);
```

Listing 4.2: Formal specification of `find`

At this point, the contract only guarantees that `find` returns a non-negative index that is less than or equal to the array length `n`. More precise statements about the return value can be made once additional assumptions about the contents of the array are introduced. For this reason, the specification of `find` is subdivided into two *behaviors*, named `some` and `none`.

- The behavior `some` applies when the sought-after value is contained in the array. This situation is characterized by the `assumes`-clause of the behavior. Under this assumption, the contract ensures that `find` returns a valid index `i` such that `a[i] == v`. Moreover, the final postcondition of this behavior states that the returned index is the *smallest* index with this property, thereby capturing the fact that `find` returns the first occurrence of the value in the array.
- The behavior `none` covers the complementary case in which the sought-after value does not occur in the array, as expressed by its `assumes`-clause. In this case, the contract requires `find` to return the value `n`, which indicates that the search was unsuccessful.

The formulas used in the `assumes`-clauses of the behaviors `some` and `none` are logical negations of each other. Consequently, the two behaviors are *disjoint* and *complete*: for every input satisfying the preconditions, exactly one of the two behaviors applies.

4.1.2. Implementation of find

The implementation of find [4.3] follows a simple linear search scheme.

```
size_type find(const value_type* a, size_type n, value_type v)
{
  /*@
    loop invariant 0 <= i <= n;
    loop invariant \forall integer k; 0 <= k < i ==> a[k] != v;
    loop assigns i;
    loop variant n-i;
   */
  for (size_type i = 0u; i < n; i++) {
    if (a[i] == v) {
      return i;
    }
  }

  return n;
}
```

Listing 4.3: Implementation of find

The array a is traversed from left to right until either an occurrence of the sought-after value is found or the end of the array is reached. If the value is found, the function returns its index; otherwise it indicates failure in accordance with the contract of find [4.2]. The algorithm performs no modification of the input array and executes at most n iterations.

The noteworthy elements of the implementation are the *loop annotations*. The first loop *invariant* establishes that all accesses to a are performed with valid indices. The second loop *invariant* supports the proof of the postconditions of the behavior some in the contract of find [4.2] by expressing that, at each iteration step, the sought-after value has not been encountered in the already inspected prefix of the array. Finally, the loop *variant* n-i is required to generate the verification conditions that establish termination of the loop.

4.2. The find2 algorithm—reuse of specification elements

In this section we specify find in a slightly different way. The approach is motivated by the observation that a considerable number of closely related ACSL formulas occur both in the contract of find [4.2] and in the implementation find [4.3]. Typical examples are the following formulas:

```
\exists integer i; 0 <= i < n        &&   a[i] == v;

\forall integer i; 0 <= i < \result  ==>  a[i] != v;

\forall integer i; 0 <= i < n        ==>  a[i] != v;

\forall integer k; 0 <= k < i        ==>  a[k] != v;
```

Observe that the first formula is the logical negation of the third one. More generally, these formulas express two complementary situations: either some element of the array equals the searched value, or no such element exists in a given range.

4.2.1. The predicates SomeEqual and NoneEqual

To make these common patterns explicit and reusable, we introduce in the following listing two predicates, SomeEqual [4.4] and NoneEqual [4.4]. Intuitively, SomeEqual expresses that there exists a valid index i such that the array element a[i] equals the value v, whereas NoneEqual states that no such index exists. These predicates abstract recurring existential and universal quantification into named logical concepts that can be used uniformly in both specifications and proofs.

```
/*@
  predicate SomeEqual{A}(value_type* a, integer m, integer n, value_type v) =
    \exists integer i; m <= i < n && a[i] == v;

  predicate SomeEqual{A}(value_type* a, integer n, value_type v) =
    SomeEqual(a, 0, n, v);

  predicate NoneEqual(value_type* a, integer m, integer n, value_type v) =
    \forall integer i; m <= i < n  ==>  a[i] != v;

  predicate NoneEqual(value_type* a, integer n, value_type v) =
    NoneEqual(a, 0, n, v);

  lemma NotSomeEqual_NoneEqual:
    \forall value_type *a, v, integer m, n;
      !SomeEqual(a, m, n, v)  ==>  NoneEqual(a, m, n, v);

  lemma NoneEqual_NotSomeEqual:
    \forall value_type *a, v, integer m, n;
     NoneEqual(a, m, n, v)   ==>  !SomeEqual(a, m, n, v);
*/
```

Listing 4.4: SomeNone (ACSL)

Note that both predicates are provided in overloaded variants. One variant applies to array sections, while the other applies to complete arrays. This avoids the need for separate predicate names while still covering both use cases.

Finally, note that the predicate `SomeEqual` is equipped with an explicit label, namely `A`. This label indicates that the evaluation of the predicate depends on a specific memory state, namely the contents of `a[0..n-1]` at label `A`. Formally, the defining formula corresponds to

```
\exists integer i; 0 <= i < n && \at(a[i],A) == v;
```

In accordance with ACSL conventions, the explicit use of `\at` can be omitted whenever the label `A` is the only label in scope. For this reason, the abbreviated form `a[i]` is used in the definition of `SomeEqual` and `NoneEqual`, and the label is omitted entirely in the overloaded versions for complete arrays.

4.2.2. Formal specification of `find2`

With the predicates `SomeEqual` [4.4] and `NoneEqual` [4.4] we encapsulate all occurrences of existential and universal quantification in both the specification and the implementation of `find2` [4.5].

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior some:
    assumes           SomeEqual(a, n, v);
    assigns           \nothing;
    ensures  bound:   0 <= \result < n;
    ensures  result:  a[\result] == v;
    ensures  first:   NoneEqual(a, \result, v);

  behavior none:
    assumes           NoneEqual(a, n, v);
    assigns           \nothing;
    ensures  result:  \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type find2(const value_type* a, size_type n, value_type v);
```

Listing 4.5: Formal specification of `find2`

As a result, the revised contract is more concise and more readable than that of `find` [4.2]. In particular, the assumptions of the two behaviors `some` and `none` are manifestly mutually exclusive, since one is the literal negation of the other. This mutual exclusivity, which was previously implicit in the structure of quantified formulas, is now expressed explicitly through predicate usage.

Moreover, the requirement that `find` returns the smallest index at which the searched value occurs can be expressed naturally using the `NoneEqual` [4.4] predicate. This is reflected in the final postcondition of behavior `some`, which states that no occurrence of the value precedes the returned index.

Finally, the specification of `find2` makes systematic use of user-defined names for `requires` and `ensures` clauses (sometimes also called *labels*, the distinction from program-state identifiers being clear from the context). We strongly recommend this practice, especially for more complex annotations. Among other

benefits, Frama-C can be instructed to verify only clauses with a given name, which significantly improves proof control and diagnosability.

4.2.3. Implementation of `find2`

The predicate `NoneEqual` is also used in the loop annotations of the implementation `find2` [4.6].

```
size_type find2(const value_type* a, size_type n, value_type v)
{
  /*@
    loop invariant bound:     0 <= i <= n;
    loop invariant not_found: NoneEqual(a, i, v);
    loop assigns i;
    loop variant n-i;
   */
  for (size_type i = 0u; i < n; i++) {
    if (a[i] == v) {
      return i;
    }
  }

  return n;
}
```

Listing 4.6: Implementation of `find2`

As in the function contract, this allows us to avoid explicit universal quantification in the annotations and to express the relevant properties in a uniform and declarative way. In particular, `NoneEqual` is used to state that all elements examined so far differ from the searched value. This closely mirrors the informal loop invariant of the algorithm and aligns the structure of the implementation annotations with that of the formal specification.

As in the specification, we systematically use labels to name individual annotations. This improves readability and enables selective verification of specific invariants or assertions.

4.3. The find3 algorithm—using a logic function

In this section we present a third specification of linear search. While this approach requires additional preparatory work, it results in a significantly more concise and transparent function contract. At the end of this section we will discuss in what sense the contracts of find2 and find3 are equivalent.

4.3.1. The logic function Find

We begin with a *recursive* definition of the ACSL logic function Find. Due to the number of lemmas associated with Find, its definition is distributed over the Listings 4.7 and 4.8. The function is provided in two *overloaded* variants: the first operates on *array sections*, whereas the second is intended for *complete arrays*.

The first group of lemmas captures elementary properties of Find that are directly related to extending the search range from a[0..n-1] to a[0..n]. Lemmas of the first group characterize how the value of Find changes when the search interval is extended by one element.

```
/*@
  logic integer
  Find(value_type* a, integer m, integer n, value_type v) =
    (n <= m) ?
     0 : ((0 <= Find(a, m, n-1, v) < n-m-1) ?
       Find(a, m, n-1, v) : ((a[n-1] == v) ? n-m-1 : n-m));

  logic integer
  Find(value_type* a, integer n, value_type v) = Find(a, 0, n, v);

  lemma Find_Empty:
    \forall value_type *a, v, integer m, n;
      n <= m  ==>  Find(a, m, n, v) == 0;

  lemma Find_Hit:
    \forall value_type *a, v, integer m, n;
      m <= n                  ==>
      Find(a, m, n, v) < n-m  ==>
      Find(a, m, n+1, v) == Find(a, m, n, v);

  lemma Find_MissHit:
    \forall value_type *a, v, integer m, n;
      m <= n                      ==>
      a[n] == v                   ==>
      Find(a, m, n, v)   == n-m   ==>
      Find(a, m, n+1, v) == n-m;

  lemma Find_MissMiss:
    \forall value_type *a, v, integer m, n;
      m <= n                      ==>
      a[n] != v                   ==>
      Find(a, m, n, v)   == n-m   ==>
      Find(a, m, n+1, v) == (n+1)-m;
```

Listing 4.7: The logic function Find (1)

Lemmas of the second group relate Find to the predicates SomeEqual [4.4] and NoneEqual [4.4], thereby connecting the functional view of search with its predicate-based formulation. Further lemmas establish equivalences between the two overloaded versions of Find and enable its use in function contracts.

```
  lemma Find_Lower:
    \forall value_type *a, v, integer m, n;
      0 <= Find(a, m, n, v);

  lemma Find_Upper:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  Find(a, m, n, v) <= n-m;

  lemma Find_WeaklyIncreasing:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  Find(a, m, n, v) <= Find(a, m, n+1, v);

  lemma Find_Increasing:
    \forall value_type *a, v, integer k, m, n;
      m <= k <= n  ==>
      Find(a, m, k, v) <= Find(a, m, n, v);

  lemma Find_Limit:
    \forall value_type *a, v, integer k, m, n;
      m <= k < n  ==>
      a[k] == v   ==>
      Find(a, m, n, v) <= k-m;

  lemma Find_NoneEqual:
    \forall value_type *a, v, integer m, n;
      m <= n                ==>
      NoneEqual(a, m, n, v)  ==>
      Find(a, m, n, v) == n-m;

  lemma Find_SomeEqual:
    \forall value_type *a, v, integer k, m, n;
      m <= k < n            ==>
      a[k] == v             ==>
      NoneEqual(a, m, k, v)  ==>
      Find(a, m, n, v) == k-m;

  lemma Find_ResultNoneEqual:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  NoneEqual(a, m, m + Find(a, m, n, v), v);

  lemma Find_ResultEqual:
    \forall value_type *a, v, integer m, n;
      0 <= Find(a, m, n, v) < n-m  ==>
      a[m + Find(a, m, n, v)] == v;
*/
```

Listing 4.8: The logic function `Find` (2)

4.3.2. Formal specification of `find3`

Using the logic function `Find` we can now give a third formal specification of linear search. In contrast to `find2` [4.5], all quantification over the input array is encapsulated in the logic function itself. As a result, the contract of `find3` [4.9] is significantly more compact and its logical structure is easier to recognize at a glance. This conciseness is achieved at the cost of a more elaborate logical infrastructure: the burden of reasoning about existence, minimality, and absence of matching elements is shifted from the function contract to the definition of the ACSL logic function `Find` [4.7] and its associated lemmas.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;
  ensures    result:  \result == Find(a, n, v);
*/
size_type find3(const value_type* a, size_type n, value_type v);
```

Listing 4.9: Formal specification of `find3`

4.3.3. Implementation of `find3`

The following listing shows the implementation of `find3` [4.10]. Operationally, the code is similar to the previous implementations of linear search. However, the verification now relies on relating the operational behavior of the loop to the abstract specification given by `Find` [4.7]. To this end, we introduce the assertion `found`, which makes explicit the connection between the current loop index and the value returned by `Find`. This assertion serves as a bridge between the concrete control flow of the implementation and the functional characterization used in the contract.

```
size_type find3(const value_type* a, size_type n, value_type v)
{
  /*@
    loop invariant bound:     0 <= i <= n;
    loop invariant not_found: Find(a, i, v) == i;
    loop assigns i;
    loop variant n-i;
   */
  for (size_type i = 0u; i < n; i++) {
    if (a[i] == v) {
      //@ assert found: Find(a, n, v) == i;
      return i;
    }
  }

  return n;
}
```

Listing 4.10: Implementation of `find3`

4.3.4. The equivalence of `find2` and `find3`

A natural question at this point is in what precise sense the contracts of `find2` [4.5] and `find3` [4.9] can be regarded as equivalent. In this section we make this notion of equivalence explicit and show how it can be established by verification.

We call two function contracts *equivalent* if each of them is sufficiently strong to verify an implementation that is specified using the other. In other words, either contract can be used as a drop-in replacement for the other in a verified context.

To demonstrate this mutual sufficiency, we introduce two auxiliary examples, `find4` and `find5`. Both functions are trivial wrappers whose implementations consist of a single call to an already verified linear search routine, while their contracts are taken from the alternative specification.

The implementation of `find4` [4.11] consists solely of a call to `find3`:

```
size_type find4(const value_type* a, size_type n, value_type v)
{
  return find3(a, n, v);
}
```

Listing 4.11: Implementation of `find4`

The contract of `find4` [4.12], however, is identical to that of `find2` [4.5]:

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior some:
    assumes           SomeEqual(a, n, v);
    assigns           \nothing;
    ensures  bound:   0 <= \result < n;
    ensures  result:  a[\result] == v;
    ensures  first:   NoneEqual(a, \result, v);

  behavior none:
    assumes           NoneEqual(a, n, v);
    assigns           \nothing;
    ensures  result:  \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type find4(const value_type* a, size_type n, value_type v);
```

Listing 4.12: Formal specification of `find4`

Analogously, the implementation of `find5` [4.13] is a direct call to `find2`:

```
size_type find5(const value_type* a, size_type n, value_type v)
{
  return find2(a, n, v);
}
```

Listing 4.13: Implementation of `find5`

The contract of `find5` [4.14], on the other hand, coincides with that of `find3` [4.9]. Thus, `find5` reverses the roles of implementation and specification compared to `find4`.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;
  ensures    result:  \result == Find(a, n, v);
*/
size_type find5(const value_type* a, size_type n, value_type v);
```

Listing 4.14: Formal specification of `find5`

The successful verification of both wrapper functions (cf. Table A.2) shows that each contract is sufficient to justify an implementation written against the other. This establishes the equivalence of the contracts of `find2` [4.5] and `find3` [4.9].

The notion of equivalence used here is inherently *contextual*. It is defined relative to the underlying specification language, the available logical infrastructure, and the verification framework. Two contracts are considered equivalent not in an abstract semantic sense, but insofar as each can be used to verify implementations specified with the other within the same formal setting.

4.4. The find_if_not algorithm

Many algorithms in the C++ standard library are parameterized not only by the type of the input sequence but also by so-called *function objects*. A representative example is the `find_if_not` algorithm, which takes a *predicate function object* P and returns the first position i in the input sequence for which $P(i)$ does *not* hold.

Although function objects could in principle be emulated in C by means of *function pointers*, we deliberately do not pursue this approach here. At present, function pointers are only partially supported by Frama-C, and there is currently no support for parameterized ACSL predicates. For these reasons, our implementation of `find_if_not` is restricted to the special case of locating the first position in an array at which a given value does *not* occur. The resulting function signature is therefore

```
size_type find_if_not(const value_type* a, size_type n, value_type v);
```

From an algorithmic point of view, this is not a particularly exciting addition to our collection of verified algorithms. Its value lies instead in providing a clean setting for introducing the predicates `AllEqual` and `SomeNotEqual` [4.15], and, more importantly, the logic function `FindNotEqual` [4.16]. These definitions will later play a central role in the specification of the algorithm `remove_copy`, and in particular of its variant `remove_copy3` [7.53].

```
/*@
    predicate AllEqual(value_type* a, integer m, integer n, value_type v) =
      \forall integer i; m <= i < n  ==>  a[i] == v;

    predicate AllEqual(value_type* a, integer m, integer n) =
      AllEqual(a, m, n, a[m]);

    predicate AllEqual(value_type* a, integer n, value_type v) =
      AllEqual(a, 0, n, v);

    predicate SomeNotEqual{A}(value_type* a, integer m, integer n, value_type v) =
      \exists integer i; m <= i < n && a[i] != v;

    predicate SomeNotEqual{A}(value_type* a, integer n, value_type v) =
      SomeNotEqual(a, 0, n, v);

    lemma NotAllEqual_SomeNotEqual:
      \forall value_type *a, v, integer m, n;
        !AllEqual(a, m, n, v)  ==>  SomeNotEqual(a, m, n, v);

    lemma SomeNotEqual_NotAllEqual:
      \forall value_type *a, v, integer m, n;
       SomeNotEqual(a, m, n, v)   ==>  !AllEqual(a, m, n, v);
*/
```

Listing 4.15: `AllSomeNot` (ACSL)

The predicate `AllEqual` expresses that every element of the array segment `a[m..n-1]` is equal to the value `v`. The predicate `SomeNotEqual` is defined as its logical negation. Together, these predicates naturally complement the previously introduced predicates `SomeEqual` [4.4] and `NoneEqual`.

In addition, two overloaded variants of `AllEqual` are provided. The first variant uses the value `a[m]` itself as the reference value `v`. The second variant is a convenient abbreviation for the common case where the lower bound `m` is equal to 0.

4.4.1. The logic function `FindNotEqual`

The definition of the overloaded logic function `FindNotEqual` is given in Listings 4.16. This function is closely related to the logic function `Find` [4.7]. While `Find` [4.7] returns the first position in a sequence at which a given value occurs, `FindNotEqual` returns the first position at which the array element *differs* from a given reference value.

More precisely, `FindNotEqual` characterizes the smallest index i in the range under consideration such that `a[i] != v` holds. If no such position exists, the function yields the upper bound of the considered range. This behavior mirrors that of `Find` [4.7] and allows the two functions to be used in a largely symmetric manner in specifications.

```
/*@
  logic integer
  FindNotEqual(value_type* a, integer m, integer n, value_type v) =
    (n <= m) ?
     0 : ((0 <= FindNotEqual(a, m, n-1, v) < n-m-1) ?
       FindNotEqual(a, m, n-1, v) : ((a[n-1] != v) ? n-m-1 : n-m));

  logic integer
  FindNotEqual(value_type* a, integer n, value_type v) =
    FindNotEqual(a, 0, n, v);

  lemma FindNotEqual_Empty:
    \forall value_type *a, v, integer m, n;
      n <= m  ==>  FindNotEqual(a, m, n, v) == 0;

  lemma FindNotEqual_Hit:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  FindNotEqual(a, m, n, v) < n-m  ==>
      FindNotEqual(a, m, n+1, v) == FindNotEqual(a, m, n, v);

  lemma FindNotEqual_MissHit:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  a[n] != v             ==>
      FindNotEqual(a, m, n, v)   == n-m  ==>
      FindNotEqual(a, m, n+1, v) == n-m;

  lemma FindNotEqual_MissMiss:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  a[n] == v             ==>
      FindNotEqual(a, m, n, v)   == n-m  ==>
      FindNotEqual(a, m, n+1, v) == (n+1)-m;

  lemma FindNotEqual_Lower:
    \forall value_type *a, v, integer m, n;
      0 <= FindNotEqual(a, m, n, v);

  lemma FindNotEqual_Upper:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  FindNotEqual(a, m, n, v) <= n-m;
```

Listing 4.16: The logic function `FindNotEqual` (1)

In addition to the core definition, several lemmas establish basic properties of `FindNotEqual`. Among them is the lemma `FindNotEqual_Unchanged`, which expresses the stability of the function under modifications of array segments that are known to be unchanged. This lemma relies on the predicate `Unchanged` [7.1], which will be introduced formally in a later chapter. The forward reference is intentional, as the lemma already reflects a recurring verification pattern that will be used extensively in subsequent algorithm specifications.

```
  lemma FindNotEqual_Unchanged{K,L}:
    \forall value_type *a, v, integer m, n;
      Unchanged{K,L}(a, m, n)  ==>
      FindNotEqual{K}(a, m, n, v) == FindNotEqual{L}(a, m, n, v);

  lemma FindNotEqual_WeaklyIncreasing:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  FindNotEqual(a, m, n, v) <= FindNotEqual(a, m, n+1, v);

  lemma FindNotEqual_Increasing:
    \forall value_type *a, v, integer k, m, n;
      m <= k <= n  ==>  FindNotEqual(a, m, k, v) <= FindNotEqual(a, m, n, v);

  lemma FindNotEqual_Limit:
    \forall value_type *a, v, integer k, m, n;
      m <= k < n  ==>
      a[k] != v   ==>  FindNotEqual(a, m, n, v) <= k-m;

  lemma FindNotEqual_AllEqual:
    \forall value_type *a, v, integer m, n;
      m <= n                ==>
      AllEqual(a, m, n, v)  ==>  FindNotEqual(a, m, n, v) == n-m;

  lemma FindNotEqual_SomeNotEqual:
    \forall value_type *a, v, integer k, m, n;
      m <= k < n            ==>  a[k] != v  ==>
      AllEqual(a, m, k, v)  ==>  FindNotEqual(a, m, n, v) == k-m;

  lemma FindNotEqual_ResultAllEqual:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  AllEqual(a, m, m + FindNotEqual(a, m, n, v), v);

  lemma FindNotEqual_ResultNotEqual:
    \forall value_type *a, v, integer m, n;
      0 <= FindNotEqual(a, m, n, v) < n-m  ==>
      a[m + FindNotEqual(a, m, n, v)] != v;
*/
```

Listing 4.17: The logic function `FindNotEqual` (2)

4.4.2. Formal specification of `find_if_not`

The contract of `find_if_not` [4.18] is, unsurprisingly, very similar to that of `find3` [4.9]. Both specifications follow the same overall structure and rely on a logic function that characterizes the first position in the array at which a distinguishing property holds.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;
  ensures    result:  \result == FindNotEqual(a, n, v);
*/
size_type find_if_not(const value_type* a, size_type n, value_type v);
```

Listing 4.18: Formal specification of `find_if_not`

The only substantive difference is that the logic function `Find` [4.7] is replaced by `FindNotEqual` [4.16]. As a consequence, the postcondition no longer characterizes the first occurrence of a given value, but instead the first position at which the array element differs from the reference value. All other aspects of the contract — in particular the treatment of the case where no such position exists — remain unchanged.

4.4.3. Implementation of `find_if_not`

The implementation of `find_if_not` [4.19] closely mirrors that of `find3` [4.10]. The control structure, loop invariants, and termination argument are identical. As in the specification, the only essential change is the systematic replacement of `Find` by `FindNotEqual`.

```
size_type find_if_not(const value_type* a, size_type n, value_type v)
{
  /*@
    loop invariant bound:     0 <= i <= n;
    loop invariant not_found: FindNotEqual(a, i, v) == i;
    loop assigns i;
    loop variant n-i;
   */
  for (size_type i = 0u; i < n; i++) {
    if (a[i] != v) {
      //@ assert found: FindNotEqual(a, n, v) == i;
      return i;
    }
  }

  return n;
}
```

Listing 4.19: Implementation of `find_if_not`

In the loop body, this change manifests itself in the condition that is checked at each iteration: instead of testing whether `a[i]` equals the given value, we now test that it *differs* from `v`. The proof obligations generated by this implementation are discharged in the same way as for `find3` [4.10], using the corresponding lemmas for `FindNotEqual`.

4.5. The `find_first_of` algorithm

The algorithm `find_first_of` [18, §28.5.7] can be viewed as a natural generalization of `find` (see §4.1 and §4.2). Whereas `find` performs a sequential search for a single, fixed value, `find_first_of` searches for the first occurrence of *any* value drawn from a given set.

```
size_type find_first_of(const value_type* a, size_type m,
                        const value_type* b, size_type n);
```

Operationally, `find_first_of` performs a linear scan of the array `a[0..m-1]` and returns the least index `i` such that the element `a[i]` is equal to at least one element of the array `b[0..n-1]`. If no such index exists, the function indicates failure in the same way as `find`. The algorithm thus combines an outer sequential search over `a` with an implicit membership test against the array `b`.

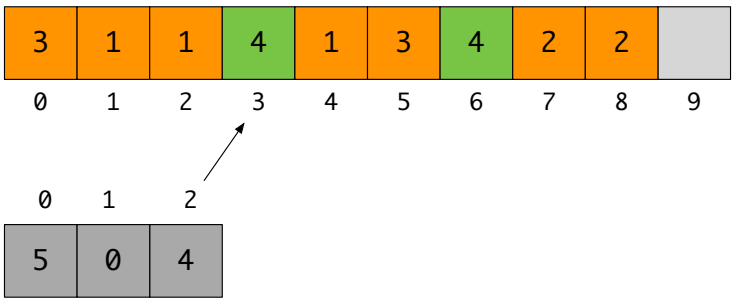


Figure 4.20.: A simple example of `find_first_of`

Figure 4.20 illustrates this behavior. Among the elements of the left-hand array, the arrow marks the smallest index at which a value occurs that is also present in the three-element array on the right.

4.5.1. The predicate `HasValueOf`

In order to express this membership test at the logic level, we introduce the predicate `HasValueOf`. Intuitively, `HasValueOf` expresses that the two arrays `a` and `b` share at least one common value at valid indices. More precisely, it asserts the existence of indices `i` and `j` within bounds such that `a[i] == b[j]` holds.

```
/*@
  predicate HasValueOf{A}(value_type* a, integer m, value_type* b, integer n) =
    \exists integer i; 0 <= i < m && SomeEqual{A}(b, n, a[i]);
*/
```

Listing 4.21: `HasValueOf` (ACSL)

As in our treatment of `find2` (§4.2), we deliberately encapsulate this existential quantification in a named predicate. This keeps the function contract of `find_first_of` free of explicit quantifiers and clarifies the logical structure: the algorithm succeeds exactly when `HasValueOf` holds.

Rather than introducing a new primitive notion of equality, `HasValueOf` is defined in terms of the established predicate `SomeEqual` [4.4]. This highlights the close conceptual relationship between `find_first_of` and earlier variants of `find`, while avoiding unnecessary duplication at the specification level.

4.5.2. Formal specification of `find_first_of`

The following listing presents the formal specification of `find_first_of`. As in the specification of `find2` [4.5], the function contract is formulated in terms of named predicates rather than explicit quantification. In particular, it relies on the predicates `HasValueOf` [4.21] and `SomeEqual` [4.4] to characterize the existence of matching elements between the two input arrays.

```
/*@
  requires   valid:  \valid_read(a + (0..m-1));
  requires   valid:  \valid_read(b + (0..n-1));

  terminates        \true;
  exits             \false;
  assigns           \nothing;

  ensures    result:   0 <= \result <= m;

  behavior found:
    assumes          HasValueOf(a, m, b, n);
    assigns          \nothing;
    ensures bound:   0 <= \result < m;
    ensures result:  SomeEqual(b, n, a[\result]);
    ensures first:   !HasValueOf(a, \result, b, n);

  behavior not_found:
    assumes          !HasValueOf(a, m, b, n);
    assigns          \nothing;
    ensures result:  \result == m;

  complete behaviors;
  disjoint behaviors;
*/
size_type find_first_of(const value_type* a, size_type m,
                        const value_type* b, size_type n);
```

Listing 4.22: Formal specification of `find_first_of`

This choice makes the logical structure of the contract immediately apparent. The successful behavior corresponds to the case where `HasValueOf` holds, that is, where at least one element of `a` occurs in `b`. Conversely, the failure behavior captures the absence of any such common value. By reusing `SomeEqual` [4.4], the specification emphasizes the close relationship between `find_first_of` and the earlier variants of `find`, while avoiding a proliferation of ad hoc predicates.

4.5.3. Implementation of `find_first_of`

The implementation of `find_first_of` [4.23] is deliberately structured as a thin wrapper around the previously verified function `find2` [4.5]. Rather than performing a nested search explicitly, the algorithm delegates the inner membership test to `find2`, thereby making reuse an explicit design choice.

```
size_type find_first_of (const value_type* a, size_type m,
                         const value_type* b, size_type n)
{
  /*@
    loop invariant bound:      0 <= i <= m;
    loop invariant not_found: !HasValueOf(a, i, b, n);
    loop assigns i;
    loop variant m-i;
  */
  for (size_type i = 0u; i < m; i++) {
    if (find2(b, n, a[i]) < n) {
      return i;
    }
  }

  return m;
}
```

Listing 4.23: Implementation of `find_first_of`

This approach has two immediate benefits. First, it leads to a concise and readable implementation whose control flow closely mirrors the informal description of the algorithm. Second, and more importantly from a verification perspective, it significantly reduces the annotation burden: no additional loop invariants are required beyond those already established for `find2`. As a result, the correctness of `find_first_of` follows largely from the existing specification and proof of `find2`, illustrating how verified components can be composed to obtain new, verified functionality with minimal additional effort.

From an algorithmic point of view, this implementation realizes `find_first_of` as a linear scan over the array `a`. At each position, the current element `a[i]` is tested for membership in the array `b` by invoking `find2`. The function thus combines an outer sequential traversal with a logically specified membership test, closely reflecting the structure captured by the predicate `HasValueOf` [4.21] at the specification level.

4.6. The adjacent_find algorithm

The algorithm adjacent_find, as specified in the C++ Standard Library [18, §28.5.8], searches an array for the first occurrence of two equal adjacent elements.

```
size_type adjacent_find(const value_type* a, size_type n);
```

More precisely, adjacent_find returns the smallest index i such that both i and i+1 are valid indices of the array and

```
a[i] == a[i+1]
```

holds. If no such index exists, the algorithm returns n.

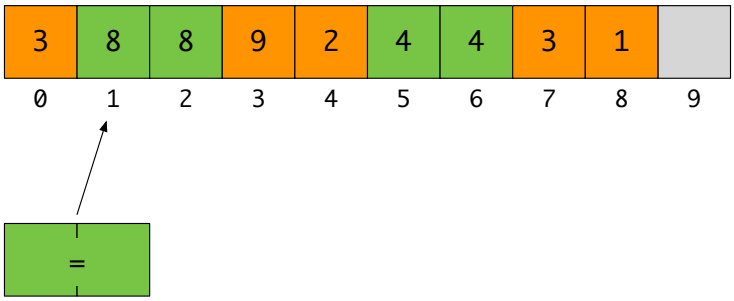


Figure 4.24.: A simple example of adjacent_find

Figure 4.24 illustrates this behavior. The arrow marks the smallest index at which two adjacent elements of the array are equal.

4.6.1. The predicate HasEqualNeighbors

As in the case of the other search algorithms considered so far, we first introduce a predicate that captures the essential logical property underlying the algorithm. The predicate HasEqualNeighbors [4.25] formalizes the existence of two adjacent valid indices at which the array holds equal values.

```
/*@
  predicate HasEqualNeighbors{L}(value_type* a, integer n) =
    \exists integer i; 0 <= i < n-1 && a[i] == a[i+1];
*/
```

Listing 4.25: The predicate HasEqualNeighbors

By encapsulating this property in a named predicate, we avoid explicit quantification in the function contract and make the specification more readable and structured.

4.6.2. Formal specification of `adjacent_find`

Our specification of `adjacent_find` [4.26] relies on the predicate `HasEqualNeighbors` [4.25]. This predicate captures the essential property underlying the algorithm, namely the existence of two adjacent valid indices at which the array contains equal values.

```
/*@
  requires   valid:     \valid_read(a + (0..n-1));

  terminates            \true;
  exits                 \false;
  assigns               \nothing;

  ensures   result:     0 <= \result <= n;

  behavior some:
    assumes             HasEqualNeighbors(a, n);
    assigns             \nothing;
    ensures  result:    0 <= \result < n-1;
    ensures  adjacent:  a[\result] == a[\result+1];
    ensures  first:     !HasEqualNeighbors(a, \result);

  behavior none:
    assumes             !HasEqualNeighbors(a, n);
    assigns             \nothing;
    ensures  result:    \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type adjacent_find(const value_type* a, size_type n);
```

Listing 4.26: Formal specification of `adjacent_find`

As in the specifications of the other search algorithms, the function contract is structured into distinct behaviors. The successful behavior corresponds exactly to the case where `HasEqualNeighbors` holds, while the failure behavior captures its negation. By formulating the contract in terms of this predicate, the specification avoids explicit quantification and makes the intended meaning of the postcondition immediately apparent.

4.6.3. Implementation of `adjacent_find`

The implementation of `adjacent_find` [4.27] follows closely the informal description of the algorithm. As a preliminary step, it checks whether the array contains at least two elements. If this is not the case, no pair of adjacent elements can exist, and the function can immediately return failure.

```
size_type adjacent_find(const value_type* a, size_type n)
{
  if (1u < n) {
    /*@
      loop invariant bound:  0 <= i < n;
      loop invariant none:   !HasEqualNeighbors(a, i+1);
      loop assigns i;
      loop variant n-i;
    */
    for (size_type i = 0u; i + 1u < n; ++i) {
      if (a[i] == a[i + 1u]) {
        return  i;
      }
    }
  }

  return n;
}
```

Listing 4.27: Implementation of `adjacent_find`

For the general case, the algorithm performs a linear scan of the array, comparing each element with its immediate successor. The loop invariant makes explicit use of the predicate `HasEqualNeighbors` [4.25] to relate the progress of the scan to the postcondition of the successful behavior. In particular, it ensures that, at each iteration, either a matching pair has already been encountered or no such pair exists in the portion of the array examined so far.

This close correspondence between the loop invariant and the function postcondition allows the correctness of the implementation to be established in a direct and transparent manner.

4.7. The equal and mismatch algorithms

The algorithms equal [18, §28.5.11] and mismatch [18, §28.5.10] of the C++ Standard Library compare two sequences element by element. For our purposes, we restrict attention to arrays of a fixed value type value_type and adapt the generic interfaces accordingly. The resulting function signatures are

```
bool       equal(const value_type* a, size_type n, const value_type* b);

size_type  mismatch(const value_type* a, size_type n, const value_type* b);
```

The function equal returns **true** if and only if a[i] == b[i] holds for every index 0 <= i < n. If this condition fails for at least one index, the function returns **false**.

The algorithm mismatch is slightly more informative. Rather than merely reporting whether the two ranges are equal, it returns the smallest index at which the arrays a and b differ. If no such index exists—that is, if the arrays agree at all positions 0 <= i < n—then mismatch returns the common length n of the two ranges.

4.7.1. The Equal predicate

The fact that two arrays a[0]..a[n-1] and b[0]..b[n-1] coincide element by element is a basic property that arises naturally in many specifications. Rather than restating this condition repeatedly, it is preferable to capture it once in a dedicated predicate. To this end, we define several *overloaded* versions of the predicate Equal [4.28], shown in the following listing.

```
/*@
  predicate Equal{K,L}(value_type* a, integer m, integer n, value_type* b) =
    \forall integer i; m <= i < n  ==>  \at(a[i],K) == \at(b[i],L);

  predicate Equal{K,L}(value_type* a, integer n, value_type* b) =
    Equal{K,L}(a, 0, n, b);

  predicate Equal{K,L}(value_type* a, integer m, integer n,
                       value_type* b, integer p) =
    \forall integer k; 0 <= k < n-m ==> \at(a[m+k],K) == \at(b[p+k],L);

  predicate Equal{K,L}(value_type* a, integer m, integer n, integer p) =
      Equal{K,L}(a, m, n, a, p);
*/
```

Listing 4.28: Equal (ACSL)

The parameters K and L in the definition of Equal are so-called *labels*. They denote the program states in which the arrays a[..] and b[..] are to be evaluated. The Frama-C tool chain provides several standard labels, such as Old and Post, which refer to the pre-state and post-state of a function, respectively. User-defined labels may be introduced to name intermediate program points. For a detailed discussion of labels and their semantics, we refer to the ACSL specification [9, §2.6.9].

Mathematical description For integers m and n with $0 \leqslant m \leqslant n$, we denote by

$$[m \mathinner{..} n) \tag{4.1}$$

the right-open integer interval $\{i \in \mathbb{Z} \mid m \leqslant i < n\}$.

For an array a and integers m and n with $0 \leqslant m \leqslant n$, we denote by

$$a[m \mathinner{..} n) \tag{4.2}$$

the following *array slice*

$$\big(a[m], a[m+1], \ldots, a[n-1]\big).$$

For a label K, the notation $a^K[m \mathinner{..} n)$ denotes the same slice, with each element evaluated in the program state K, that is,

$$a^K[i] = \mathrm{at}(a[i], K). \tag{4.3}$$

Here, $\mathrm{at}(x, K)$ denotes the value of the expression x in the program state K, corresponding to the ACSL construct `\at`.

The following equations provide a concise mathematical description of the overloaded variants of the predicate `Equal` defined in Listing 4.28.

`Equal`$\{K, L\}(a, m, n, b)$ expresses equality of two array slices over the same index range:

$$a^K[m \mathinner{..} n) = b^L[m \mathinner{..} n), \qquad \textbf{Base variant} \tag{4.4a}$$

that is,

$$a^K[i] = b^L[i] \qquad \forall i \in [m \mathinner{..} n).$$

`Equal`$\{K, L\}(a, n, b)$ is a shorthand for (4.4a) with $m = 0$:

$$a^K[0 \mathinner{..} n) = b^L[0 \mathinner{..} n), \qquad \textbf{Prefix variant} \tag{4.4b}$$

`Equal`$\{K, L\}(a, m, n, b, p)$ generalizes (4.4a) by starting the right-hand slice at index p:

$$a^K[m \mathinner{..} n) = b^L\big[p \mathinner{..} p + (n - m)\big), \qquad \textbf{Shifted-slice variant} \tag{4.4c}$$

`Equal`$\{K, L\}(a, m, n, p)$ instantiates (4.4c) for the case $a = b$:

$$a^K[m \mathinner{..} n) = a^L\big[p \mathinner{..} p + (n - m)\big), \qquad \textbf{Same-array variant} \tag{4.4d}$$

4.7.2. Formal specification of `equal` and `mismatch`

Using the predicate `Equal` [4.28], we can give a concise and precise specification of `equal` [4.29].

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));
  requires   valid:   \valid_read(b + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  \result <==> Equal{Here,Here}(a, n, b);
*/
bool equal(const value_type* a, size_type n, const value_type* b);
```

Listing 4.29: Formal specification of `equal`

In the `ensures` clause, `Equal` is instantiated with the predefined label `Here`, which refers to the post-state of the function. This states that, upon termination, the two input ranges are equal element by element. Note that the postcondition is formulated as a logical equivalence. An equality would not be well-formed in ACSL, since `Equal` is a predicate, not a logic function, and therefore cannot appear as the argument of the equality operator.

The formal specification of `mismatch` [4.30] is necessarily more elaborate. Unlike `equal`, which merely reports whether the two arrays coincide, `mismatch` returns additional information in the form of an index.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));
  requires   valid:   \valid_read(b + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior all_equal:
    assumes           Equal{Here,Here}(a, n, b);
    assigns           \nothing;
    ensures result:   \result == n;

  behavior some_not_equal:
    assumes           !Equal{Here,Here}(a, n, b);
    assigns           \nothing;
    ensures bound:    0 <= \result < n;
    ensures result:   a[\result] != b[\result];
    ensures first:    Equal{Here,Here}(a, \result, b);

  complete behaviors;
  disjoint behaviors;
*/
size_type mismatch(const value_type* a, size_type n, const value_type* b);
```

Listing 4.30: Formal specification of `mismatch`

Conceptually, the specification of `mismatch` closely parallels that of `find2` [4.5]. While `find2` returns the smallest index `i` such that `a[i] == v` holds, `mismatch` returns the smallest index `i` such that `a[i] != b[i]`. In both cases, the returned index identifies the first position at which a distinguishing condition becomes true.

As in the specification of `find2`, the behaviors of `mismatch` are both complete and mutually exclusive. This is apparent from the `assumes` clauses of the behaviors `all_equal` and `some_not_equal`, which are exact negations of one another. The predicate `Equal` [4.28] plays a central role here, both in expressing the case where no mismatch exists and in delimiting the prefix of equal elements in the case where a mismatch is found.

4.7.3. Implementation of `equal` and `mismatch`

The implementation of `equal` [4.31] is deliberately minimal: it consists solely of a call to `mismatch` and a comparison of the returned index with the array length `n`. This directly reflects the relationship between the two algorithms established at the level of their specifications.

```
bool equal(const value_type* a, size_type n, const value_type* b)
{
  return mismatch(a, n, b) == n;
}
```

Listing 4.31: Implementation of `equal`

The implementation of `mismatch` [4.32] follows the expected linear scan of the two arrays, but is enriched with loop annotations to support deductive verification.

```
size_type mismatch(const value_type* a, size_type n, const value_type* b)
{
  /*@
    loop invariant bound:  0 <= i <= n;
    loop invariant equal:  Equal{Here,Here}(a, i, b);
    loop assigns i;
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; i++) {
    if (a[i] != b[i]) {
      return i;
    }
  }

  return n;
}
```

Listing 4.32: Implementation of `mismatch`

As in the specification, we employ the predicate `Equal` [4.28] in the loop invariant to express that all indices `k` strictly smaller than the current index `i` satisfy `a[k] == b[k]`. This invariant captures the essential progress property of the algorithm: at each iteration, the examined prefix of the arrays is known to be equal.

The presence of this invariant is crucial for proving that the value returned by `mismatch` is indeed the smallest index at which the two ranges differ, or `n` if no such index exists.

4.8. The search algorithm

The `search` algorithm of the C++ Standard Library [18, §28.5.13] searches for the first occurrence of a sequence as a contiguous subsequence of another sequence, using element-by-element comparison. For our purposes, we adapt the generic formulation to arrays of type `value_type`. The resulting function signature is as follows:

```
size_type search(const value_type* a, size_type n,
                 const value_type* b, size_type p);
```

The function `search` returns the smallest index `s` such that the subrange `a[s..s+p-1]` is element-wise equal to the range `b[0..p-1]`, that is, `a[s+k] == b[k]` holds for every index `k` with `0 <= k < p` (see Figure 4.33). If no such index exists, the function returns the end of the searched range, represented here by the length `n` of the array `a`.

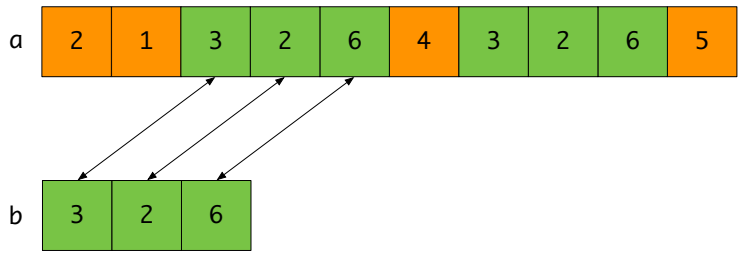


Figure 4.33.: A simple example of `search`

4.8.1. The predicate HasSubRange

The formal specification of `search` begins with the introduction of the predicate `HasSubRange`. This predicate expresses that the array `a` contains `b` as a *contiguous* subrange. Using the predicate `Equal` [4.28], it formalizes the existence of an index `s` such that the ranges `a[s..s+p-1]` and `b[0..p-1]` are equal element by element.

```
/*@
  predicate HasSubRange{L}(value_type* a, integer m, integer n, value_type* b,
      integer p) =
    \exists integer k; (m <= k <= n-p) && Equal{L,L}(a+k, p, b);

  predicate HasSubRange{L}(value_type* a, integer n, value_type* b, integer p) =
    HasSubRange{L}(a, 0, n, b, p);

  lemma HasSubRange_Sizes:
    \forall value_type *a, *b, integer m, n, p;
      HasSubRange(a, m, n, b, p)  ==>  p <= n-m;
*/
```

Listing 4.34: `HasSubRange` (ACSL)

For such a subrange of length `p` to exist, the array `a` must be long enough to accommodate it. This necessary size constraint is captured by the lemma `HasSubRange_Sizes`.

4.8.2. Formal specification of search

The following listing shows the formal specification of `search` [4.35].

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));
  requires   valid:   \valid_read(b + (0..p-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior has_match:
    assumes          HasSubRange(a, n, b, p);
    assigns          \nothing;
    ensures bound:   0 <= \result <= n-p;
    ensures result:  Equal{Here,Here}(a+\result, p, b);
    ensures first:   !HasSubRange(a, \result+p-1, b, p);

  behavior no_match:
    assumes          !HasSubRange(a, n, b, p);
    assigns          \nothing;
    ensures result:  \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type search(const value_type* a, size_type n,
                 const value_type* b, size_type p);
```

Listing 4.35: Formal specification of `search`

At a conceptual level, the specification of `search` closely resembles that of `find` [4.2]. As before, we distinguish two behaviors in order to capture the essential cases.

- The behavior `has_match` applies when the array `a` contains the sequence `b` as a contiguous subrange. This situation is characterized in the `assumes` clause by the predicate `HasSubRange` [4.34].

  The ensures clause `bound` of behavior `has_match` states that the returned index lies in the range `[0..n-p]`. Clause `result` expresses that `search` returns an index at which a copy of `b` occurs in `a`. Finally, clause `first` enforces minimality: the returned index is the least one with this property, i.e. the sequence `b` does not occur in the prefix `a[0..\result+p-2]`.

- The behavior `no_match` covers the complementary case in which the array `a` does not contain `b` as a subrange. In this situation, `search` is required to return the length `n` of the array `a`. If either of the ranges `a` or `b` is empty, the return value is `0`, in accordance with the definition of the algorithm.

The `assumes` clause of behavior `has_match` is the logical negation of the `assumes` clause of behavior `no_match`. Consequently, the two behaviors are explicitly specified as *complete* and *disjoint*.

4.8.3. Implementation of search

The implementation of `search` [4.36] is simple and easy to follow: it checks every possible starting position in `a` until it finds a match for `b` or reaches the end of `a`. Because it may compare up to `p` elements at each of the `n-p+1` starting positions, its worst-case number of comparisons grows proportionally to $n \cdot p$.

More advanced string search algorithms, such as the classic method of Knuth, Morris, and Pratt [19], avoid this quadratic behavior and guarantee at most $n + p$ comparisons. [8]

```
size_type search(const value_type* a, size_type n,
                 const value_type* b, size_type p)
{
  if (p <= n) {
    /*@
      loop invariant bound:     i <= n-p+1;
      loop invariant not_found: !HasSubRange(a, p+i-1, b, p);
      loop assigns i;
      loop variant n-i;
    */
    for (size_type i = 0u; i <= n - p; ++i) {
      if (equal(a + i, p, b)) {
        //@ assert has_match: HasSubRange(a, n, b, p);
        return i;
      }
    }
  }

  //@ assert no_match: !HasSubRange(a, n, b, p);
  return n;
}
```

Listing 4.36: Implementation of `search`

Depending on the version of Frama-C/WP, the loop invariant `not_found` may be essential for establishing the postconditions of the behavior `has_match`. It expresses that no occurrence of the sequence `b` has been found in `a` up to the current iteration.

Neither the case `p == 0` nor the case `n == 0` requires special treatment, not even for efficiency reasons. In the former case, the call `equal(a+i, p, b)` succeeds immediately, while in the latter case the condition `p > n` applies.

[8] The efficiency of `search` has also been discussed by the C++ standardization committee; see http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n3905.html.

4.9. The search_n algorithm

The `search_n` algorithm of the C++ Standard Library [18, §28.5.13] locates the first position at which a given value occurs a prescribed number of consecutive times in a sequence. For our purposes, we adapt the generic interface to operate on an array of elements of type `value_type`. The resulting function signature is

```
size_type
search_n(const value_type* a, size_type n,
         size_type p, value_type v);
```

This interface closely resembles that of `search` (Section 4.8). The only difference is that `search_n` takes a single value `v` instead of a pattern array.

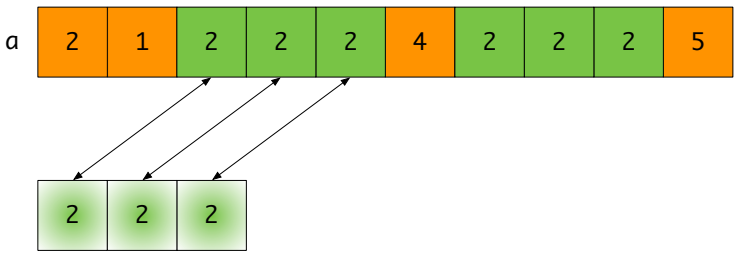


Figure 4.37.: A simple example of `search_n`

The function `search_n` returns the smallest index `s` such that `a[s+k] == v` holds for all indices `k` with `0 <= k < p` (see Figure 4.37). If no such index exists, the function returns the array length `n`.

4.9.1. The predicate HasConstantSubRange

The specification of `search_n` is based on the predicate `HasConstantSubRange` [4.38]. This predicate expresses that the array `a` of length `n` contains a contiguous subsequence consisting of `p` occurrences of the value `v`. Its definition reuses the predicate `AllEqual` [4.15] to characterize constant segments.

```
/*@
  predicate HasConstantSubRange{L}(value_type* a, integer m, integer n, value_type
      v, integer p) =
    \exists integer k; m <= k <= n-p && AllEqual(a, k, k+p, v);

  predicate HasConstantSubRange{L}(value_type* a, integer n, value_type v, integer
      p) =
    HasConstantSubRange(a, 0, n, v, p);

  lemma HasConstantSubRange_Sizes:
    \forall value_type *a, v, integer n, p;
      HasConstantSubRange(a, n, v, p)  ==>  p <= n;
*/
```

Listing 4.38: `HasConstantSubRange` (ACSL)

As with `HasSubRange` [4.34], the existence of such a subsequence requires that the array be sufficiently long. This size constraint is made explicit by the lemma `HasConstantSubRange_Sizes` [4.38].

4.9.2. Formal specification of search_n

As for search [4.35], the specification of search_n [4.39] closely resembles that of find2 [4.5].

```
/*@
  requires   valid:    \valid_read(a + (0..n-1));

  terminates           \true;
  exits                \false;
  assigns              \nothing;

  ensures    result:   0 <= \result <= n;

  behavior has_match:
    assumes            HasConstantSubRange(a, n, v, p);
    assigns            \nothing;
    ensures result:    0 <= \result <= n-p;
    ensures match:     AllEqual(a, \result, \result+p, v);
    ensures first:     !HasConstantSubRange(a, \result+p-1, v, p);

  behavior no_match:
    assumes            !HasConstantSubRange(a, n, v, p);
    assigns            \nothing;
    ensures result:    \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type search_n(const value_type* a, size_type n, value_type v, size_type p);
```

Listing 4.39: Formal specification of search_n

We again distinguish two behaviors capturing the essential cases.

- The behavior has_match applies if the array a contains a p-fold repetition of the value v. This condition is expressed in the assumes clause using the predicate HasConstantSubRange [4.38]. The ensures clauses state that the return value lies in [0..n-p], that p or more consecutive occurrences of v start at this position, and that this position is minimal.
- The behavior no_match covers the complementary case in which no such subsequence exists. Here, search_n returns the length n of the range a.

4.9.3. Implementation of search_n

Although the specification of search_n [4.39] closely mirrors that of search [4.35], their implementations differ substantially. While search [4.36] uses a simple but non-optimal $\mathcal{O}(n \cdot p)$ algorithm, the implementation of search_n [4.40] runs in linear time $\mathcal{O}(n)$.

The implementation tracks, in the variable start, the beginning of the current maximal block of consecutive values v. The loop invariant not_found expresses that no p-fold repetition has been encountered so far; once such a repetition is found, the loop terminates and start is returned.

The boundary cases n < p and p == 0 are handled explicitly in separate branches, which simplifies the verification by automatic provers.

```
size_type search_n(const value_type* a, size_type n, value_type v, size_type p)
{
  if (0u < p) {
    if (p <= n) {
      size_type start = 0u;

      /*@
        loop invariant match:     AllEqual(a, start, i, v);
        loop invariant start:     0 < start ==> a[start-1] != v;
        loop invariant bound:     start <= i + 1 <= start + p;
        loop invariant not_found: !HasConstantSubRange(a, i, v, p);
        loop assigns i, start;
        loop variant n - i;
      */
      for (size_type i = 0u; i < n; ++i) {
        if (a[i] != v) {
          start = i + 1u;
          //@ assert not_found: !HasConstantSubRange(a, i+1, v, p);
        }
        else {
          //@ assert match: a[i] == v;
          //@ assert match: AllEqual(a, start, i+1, v);
          if (p == i + 1u - start) {
            //@ assert bound: start + p == i + 1;
            //@ assert match: AllEqual(a, start, start+p, v);
            /*@ assert match: \exists integer k;
                                 0 <= k <= n-p && AllEqual(a, k, k+p, v);
             */
            //@ assert match: HasConstantSubRange(a, n, v, p);
            return start;
          }
          else {
            //@ assert bound: i + 1 < start + p;
            continue;
          }
        }

        //@ assert not_found: !HasConstantSubRange(a, i+1, v, p);
      }

      //@ assert not_found: !HasConstantSubRange(a, n, v, p);
      return n;
    }
    else {
      //@ assert not_found: n < p;
      //@ assert not_found: !HasConstantSubRange(a, n, v, p);
      return n;
    }
  }
  else {
    //@ assert bound: p == 0;
    //@ assert match: AllEqual(a, 0, 0, v);
    //@ assert match: HasConstantSubRange(a, n, v, 0);
    return 0u;
  }
}
```

Listing 4.40: Implementation of search_n

4.10. The find_end algorithm

The algorithm find_end, as specified in the C++ Standard Library [18, §28.5.6], searches for the *last* occurrence of a given sequence as a contiguous subsequence of another sequence, using element-by-element comparison. For our purposes, we specialize the generic formulation to arrays of type value_type. The resulting function signature is as follows:

```
size_type
find_end(const value_type* a, size_type n,
         const value_type* b, size_type p);
```

The function find_end returns the greatest index s such that a[s+k] == b[k] holds for all indices k with 0 <= k < p. In other words, s denotes the starting position of the last occurrence of the subarray b[0..p-1] within a[0..n-1] (see Figure 4.41). If no such index exists, the function returns the value n.

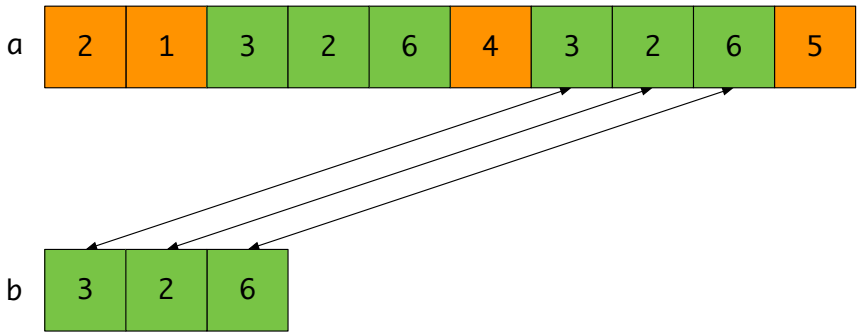


Figure 4.41.: A simple example of find_end

A special case arises when p == 0. In this situation, the empty sequence is considered to occur at the end of the array, and the function therefore returns n.

4.10.1. Formal specification of find_end

Listing 4.42 shows the formal specification of find_end [4.42]. From a conceptual point of view, the contract of find_end is very similar to that of find2 [4.5]. As before, we use behaviors to capture the essential cases of the algorithm. These behaviors are both *complete* and *disjoint*.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));
  requires   valid:   \valid_read(b + (0..p-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior has_match:
    assumes          HasSubRange(a, n, b, p);
    assigns          \nothing;
    ensures bound:   0 <= \result <= n-p;
    ensures result:  Equal{Here,Here}(a + \result, p, b);
    ensures last:    !HasSubRange(a, \result + 1, n, b, p);

  behavior no_match:
    assumes          !HasSubRange(a, n, b, p);
    assigns          \nothing;
    ensures result:  \result == n;

  complete behaviors;
  disjoint behaviors;
*/
size_type find_end(const value_type* a, size_type n,
                   const value_type* b, size_type p);
```

Listing 4.42: Formal specification of find_end

The behavior has_match applies when the array a contains a contiguous subsequence that is identical to b. This condition is expressed in the corresponding assumes clause using the predicate HasSubRange [4.34]. The ensures clause bound states that the return value lies in the range 0..n-p. The result clause of behavior has_match expresses that the returned index indeed denotes a position in a where the sequence b occurs. Finally, the clause last formalizes the defining property of find_end: it ensures that there is no occurrence of b in a starting at any index strictly greater than \result.

The behavior no_match covers the complementary case in which no subsequence of a is equal to b. In this situation, find_end must return the value n, that is, the length of the array a.

4.10.2. Implementation of find_end

The implementation of find_end [4.43] closely follows that of search [4.36], with the essential difference that the last matching position is recorded rather than the first.

```
size_type find_end(const value_type* a, size_type n,
                   const value_type* b, size_type p)
{
  size_type r = n;

  if ((0u < p) && (p <= n)) {
    /*@
      loop invariant bound   :  r <= n - p || r == n;
      loop invariant not_found: r == n ==> !HasSubRange(a, p+i-1, b, p);
      loop invariant found:     r < n  ==> Equal{Here,Here}(a+r, p, b);
      loop invariant last:      r < n  ==> !HasSubRange(a, r+1, i+p-1, b, p);
      loop assigns i, r;
      loop variant n - i;
    */
    for (size_type i = 0u; i <= n - p; ++i) {
      if (equal(a + i, p, b)) {
        r = i;
      }
    }
  }

  return r;
}
```

Listing 4.43: Implementation of find_end

The variable r stores the prospective return value according to the current state of the search. It is initially set to n, indicating that no occurrence of b has been found so far. Whenever an occurrence is detected, r is updated to the corresponding starting index.

The invariant bound states that r either retains the value n or lies in the range 0..n-p. In the former case, the invariant not_found expresses that no occurrence of b has been identified. In the latter case, the invariant found states that the subsequence b[0..p-1] occurs in a starting at index r. Finally, the invariant last ensures that no occurrence of b has been found at any index strictly greater than r.

4.11. The count algorithm

The `count` algorithm of the C++ Standard Library [18, §28.5.9] determines how often a given value occurs in a sequence. For the purposes of this chapter, we restrict attention to arrays of elements of type `value_type` and adapt the generic interface accordingly. The resulting function signature is:

```
size_type count(const value_type* a, size_type n, value_type v);
```

Given an array `a` of length `n`, the function returns the number of indices `i` with `0 <= i < n` for which `a[i] == v` holds.

4.11.1. The logic function Count

In order to specify `count`, we first need a formal notion of counting occurrences of a value in an array segment. The ACSL specification language does not provide such a notion as a primitive.[9] We therefore introduce a definition of the *logic function* `Count` [4.44], which captures the standard intuitive behavior of counting a given value in a contiguous array section. The expression `Count(a,m,n,v)` denotes the number of occurrences of `v` in the range `a[m..n-1]`.

```
/*@
  logic integer
  Count(value_type* a, integer m, integer n, value_type v) =
    n <= m ? 0 : Count(a, m, n-1, v) + (a[n-1] == v ? 1 : 0);

  logic integer
  Count(value_type* a, integer n, value_type v) = Count(a, 0, n, v);

  lemma Count_Empty:
    \forall value_type *a, v, integer m, n;
      n <= m  ==>  Count(a, m, n, v) == 0;

  lemma Count_Hit:
    \forall value_type *a, v, integer n, m;
      m < n        ==>
      a[n-1] == v  ==>  Count(a, m, n, v) == Count(a, m, n-1, v) + 1;

  lemma Count_Miss:
    \forall value_type *a, v, integer n, m;
      m < n        ==>
      a[n-1] != v  ==>  Count(a, m, n, v) == Count(a, m, n-1, v);

  lemma Count_One:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>
     Count(a, m, n+1, v) == Count(a, m, n, v) + Count(a, n, n+1, v);

  lemma Count_Single{K,L}:
    \forall value_type *a, *b, v, integer m, n;
      \at(a[m],K) == \at(b[n],L)  ==>
      Count{K}(a, m, m+1, v) == Count{L}(b, n, n+1, v);
```

Listing 4.44: The logic function `Count` (1)

[9] Strictly speaking, the ACSL documentation mentions the operator `numof` as part of a collection of so-called *higher-order logic constructions* [9, §2.6.7]. These *extended quantifiers*, however, have originally been described as experimental features and are therefore not used here.

- The bounds `m` and `n`, as well as the return value of `Count`, are of type `integer`.

- The logic function `Count` is defined recursively. The definition distinguishes the empty range from the non-empty case and, in the latter, inspects the value of the current array element. The recursion decreases the length of the range. An overloaded variant of `Count` is provided that takes only the array length, corresponding to the common case `m = 0`.

- Lemma `Count_Empty` [4.44] characterizes the value of `Count` on empty ranges.

- Lemmas `Count_Hit` [4.44] and `Count_Miss` express how counting over a range of length `n-m` reduces to counting over a range of length `n-m-1`, depending on whether the last element matches the value `v`.

- Lemmas `Count_One` [4.44] and `Count_Single` build on `Count_Hit` and `Count_Miss`. They are convenient derived results that simplify several Coq proofs by shifting the induction scheme from `n-1 -> n` to `n -> n+1`.

Listing 4.45 shows another group of `Count` lemmas

```
lemma Count_Equal{K,L}:
  \forall value_type *a, v, integer m, n, p;
    0 <= m <= n              ==>
    Equal{K,L}(a, m, n, p)   ==>
    Count{K}(a, m, n, v) == Count{L}(a, p, p + (n-m), v);

lemma Count_Unchanged{K,L}:
  \forall value_type *a, v, integer m, n;
    Unchanged{K,L}(a, m, n)  ==>
    Count{K}(a, m, n, v) == Count{L}(a, m, n, v);

lemma Count_Union:
  \forall value_type *a, v, integer k, m, n;
    0 <= k <= m <= n  ==>
    Count(a, k, n, v) == Count(a, k, m, v) + Count(a, m, n, v);

lemma Count_Cut:
  \forall value_type *a, v, integer k, m, n;
    0 <= k <= m < n  ==> Count(a, k, n, v) ==
      Count(a, k, m, v) + Count(a, m, m+1, v) + Count(a, m+1, n, v);

lemma Count_Single_Bounds:
  \forall value_type *a, v, integer n;
    0 <= Count(a, n, n+1, v) <= 1;
```

Listing 4.45: The logic function `Count` (2)

- The logic function `Count` depends only on the values in the memory locations `a[m..n-1]`. Lemma `Count_Unchanged` makes this explicit by stating that `Count` yields the same result in two program states `K` and `L`, provided that the values of `a[m..n-1]` are unchanged. The predicate `Unchanged` is used to express this premise.

- Lemma `Count_Equal` generalizes `Count_Unchanged` to the comparison of two arrays.

- Lemmas `Count_Union` and `Count_Cut` support reasoning about partitions of array ranges.

The final group of initial `Count` lemmas are shown in Listing 4.46.

- Lemmas `Count_Single_Bounds` and `Count_Bounds` establish lower and upper bounds for the value of `Count`. Lemma `Count_Increasing` states that `Count` is monotonic with respect to range extension.
- Finally, lemmas `Count_Single_Shift` and `Count_Shift` express that `Count` is invariant under shifts of the array index.

```
  lemma Count_Bounds:
    \forall value_type *a, v, integer m, n;
      0 <= m <= n  ==>  0 <= Count(a, m, n, v) <= n-m;

  lemma Count_Increasing:
    \forall value_type *a, v, integer m, n, p;
      m <= n <= p  ==>  Count(a, m, n, v) <= Count(a, m, p, v);

  lemma Count_Single_Shift:
    \forall value_type *a, v, integer n;
      0 <= n  ==>  Count(a+n, 0, 1, v) == Count(a, n, n+1, v);

  lemma Count_Shift:
    \forall value_type *a, v, integer m, n;
      0 <= m  ==>  0 <= n  ==>  Count(a+m, 0, n, v) == Count(a, m, m+n, v);
*/
```

Listing 4.46: The logic function `Count` (3)

4.11.2. Relating `Count` and `Find`

So far, the logic function `Count` has been treated independently of the predicates used in the specifications of searching algorithms. In those specifications, existential properties are expressed using the predicate `SomeEqual` [4.4], introduced earlier in the discussion of the various `find` variants.

Lemma `Count_SomeEqual` [4.47], shown below, establishes a direct connection between these two notions. It states that, for a non-empty array range `[m,n)`, a strictly positive value of `Count` implies that `SomeEqual` holds on the same range. In other words, if the count of occurrences of a value `v` in `a[m..n-1]` is positive, then `v` occurs at least once in that range.

```
/*@
  lemma Count_SomeEqual:
    \forall value_type *a, v, integer m, n;
      0 <= m < n              ==>
      0 < Count(a, m, n, v)   ==>
      SomeEqual(a, m, n, v);
*/
```

Listing 4.47: `CountFind` (ACSL)

This lemma makes explicit a basic consistency property between counting and existence. It is stated only in this direction; the converse implication, although expected from the definition of `Count`, is not required here and is therefore not included.

4.11.3. Formal specification of count

The contract of `count` [4.48] is formulated using the logic function `Count` [4.44], which expresses the number of occurrences of a given value in an array range. In addition, the specification explicitly states that the result returned by `count` is non-negative and does not exceed the length of the array.

```
/*@
  requires   valid:  \valid_read(a + (0..n-1));

  terminates         \true;
  exits              \false;
  assigns            \nothing;

  ensures    bound:  0 <= \result <= n;
  ensures    count:  \result == Count(a, n, v);
*/
size_type count(const value_type* a, size_type n, value_type v);
```

Listing 4.48: Formal specification of `count`

4.11.4. Implementation of count

The following listing shows a straightforward implementation of `count` [4.49]. One of the loop invariants refers to the logic function `Count`, thereby establishing a direct connection between the operational behavior of the loop and the abstract counting property used in the specification.

```
size_type count(const value_type* a, size_type n, value_type v)
{
  size_type counted = 0u;

  /*@
    loop invariant bound: 0 <= i <= n;
    loop invariant bound: 0 <= counted <= i;
    loop invariant count: counted == Count(a, i, v);
    loop assigns i, counted;
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    if (a[i] == v) {
      counted++;
    }
  }

  return counted;
}
```

Listing 4.49: Implementation of `count`

5. Minimum and maximum algorithms

In this chapter, we study the formal specification of algorithms of the C++ Standard Library [18, §28.7.8] that compute minimum or maximum values of their arguments. Like the algorithms discussed in Chapter 4, these algorithms do not modify any memory locations and are therefore specified uniformly using nonmutating contracts.

The algorithms considered here are characterized by their essential reliance on comparisons between values. Their specifications are formulated in terms of binary relational operators such as <, and the mathematical properties of these operators play a central role in the correctness arguments.

The chapter is organized to reflect this structure. We begin by introducing auxiliary notions that support precise, reusable, and modular specifications of comparison-based algorithms. We then turn to the specification and verification of the minimum and maximum algorithms provided by the library.

- We first establish the necessary foundations. §5.1 discusses basic properties that relational operators must satisfy in order to be suitable for use in minimum and maximum algorithms. §5.2 then introduces logical predicates that describe order-related properties of arrays with elements of type `value_type`, which are used throughout the remainder of the chapter.
- §5.3 introduces the type `size_type_pair` together with the function `make_pair`, providing a uniform way of representing pairs of indices in specifications. These definitions are used in particular for the specification of `minmax_element` [5.17].
- We then study individual algorithms. `clamp` (§5.4) serves as a simple introductory example that restricts a value to lie within given boundary values.
- `max_element` (§5.5) returns an index to a maximum element of a range, choosing the smallest index among all valid candidates. An alternative specification based on user-defined predicates, `max_element2`, is presented in §5.6.
- `max_seq` (§5.7) is closely related to `max_element` and illustrates modular verification. Unlike `max_element`, it returns the maximum value itself rather than an index.
- `min_element` (§5.8) computes an index to a smallest element of a range.
- The chapter concludes with `minmax_element` (§5.9), which computes both the smallest and the largest element of a range simultaneously. Its specification and implementation rely on the auxiliary function `make_pair` introduced earlier.

We begin with a discussion of relational operators in §5.1, which provides the foundation for all subsequent specifications.

5.1. A note on relational operators

Note that in order to compare values, algorithms in the C++ Standard Library [18, §28.7.8] usually rely solely on the *less than* operator < or on special function objects. To be precise, the operator < must induce a *strict partial order*, which means that the following rules must hold.[10]

$$\textit{Irreflexivity}: \quad \forall x: \quad \neg(x < x) \tag{5.1a}$$

$$\textit{Asymmetry}: \quad \forall x, y: \quad x < y \implies \neg(y < x) \tag{5.1b}$$

$$\textit{Transitivity}: \quad \forall x, y, z: \quad x < y \implies y < z \implies x < z \tag{5.1c}$$

We adopt the usual convention that the implication symbol $\implies$ associates to the right. Thus an expression of the form $A \implies B \implies C$ is understood as $A \implies (B \implies C)$.

If one wishes to check that the operator < of our `value_type` (see §2). satisfies these properties, the corresponding lemmas of [5.1] can be formulated and verified with Frama-C.

```
/*@
  lemma Less_Irreflexivity:
    \forall value_type a; !(a < a);

  lemma Less_Antisymmetry:
    \forall value_type a, b; (a < b)     ==>  !(b < a);

  lemma Less_Transitivity:
    \forall value_type a, b, c; (a < b)  ==>  (b < c)  ==>  (a < c);

  lemma Greater_Less:
    \forall value_type a, b;  (a > b)   <==>  (b < a);

  lemma LessOrEqual_Less:
    \forall value_type a, b;  (a <= b)  <==>  !(b < a);

  lemma GreaterOrEqual_Less:
    \forall value_type a, b;  (a >= b)  <==> !(a < b);
*/
```

Listing 5.1: `LessThanComparable` (ACSL)

It is possible to specify and implement the algorithms of this chapter by using only operator <. For example, `a <= b` can be written as `a < b || a == b`, or, for our ordering on `value_type`, as `!(b < a)`. Listing [5.1] therefore also contains lemmas on representing the operators >, <=, and >= in terms of operator <.

Remark on `StrictWeakOrdering`. The C++ Standard formally requires comparison predicates used by most algorithms to satisfy the concept of a *strict weak ordering*. This notion slightly relaxes that of a strict partial order by allowing distinct elements to be considered *equivalent* with respect to the ordering, that is, neither $x < y$ nor $y < x$ holds. In this chapter, we restrict attention to orderings that are strict partial orders. This simplification is sufficient for the algorithms considered here and avoids the additional technical overhead associated with equivalence classes induced by weak orderings.

[10] See http://en.wikipedia.org/wiki/Partially_ordered_set

5.2. Predicates for bounds and extrema of arrays

We begin by introducing predicates that express how all elements of an array segment compare to a given value. The predicates `LowerBound` [5.2] and `UpperBound`, together with their strict variants `StrictLowerBound` and `StrictUpperBound`, are shown in the following listing.

```
/*@
    predicate LowerBound{L}(value_type* a, integer m, integer n, value_type v) =
      \forall integer i; m <= i < n  ==>  v <= a[i];

    predicate LowerBound{L}(value_type* a, integer n, value_type v) =
      LowerBound{L}(a, 0, n, v);

    predicate StrictLowerBound{L}(value_type* a, integer m, integer n, value_type
        v) =
      \forall integer i; m <= i < n  ==>  v < a[i];

    predicate StrictLowerBound{L}(value_type* a, integer n, value_type v) =
      StrictLowerBound{L}(a, 0, n, v);

    predicate UpperBound{L}(value_type* a, integer m, integer n, value_type v) =
      \forall integer i; m <= i < n  ==>  a[i] <= v;

    predicate UpperBound{L}(value_type* a, integer n, value_type v) =
      UpperBound{L}(a, 0, n, v);

    predicate StrictUpperBound{L}(value_type* a, integer m, integer n, value_type
        v) =
      \forall integer i; m <= i < n  ==>  a[i] < v;

    predicate StrictUpperBound{L}(value_type* a, integer n, value_type v) =
      StrictUpperBound{L}(a, 0, n, v);
*/
```

Listing 5.2: `ArrayBounds` (ACSL)

These predicates provide a concise and uniform way to state global ordering properties of array segments. They will be used extensively throughout this chapter and again in Chapter 6.

Using these bounds predicates as building blocks, we now define predicates that characterize extrema of array segments. An element is a minimum (respectively maximum) if it constitutes an lower (respectively upper) bound for all elements in the range. This idea is captured by the predicates `MaxElement` [5.3] and `MinElement` [5.3] shown below.

```
/*@
  predicate MaxElement{L}(value_type* a, integer n, integer max) =
    0 <= max < n && UpperBound(a, n, a[max]);

  predicate MinElement{L}(value_type* a, integer n, integer min) =
    0 <= min < n && LowerBound(a, n, a[min]);
*/
```

Listing 5.3: `ArrayExtrema` (ACSL)

We will discuss these extrema predicates in more detail in §5.6 and §5.8.

5.3. The auxiliary function `make_pair`

In several specifications we need to reason about *pairs of indices* rather than individual positions. For this purpose, and purely as a technical convenience, we introduce a small auxiliary data type. This construction is independent of any particular algorithm and, in particular, is *not* conceptually related to algorithms computing minimum or maximum elements.

The following listing introduces the type `size_type_pair`, which simply bundles two values of type `size_type` into a single object. It serves as a lightweight vehicle for passing index pairs through specifications and implementations.

```
struct size_type_pair {
  size_type first;
  size_type second;
};

typedef struct size_type_pair size_type_pair;
```

Listing 5.4: The auxiliary type `size_type_pair`

To construct values of type `size_type_pair`, we provide the auxiliary function `make_pair`, which combines two indices `first` and `second` into a single object. The function serves purely as a technical convenience. The formal specification of `make_pair` [5.5] is shown in the following listing.

```
/*@
    terminates          \true;
    exits               \false;
    assigns             \nothing;

    ensures   result:  \result.first  == first;
    ensures   result:  \result.second == second;
*/
size_type_pair make_pair(size_type first, size_type second);
```

Listing 5.5: Formal specification of `make_pair`

The next listing gives a straightforward implementation that directly reflects the specification.

```
size_type_pair make_pair(size_type first, size_type second)
{
  size_type_pair pair = {first, second};
  return pair;
}
```

Listing 5.6: Implementation of `make_pair`

5.4. The clamp algorithm

The clamp algorithm of the C++ Standard Library [18, §28.7.9] restricts a value to a given closed interval. If the value of v is within the interval [lower, upper], then clamp returns v; otherwise returns the nearest boundary. The standard also states that if upper is less than lower, then the behaviour of clamp is undefined. The signature of our version of clamp is as follows:

```
value_type clamp(value_type v, value_type lower, value_type upper);
```

More formally, the specification of clamp can be stated as follows:

- If lower <= v <= upper, the function returns v.
- If v < lower, the function returns lower.
- If v > upper, the function returns upper.

5.4.1. Formal specification of clamp

The following listing shows the formal specification of clamp [5.7]. As a precondition, we require that the lower bound is strictly smaller than the upper bound, that is, lower < upper. This condition excludes degenerate or ill-formed intervals and reflects the intended use of clamp as an operation on a proper closed interval.

```
/*@
  requires   bound:  lower < upper;

  terminates         \true;
  exits              \false;
  assigns            \nothing;

  ensures    bound:  lower <= \result <= upper;

  behavior lower_bound:
    assumes          v < lower;
    ensures result:  \result == lower;

  behavior between:
    assumes          lower <= v <= upper;
    ensures result:  \result == v;

  behavior upper_bound:
    assumes          upper < v;
    ensures result:  \result == upper;

  complete behaviors;
  disjoint behaviors;
*/
value_type clamp(value_type v, value_type lower, value_type upper);
```

Listing 5.7: Formal specification of clamp

The function is specified as terminating normally, without exceptional exits, and as having no side effects. In particular, the assigns clause expresses that clamp is a pure function whose result depends solely on its input parameters.

The global postcondition states that the returned value always lies within the interval defined by the bounds. Formally, the result satisfies the inequality `lower <= \result <= upper`, independently of the relative position of the input value `v`. The specification then refines this general guarantee by distinguishing three exclusive behaviors that cover all possible cases.

5.4.2. Implementation of `clamp`

The implementation of `clamp` [5.8] is straightforward. Due to its simple control flow, the correctness of the implementation can be verified without introducing any additional annotations.

```
value_type clamp(value_type v, value_type lower, value_type upper)
{
  return (v < lower) ? lower : (upper < v) ? upper : v;
}
```

Listing 5.8: Implementation of `clamp`

5.5. The max_element algorithm

The max_element algorithm of the C++ Standard Library [18, §28.7.8] determines the position of a maximal element in a finite sequence. The signature of our version of max_element is

```
size_type max_element(const value_type* a, size_type n);
```

The algorithm distinguishes two conceptually simple cases. If the range a[0..n-1] is empty (n == 0), no maximum exists and the function returns the sentinel value n. Otherwise (n > 0), the algorithm scans the array from left to right and determines the first occurrence of a maximum value.

5.5.1. Formal specification of max_element

The following listing shows the formal ACSL specification of max_element.

```
/*@
  requires   valid:  \valid_read(a + (0..n-1));

  terminates         \true;
  exits              \false;
  assigns            \nothing;

  ensures    result:  0 <= \result <= n;

  behavior empty:
    assumes          n == 0;
    assigns          \nothing;
    ensures result:  \result == 0;

  behavior not_empty:
    assumes          0 < n;
    assigns          \nothing;
    ensures result:  0 <= \result < n;
    ensures upper:   \forall integer i; 0 <= i < n       ==> a[i] <= a[\result];
    ensures first:   \forall integer i; 0 <= i < \result ==> a[i] <  a[\result];

  complete behaviors;
  disjoint behaviors;
*/
size_type max_element(const value_type* a, size_type n);
```

Listing 5.9: Formal specification of max_element

The specification distinguishes two behaviors, empty and not_empty, corresponding to the cases n == 0 and n > 0, respectively.

In behavior not_empty, the ensures clauses max and first state that the returned index refers to a maximal element and denotes its first occurrence.

5.5.2. Implementation of `max_element`

The following listing shows a straightforward implementation of `max_element` [5.10] together with its loop annotations.

```
size_type max_element(const value_type* a, size_type n)
{
  if (0u < n) {
    size_type max = 0u;

    /*@
      loop invariant bound:  0 <= i <= n;
      loop invariant max:    0 <= max <  n;
      loop invariant upper:  \forall integer k; 0 <= k < i   ==> a[k] <= a[max];
      loop invariant first:  \forall integer k; 0 <= k < max ==> a[k] <  a[max];
      loop assigns max, i;
      loop variant n-i;
    */
    for (size_type i = 1u; i < n; i++) {
      if (a[max] < a[i]) {
        max = i;
      }
    }

    return max;
  }

  return n;
}
```

Listing 5.10: Implementation of `max_element`

The implementation reflects the same two cases as the specification. The case `n == 0` is handled explicitly before the loop. For `n > 0`, the loop traverses the array and maintains the index of a candidate maximum.

The loop invariant `max` anticipates the postcondition `max` of behavior `not_empty` by stating that the current candidate index refers to a maximal element of the prefix inspected so far. The invariant `upper` ensures that the candidate index remains within bounds. Finally, the invariant `first` records that the current candidate index refers to the first occurrence of the maximal value seen so far, which allows the postcondition `first` to be proved at loop termination.

5.6. The max_element2 algorithm (predicate-based specification)

In this section we present an alternative specification of the `max_element` algorithm. Instead of expressing maximality directly through quantified conditions, this variant relies on previously introduced predicates.

In particular, we make use of the predicate `UpperBound` [5.2], which expresses that a given value is greater than or equal to all elements of a given array segment. Its strict counterpart, `StrictUpperBound` [5.2], is used to capture the notion of a *first* maximum.

In addition, we employ the predicate `MaxElement` [5.3]. This predicate characterizes an index `max` whose associated element is an upper bound of the range `a[0..n-1]` and, at the same time, belongs to that range. Together, these predicates allow a more declarative formulation of the specification.

5.6.1. Formal specification of max_element2

The following listing shows the formal specification of `max_element2` [5.11].

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior empty:
    assumes          n == 0;
    assigns          \nothing;
    ensures result:  \result == 0;

  behavior not_empty:
    assumes          0 < n;
    assigns          \nothing;
    ensures result:  0 <= \result < n;
    ensures max:     MaxElement(a, n, \result);
    ensures first:   StrictUpperBound(a, \result, a[\result]);

  complete behaviors;
  disjoint behaviors;
*/
size_type max_element2(const value_type* a, size_type n);
```

Listing 5.11: Formal specification of `max_element2`

The predicate `MaxElement` [5.3] is used to express that the returned index refers to a maximum element of the array. The predicate `StrictUpperBound` [5.2] is employed to specify that this index denotes the first occurrence of such a maximum.

5.6.2. Implementation of `max_element2`

The implementation of `max_element2` [5.12] is structurally identical to that of `max_element` [5.10]. The difference lies solely in the loop annotations, which now make use of the predicate-based characterizations introduced above.

```
size_type max_element2(const value_type* a, size_type n)
{
  if (0u < n) {
    size_type max = 0u;

    /*@
      loop invariant bound:   0 <= i <= n;
      loop invariant max:     0 <= max < n;
      loop invariant upper:   UpperBound(a, i, a[max]);
      loop invariant first:   StrictUpperBound(a, max, a[max]);
      loop assigns max, i;
      loop variant n-i;
    */
    for (size_type i = 0u; i < n; i++) {
      if (a[max] < a[i]) {
        max = i;
      }
    }

    return max;
  }

  return n;
}
```

Listing 5.12: Implementation of `max_element2`

5.7. The max_seq algorithm

In this section we consider the function max_seq [8, Ch. 3], which is closely related to max_element . The essential difference is that max_seq returns the maximum value itself, rather than its position. Consequently, max_seq requires a *non-empty* input range.

The function max_seq can easily be implemented using max_element2 [5.12]. Moreover, relying solely on the formal specification of max_element2 [5.11], we can deductively verify the correctness of this implementation. This provides a simple example of *modular verification* in the following sense:

> Any implementation of max_element2 that is separately proven to satisfy the contract max_element2 [5.11] guarantees the correctness of max_seq. Once the contracts have been defined, the function max_element2 may be implemented independently of max_seq, without affecting its verification.

5.7.1. Formal specification of max_seq

The following listing shows the formal specification of max_seq [5.13].

```
/*@
  requires    0 < n;
  requires    \valid_read(p + (0..n-1));

  terminates  \true;
  exits       \false;
  assigns     \nothing;

  ensures     \forall integer i; 0 <= i <= n-1 ==> \result >= p[i];
  ensures     \exists integer e; 0 <= e <= n-1 &&  \result == p[e];
*/
value_type max_seq(const value_type* p, size_type n);
```

Listing 5.13: Formal specification of max_seq

The first requires-clause expresses that max_seq expects a *non-empty* range. The postconditions formalize that the returned value is indeed a maximum of the range.

5.7.2. Implementation of max_seq

The implementation of max_seq [5.14] consists of a single call to max_element2 [5.12]. Since max_seq requires a non-empty range, this call returns an index of a maximum element. The fact that max_element2 returns the smallest such index is irrelevant in this context.

```
value_type max_seq(const value_type* p, size_type n)
{
  return p[max_element2(p, n)];
}
```

Listing 5.14: Implementation of max_seq

5.8. The min_element algorithm

The min_element algorithm of the C++ Standard Library [18, §28.7.8] determines the position of a minimal element in a finite sequence. The signature of our version of min_element is

```
size_type min_element(const value_type* a, size_type n);
```

The function operates on the range a[0..n-1]. If the range is non-empty, it returns the unique index i such that a[i] is minimal among all elements of the range and i denotes the first position with this property. If the range is empty (n == 0), the function returns n.

Conceptually, min_element is the natural dual of the max_element algorithm. Accordingly, its specification and implementation closely follow the predicate-based variant of max_element discussed in §5.6.

5.8.1. Formal specification of min_element

The following listing shows the formal specification of min_element [5.15].

```
/*@
  requires   valid:  \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  0 <= \result <= n;

  behavior  empty:
    assumes          n == 0;
    assigns          \nothing;
    ensures  result: \result == 0;

  behavior  not_empty:
    assumes          0 < n;
    assigns          \nothing;
    ensures result:  0 <= \result < n;
    ensures min:     MinElement(a, n, \result);
    ensures first:   StrictLowerBound(a, \result, a[\result]);

  complete behaviors;
  disjoint behaviors;
*/
size_type min_element(const value_type* a, size_type n);
```

Listing 5.15: Formal specification of min_element

As in the predicate-based specification of max_element, the contract is formulated using previously introduced predicates. The predicate LowerBound [5.2] expresses that a given value is less than or equal to all elements of a range, while StrictLowerBound [5.2] is used to characterize the first occurrence of a minimum. The predicate MinElement [5.3] combines these properties with the requirement that the value occurs in the range.

5.8.2. Implementation of `min_element`

The implementation of `min_element` [5.16] closely follows that of `max_element` [5.10]. Its loop annotations make systematic use of the predicates `LowerBound` [5.2] and `StrictLowerBound` [5.2] to establish the postconditions of the specification.

```
size_type min_element(const value_type* a, size_type n)
{
  if (0u < n) {
    size_type min = 0u;

    /*@
      loop invariant bound:  0 <= i   <= n;
      loop invariant min:    0 <= min <  n;
      loop invariant lower:  LowerBound(a, i, a[min]);
      loop invariant first:  StrictLowerBound(a, min, a[min]);
      loop assigns min, i;
      loop variant n-i;
    */
    for (size_type i = 0u; i < n; i++) {
      if (a[i] < a[min]) {
        min = i;
      }
    }

    return min;
  }

  return n;
}
```

Listing 5.16: Implementation of `min_element`

5.9. The minmax_element algorithm

The minmax_element algorithm of the C++ Standard Library [18, §28.7.8] determines both a minimum and a maximum element in a finite sequence. The signature of our version of minmax_element is

```
size_type_pair minmax_element(const value_type* a, size_type n);
```

The function returns a pair of indices (see §5.3). The first component denotes the first position at which a minimum value occurs in the range a[0..n-1], while the second component denotes the last position at which a maximum value occurs.

The properties of the index identifying the minimum are identical to those of min_element [5.15]. By contrast, the properties of the index identifying the maximum differ slightly from those specified for max_element2 [5.11]. While max_element returns the first occurrence of a maximum value, minmax_element returns the last occurrence of such a value.

5.9.1. Formal specification of minmax_element

The following listing shows the formal ACSL specification of minmax_element [5.17].

```
/*@
  requires   valid:  \valid_read(a + (0..n-1));

  terminates         \true;
  exits              \false;
  assigns            \nothing;

  ensures    result:  0 <= \result.first  <= n;
  ensures    result:  0 <= \result.second <= n;

  behavior empty:
    assumes          0 == n;
    assigns          \nothing;
    ensures result:  \result.first == 0;
    ensures result:  \result.second == 0;

  behavior not_empty:
    assumes          0 < n;
    assigns          \nothing;
    ensures result:  0 <= \result.first < n;
    ensures result:  0 <= \result.second < n;

    ensures min:     MinElement(a, n, \result.first);
    ensures first:   StrictLowerBound(a, \result.first, a[\result.first]);
    ensures max:     MaxElement(a, n, \result.second);
    ensures last:    StrictUpperBound(a, \result.second+1, n, a[\result.second]);
*/
size_type_pair minmax_element(const value_type* a, size_type n);
```

Listing 5.17: Formal specification of minmax_element

The specification combines the ideas used for `min_element` and `max_element`. The predicates `MinElement` [5.3] and `MaxElement` [5.3] are used to characterize minimality and maximality, respectively. The predicate `StrictLowerBound` [5.2] specifies that the minimum index refers to the first occurrence of a minimum value, while `StrictUpperBound` [5.2] is adapted to express that the returned maximum index denotes the last occurrence of a maximum value.

Accordingly, the postcondition `last` differs from the corresponding postcondition of `max_element`: it states that no element at or after the returned index is greater than or equal to the maximum value.

5.9.2. Implementation of `minmax_element`

The implementation of `minmax_element` [5.18] uses the auxiliary function `make_pair` [5.5] to construct the result. Apart from this, it closely follows the structure of the implementations of `min_element` and `max_element`.

```
size_type_pair minmax_element(const value_type* a, size_type n)
{
  if (0u < n) {
    size_type min = 0u;
    size_type max = 0u;

    /*@
      loop invariant bound: 0 <= i    <= n;
      loop invariant min:   0 <= min  <  n;
      loop invariant max:   0 <= max  <  n;
      loop invariant lower: LowerBound(a, i, a[min]);
      loop invariant upper: UpperBound(a, i, a[max]);
      loop invariant first: StrictLowerBound(a, min, a[min]);
      loop invariant last:  StrictUpperBound(a, max+1, i, a[max]);
      loop assigns min, max, i;

      loop variant n-i;
    */
    for (size_type i = 0u; i < n; i++) {
      if (a[i] >= a[max]) {
        max = i;
      }

      if (a[i] < a[min]) {
        min = i;
      }
    }

    return make_pair(min, max);
  }

  return make_pair(n, n);
}
```

Listing 5.18: Implementation of `minmax_element`

We focus on the loop invariant `last`, which is the only invariant that differs from the earlier implementations. It reflects the modified use of the predicate `StrictUpperBound` [5.2], whose range is adjusted to capture the fact that the algorithm returns the last occurrence of a maximum value.

6. Binary search algorithms

In this chapter, we study the formal specification of the binary search algorithms of the C++ Standard Library [18, §28.7.3]. These algorithms determine whether a given value is contained in a sorted range, and, if so, identify positions at which it may occur.

As in the case of the maximum and minimum algorithms discussed in Chapter 5, the binary search algorithms rely essentially on comparisons between values. Their specifications are formulated primarily in terms of the less-than operator < and the relational operators derived from it, while the equality operator == plays only a supporting role.

A distinctive aspect of binary search is that its correctness depends crucially on structural assumptions about the input range. In particular, the range is required to be ordered, and this monotonicity assumption is made explicit in the form of logical predicates introduced in §6.1 and §6.2. These predicates provide the foundation for the specifications and proofs of the binary search algorithms considered in this chapter.

The chapter then turns to the specification and verification of the individual binary search algorithms provided by the library.

- `lower_bound` in §6.3 computes the first position at which a given value could be inserted without violating the ordering of the range.
- `upper_bound` in §6.4 computes the first position at which a value strictly greater than the given one occurs.
- `equal_range`, discussed in §6.5, is presented in two implementation variants that illustrate alternative ways of combining `lower_bound` and `upper_bound`.
- `binary_search` in §6.6 decides whether a given value is contained in the range.

6.1. The predicate `Increasing`

We begin by introducing a logic predicate that expresses an ordering property of array segments. This predicate abstracts a recurring universal quantification pattern over array indices and allows us to state ordering assumptions in a concise and uniform manner. It serves as a foundational precondition for the formal specification of all binary search algorithms discussed in this chapter.

The following listing introduces the overloaded predicate `Increasing` [6.1].

```
/*@
  predicate Increasing{L}(value_type* a, integer m, integer n) =
    \forall integer i, j; m <= i < j < n  ==>  a[i] <= a[j];

  predicate Increasing{L}(value_type* a, integer n) = Increasing{L}(a, 0, n);
*/
```

Listing 6.1: `Increasing` (ACSL)

This predicate characterizes array segments or entire arrays whose elements are *increasing*, in the sense that they are ordered with respect to the non-strict order <=. By convention, order properties based on the strict relation < are referred to as *strictly increasing*.

More precisely, the predicate `Increasing` requires that, for all indices within the specified segment, each element is smaller than or equal to any element at a greater index. This captures the notion of an increasing range at the logic level. In this chapter, the predicate `Increasing` is used primarily as a *precondition* that formalizes the ordering assumptions under which binary search algorithms operate. In later chapters, for example in the specification of `sort_heap` [9.40], the same predicate appears as an essential *postcondition*, expressing that an algorithm has produced an increasing array (see in particular Chapter 10).

6.2. Global vs. local monotonicity

In the verification of algorithms that construct or transform arrays, it is often inconvenient to maintain global ordering properties of the form expressed by the predicate `Increasing` as loop invariants. Instead, such algorithms typically establish ordering properties through local comparisons between neighbouring elements.

For this reason, we introduce the predicate `WeaklyIncreasing` [6.2], which provides a weaker, neighbour-based formulation of monotonicity. This predicate is particularly well suited for use in assertions and loop invariants, where it allows us to express intermediate ordering properties that arise naturally during algorithm execution.

```
/*@
  predicate WeaklyIncreasing{L}(value_type* a, integer m, integer n) =
    \forall integer i; m <= i < n-1  ==>  a[i] <= a[i+1];

  predicate WeaklyIncreasing{L}(value_type* a, integer n) = WeaklyIncreasing{L}(a,
      0, n);
*/
```

Listing 6.2: `WeaklyIncreasing` (ACSL)

Users inexperienced in formal verification often underestimate the difference between `Increasing` [6.1] and `WeaklyIncreasing` [6.2]. Although both predicates characterize non-decreasing order of array segments, they differ in the scope of the comparisons they express: `WeaklyIncreasing` only compares direct neighbours, whereas `Increasing` relates all pairs of elements at increasing indices.

As a consequence, `Increasing` immediately implies `WeaklyIncreasing`, while the converse implication typically requires an induction over the length of the array segment. Such reasoning relies on the transitivity of the operator <= and therefore exceeds the capabilities of automated provers. For this reason, both directions of the relationship between `Increasing` and `WeaklyIncreasing` are made explicit by dedicated auxiliary lemmas (see, in particular, `Increasing_WeaklyIncreasing` [6.3] and `WeaklyIncreasing_Increasing`).

In addition to relating the predicates `Increasing` and `WeaklyIncreasing`, the lemma collection also contains auxiliary results that support the reuse of `Increasing` under common transformations. The lemma `Increasing_Shift` expresses that the increasing property is preserved under shifts of the index range, which is essential when reasoning about subranges addressed via pointer arithmetic. The lemma `Increasing_Equal` captures the fact that equality of array elements at corresponding positions preserves the increasing property, and is used to transfer ordering assumptions between logically equivalent ranges. Both lemmas also play an essential role in the verification of sorting algorithms, where ordering properties must be maintained and propagated through successive transformations of the array.

```
/*@
  lemma Increasing_Shift{L}:
    \forall value_type *a, integer l, r;
      0 <= l <= r             ==>
      Increasing{L}(a, l, r)  ==>
      Increasing{L}(a+l, r-l);

  lemma Increasing_Equal{K,L}:
    \forall value_type* a, integer m, n, p;
      Increasing{K}(a, m, n)    ==>
      Equal{K,L}(a, m, n, m+p)  ==>
      Increasing{L}(a, m+p, n+p);

  lemma WeaklyIncreasing_Increasing{L}:
    \forall value_type* a, integer m, n;
      0 <= m <= n                ==>
      WeaklyIncreasing(a, m, n)  ==>
      Increasing(a, m, n);

  lemma Increasing_WeaklyIncreasing{L}:
    \forall value_type* a, integer m, n;
      0 <= m <= n                 ==>
      Increasing(a, m, n)         ==>
      WeaklyIncreasing(a, m, n);
*/
```

Listing 6.3: `IncreasingLemmas` (ACSL)

The auxiliary lemmas provided above make the necessary reasoning steps explicit at the logic level. They are reused throughout the verification of binary search algorithms and, as discussed earlier, also play a central role in the verification of sorting algorithms.

In practice, we exploit the relationship between the predicates `Increasing` and `WeaklyIncreasing` in a systematic manner:

- The predicate `Increasing` [6.1] is used in preconditions and postconditions of function contracts, where global ordering assumptions on array segments are required.
- The predicate `WeaklyIncreasing` [6.2] is employed in assertions and loop invariants whenever we need to express that an algorithm (typically a sorting algorithm) maintains or produces a weakly increasing array through local comparisons.
- When it is necessary to conclude that a *weakly increasing* array is in fact *increasing*, we rely on the lemma `WeaklyIncreasing_Increasing` [6.3].

6.3. The lower_bound algorithm

The lower_bound algorithm is one of the four binary search algorithms of the C++ Standard Library [18, §28.7.3.1]. For our purposes we have modified the generic implementation to that of an array of type value_type. The signature now reads:

```
size_type
lower_bound(const value_type* a, size_type n, value_type v);
```

As with the other binary search algorithms lower_bound requires that its input array is in increasing order. The index lb, that lower_bound returns satisfies the inequality

$$0 \leqslant lb \leqslant n \tag{6.1}$$

and has the following properties for any valid index k of the array.

$$0 \leqslant k < lb \quad \Longrightarrow \quad a[k] < v \tag{6.2}$$

$$lb \leqslant k < n \quad \Longrightarrow \quad v \leqslant a[k] \tag{6.3}$$

Conditions (6.2) and (6.3) imply that v can only occur in the array section a[lb..n-1]. In this sense lower_bound returns a *lower bound* for the potential indices.

As an example, we consider in Figure 6.4 an increasingly ordered array. The arrows indicate which indices are returned by lower_bound for a given value. Note that the index 9 points *one past the end* of the array. Values that are not contained in the array are colored in gray.

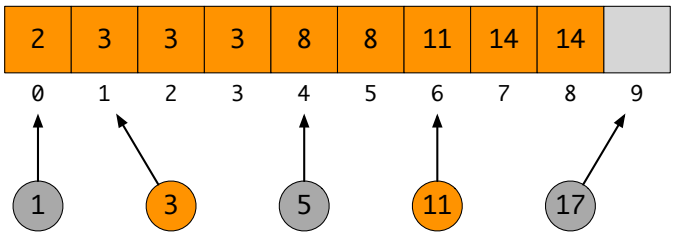


Figure 6.4.: Some examples for lower_bound

Figure 6.4 also clarifies that care must be taken when interpreting the return value of lower_bound. An important difference to the algorithms in Chapter 4 is that a return value of lower_bound that is less than n does not necessarily imply a[lb] == v. We can only be sure that v <= a[lb] holds.

6.3.1. Formal specification of lower_bound

The specification of lower_bound [6.5] is shown in the following listing. The precondition increasing formalizes the requirement that the array values are in increasing order. The postconditions reflect the conditions listed above and are expressed using the predicates LowerBound [5.2] and StrictUpperBound [5.2].

- Condition (6.1) is reflected by the postcondition result.
- Condition (6.2) is reflected by the postcondition left.
- Condition (6.3) is reflected by the postcondition right.

```
/*@
  requires   valid:       \valid_read(a + (0..n-1));
  requires   increasing:  Increasing(a, n);

  terminates              \true;
  exits                   \false;
  assigns                 \nothing;

  ensures    result:      0 <= \result <= n;
  ensures    left:        StrictUpperBound(a, 0, \result, v);
  ensures    right:       LowerBound(a, \result, n, v);
*/
size_type lower_bound(const value_type* a, size_type n, value_type v);
```

Listing 6.5: Formal specification of `lower_bound`

6.3.2. Implementation of `lower_bound`

Our implementation of `lower_bound` [6.6] follows the standard binary search scheme and maintains a shrinking search interval `[left,right)` that is known to contain the result.

```
size_type lower_bound(const value_type* a, size_type n, value_type v)
{
  size_type left  = 0u;
  size_type right = n;

  /*@
    loop invariant bound:  0 <= left <= right <= n;
    loop invariant left:   StrictUpperBound(a, 0,  left, v);
    loop invariant right:  LowerBound(a, right, n, v);

    loop assigns left, right;
    loop variant right - left;
  */
  while (left < right) {
    const size_type middle = left + (right - left) / 2u;

    if (a[middle] < v) {
      left = middle + 1u;
    }
    else {
      right = middle;
    }
  }

  return left;
}
```

Listing 6.6: Implementation of `lower_bound`

Each iteration step narrows down the range that contains the sought-after result. The loop invariants express that in each iteration step all indices less than the temporary left bound `left` contain values that are less than `v` and all indices not less than the temporary right bound `right` contain values that are greater or equal than `v`. The expression to compute `middle` is slightly more complex than the naïve `(left+right)/2`, but it avoids potential overflows.

6.4. The upper_bound algorithm

The upper_bound algorithm of the C++ Standard Library [18, §28.7.3.2] is a variant of binary search and is closely related to lower_bound [6.5]. The signature reads:

```
size_type
upper_bound(const value_type* a, size_type n, value_type v);
```

As with the other binary search algorithms, upper_bound requires that its input array is in increasing order. The index ub returned by upper_bound satisfies the inequality

$$0 \leqslant ub \leqslant n \tag{6.4}$$

and is involved in the following implications for any valid index k of the array under consideration

$$0 \leqslant k < ub \quad \Longrightarrow \quad a[k] \leqslant v \tag{6.5}$$

$$ub \leqslant k < n \quad \Longrightarrow \quad v < a[k] \tag{6.6}$$

Conditions (6.5) and (6.6) imply that v can only occur in the array section a[0..ub-1]. In this sense, upper_bound returns an *upper bound* for the potential indices at which v can occur. It also follows that the searched-for value v can *never* be located at the index ub.

Figure 6.7 is a variant of Figure 6.4 for the upper_bound algorithm, using the same example array. The arrows indicate which indices are returned by upper_bound for a given value. Compared to Figure 6.4, only the arrows corresponding to values that *are present* in the array change their target index.

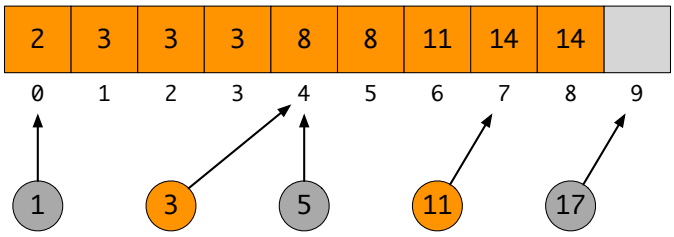


Figure 6.7.: Some examples for upper_bound

6.4.1. Formal specification of upper_bound

The following listing shows the specification of upper_bound [6.8], which is closely related to the specification of lower_bound [6.5]. The precondition increasing formalizes the requirement that the array values are in increasing order.

The postconditions reflect the conditions listed above and are expressed using the predicates UpperBound [5.2] and StrictLowerBound [5.2], namely,

- Condition (6.4) is reflected by the postcondition result.
- Condition (6.5) is reflected by the postcondition left.
- Condition (6.6) is reflected by the postcondition right.

```
/*@
  requires   valid:       \valid_read(a + (0..n-1));
  requires   increasing:  Increasing(a, n);

  terminates              \true;
  exits                   \false;
  assigns                 \nothing;

  ensures    result:      0 <= \result <= n;
  ensures    left:        UpperBound(a, 0, \result, v);
  ensures    right:       StrictLowerBound(a, \result, n, v);
*/
size_type upper_bound(const value_type* a, size_type n, value_type v);
```

Listing 6.8: Formal specification of upper_bound

6.4.2. Implementation of upper_bound

Our implementation of upper_bound [6.9] follows the standard binary search scheme and maintains a shrinking search interval [left,right) that is known to contain the sought-after result.

```
size_type upper_bound(const value_type* a, size_type n, value_type v)
{
  size_type left  = 0u;
  size_type right = n;

  /*@
    loop invariant bound:  0 <= left <= right <= n;
    loop invariant left:   UpperBound(a, 0, left, v);
    loop invariant right:  StrictLowerBound(a, right, n, v);

    loop assigns left, right;
    loop variant right - left;
  */
  while (left < right) {
    const size_type middle = left + (right - left) / 2u;

    if (a[middle] <= v) {
      left = middle + 1u;
    }
    else {
      right = middle;
    }
  }

  return right;
}
```

Listing 6.9: Implementation of upper_bound

The loop invariants express that, throughout the loop, all indices less than the temporary left bound left contain values not greater than v, and all indices not less than the temporary right bound right contain values greater than v.

6.5. The equal_range algorithm

The equal_range algorithm is one of the four binary search algorithms provided by the C++ Standard Library [18, §28.7.3.3]. As with the other binary search algorithms, equal_range requires that its input range is sorted in increasing order.

The specification of this algorithm in the C++ standard is particularly simple, since it explicitly states that equal_range combines the results of lower_bound [6.5] and upper_bound [6.8]. In C++, equal_range therefore returns a pair of iterators that delimit the subrange of elements equal to a given value.

As usual for our purposes, we adapt equal_range to operate on an array of type value_type. Instead of returning a pair of iterators, our version returns a pair of indices. The return type of equal_range is the struct size_type_pair shown in Listing 5.4, and its signature reads as follows:

```
size_type_pair
equal_range(const value_type* a, size_type n, value_type v);
```

In accordance with the preceding discussion, Figure 6.10 combines Figures 6.4 and 6.7 in order to visualize the behavior of equal_range for selected test cases. The two types of arrows, $\rightarrow$ and $\dashrightarrow$, represent the respective fields first and second of the return value. For values that are not contained in the array, the two arrows point to the same index.

More generally, if equal_range returns the pair (lb, ub), then the difference $ub - lb$ is equal to the number of occurrences of the argument v in the array.

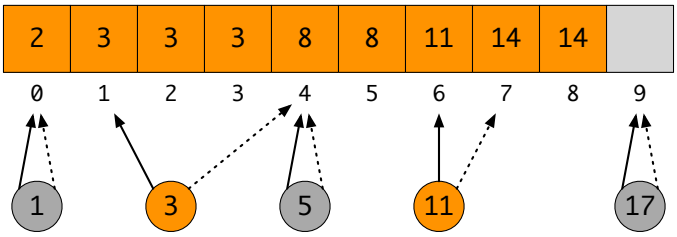


Figure 6.10.: Some examples for equal_range

We present two different implementations of equal_range and verify both of them against the *same formal specification*. Although the implementations differ substantially in structure, they are intended to be observationally equivalent.

The first implementation, equal_range [6.12], is deliberately straightforward: it invokes lower_bound and upper_bound separately and returns the pair consisting of their respective results. The second implementation, equal_range2 [6.13], is more elaborate and follows an efficiency-oriented strategy by attempting to minimize duplicate computations.

6.5.1. Formal specification of `equal_range`

Let (lb, ub) be the return value of `equal_range`. Then the conditions (6.1)–(6.6) can be combined into the inequality

$$0 \leqslant lb \leqslant ub \leqslant n. \tag{6.7}$$

Moreover, for any valid index k of the array under consideration, the following three implications hold:

$$0 \leqslant k < lb \quad \Longrightarrow \quad a[k] < v, \tag{6.8}$$
$$lb \leqslant k < ub \quad \Longrightarrow \quad a[k] = v, \tag{6.9}$$
$$ub \leqslant k < n \quad \Longrightarrow \quad a[k] > v. \tag{6.10}$$

Here are some justifications for these conditions.

- Conditions (6.8) and (6.10) are exactly Conditions (6.2) and (6.6), respectively.
- The inequality (6.7) follows from (6.1) and (6.4) together with the following observation. If ub were smaller than lb, then by (6.8) we would have $a[ub] < v$. On the other hand, (6.10) implies $v < a[ub]$. This contradiction shows that $lb \leqslant ub$ must hold.
- Condition (6.9) follows from the combination of (6.3) and (6.5), together with the fact that $\leqslant$ is a total order on the integers.

The following listing shows the shared ACSL specification of `equal_range` [6.11] and `equal_range2`. The precondition `increasing` expresses that the array values are required to be in increasing order.

```
/*@
  requires   valid:       \valid_read(a + (0..n-1));
  requires   increasing:  Increasing(a, n);

  terminates              \true;
  exits                   \false;
  assigns                 \nothing;

  ensures    result:      0 <= \result.first <= \result.second <= n;
  ensures    left:        StrictUpperBound(a, 0, \result.first, v);
  ensures    middle:      AllEqual(a, \result.first, \result.second, v);
  ensures    right:       StrictLowerBound(a, \result.second, n, v);
 */
size_type_pair equal_range(const value_type* a, size_type n, value_type v);
```

Listing 6.11: Formal specification of `equal_range`

The postconditions reflect the conditions listed above and can be expressed using the already introduced predicates `AllEqual` [4.15], `StrictUpperBound` [5.2], and `StrictLowerBound`. In particular, we observe:

- Condition (6.7) becomes postcondition `result`.
- Condition (6.8) becomes postcondition `left`.
- Condition (6.9) becomes postcondition `middle`.
- Condition (6.10) becomes postcondition `right`.

6.5.2. Implementation of equal_range

Our first implementation of equal_range [6.12] is shown in the following listing. It serves as a baseline implementation and closely mirrors the structure suggested by the specification: it invokes lower_bound [6.5] and upper_bound [6.8] independently and returns the resulting pair.

```
size_type_pair equal_range(const value_type* a, size_type n, value_type v)
{
  size_type first  = lower_bound(a, n, v);
  size_type second = upper_bound(a, n, v);
  //@ assert aux: second < n  ==>  v < a[second];
  return make_pair(first, second);
}
```

Listing 6.12: Implementation of equal_range

In an early version of this document, the relation first <= second was established using the interactive theorem prover Coq. After revisiting this proof, we reformulated the argument by introducing the auxiliary assertion aux, which exploits a property already guaranteed by the postcondition of upper_bound [6.8]. As a result, both the assertion aux and the postcondition first <= second can now be verified fully automatically.

6.5.3. Implementation of equal_range2

The first implementation of equal_range [6.12] performs more work than necessary, since it computes the lower and upper bounds independently. The second implementation, equal_range2 [6.13], follows a more typical and efficient C++ approach.

The algorithm first performs a binary-search–style range reduction that narrows the search interval as much as possible with respect to the argument v. Only once this reduced range has been determined are upper_bound and lower_bound applied to the corresponding subranges. In this way, duplicate comparisons are avoided while preserving the same observable behavior and the same formal specification.

Due to the higher code complexity of the second implementation, additional assertions had to be inserted in order to ensure that Frama-C/WP is able to verify the correctness of the code. In contrast to the first implementation, these assertions do not concern the algorithmic behavior itself, but rather properties of pointer arithmetic and shifting base pointers. All of these assertions fall into three closely related groups, which are briefly discussed after the listing of equal_range2.

```
size_type_pair equal_range2(const value_type* a, size_type n, value_type v)
{
  size_type first  = 0u;
  size_type middle = 0u;
  size_type last   = n;

  /*@
    loop invariant bounds: 0 <= first <= last <= n;
    loop invariant left:   StrictUpperBound(a, 0, first, v);
    loop invariant right:  StrictLowerBound(a, last, n, v);
    loop assigns first, last, middle;
    loop variant last - first;
   */
  while (last > first) {
    middle = first + (last - first) / 2u;

    if (a[middle] < v) {
      first = middle + 1u;
    }
    else if (v < a[middle]) {
      last = middle;
    }
    else {
      break;
    }
  }

  if (first < last) {
    //@ assert increasing: Increasing(a, first, middle);
    size_type left = first + lower_bound(a + first, middle - first, v);
    //@ assert middle: LowerBound(a, left, middle, v);
    //@ assert left:   StrictUpperBound(a, first, left, v);
    ++middle;
    //@ assert increasing: Increasing(a, middle, last);
    size_type right = middle + upper_bound(a + middle, last - middle, v);
    //@ assert middle: UpperBound(a, middle, right, v);
    //@ assert right:  StrictLowerBound(a, right, last, v);
    //@ assert middle: AllEqual(a, left, right, v);
    return make_pair(left, right);
  }
  else {
    return make_pair(first, first);
  }
}
```

Listing 6.13: Implementation of `equal_range2`

The `increasing` properties. Both `lower_bound` and `upper_bound` require that they operate on increasingly ordered arrays. This requirement also applies to `equal_range`. However, in the second implementation a more specific formulation is needed for the subarrays on which these functions are invoked. In particular, using the predicate `Increasing` to write

```
Increasing(a, middle, last)
```

expresses the required ordering property for an intermediate subarray. Making this property explicit allows the automatic provers to discharge the preconditions of the call to `lower_bound`. A corresponding assertion is used before the call to `upper_bound`.

To enable the automatic verification of the corresponding proof obligations, we introduce a small collection of auxiliary lemmas, summarized under the name [6.14]. These lemmas capture general facts about array bounds under pointer shifts and can be reused independently of the present algorithm.

```
/*@
  lemma LowerBound_Shift{L}:
    \forall value_type *a, v, integer b, c, d;
      LowerBound{L}(a+b, c, d, v)  ==>  LowerBound{L}(a, c+b, d+b, v);

  lemma StrictLowerBound_Shift{L}:
    \forall value_type *a, v, integer b, c, d;
      StrictLowerBound{L}(a+b, c, d, v)  ==> StrictLowerBound{L}(a, c+b, d+b, v);

  lemma UpperBound_Shift{L}:
    \forall value_type *a, v, integer b, c;
      UpperBound{L}(a+b, 0, c-b, v)  ==>  UpperBound{L}(a, b, c, v);

  lemma StrictUpperBound_Shift{L}:
    \forall value_type *a, v, integer b, c;
      StrictUpperBound{L}(a+b, 0, c-b, v)  ==> StrictUpperBound{L}(a, b, c, v);
*/
```

Listing 6.14: `ArrayBoundsShift` (ACSL)

The `strict` and `constant` properties. Part of the postconditions of `equal_range` [6.11] is that the argument `v` is both a strict upper bound and a strict lower bound for the corresponding array segments. However, the calls to `lower_bound` and `upper_bound` only establish the following properties:

```
StrictUpperBound(a + first, 0, left - first, v)
StrictLowerBound(a + middle, right - middle, last - middle, v)
```

Taken in isolation, these properties are not sufficient to derive the required postconditions automatically. In each case, a single intermediate assertion is introduced to bridge this gap and to guide the prover towards the desired result.

Closely related to the `strict` properties are the `constant` properties, which capture bounds that are invariant with respect to the current subarray. They guide the prover towards the following assertions:

```
LowerBound(a, left, n, v)
UpperBound(a, 0, right, v)
```

Together, the `strict` and `constant` properties allow the postcondition `middle` to be derived automatically.

6.6. The binary_search algorithm

The binary_search algorithm is one of the four binary search algorithms of the C++ Standard Library [18, §28.7.3.4]. For the purposes of this chapter, we adapt the generic, iterator-based interface of the standard algorithm to operate on a contiguous array of elements of type value_type. Its signature therefore reads:

```
bool binary_search(const value_type* a, size_type n, value_type v);
```

The function binary_search requires that the input array a[0..n-1] is sorted in increasing order. It returns **true** if and only if there exists an index i such that a[i] == v. More precisely, the C++ Standard Library specifies this condition in terms of the ordering relation, requiring that both a[i] <= v and v <= a[i] hold. For our definition of value_type (see §2), this is equivalent to the equality a[i] == v.

The following figure illustrates the behavior of binary_search on several example arrays.

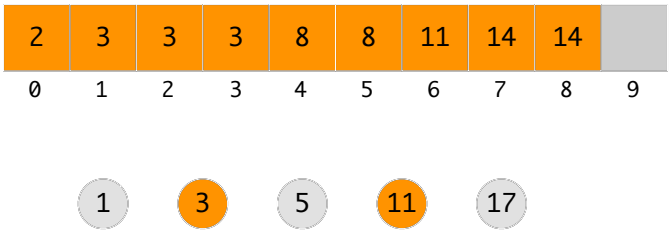


Figure 6.15.: Some examples for binary_search

In contrast to the other binary search algorithms discussed in this chapter, binary_search does not return a specific array position or range, but only a boolean result. For this reason, no arrows are required in Figure 6.15. Instead, the colors orange and grey of the sought-after values indicate whether the algorithm returns **true** or **false**, respectively.

6.6.1. Formal specification of binary_search

The ACSL specification of binary_search is shown in the following listing.

```
/*@
  requires   valid:       \valid_read(a + (0..n-1));
  requires   increasing:  Increasing(a, n);

  terminates              \true;
  exits                   \false;
  assigns                 \nothing;

  ensures    result:      \result <==> SomeEqual(a, n, v);
*/
bool binary_search(const value_type* a, size_type n, value_type v);
```

Listing 6.16: Formal specification of binary_search

Conceptually, the result of `binary_search` must express the existence of an index at which the searched value occurs in the input array. At the level of ACSL formulas, this condition can be written as

```
\exists integer i; 0 <= i < n && a[i] == v;
```

While this formulation is logically correct, repeating such existential expressions verbatim across multiple specifications is undesirable. As discussed earlier in this report, careful *specification and proof engineering* aims at factoring out recurring logical patterns into named predicates. This improves readability, promotes uniformity across specifications, and reduces the effort required to maintain and adapt proofs.

For this reason, we reuse the predicate `SomeEqual` [4.4], which captures the above existential condition once and for all. Using this predicate leads to a more concise and robust specification of `binary_search` and serves as a further illustration of the benefits of systematic abstraction at the specification level.

It is instructive to compare the specification of `binary_search` [6.16] with that of `find2` [4.5]. At the level of functional behavior, both algorithms serve the same purpose: they determine whether a given value is contained in an array. Consequently, their ACSL specifications are closely related.

By contrast, the essential difference between the two algorithms lies in their complexity guarantees. The C++ Standard Library requires `find` to have *linear* complexity, whereas `binary_search` must have *logarithmic* complexity. Such asymptotic complexity requirements, however, are beyond the *current expressive power* of ACSL and therefore cannot be captured directly in the specification.

6.6.2. Implementation of `binary_search`

Our implementation of `binary_search` [6.17] is deliberately simple and closely follows the structure suggested by the specification. It relies on a single call to `lower_bound` [6.5] to identify a candidate position at which the searched value may occur.

```
bool binary_search(const value_type* a, size_type n, value_type v)
{
  const size_type i = lower_bound(a, n, v);
  return (i < n) && (a[i] <= v);
}
```

Listing 6.17: Implementation of `binary_search`

Recall that `lower_bound` returns the smallest index `i` such that `v <= a[i]` holds, or `n` if no such index exists. Consequently, if `lower_bound` returns an index `0 <= i < n` and, in addition, `a[i] <= v` holds, then both ordering relations `a[i] <= v` and `v <= a[i]` are satisfied, which implies that `a[i] == v`.

The implementation therefore checks whether the index returned by `lower_bound` lies within the array bounds and whether the corresponding array element is equal to the searched value. Based on this test, it returns either **`true`** or **`false`**, in accordance with the specification.

Part III.

Mutating and Numeric Algorithms

Introduction

This part is devoted to algorithms that modify program state or compute numeric results derived from the elements of a sequence. In contrast to the nonmutating algorithms studied earlier, the procedures considered here actively update memory locations or accumulate values during traversal.

Chapter 7 introduces mutating algorithms. These algorithms transform a sequence in place, for example by overwriting elements or rearranging their order. From a verification perspective, mutation requires more detailed reasoning about memory updates, frame conditions, and the preservation of unaffected data.

Chapter 8 focuses on numeric algorithms. These algorithms compute aggregated numeric values, such as sums or products, from the elements of one sequence or several sequences. Although they may or may not modify the input data, their defining characteristic is the systematic accumulation of intermediate results, which gives rise to characteristic loop invariants and correctness arguments.

Together, these chapters introduce algorithmic patterns in which state changes and numeric computation play a central role. They prepare the ground for the subsequent parts of the document, where mutation is combined with reordering and more complex structural properties of data.

7. Mutating algorithms

Let us now turn our attention to another class of algorithms, viz. *mutating* algorithms of the C++ Standard Library [18, §28.6], i.e., algorithms that change one or more ranges. In Frama-C, you can explicitly specify that, e.g., entries in an array a may be modified by a function f, by including the following *assigns clause* into the f's specification:

```
assigns a[0..length-1];
```

The expression length-1 refers to the value of length when f is entered, see [9, §2.3.2].

The sections of this chapter are organized to progressively develop techniques for specifying and verifying mutating algorithms.

- We begin with general specification mechanisms for array mutation. §7.1 introduces the predicate Unchanged for expressing preservation properties, while §7.2 adds the logic function At and the predicate ArrayUpdate to reason about localized updates at the level of individual array elements.
- We then study basic mutating operations. fill (§7.3) initializes an array, and swap (§7.4) exchanges two values.
- More complex algorithms are obtained by composition. swap_ranges (§7.5) exchanges two arrays element by element and serves as a first example of modular verification through reuse of the verified swap operation.
- Copying algorithms are treated next. copy (§7.6) and copy_backward (§7.7) copy arrays under different separation assumptions.
- Reordering algorithms follow. reverse_copy and reverse (§7.8, §7.9) reverse arrays, while rotate_copy and rotate (§7.10, §7.11) perform rotations.
- Replacement and filtering algorithms are then considered. replace_copy and replace (§7.12, §7.13) substitute values, whereas remove_copy and remove (§7.14–§7.17) filter values. The multiple specifications of remove_copy highlight the iterative nature of contract development.
- The chapter concludes with shuffle (§7.18), which randomly reorders an array and relies on the random number generator random_number (§7.19).

7.1. The predicate Unchanged

Many of the algorithms in this section iterate sequentially over one or several sequences. For the verification of such algorithms, it is often important to express that a section of an array, or even the complete array, has remained *unchanged*. Since this property cannot always be expressed using an `assigns` clause alone, we introduce in the following listing the overloaded predicate `Unchanged` [7.1]. The expression `Unchanged{K,L}(a,m,n)` holds if the range `a[m..n-1]` in state `K` is element-wise equal to the corresponding range in state `L`.

```
/*@
  predicate Unchanged{K,L}(value_type* a, integer m, integer n) =
    \forall integer i; m <= i < n ==>  \at(a[i],K) == \at(a[i],L);

  predicate Unchanged{K,L}(value_type* a, integer n) = Unchanged{K,L}(a, 0, n);
*/
```

Listing 7.1: `Unchanged` (ACSL)

Using the slice notation introduced with Equations (4.4), the predicate **Unchanged**$\{K, L\}(a, m, n)$ expresses that the array slice $a[m \mathinner{..} n)$ has the same values in program states K and L, that is,

$$a^K[m \mathinner{..} n) = a^L[m \mathinner{..} n). \tag{7.1}$$

Although the predicate `Unchanged` is closely related to `Equal`, it serves a different purpose. The predicate `Equal` is used to express equality between two array segments, either belonging to different arrays or to different segments of the same array. By contrast, `Unchanged` always refers to the same array segment, but compares its values across two different program states. It therefore captures preservation across program states and can be combined naturally with operational `assigns` clauses in loop invariants and postconditions. The following example illustrates why this distinction is essential.

Operational vs. semantic preservation

Consider the following function, which explicitly writes back each element of an array:

```
void rewrite_array(value_type *a, size_type n)
{
  for (int i = 0; i < n; i++) {
    a[i] = a[i];
  }
}
```

From a semantic point of view, the array is not changed: every element in the post-state is equal to the corresponding element in the pre-state. It is therefore tempting to specify the function as follows:

```
/*@
  requires \valid(a + (0..n-1));
  assigns  \nothing;
  ensures  Unchanged{Pre,Post}(a,0,n);
*/
void rewrite_array(value_type* a, size_type n);
```

However, this specification is incorrect. Although the values stored in the array remain the same, the function performs write operations to memory. More generally, an `assigns` clause does not specify

precisely which memory locations are modified (only which are permitted to be modified). In particular, the clause `assigns \nothing;` forbids *any* write, even if the written value coincides with the previous one.

A correct specification must therefore allow the writes explicitly:

```
/*@
  requires \valid(a + (0..n-1));
  assigns  a[0..n-1];
  ensures  Unchanged{Pre,Post}(a,0,n);
*/
void rewrite_array(value_type* a, size_type n);
```

The next listing shows the ACSL-annotated implementation of the correct specification. In particular:

- The two invariants `array_prefix` and `array_suffix` split the array into a prefix `a[0..i-1]` and a suffix `a[i..n-1]`. The invariant `array_prefix` asserts that all elements already visited by the loop have the same value as in the pre-state. The invariant `array_suffix` asserts the same for the elements not yet visited.
- The `loop assigns` clause specifies which locations the loop body is allowed to write to. Since the statement `a[i] = a[i];` performs a write, we must permit writes to (parts of) the array. At the same time, the invariants ensure that these writes do not change the values, yielding the postcondition `Unchanged{Pre,Post}(a,0,n)` after loop termination.

```
void rewrite_array(value_type* a, size_type n)
{
  /*@
    loop invariant 0u <= i <= n;
    loop invariant array_prefix: Unchanged{Pre,Here}(a, 0, i);
    loop invariant array_suffix: Unchanged{Pre,Here}(a, i, n);
    loop assigns i, a[0..n-1];
    loop variant n - i;
  */
  for (size_type i = 0u; i < n; i++) {
    a[i] = a[i];
  }
}
```

In summary, this pair of specifications highlights an essential distinction. The `assigns` clause is an operational statement about memory writes, whereas `Unchanged` [7.1] is a semantic statement about equality of values between program states.

Lemmas for Unchanged

The following listing collects a small number of auxiliary lemmas about the predicate Unchanged [7.1] that are used repeatedly in the verification of subsequent algorithms.

```
/*@
  lemma Unchanged_Shrink{K,L}:
    \forall value_type *a, integer m, n, p, q;
       m <= p <= q <= n          ==>
       Unchanged{K,L}(a, m, n)   ==>
       Unchanged{K,L}(a, p, q);

  lemma Unchanged_Extend{K,L}:
    \forall value_type *a, integer n;
      Unchanged{K,L}(a, n)         ==>
      \at(a[n],K) == \at(a[n],L)   ==>
      Unchanged{K,L}(a, n+1);

  lemma Unchanged_Shift{K,L}:
    \forall value_type *a, integer p, q, r;
      Unchanged{K,L}(a+p, q, r)  ==>  Unchanged{K,L}(a, p+q, p+r);

  lemma Unchanged_Symmetric{K,L}:
    \forall value_type *a, integer m, n;
      Unchanged{K,L}(a, m, n)  ==>
      Unchanged{L,K}(a, m, n);

  lemma Unchanged_Transitive{K,L,M}:
    \forall value_type *a, integer m, n;
      Unchanged{K,L}(a, m, n)  ==>
      Unchanged{L,M}(a, m, n)  ==>
      Unchanged{K,M}(a, m, n);
*/
```

Listing 7.2: UnchangedLemmas (ACSL)

Using the array slice notation used in Equation (7.1) to characterize the predicate Unchanged, the above lemmas express the following.

- Lemma Unchanged_Shrink [7.2] states that equality of array slices is preserved under restriction to subranges. If the slice $a^K[m \mathinner{..} n)$ coincides with $a^L[m \mathinner{..} n)$, then any sub-slice $a^K\big[p \mathinner{..} q\big)$ coincides with $a^L\big[p \mathinner{..} q\big)$ as well, provided $0 \leqslant m \leqslant p \leqslant q \leqslant n$.
- Lemma Unchanged_Extend expresses that equality of adjacent slices is preserved under concatenation. If $a^K[0 \mathinner{..} n) = a^L[0 \mathinner{..} n)$ and $a^K[n] = a^L[n]$, then $a^K[0 \mathinner{..} n+1) = a^L[0 \mathinner{..} n+1)$. This property allows equality of prefixes to be established inductively.
- Lemma Unchanged_Shift describes how slice equality behaves under pointer arithmetic. It relates equality of the slice $(a+p)^K\big[q \mathinner{..} r\big)$ to equality of the slice $(a+p)^L\big[q \mathinner{..} r\big)$, or equivalently, equality of $a^K\big[p+q \mathinner{..} p+r\big)$ to equality of $a^L\big[p+q \mathinner{..} p+r\big)$.
- Lemmas Unchanged_Symmetric and Unchanged_Transitive establish that slice equality is symmetric and transitive with respect to program states.

7.2. Reasoning about local array updates

When updating a single element of an array, verification typically requires expressing two complementary properties. One distinguished element may change its value. All remaining elements must be shown to remain unchanged. Such properties are sometimes difficult to express directly using `assigns` clauses alone. This is particularly the case when reasoning about intermediate program points or when the relevant program state cannot be named explicitly.

To simplify comparisons of array elements across program states, we introduce the auxiliary logic function `At` [7.3]. The notation `\At{L}(a,i)` denotes the value of the array element `a[i]` in the program state labeled `L`. Compared to the built-in notation `\at(a[i],L)`, this formulation is slightly more concise. It also reads more fluently in specifications, reflecting the intended meaning "at `L`, consider `a` at index `i`".

```
/*@
  logic value_type At{L}(value_type* x, integer i) = \at(x[i],L);
*/
```

Listing 7.3: `At` (ACSL)

In order to describe updates of individual array elements more concisely, we also introduce the predicate `ArrayUpdate` [7.4]. This predicate relies on `Unchanged` [7.1] and uses the logic function `At`. It characterizes the effect of updating one array element while preserving all others. The accompanying lemmas `ArrayUpdate_Shrink` and `ArrayUpdate_UpperBound` are intended to support common proof patterns involving localized array updates.

```
/*@
  predicate ArrayUpdate{K,L}(value_type* a, integer n, integer i, value_type v) =
      0 <= i < n                  &&
      Unchanged{K,L}(a, 0, i)     &&
      Unchanged{K,L}(a, i+1, n)   &&
      At{K}(a, i) != v            &&
      At{L}(a, i) == v;

  lemma ArrayUpdate_Shrink{K,L}:
    \forall value_type *a, v, integer n, i;
      0 <= i < n-1                  ==>
      ArrayUpdate{K,L}(a, n, i, v)  ==>
      ArrayUpdate{K,L}(a, n-1, i, v);

  lemma ArrayUpdate_UpperBound{K,L}:
    \forall value_type *a, v, w, integer n, i;
      ArrayUpdate{K,L}(a, n, i, v)  ==>
      v <= w                        ==>
      UpperBound{K}(a, n, w)        ==>
      UpperBound{L}(a, n, w);
*/
```

Listing 7.4: `ArrayUpdate` (ACSL)

7.3. The `fill` algorithm

The `fill` algorithm of the C++ Standard Library [18, §28.6.6] initializes a sequence by assigning a fixed value to each of its elements. The signature of the modified variant considered here is

```
void fill(value_type* a, size_type n, value_type v);
```

7.3.1. Formal specification of `fill`

The following listing presents the formal specification of `fill` [7.5]. The postcondition of `fill` can be expressed concisely by means of the overloaded predicate `AllEqual` [4.15]. Note that is postcondition could equivalently be written using an explicit universal quantification, stating

```
    ensures constant: \forall integer i; 0 <= i < n  ==>  a[i] == v;
```

While this formulation is perfectly adequate, the use of the predicate `AllEqual` [4.15] avoids repeating this quantifier structure verbatim. More importantly, it allows the same logical concept to be reused uniformly in postconditions, loop invariants, and auxiliary lemmas, thereby keeping the specification concise and conceptually consistent.

```
/*@
  requires   valid:   \valid(a + (0..n-1));

  terminates           \true;
  exits                \false;
  assigns              a[0..n-1];

  ensures    constant: AllEqual(a, n, v);
*/
void fill(value_type* a, size_type n, value_type v);
```

Listing 7.5: Formal specification of `fill`

The `assigns` clauses specify that `fill` may modify only the elements of the range `a[0..n-1]`. In general, if more than one *assigns clause* appears in a function contract, the function is permitted to modify any of the referenced memory locations. If, however, no *assigns clause* is present at all, the function is allowed to modify arbitrary memory locations, see [9, §2.3.2]. To explicitly forbid any modification outside the intended scope, an `assigns \nothing;` clause must be used, as already practised in the example specifications of Chapter 4.

7.3.2. Implementation of `fill`

The implementation of `fill` [7.6] is annotated with a loop invariant named `constant`. This invariant expresses that, at each iteration, the array prefix up to the current loop index `i` is entirely filled with the value `v`. Here again, the predicate `AllEqual` [4.15] is used to formulate this property.

```
void fill(value_type* a, size_type n,  value_type v)
{
  /*@
    loop invariant bound:    0 <= i <= n;
    loop invariant constant: AllEqual(a, i, v);
    loop assigns i, a[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    a[i] = v;
  }
}
```

Listing 7.6: Implementation of `fill`

7.4. The swap algorithm

The swap algorithm [18, § 28.6.3] in the C++ Standard Library exchanges the contents of two variables. Closely related is the algorithm iter_swap [18, § 28.6.3], which exchanges the contents referenced by two iterators. Since C and hence ACSL do not support C++ references, we restrict ourselves to a pointer-based variant and refer to it simply as swap.

7.4.1. Formal specification of swap

The contract of swap [7.7] is shown in the following listing. The preconditions require that both pointer arguments are valid and dereferenceable. No further assumptions are needed.

```
/*@
  requires   valid:     \valid(p);
  requires   valid:     \valid(q);

  terminates            \true;
  exits                 \false;
  assigns               *p, *q;

  ensures    exchange:  *p == \old(*q);
  ensures    exchange:  *q == \old(*p);
*/
void swap(value_type* p, value_type* q);
```

Listing 7.7: Formal specification of swap

The postcondition specifies that the values stored at the two memory locations have been exchanged. The expression \old(*p) denotes the value of *p in the pre-state of the function call. Unless stated otherwise, postconditions are interpreted with respect to the state after normal termination of the function.

7.4.2. Implementation of swap

The following listing shows a straightforward implementation of swap [7.8]. It follows the conventional three-step pattern using a temporary variable.

```
void swap(value_type* p, value_type* q)
{
  value_type save = *p;
  *p = *q;
  *q = save;
}
```

Listing 7.8: Implementation of swap

No auxiliary annotations are required. The implementation satisfies the contract directly and can be verified by Frama-C/WP without additional assertions or loop invariants.

7.5. The swap_ranges algorithm

The swap_ranges algorithm in the C++ Standard Library [18, §28.6.3] exchanges the contents of two ranges element-wise. After translating C++ reference types and iterators to C, our version of the signature reads:

```
void swap_ranges(value_type* a, size_type n, value_type* b);
```

We do not return a value. In the original interface, the return value merely identifies the end position, which in our setting is determined by n anyway.

7.5.1. Formal specification of swap_ranges

The following listing shows a specification for swap_ranges [7.9].

```
/*@
  requires   valid:  \valid(a + (0..n-1));
  requires   valid:  \valid(b + (0..n-1));
  requires   sep:    \separated(a+(0..n-1), b+(0..n-1));

  terminates         \true;
  exits              \false;
  assigns            a[0..n-1];
  assigns            b[0..n-1];

  ensures    equal:  Equal{Old,Here}(a, n, b);
  ensures    equal:  Equal{Old,Here}(b, n, a);
*/
void swap_ranges(value_type* a, size_type n, value_type* b);
```

Listing 7.9: Formal specification of swap_ranges

The algorithm is well-defined only if the two ranges do not overlap. This requirement is expressed by the precondition labeled sep, which states that the memory regions a[0..n-1] and b[0..n-1] are separated.

The assigns clauses specify that swap_ranges may modify exactly the two ranges a[0..n-1] and b[0..n-1], and no other memory locations.

The postconditions labeled equal specify the mutual exchange of the two ranges. They are formulated using the predicate Equal [4.28] together with the labels Old and Here. Since these formulas occur in a postcondition, Here refers to the post-state of swap_ranges, whereas Old refers to its pre-state. Together, the two equal postconditions express that each element of a is replaced by the corresponding pre-state element of b, and vice versa.

7.5.2. Implementation of swap_ranges

The implementation of swap_ranges [7.10] together with the loop annotations is shown in the following listing.

```
void swap_ranges(value_type* a, size_type n, value_type* b)
{
  /*@
    loop invariant bound:      0 <= i <= n;
    loop invariant equal:      Equal{Pre,Here}(a, i, b);
    loop invariant equal:      Equal{Pre,Here}(b, i, a);

    loop invariant unchanged:  Unchanged{Pre,Here}(a, i, n);
    loop invariant unchanged:  Unchanged{Pre,Here}(b, i, n);

    loop assigns i, a[0..n-1], b[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    swap(a + i, b + i);
  }
}
```

Listing 7.10: Implementation of swap_ranges

The loop repeatedly calls swap [7.7] to exchange the current pair of elements. To establish the postconditions labeled equal, the loop invariants must capture both the progress of the swap and the preservation of the untouched suffixes.

- The loop invariants labeled equal state that, at each iteration, the prefixes a[0..i-1] and b[0..i-1] have already been exchanged. They use the predicate Equal [4.28] with the labels Pre and Here, where Pre denotes the loop-entry state and Here the current state.
- In addition, the loop invariants labeled unchanged state that the suffixes a[i..n-1] and b[i..n-1] still coincide with their values at loop entry. This property is expressed using the predicate Unchanged.

These unchanged invariants are essential because the loop assigns clause allows modification of the full ranges a[0..n-1] and b[0..n-1]. They recover the finer-grained information needed to conclude the postconditions labeled equal.

7.6. The copy algorithm

The copy algorithm in the C++ Standard Library [18, §28.6.1] implements the duplication of a finite sequence. In its generic form, the algorithm operates on iterator pairs and supports a wide range of value types. For the purposes of this presentation, we restrict attention to contiguous ranges of elements of type `value_type`. This specialization allows us to focus on the essential behaviour of the algorithm without distraction from iterator abstractions. After this simplification, the signature of `copy` reads:

```
void copy(const value_type* a, size_type n, value_type* b);
```

Informally, the function copies each element of the source range `a[0..n-1]` to the corresponding position in the destination range `b[0..n-1]`. No other memory locations are affected. The overall effect of the algorithm is illustrated in Figure 7.11.

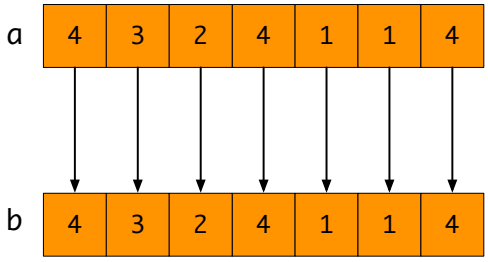


Figure 7.11.: Effects of `copy`

7.6.1. Formal specification of copy

The formal specification of `copy` is shown in the following listing. As usual, the preconditions require that the source range `a[0..n-1]` is valid for reading and that the destination range `b[0..n-1]` is valid for writing. The distinctive precondition is `sep`, which captures the non-overlapping property discussed below and rules out only those memory configurations that would invalidate the left-to-right copying semantics.

```
/*@
  requires   valid:  \valid_read(a + (0..n-1));
  requires   valid:  \valid(b + (0..n-1));
  requires   sep:    \separated(a + (0..n-1), b);

  terminates         \true;
  exits              \false;
  assigns            b[0..n-1];

  ensures    equal:  Equal{Old,Here}(a, n, b);
*/
void copy(const value_type* a, const size_type n, value_type* b);
```

Listing 7.12: Formal specification of `copy`

At first glance, Figure 7.11 may suggest that the source and destination ranges `a[0..n-1]` and `b[0..n-1]` must be completely disjoint. This requirement would be natural for a specification that abstracts away from the operational behaviour of the algorithm. However, the informal description of `copy` is more precise: elements are copied sequentially in order of increasing indices. As a consequence, a strictly weaker separation condition is sufficient. More precisely, it is only required that the destination pointer `b` does not point into the source range `a[0..n-1]`. In this situation, copying from left to right cannot overwrite values that are still needed. An example of such a permitted overlap is illustrated in Figure 7.13.

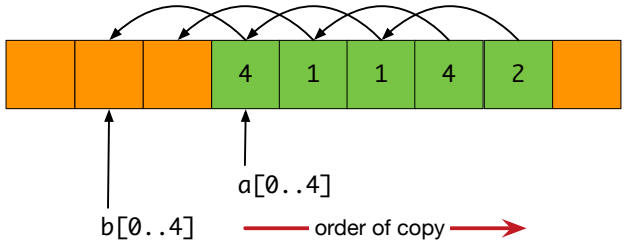


Figure 7.13.: Possible overlap of copy ranges

The postcondition expresses that, upon termination, the destination range contains an element-wise copy of the source range as it was in the pre-state. This is concisely stated using the predicate `Equal` [4.28]. The `assigns`-clause restricts modifications to the destination range `b[0..n-1]` only. In particular, no guarantee is made that the source range `a[0..n-1]` remains unchanged. This asymmetry is intentional and reflects the weak separation requirement discussed above: in the situation illustrated in Figure 7.13, writing to the destination range may overwrite parts of the source range without violating the intended left-to-right copying semantics.

7.6.2. Implementation of copy

The following listing shows an implementation of the `copy` function together with its ACSL annotations.

```
void copy(const value_type* a, size_type n, value_type* b)
{
  /*@
    loop invariant bound:     0 <= i <= n;
    loop invariant equal:     Equal{Pre,Here}(a, i, b);
    loop invariant unchanged: Unchanged{Pre,Here}(a, i, n);
    loop assigns   i, b[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    b[i] = a[i];
  }
}
```

Listing 7.14: Implementation of `copy`

In order for the postcondition `equal` to hold, it must be guaranteed that, for every index `i`, the value `a[i]` is read before it can be modified by any write operation performed by the algorithm. This requirement follows directly from the left-to-right copying semantics and is essential in the presence of the weak separation condition discussed in the previous subsection.

Within the loop, this property is expressed by means of the predicate `Unchanged`, which asserts that the relevant part of the source range has not been altered at the current iteration. The invariant therefore connects the operational order of the copy with the logical requirement needed to establish the postcondition.[11]

The function-level `assigns` clause restricts side effects to the destination range `b[0..n-1]` and to the loop variable `i`. As in the specification, no attempt is made to preserve the source range `a[0..n-1]`. If the ranges overlap as in Figure 7.13, writes to the destination range may overwrite elements of the source range without violating either the annotations or the intended behaviour of the algorithm.

[11] Alternatively, the same effect could be achieved by strengthening the `loop assigns` clause to `i, b[0..i-1]`. At present, however, Frama-C does not support `loop assigns` clauses that refer to the loop variable itself.

7.7. The copy_backward algorithm

The `copy_backward` algorithm in the C++ Standard Library [18, §28.6.1] implements a duplication algorithm that is complementary to `copy`. As before, we restrict the generic iterator-based interface to contiguous ranges of elements of type `value_type`. The resulting signature reads:

```
void copy_backward(const value_type* a, size_type n, value_type* b);
```

The primary motivation for `copy_backward` is to support copying in memory configurations that violate the separation requirement of `copy`. In particular, `copy_backward` can be applied when the beginning of the destination range `b[0..n-1]` lies inside the source range `a[0..n-1]`. In such situations, the precondition `sep` of `copy` [7.12] is violated, and left-to-right copying would overwrite values that are still required.

To address this issue, the informal specification of `copy_backward` prescribes copying in reverse order, starting from the end of the source range. For this strategy to be safe, it is required that the pointer `b+n` does not point into the source range. Figure 7.15 illustrates a typical overlap pattern in which `copy_backward`, but not `copy`, can be applied.

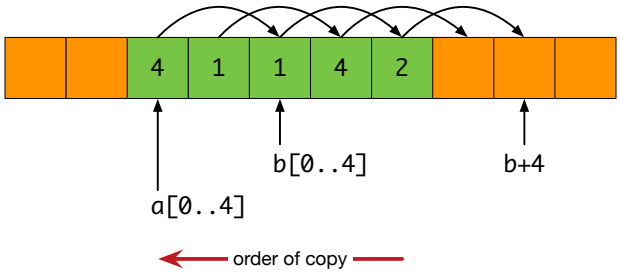


Figure 7.15.: Possible overlap of `copy_backward` ranges

In the original C++ interface, the destination iterator passed to `copy_backward` refers to one past the end of the destination range. This convention is motivated by the use of bidirectional iterators, which do not support random access. Since our C formulation operates directly on pointers, we instead let `b` denote the start of the destination range. This avoids the need for additional pointer arithmetic in both the specification and the implementation.

7.7.1. Formal specification of copy_backward

The formal specification of `copy_backward` is shown in the following listing. As in the case of `copy`, the source range `a[0..n-1]` must be valid for reading and the destination range `b[0..n-1]` must be valid for writing. The precondition `sep` captures the admissible overlap between the two ranges and reflects the reverse copying order discussed above.

```
/*@
  requires   valid:  \valid_read(a + (0..n-1));
  requires   valid:  \valid(b + (0..n-1));
  requires   sep:    \separated(a + (0..n-1), b + n);

  terminates         \true;
  exits              \false;
  assigns            b[0..n-1];

  ensures    equal:  Equal{Old,Here}(a, n, b);
*/
void copy_backward(const value_type* a, size_type n, value_type* b);
```

Listing 7.16: Formal specification of `copy_backward`

The postcondition states that, upon termination, the destination range contains an element-wise copy of the source range as it was in the pre-state. This property is concisely expressed using the predicate `Equal` [4.28]. As with `copy`, no guarantee is made that the source range `a[0..n-1]` remains unchanged, since parts of it may be overwritten in the presence of overlapping ranges.

7.7.2. Implementation of copy_backward

The following listing shows an implementation of the `copy_backward` function together with its ACSL annotations.

```
void copy_backward(const value_type* a, size_type n, value_type* b)
{
  /*@
    loop invariant bound:     0 <= i <= n;
    loop invariant equal:     Equal{Pre,Here}(a, i, n, b);
    loop invariant unchanged: Unchanged{Pre,Here}(a, i);
    loop assigns i, b[0..n-1];
    loop variant i;
   */
  for (size_type i = n; i > 0u; --i) {
    b[i - 1u] = a[i - 1u];
  }
}
```

Listing 7.17: Implementation of `copy_backward`

The loop invariants are closely related to those used for `copy`. They constrain the range of the loop variable using `bound` and express which part of the destination range has already been filled using the predicate `equal`. In addition, an invariant based on `Unchanged` ensures that, at each iteration, those elements of the source range that still need to be read have not yet been modified. This invariant is essential for establishing the postcondition in the presence of overlapping ranges and reflects the right-to-left traversal order of the algorithm.

7.8. The reverse_copy algorithm

The `reverse_copy` algorithm of the C++ Standard Library [18, §28.6.10] copies the elements of a source range into a destination range such that the order of elements is reversed. The source and destination ranges are required to be non-overlapping. As in the previous sections, we restrict the generic interface to contiguous ranges of elements of type `value_type`. The resulting signature reads:

```
void reverse_copy(const value_type* a, size_type n, value_type* b);
```

Figure 7.18 illustrates the effect of applying `reverse_copy` to an array $a[0 \,..\, n)$, producing a reversed copy in the array $b[0 \,..\, n)$.

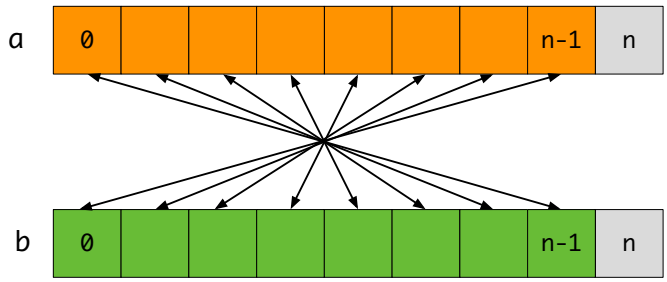


Figure 7.18.: Effects of `reverse_copy`

7.8.1. The predicate Reverse

In order to concisely formalize the reversal relation employed by the `reverse_copy` algorithm, we introduce in the following listing several overloaded versions of the predicate `Reverse` [7.20]. These variants provide convenient default parameters and allow reversal properties to be expressed succinctly in subsequent ACSL annotations.

The first predicate in this family constitutes the core definition of `Reverse`. It relates two array slices $a[m \,..\, n)$ and $b\big[q-1-(n-m) \,..\, q\big)$ and captures their element-wise reversal property—see Figure 7.19. In this formulation, the parameter q denotes the one-past-end index of the destination slice. Thus, both source and destination ranges are described in the same half-open form, which is convenient for reasoning in ACSL. The first index $q-1-(n-m)$ of the second slice is determined implicitly by the integer parameters m, n, and q.

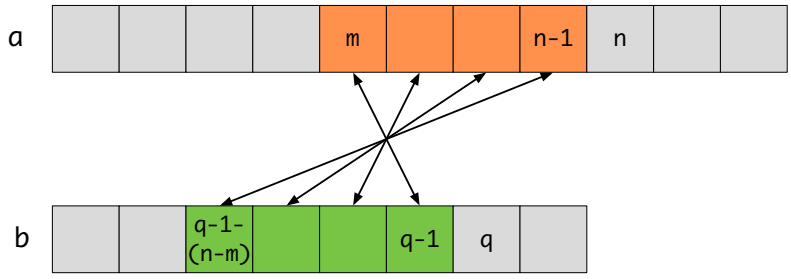


Figure 7.19.: Sketch of predicate `Reverse`

The subsequent overloaded versions of `Reverse` [7.20] are all defined in terms of this first, foundational predicate. They introduce no new semantics, but merely provide convenient default values for certain parameters.

```
/*@
  predicate Reverse{K,L}(value_type* a, integer m, integer n, value_type* b,
      integer q) =
    \forall integer i; m <= i < n  ==>  \at(a[i],K) == \at(b[q-1 - (i-m)],L);

  predicate Reverse{K,L}(value_type* a, integer m, integer n, value_type* b) =
    Reverse{K,L}(a, m, n, b, n);

  predicate Reverse{K,L}(value_type* a, integer n, value_type* b) =
    Reverse{K,L}(a, 0, n, b, n);

  predicate Reverse{K,L}(value_type* a, integer m, integer n, integer q) =
    Reverse{K,L}(a, m, n, a, q);

  predicate Reverse{K,L}(value_type* a, integer m, integer n) =
    Reverse{K,L}(a, m, n, n);

  predicate Reverse{K,L}(value_type* a, integer n) = Reverse{K,L}(a, 0, n);
*/
```

Listing 7.20: `Reverse` (ACSL)

The first three predicates cover the general case of two explicit array slices, possibly with different starting indices. The remaining three predicates specialize this relation to the frequent case in which a slice of a single array is reversed in place.

We also add `Reverse_Shift` [7.21] in the following listing. This lemma captures the semantic invariance of the `Reverse` [7.20] predicate under pointer shifts and allows us to transfer reversal properties between shifted and unshifted views of the same memory region.

```
/*@
  lemma Reverse_Shift{K,L}:
    \forall value_type* a, integer n, p;
    0 <= p <= n ==>
    (Reverse{K,L}(a + p, 0, n - p) ==> Reverse{K,L}(a, p, n));
*/
```

Listing 7.21: `ReverseLemmas` (ACSL)

7.8.2. Formal specification of `reverse_copy`

The formal specification of `reverse_copy` is given in the following listing. Its central postcondition `reverse` is expressed using a suitable variant of the predicate `Reverse` [7.20], which directly captures the intended element-wise reversal relation between the source and destination arrays.

The preconditions `valid` ensure that the source range is readable and that the destination range is writable for the specified number of elements. In addition, the precondition `sep` expresses that the two ranges do not overlap. This separation condition is essential, since `reverse_copy` performs the copy operation without any precautions against aliasing between source and destination.

```
/*@
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                b[0..(n-1)];

  ensures    reverse:    Reverse{Old,Here}(a, n, b);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
*/
void reverse_copy(const value_type* a, size_type n, value_type* b);
```

Listing 7.22: Formal specification of `reverse_copy`

Besides the reversal property, the specification includes a postcondition `unchanged` stating that the source range is not altered by the algorithm. Together with the `assigns` clause, this precisely characterizes the observable effects of `reverse_copy`: the destination range is written according to the reversal relation, while all other memory locations are preserved.

7.8.3. Implementation of `reverse_copy`

The implementation of `reverse_copy` [7.23] is shown in the following listing. The algorithm performs a single traversal of the *destination* range and, at each index `i`, assigns the value `a[n-1-i]` from the source range.

```
void reverse_copy(const value_type* a, size_type n, value_type* b)
{
  /*@
    loop invariant bound:   0 <= i <= n;
    loop invariant reverse: Reverse{Here,Pre}(b, 0, i, a, n);
    loop assigns i, b[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    b[i] = a[n - 1u - i];
  }
}
```

Listing 7.23: Implementation of `reverse_copy`

To establish the postcondition `reverse`, the loop maintains an invariant of the same name, which employs the first (core) version of `Reverse` [7.20]. At loop counter value `i`, this invariant states that the already written prefix $b\big[0 \,..\, i\big)$ equals the reverse of the corresponding suffix $a\big[(n-i) \,..\, n\big)$ of the source range. Upon loop termination, the invariant therefore yields the desired reversal property for the full ranges.

The remaining loop annotations are standard. They ensure that index bounds are respected, restrict the set of memory locations that may be modified, and guarantee termination of the loop.

7.9. The reverse algorithm

The `reverse` algorithm of the C++ Standard Library [18, § 28.6.10] reverses the order of elements *in place* within a given sequence. In contrast to `reverse_copy`, the reversal is performed directly on the input range and therefore does not require a separate destination range.

As before, we restrict the generic interface to a contiguous range of elements of type `value_type`. The resulting function signature is

```
void reverse(value_type* a, size_type n);
```

Figure 7.24 illustrates the effect of applying `reverse` *in place* to an array slice $a[0 \,..\, n)$.

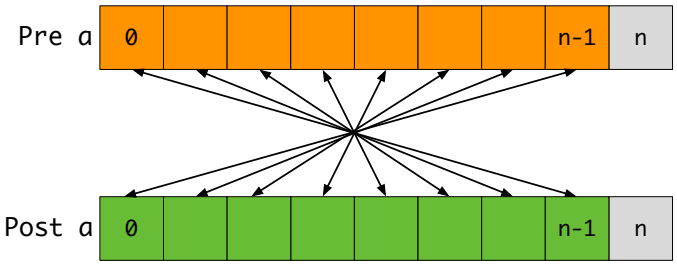


Figure 7.24.: Effect of `reverse`

7.9.1. Formal specification of reverse

The formal specification of `reverse` [7.25] is shown in the following listing.

```
/*@
  requires   valid:    \valid(a + (0..n-1));

  terminates           \true;
  exits                \false;
  assigns              a[0..n-1];

  ensures    reverse:  Reverse{Old,Here}(a, n);
*/
void reverse(value_type* a, size_type n);
```

Listing 7.25: Formal specification of `reverse`

Since `reverse` operates *in place*, its postcondition relates the final contents of the array to its own pre-state. This relation is expressed using an appropriate variant of the predicate `Reverse` [7.20], instantiated such that both ranges refer to the same array but are evaluated in different program states.

The precondition `valid` ensures that the array is valid for both reading and writing over the specified range. The `assigns` clause restricts the effects of the function to the elements of the array itself.

7.9.2. Implementation of reverse

The implementation of `reverse` [7.26] is shown in the following listing. Since the algorithm operates *in place*, it reverses the sequence by pairwise exchanging elements. To this end, the loop iterates over the first half of the array and invokes `swap` [7.7] to exchange $a[i]$ with its symmetric counterpart $a[n-1-i]$. Iterating beyond this point would merely undo exchanges that have already been performed.

```
void reverse(value_type* a, size_type n)
{
  const size_type half = n / 2u;

  //@ assert half:    half <= n - half;
  //@ assert half:  2*half <= n <= 2*half + 1;
  /*@
    loop invariant bound:    0 <= i <= half <= n-i;
    loop invariant left:     Reverse{Pre,Here}(a, 0, i, n);
    loop invariant middle:   Unchanged{Pre,Here}(a, i, n-i);
    loop invariant right:    Reverse{Pre,Here}(a, n-i, n, i);
    loop assigns i, a[0..n-1];
    loop variant half - i;
  */
  for (size_type i = 0u; i < half; ++i) {
    swap(&a[i], &a[n - 1u - i]);
  }
}
```

Listing 7.26: Implementation of `reverse`

The central loop invariants `left` and `right` employ an appropriate variant of the predicate `Reverse` [7.20] to relate the already processed prefix and suffix of the array to the pre-state. At iteration counter `i`, they express that the elements in $a\big[0 \,..\, i\big)$ and $a\big[(n-i) \,..\, n\big)$ appear in reversed order with respect to their initial values. In addition, the predicate `Unchanged` [7.1] is used to state that the remaining middle segment $a\big[i \,..\, (n-i)\big)$ has not been modified yet.

Taken together, these invariants capture the characteristic structure of the in-place reversal: the algorithm progresses symmetrically from both ends of the array, and termination occurs once the two halves meet.

7.10. The rotate_copy algorithm

The `rotate_copy` algorithm of the C++ Standard Library [18, §28.6.11] copies a sequence of length n into a separate sequence while performing a rotation by p positions. We adapt the generic iterator-based formulation to contiguous ranges of type `value_type`, yielding the signature:

```
void rotate_copy(const value_type* a, size_type p, size_type n, value_type* b);
```

The parameter p, with $0 \leqslant p \leqslant n$, denotes the rotation point. The source array $a[0 \,..\, n)$ is split into a prefix $a\big[0 \,..\, p\big)$ and a suffix $a\big[p \,..\, n\big)$, which are copied to $b[0 \,..\, n)$ in order: the suffix becomes the prefix of b, and the prefix becomes its suffix. Figure 7.27 illustrates this mapping.

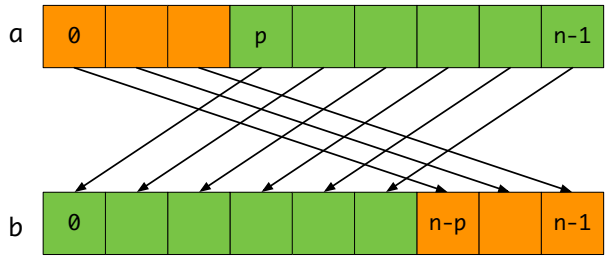


Figure 7.27.: Effects of `rotate_copy`

7.10.1. The predicate Rotate

The overloaded versions of the predicate `Rotate` in the following listing are defined in terms of appropriate overloaded versions of the predicate `Equal` [4.28]. They express that one array slice is obtained from another by a cyclic shift over a fixed index range.

```
/*@
  predicate Rotate{K,L}(value_type* a, integer p, integer n, value_type* b) =
     Equal{K,L}(a, 0, p, b, n-p) &&
     Equal{K,L}(a, p, n, b, 0);

  predicate Rotate{K,L}(value_type* a, integer m, integer p, integer n) =
    Equal{K,L}(a, m, p, m+n-p) &&
    Equal{K,L}(a, p, n, m);

  predicate Rotate{K,L}(value_type* a, integer p, integer n) =
    Rotate{K,L}(a, 0, p, n);
*/
```

Listing 7.28: `Rotate` (ACSL)

Using the slice notation introduced in (4.4), the predicate $\mathrm{Rotate}\{K, L\}(a, p, n, b)$ expresses that the array slice $a[0 \,..\, n)$ in state K is rotated by p positions to form the slice $b[0 \,..\, n)$ in state L. That is, the prefix of a of length p is mapped to the corresponding suffix of b, while the suffix of a starting at index p is mapped to the prefix of b. Formally, this predicate holds if and only if

$$\mathrm{Rotate}\{K, L\}(a, p, n, b) \quad \Longleftrightarrow \quad \begin{cases} a^K\big[0 \,..\, p\big) = b^L\big[(n-p) \,..\, n\big), \\ a^K\big[p \,..\, n\big) = b^L\big[0 \,..\, (n-p)\big). \end{cases} \tag{7.2}$$

The second version of Rotate will be explained in §7.11.

7.10.2. Formal specification of `rotate_copy`

The contract of `rotate_copy` is shown in the following listing. It formalizes the fact that the algorithm performs a pure copy operation with a well-defined reordering of elements.

```
/*@
  requires   bound:      0 <= p <= n;
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                b[0..(n-1)];

  ensures    rotate:     Rotate{Old,Here}(a, p, n, b);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
*/
void rotate_copy(const value_type* a, size_type p, size_type n, value_type* b);
```

Listing 7.29: Formal specification of `rotate_copy`

As for the other `_copy` algorithms discussed in this chapter, the preconditions require that the source and destination pointers are valid for ranges of length n and that the corresponding ranges do not overlap. The `assigns` clause restricts all modifications to the destination range $b[0 \,..\, n)$, thereby formalizing that the source array $a[0 \,..\, n)$ is accessed in a read-only manner.

The essential effect of the algorithm is expressed by the predicate `Rotate` [7.28]. This predicate provides a logical characterization of rotation from one array into another and captures the complete postcondition of `rotate_copy` at the level of ACSL. It states that the array `b` represents a rotation of the array `a` by p positions over a range of length n, independently of any particular implementation strategy.

By using `Rotate` [7.28], the contract directly reflects the abstract operation performed by `rotate_copy`. The element-wise correspondence between source and destination arrays is encapsulated in a single predicate, which keeps the specification concise and emphasizes the conceptual intent of the algorithm rather than its low-level realization.

7.10.3. Implementation of `rotate_copy`

The following listing shows a straightforward implementation of the `rotate_copy` function. The algorithm is realized by decomposing the rotation into two applications of `copy`, in accordance with the prefix–suffix decomposition introduced earlier.

First, the suffix $a\big[p \mathinner{..} n\big)$ is copied to the prefix $b\big[0 \mathinner{..} (n-p)\big)$ of the destination array. Second, the prefix $a\big[0 \mathinner{..} p\big)$ is copied to the remaining suffix $b\big[(n-p) \mathinner{..} n\big)$. Together, these two copy operations establish exactly the rotation of the two arrays.

```
void rotate_copy(const value_type* a, size_type p, size_type n, value_type* b)
{
  copy(a,  p, b + (n - p));
  copy(a + p, n - p, b);
}
```

Listing 7.30: Implementation of `rotate_copy`

The precondition that the source and destination ranges do not overlap ensures that both calls to `copy` satisfy their separation requirements. Moreover, since `copy` accesses its source range in a read-only manner and writes exclusively to its destination range, the implementation preserves the intended access discipline of `rotate_copy`.

From a verification perspective, the effect of the two calls to `copy` composes naturally into the predicate `Rotate` [7.28]. Rather than reasoning about individual subranges, the implementation can be shown to establish the abstract rotation property expressed by `Rotate` [7.28], which directly matches the postcondition of `rotate_copy`.

7.11. The rotate algorithm

The algorithm `rotate` is an *in-place* variant of `rotate_copy` [7.29]. As before, the elements of a sequence of length n are rotated by p positions, but in contrast to `rotate_copy`, no auxiliary destination range is used. Instead, the rearrangement is performed directly within the array $a[0 \,..\, n)$. As in the previous section, we adapt the generic iterator-based formulation from the C++ Standard Library [18, §28.6.11] to contiguous ranges of objects of type `value_type`. The resulting signature reads:

```
size_type rotate(value_type* a, size_type p, size_type n);
```

The parameter p again denotes the rotation point and satisfies $0 \leqslant p \leqslant n$. Conceptually, the array is decomposed into a prefix $a\big[0 \,..\, p\big)$ and a suffix $a\big[p \,..\, n\big)$, which exchange their relative positions. Unlike `rotate_copy`, however, this exchange must be realized without overwriting elements that are still needed. Figure 7.31 illustrates the effect of `rotate`. The figure closely resembles that of `rotate_copy` (see Figure 7.27), but now both the prefix and the suffix are rearranged *in place* within the same array.

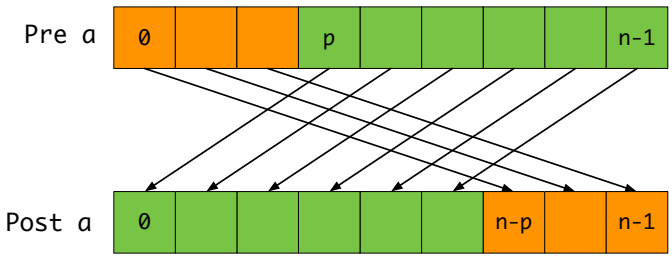


Figure 7.31.: Effects of `rotate`

7.11.1. Mathematical description of in-place rotation

In contrast to `rotate_copy`, where rotation is expressed between two distinct arrays, the specification of `rotate` relies on a second and a third version of the predicate `Rotate` [7.28], both of which refer to a single array across two program states.

1. Following the mathematical description (7.2), the second version of `Rotate` expresses a general in-place rotation. It states that the slice $a[m \,..\, n)$ in state K is rotated by $p - m$ positions to form the same slice $a[m \,..\, n)$ in state L. That is, the prefix $a\big[m \,..\, p\big)$ is mapped to the corresponding suffix of $a[m \,..\, n)$, while the suffix $a\big[p \,..\, n\big)$ is mapped to the corresponding prefix of $a[m \,..\, n)$.

   Formally, this predicate holds if and only if

$$\mathrm{Rotate}\{K, L\}(a, m, p, n) \quad \Longleftrightarrow \quad \begin{cases} a^K\big[m \,..\, p\big) = a^L\big[(m+n-p) \,..\, n\big), \\ a^K\big[p \,..\, n\big) = a^L\big[m \,..\, (m+n-p)\big). \end{cases} \tag{7.3}$$

   This general form will be used in the verification of `insertion_sort` [10.18].

2. The third version of the predicate `Rotate` [7.28] is the special case of the second version obtained by fixing $m = 0$. This version serves as the basis for the formal specification of `rotate`. By instantiating (7.3) with $m = 0$, we see that it expresses that the slice $a[0 \mathinner{..} n)$ in state K is rotated in place by p positions to form the same slice $a[0 \mathinner{..} n)$ in state L. Formally, this predicate holds if and only if

$$\mathrm{Rotate}\{K, L\}(a, p, n) \quad \iff \quad \begin{cases} a^K\big[0 \mathinner{..} p\big) = a^L\big[(n-p) \mathinner{..} n\big), \\ a^K\big[p \mathinner{..} n\big) = a^L\big[0 \mathinner{..} (n-p)\big). \end{cases} \tag{7.4}$$

7.11.2. Formal specification of `rotate`

The specification of `rotate` is shown in the following listing. It specifies an in-place rotation of the array $a[0 \mathinner{..} n)$ by p positions.

```
/*@
  requires   valid:   \valid(a + (0..n-1));
  requires   bound:   p <= n;

  terminates          \true;
  exits               \false;
  assigns             a[0..n-1];

  ensures    result:  \result == n-p;
  ensures    rotate:  Rotate{Old,Here}(a, p, n);
*/
size_type rotate(value_type* a, size_type p, size_type n);
```

Listing 7.32: Formal specification of `rotate`

The postconditions relate the final contents of `a` to its initial contents using the third version of the predicate `Rotate` [7.28], as characterized in Equation (7.4). They assert that the suffix $a\big[p \mathinner{..} n\big)$ in the initial state becomes the prefix of the array in the final state, while the initial prefix $a\big[0 \mathinner{..} p\big)$ becomes the final suffix.

As is typical for in-place algorithms, the preconditions require that the pointer `a` is valid for a range of length n and that the rotation point p satisfies $0 \leqslant p \leqslant n$. The `assigns` clause permits modification of the entire range $a[0 \mathinner{..} n)$, while excluding any writes outside this range. This reflects that all elements may be moved during the rotation, but that no auxiliary memory is used.

Finally, the return value of `rotate` is specified to be $n - p$. It denotes the index of the first element of the original suffix $a\big[p \mathinner{..} n\big)$ in the rotated array, in accordance with the standard interface.

7.11.3. Implementation of rotate

The following listing shows an implementation of the `rotate` function together with its ACSL annotations. Although there are several possible ways to implement `rotate`, we have chosen a well-established variant derived from an implementation of `std::rotate` for *bidirectional iterators* [18, §27.2.6]. This algorithm is concise and efficient, but its control structure is somewhat subtle and therefore benefits from a careful formal annotation.

```
size_type rotate(value_type* a, size_type p, size_type n)
{
  // if one subrange is empty, then nothings needs to be done
  if ((0u < p) && (p < n)) {
    reverse(a, p);
    reverse(a + p, n - p);
    //@ assert reverse:  Reverse{Pre,Here}(a, p, n);
    reverse(a, n);
  }

  return n - p;
}
```

Listing 7.33: Implementation of `rotate`

The implementation reduces the rotation to a sequence of calls to the algorithm `reverse` [7.25]. This classical decomposition avoids auxiliary storage and expresses the entire algorithm solely in terms of a single, already verified primitive.

The diagram in (7.5) illustrates how the implementation of `rotate` [7.33] operates on an array of seven elements, namely `a = {0, 1, 2, 3, 4, 5, 6}`. On the left-hand side, it shows the effect of calling `rotate(a, 3, 7)`. The right-hand side depicts the successive applications of `reverse` to the corresponding array slices. This example makes concrete how the implementation achieves the rotation by composing three reversals.

$$
\begin{array}{c}
\underline{0\ \ 1\ \ 2}\ \ \underline{3\ \ 4\ \ 5\ \ 6} \\
\texttt{rotate} \\
\underline{3\ \ 4\ \ 5\ \ 6}\ \ \underline{0\ \ 1\ \ 2}
\end{array}
\qquad
\begin{array}{c}
\underline{0\ \ 1\ \ 2}\ \ \underline{3\ \ 4\ \ 5\ \ 6} \\
\texttt{reverse} \\
\underline{2\ \ 1\ \ 0}\ \ \underline{3\ \ 4\ \ 5\ \ 6} \\
\texttt{reverse} \\
\underline{2\ \ 1\ \ 0\ \ 6\ \ 5\ \ 4\ \ 3} \\
\texttt{reverse} \\
\underline{3\ \ 4\ \ 5\ \ 6}\ \ \underline{0\ \ 1\ \ 2}
\end{array}
\tag{7.5}
$$

With this example in mind, we now analyze the implementation of `rotate` [7.33] in more detail. The following itemized discussion recalls the effect of each of the three calls to `reverse` [7.25] as specified by their contracts. Although each contract describes only a local transformation of the array, their combined effect is nontrivial. The automatic provers perform the necessary reasoning to combine these effects and establish that the overall behavior of the implementation satisfies the rotation property.

- First, the prefix $a\big[0 \,..\, p\big)$ is reversed. The contract of `reverse` [7.25] ensures that, after this call, the following assertion holds implicitly:

  ```
  //@ assert reverse: Reverse{Pre,Here}(a, 0, p);
  ```

  Although it is sometimes beneficial, for the sake of automated provers, to state such consequences explicitly, no additional assertion is required in this case.

- Next, the suffix $a\big[p \,..\, n\big)$ is reversed by calling `reverse` with a shifted address. As a consequence, the contract of `reverse` ensures only the following implicit assertion:

  ```
  //@ assert reverse: Reverse{Pre,Here}(a+p, 0, n-p);
  ```

  Using the lemma `Reverse_Shift` [7.21], this postcondition can be rewritten as the following assertion for the suffix $a\big[p \,..\, n\big)$. We make this fact explicit by enriching the implementation with:

  ```
  //@ assert reverse: Reverse{Pre,Here}(a, p, n);
  ```

- the complete range $a[0 \,..\, n)$ is reversed, thereby establishing the desired in-place rotation.

From a verification perspective, this implementation is particularly remarkable. Its correctness relies entirely on the contract of algorithm `reverse` [7.25]. The `assigns` clauses ensure that each call affects only the intended subrange, while the postconditions precisely characterize the local effect of each reversal. Although the combination of these effects is nontrivial, Frama-C/WP and the involved provers are able to perform the necessary reasoning automatically and establish that the overall behavior of the implementation satisfies the rotation property specified by `Rotate` [7.28], without requiring substantial auxiliary annotations.

7.12. The replace_copy algorithm

The replace_copy algorithm of the C++ Standard Library [18, §28.6.5] copies a sequence while substituting selected elements. Here, the iterator-based interface has been adapted to process value_type ranges. The resulting signature reads:

```
size_type replace_copy(const value_type* a, size_type n,
                       value_type* b, value_type v, value_type w);
```

The replace_copy algorithm copies the elements of the range a[0..n-1] to the range b[0..n-1], substituting every occurrence of v by w. The return value is the length of the output range. Since this length equals n, which is already passed as an argument, the return value does not convey additional information.

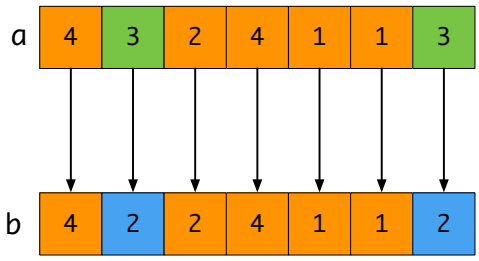


Figure 7.34.: Effects of replace_copy

Figure 7.34 illustrates the behavior of replace_copy by means of an example. Here, all occurrences of the value 3 in a[0..n-1] are replaced by the value 2 in b[0..n-1].

7.12.1. The predicate Replace

We begin by defining in the following listing the predicate Replace [7.35], which formalizes the intended element-wise relationship between an input array a[0..n-1] and an output array b[0..n-1].

```
/*@
  predicate Replace{K,L}(value_type* a, integer n,
                 value_type* b, value_type v, value_type w) =
    \forall integer i; 0 <= i < n  ==>
      \let ai = \at(a[i],K);
      \let bi = \at(b[i],L);
      (ai == v  ==>  bi == w) && (ai != v  ==>  bi == ai) ;

  predicate Replace{K,L}(value_type* a, integer n, value_type v, value_type w) =
    Replace{K,L}(a, n, a, v, w);
*/
```

Listing 7.35: Replace (ACSL)

The predicate expresses that each element of the output array equals either the corresponding input element or a designated replacement value, depending on whether the input element matches the value to be replaced. In order to keep the definition concise and readable, we introduce *local bindings* using the ACSL construct \let. In particular, the bindings \let ai = ... and \let bi = ... denote the elements a[i] and b[i] at the current index, as evaluated in the appropriate program state (see [9, §2.2]).

The listing also contains a second, overloaded variant of the predicate `Replace` [7.35]. This variant relates an array to itself across two program states and will be used in the specification of the corresponding in-place algorithm `replace` [7.38].

7.12.2. Formal specification of `replace_copy`

The formal specification of `replace_copy` [7.36] is shown in the following listing.

```
/*@
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                b[0..n-1];

  ensures    result:     \result == n;
  ensures    replace:    Replace{Old,Here}(a, n, b, v, w);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
*/
size_type replace_copy(const value_type* a, size_type n, value_type* b,
                       value_type v, value_type w);
```

Listing 7.36: Formal specification of `replace_copy`

The contract closely mirrors the informal description of the algorithm. The preconditions require that the input range `a[0..n-1]` is readable and that the output range `b[0..n-1]` is writable. In addition, the two ranges are required to be non-overlapping, as `replace_copy` performs a duplication rather than an in-place update. The `assigns` clause restricts side effects to the output range `b[0..n-1]`. The postcondition is expressed concisely using the predicate `Replace` [7.35].

7.12.3. Implementation of `replace_copy`

The implementation of `replace_copy` [7.37], together with its loop annotations, is shown in the following listing.

```
size_type replace_copy(const value_type* a, size_type n, value_type* b,
                       value_type v, value_type w)
{
  /*@
    loop invariant bounds:    0 <= i <= n;
    loop invariant replace:   Replace{Pre,Here}(a, i, b, v, w);
    loop assigns i, b[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    b[i] = (a[i] == v ? w : a[i]);
  }

  return n;
}
```

Listing 7.37: Implementation of `replace_copy`

The algorithm consists of a single loop that traverses the input range `a[0..n-1]` from left to right and constructs the output range `b[0..n-1]` element by element. At each iteration, the current input element is compared with the value `v`. If the comparison succeeds, the value `w` is written to the corresponding position in the output range; otherwise, the input element is copied unchanged.

The loop invariants mirror the contract of `replace_copy` [7.36]. The invariant `bounds` constrains the loop index to the valid range. The central invariant `replace` uses the predicate `Replace` [7.35] to express that the prefix of the output range that has already been produced satisfies the same element-wise replacement relation as required by the postcondition. The `loop assigns` clause records that all memory updates are confined to the output range. The loop variant `n-i` establishes termination by monotonically decreasing towards zero.

7.13. The `replace` algorithm

The `replace` algorithm of the C++ Standard Library [18, §28.6.5] performs an in-place substitution of selected values in a sequence. Here, the iterator-based interface of the standard algorithm is adapted to arrays of type `value_type`. The resulting signature reads:

```
void replace(value_type* a, size_type n, value_type v, value_type w);
```

The `replace` algorithm traverses the range `a[0..n-1]` and replaces every element that is equal to `v` by the value `w`. All other elements remain unchanged.

7.13.1. Formal specification of `replace`

The formal specification of `replace` [7.38] is shown in the following listing.

```
/*@
  requires   valid:    \valid(a + (0..n-1));

  terminates           \true;
  exits                \false;
  assigns              a[0..n-1];

  ensures    replace:  Replace{Old,Here}(a, n, v, w);
*/
void replace(value_type* a, size_type n, value_type v, value_type w);
```

Listing 7.38: Formal specification of `replace`

Since `replace` performs an in-place update, its contract differs from that of `replace_copy` in two important respects. First, the `assigns` clause restricts side effects to the range `a[0..n-1]`, reflecting that all modifications occur within the input array itself. Second, the postcondition relates the final state of the array to its initial state, rather than to a separate output array.

This relationship is expressed using the second, overloaded variant of the predicate `Replace` [7.35]. Unlike the version used for `replace_copy`, this variant relates a single array across two program states. It specifies that, for each index `i` in the range `0..n-1`, the value stored in `a[i]` after execution equals `w` if the corresponding value in the pre-state equals `v`, and otherwise remains unchanged. In this way, the predicate concisely captures the element-wise effect of the in-place replacement.

7.13.2. Implementation of `replace`

The implementation of `replace` [7.39], together with its loop annotations, is shown in the following listing.

```
void replace(value_type* a, size_type n, value_type v, value_type w)
{
  /*@
    loop invariant bounds:    0 <= i <= n;
    loop invariant replace:   Replace{Pre,Here}(a, i, v, w);
    loop invariant unchanged: Unchanged{Pre,Here}(a, i, n);
    loop assigns i, a[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    if (a[i] == v) {
      a[i] = w;
    }
  }
}
```

Listing 7.39: Implementation of `replace`

The algorithm traverses the array `a[0..n-1]` once from left to right and updates the elements in place. At each iteration, the current element `a[i]` is compared with the value `v`. If the comparison succeeds, the element is overwritten with the value `w`; otherwise, it is left unchanged.

The invariant `unchanged` expresses that, upon entry to iteration `i`, the suffix `a[i..n-1]` still coincides with its initial state. This invariant is characteristic for in-place algorithms that process an array sequentially and complements the use of the state-based predicate `Replace` [7.35] in the postcondition. The `loop assigns` clause makes explicit that all side effects are confined to the array `a` and the loop index.

7.14. The remove_copy algorithm (basic contract)

The `remove_copy` algorithm of the C++ Standard Library [18, §28.6.8] constructs a new sequence by copying all elements of an input range except those equal to a given value. In contrast to many of the algorithms discussed so far, the presentation of `remove_copy` will be distributed over several sections. In the present section we therefore restrict attention to the basic contract and to the externally observable behavior of the algorithm, postponing a more refined specification and the implementation details to later sections.

As before, the generic iterator-based formulation has been specialized to contiguous ranges of elements of type `value_type`. The resulting function signature reads as follows:

```
size_type
remove_copy(const value_type* a, size_type n, value_type* b, value_type v);
```

Intuitively, `remove_copy` scans the input range `a[0..n-1]`, selects those elements that are not equal to `v`, and copies them, in order, into the output range starting at `b`. The function returns the number of elements that have been copied.

The essential requirements and properties of `remove_copy` are summarized in Table 7.40. These requirements will later be reflected as precisely as currently possible in the formal specification.

Requirements	**Description**
Remove Copy Size	The output range must be large enough to hold all elements of the input range except those equal to `v`.
Remove Copy Separated	The input range and the output range do not overlap.
Remove Copy Elements	All elements of `a[0..n-1]` that are not equal to `v` are copied to the range `b[0.. \result-1]`.
Remove Copy Stability	The algorithm is stable, that is, the relative order of the copied elements in `b` is the same as in `a`.
Remove Copy Return	The return value is the length of the resulting output range.
Remove Copy Complexity	The algorithm performs exactly n comparisons.

Table 7.40.: Properties of `remove_copy`

Figure 7.41 illustrates the effect of `remove_copy` for a concrete input, showing how all elements different from 4 are copied from the input range to the output range while preserving their relative order.

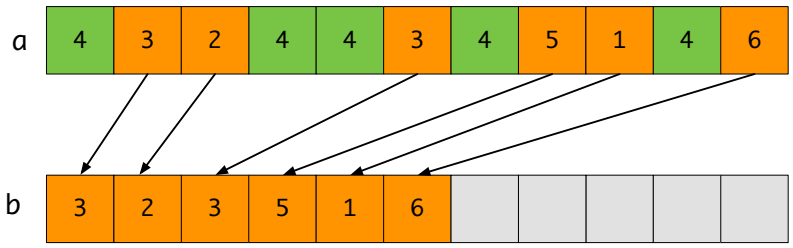


Figure 7.41.: Effects of `remove_copy`

7.14.1. Formal specification of remove_copy

The following listing presents a first attempt at a formal specification of `remove_copy`. At this stage, the contract captures only basic properties of the resulting output range and deliberately avoids committing to a more detailed structural relationship between input and output.

In the postcondition `discard` we use the predicate `NoneEqual` [4.4] to express that the value `v` does not occur in the output range `b[0..\result-1]`. This ensures that all copied elements differ from the value being removed, but it does not yet constrain which elements have actually been copied.

```
/*@
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                b[0..n-1];

  ensures    bound:      0 <= \result <= n;
  ensures    discard:    NoneEqual(b, \result, v);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
  ensures    unchanged:  Unchanged{Old,Here}(b, \result, n);
*/
size_type remove_copy(const value_type *a, size_type n, value_type *b,
                      value_type v);
```

Listing 7.42: Formal specification of `remove_copy`

One limitation of this specification is that the postcondition `bound` only provides a very coarse upper bound on the number of copied elements. While it guarantees that the result does not exceed the length of the input, it does not characterize the result size more precisely. This issue will be addressed in the refined contract of `remove_copy2` [7.46].

A more fundamental shortcoming, however, is that the present specification does not describe the relationship between the elements of the input range `a[0..n-1]` and those of the output range `b[0..\result-1]`. In particular, no statement is made about which input elements are copied or about the order in which they appear in the output. As a consequence, the stability of `remove_copy` is not yet expressible at this level. Both the precise correspondence between input and output elements and the formalization of stability will be tackled in the contract of `remove_copy3` [7.53].

7.14.2. Implementation of remove_copy

An implementation of `remove_copy` together with its ACSL annotations is shown in the following listing. The implementation follows the standard structure of a single linear scan over the input range `a[0..n-1]`. An index variable `i` traverses the input range, while a second index `k` denotes the length of the output prefix that has already been constructed in the range `b`. Whenever the current input element `a[i]` differs from the value `v`, it is copied to position `b[k]`, and the index `k` is incremented. Elements equal to `v` are simply skipped, leaving the output range unchanged.

```
size_type remove_copy(const value_type *a, size_type n, value_type *b,
                      value_type v)
{
  size_type k = 0u;

  /*@
    loop invariant bound:     0 <= k <= i <= n;
    loop invariant discard:   NoneEqual(b, k, v);
    loop invariant unchanged: Unchanged{Pre,Here}(b, k, n);
    loop assigns   k, i, b[0..n-1];
    loop variant   n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    if (a[i] != v) {
      b[k++] = a[i];
    }
  }

  return k;
}
```

Listing 7.43: Implementation of `remove_copy`

The loop invariants reflect this separation of concerns. The invariant `bound` relates the output index `k` to the progress of the input index `i` and ensures that the output range is never written out of bounds. In addition, we must explicitly state that none of the elements written so far equals the value being removed.

This property is captured by the additional loop invariant `discard`, which asserts that the value `v` does not occur in the partially constructed output range `b[0..k-1]` at any loop iteration. Without this invariant, the verifier would have no means of propagating the postcondition that excludes `v` from the output range.

At this level of specification, the loop invariants intentionally do not express any correspondence between elements of the input range and those of the output range beyond the exclusion of `v`. In particular, neither the origin nor the relative order of the copied elements is specified. This mirrors the limitations already observed in the contract and prepares the ground for later refinements, where stability and a precise input–output relationship will be formalized.

7.15. The remove_copy2 algorithm (number of copied elements)

In this section we refine the contract of `remove_copy` [7.42] by specifying the number of elements copied to the output range. Unlike the basic contract of Section 7.14, the present specification characterizes the return value precisely.

By definition, the number of copied elements equals the number of elements in the input range `a[0..n-1]` that are different from the value `v`. This quantity can be expressed conveniently by means of a logic function that counts the elements of a range not equal to `v`. One straightforward possibility is to express this number using the already available logic function `Count` [4.44]:

```
logic integer
CountNotEqual(value_type* a, integer n, value_type v) =
  n - Count(a, n, v);
```

While such a definition is sufficient from a mathematical point of view, we introduce a dedicated logic function `CountNotEqual` [7.44] together with basic properties and supporting lemmas. This allows contracts and invariants to be stated directly in terms of the quantity of interest, rather than via derived expressions. For this reason, `CountNotEqual` is introduced as an independent logic function. The relationship between `CountNotEqual` and the previously introduced logic function `Count` [4.44] is captured explicitly by the lemma `CountNotEqual_Count`, shown in Listing 7.45.

```
/*@
  logic integer
  CountNotEqual(value_type* a, integer m, integer n, value_type v) =
    n <= m ? 0 : CountNotEqual(a, m, n-1, v) + (a[n-1] == v ? 0 : 1);

  logic integer
  CountNotEqual(value_type* a, integer n, value_type v) =
    CountNotEqual(a, 0, n, v);

  lemma CountNotEqual_Empty:
    \forall value_type *a, v, integer m, n;
      n <= m  ==>  CountNotEqual(a, m, n, v) == 0;

  lemma CountNotEqual_Hit:
    \forall value_type *a, v, integer m, n;
      m <= n     ==>
      a[n] != v  ==>
      CountNotEqual(a, m, n+1, v) == CountNotEqual(a, m, n, v) + 1;

  lemma CountNotEqual_Miss:
    \forall value_type *a, v, integer m, n;
      m <= n     ==>
      a[n] == v  ==>
      CountNotEqual(a, m, n+1, v) == CountNotEqual(a, m, n, v);

  lemma CountNotEqual_Lower:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  0 <= CountNotEqual(a, m, n, v);

  lemma CountNotEqual_Upper:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  CountNotEqual(a, m, n, v) <= n-m;
```

Listing 7.44: The logic function `CountNotEqual` (1)

```
  lemma CountNotEqual_WeaklyIncreasing:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  CountNotEqual(a, m, n, v) <= CountNotEqual(a, m, n+1, v);

  lemma CountNotEqual_Increasing:
    \forall value_type *a, v, integer k, m, n;
      m <= k <= n  ==>  CountNotEqual(a, m, k, v) <= CountNotEqual(a, m, n, v
          );

  lemma CountNotEqual_Unchanged{K,L}:
    \forall value_type *a, v, integer m, n;
      Unchanged{K,L}(a, m, n)  ==>
      CountNotEqual{K}(a, m, n, v) == CountNotEqual{L}(a, m, n, v);

  lemma CountNotEqual_Count:
    \forall value_type *a, v, integer m, n;
      m <= n  ==>  CountNotEqual(a, m, n, v) == n - m - Count(a, m, n, v);

  lemma CountNotEqual_Union:
    \forall value_type *a, v, integer k, m, n;
      0 <= k <= m <= n  ==>
      CountNotEqual(a, k, n, v) ==
      CountNotEqual(a, k, m, v) + CountNotEqual(a, m, n, v);
*/
```

Listing 7.45: The logic function `CountNotEqual` (2)

7.15.1. Formal specification of `remove_copy2`

The formal specification of `remove_copy2` extends the basic contract of `remove_copy` [7.42] by precisely characterizing the number of elements copied to the output range.

```
/*@
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                b[0..n-1];

  ensures    size:       \result == CountNotEqual(a, n, v);
  ensures    bound:      0 <= \result <= n;
  ensures    discard:    NoneEqual(b, \result, v);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
  ensures    unchanged:  Unchanged{Old,Here}(b, \result, n);
*/
size_type remove_copy2(const value_type* a, size_type n, value_type* b,
                       value_type v);
```

Listing 7.46: Formal specification of `remove_copy2`

To this end, we introduce an additional postcondition `size`, which states that the return value of `remove_copy2` equals the number of elements in the input range `a[0..n-1]` that are different from `v`. This property is expressed using the logic function `CountNotEqual` [7.44].

7.15.2. Implementation of remove_copy2

The following listing shows the implementation of the extended algorithm remove_copy2 together with its ACSL annotations. Compared to the basic implementation of remove_copy, the algorithmic structure remains unchanged. The only conceptual extension concerns the precise tracking of the number of elements copied to the output range. This is reflected by the additional loop invariant size, which states that the current value of the output index equals the number of elements in the already processed prefix of the input range that are different from v. The invariant directly mirrors the postcondition of remove_copy2 [7.46] and makes the result size available throughout the loop.

```
size_type remove_copy2(const value_type* a, size_type n, value_type* b,
                       value_type v)
{
  size_type k = 0u;

  /*@
    loop invariant size:      k == CountNotEqual(a, i, v);
    loop invariant bound:     0 <= k <= i <= n;
    loop invariant discard:   NoneEqual(b, k, v);
    loop invariant unchanged: Unchanged{Pre,Here}(b, k, n);
    loop assigns   k, i, b[0..n-1];
    loop variant   n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    if (a[i] != v) {
      b[k++] = a[i];
      //@ assert unchanged: Unchanged{LoopCurrent,Here}(a, n);
      //@ assert size:      k == CountNotEqual(a, 0, i+1, v);
    }
  }

  return k;
}
```

Listing 7.47: Implementation of remove_copy2

To allow the verifier to re-establish this invariant after each iteration, the implementation is augmented with the assertions size and unchanged. These assertions make explicit the effect of each loop body execution on the output index and on the portions of the arrays that are not modified.

Despite this refinement, the implementation still does not specify how individual elements of the input range relate to those of the output range. In particular, no correspondence between input and output positions is yet expressed. This remaining gap, and with it the formalization of stability, will be addressed in the contract of remove_copy3 [7.53].

7.16. The `remove_copy3` algorithm (final contract)

In this section we complete the refinement of the contracts introduced in `remove_copy` [7.42] and `remove_copy2` [7.46]. The final step consists in introducing a logic function that formally describes the relationship between the elements of the input range `a[0..n-1]` and those of the output range `b[0..\result-1]`. Recall from the previous section that the return value `\result` equals `CountNotEqual (a, n, v)`.

7.16.1. A closer look at the properties of `remove_copy`

Figure 7.48 shows a refined version of Figure 7.41. For clarity, indices of elements that are not copied into the target array have been omitted. In addition, a dashed arrow has been added to indicate the *one-past-the-end* position of both the input and the output range.

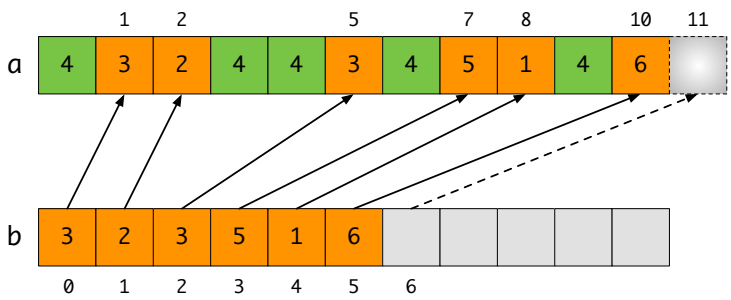


Figure 7.48.: Partitioning the input of `remove_copy`

The arrows connecting indices of the array `b` to indices of the array `a` induce a sequence p of indices. In the example shown, this sequence consists of seven indices, where the *one-past-the-end* index is underlined:

$$p = (1, 2, 5, 7, 8, 10, \underline{11}).$$

More generally, we refer to p as the *partitioning sequence* associated with an application of `remove_copy` to the array `a[0..n-1]`. Let m denote the length of this sequence. Then p is a **strictly increasing** sequence satisfying

$$0 \leqslant p_0 < \cdots < p_m = n. \tag{7.6}$$

The open index intervals (p_i, p_{i+1}) marks **consecutive ranges** of the value `v` in the source array:

$$a[k] = v \qquad \forall k : p_i < k < p_{i+1}. \tag{7.7}$$

In addition, the half-open interval $[0, p_0)$ also forms a **consecutive range** of the value `v`:

$$a[k] = v \qquad \forall k : 0 \leqslant k < p_0. \tag{7.8}$$

The elements of the input array selected by the partitioning sequence are all different from `v`:

$$a[p_i] \neq v \qquad \forall i : 0 \leqslant i < m. \tag{7.9}$$

Finally, the defining correspondence between input and output elements is given by

$$a[p_i] = b[i] \qquad \forall i : 0 \leqslant i < m. \tag{7.10}$$

Together with (7.9), this immediately implies that the output array does not contain the value v:

$$b[i] \neq v \qquad \forall i : 0 \leqslant i < m. \tag{7.11}$$

7.16.2. Additional lemmas for CountNotEqual

The formalization of the properties in Section 7.16.1 relies on the logic function CountNotEqual [7.44]. In order to reason about the correspondence between input and output indices, we additionally make use of the logic function FindNotEqual [4.16].

The accompanying lemmas collected under [7.49] establish further relationships between CountNotEqual and FindNotEqual. These results are of a technical nature: they do not introduce new conceptual ideas, but provide the auxiliary facts required to carry out the verification of the final contract.

```
/*@
  lemma CountNotEqual_AllEqual:
    \forall value_type *a, v, integer m, n;
      0 <= m <= n            ==>
      AllEqual(a, m, n, v)   ==>  CountNotEqual(a, m, n, v) == 0;

  lemma CountNotEqual_SomeNotEqual:
    \forall value_type *a, v, integer m, n;
      0 <= m < n                     ==>
      0 < CountNotEqual(a, m, n, v)  ==>  SomeNotEqual(a, m, n, v);

  lemma CountNotEqual_FindNotEqual:
    \forall value_type *a, v, integer m, n;
      0 <= m < n                     ==>
      0 < CountNotEqual(a, m, n, v)  ==>  FindNotEqual(a, m, n, v) < n-m;

  lemma CountNotEqual_Zero:
    \forall value_type *a, v, integer m, n;
      0 <= m < n   ==>
      CountNotEqual(a, m, m + FindNotEqual(a, m, n, v), v) == 0;

  lemma CountNotEqual_Decrement:
    \forall value_type *a, v, integer m, n;
      0 <= m < n  ==>
      CountNotEqual(a, m + FindNotEqual(a, m, n, v), n, v) ==
      CountNotEqual(a, 0, n, v) - CountNotEqual(a, 0, m, v);
*/
```

Listing 7.49: CountFindNotEqual (ACSL)

7.16.3. Formalizing the properties of the partitions

The logic function `IndexOfNotEqual`, whose definition is given in Listings 7.50 and 7.51, formalizes the partitioning sequence p introduced in Section 7.16.1. Intuitively, `IndexOfNotEqual (a, n, v, k)` returns the array index corresponding to the k-th element of the range `a[0..n-1]` that is not equal to `v`.

```
/*@
  logic integer
  IndexOfNotEqual(value_type* a, integer n, value_type v, integer k) =
    \let count = CountNotEqual(a, n, v);
    \let start = IndexOfNotEqual(a, n, v, k-1) + 1;
    // Negative rank: invalid request
    k < 0 ? -1 :
      // Now we know 0 <= k
      (n <= 0 ? 0 :
        // Now we know 0 < n
        k < count ? start + FindNotEqual(a, start, n, v) : n
      );

  lemma IndexOfNotEqual_Left:
    \forall value_type *a, v, integer n, k;
      k < 0  ==>  IndexOfNotEqual(a, n, v, k) == -1;

  lemma IndexOfNotEqual_Empty:
    \forall value_type *a, v, integer n, k;
      n <= 0 <= k  ==>
      IndexOfNotEqual(a, n, v, k) == 0;

  lemma IndexOfNotEqual_Right:
    \forall value_type *a, v, integer n, k;
      0 <= n                        ==>
      CountNotEqual(a, n, v) <= k  ==>  IndexOfNotEqual(a, n, v, k) == n;

  lemma IndexOfNotEqual_Next:
    \forall value_type *a, v, integer n, k;
      \let x = IndexOfNotEqual(a, n, v, k-1) + 1;
      0 <= n                            ==>
      0 <= k < CountNotEqual(a, n, v)  ==>
      IndexOfNotEqual(a, n, v, k) == x + FindNotEqual(a, x, n, v);

  lemma IndexOfNotEqual_Lower:
    \forall value_type *a, v, integer n, k;
      0 < n                             ==>
      0 <= k < CountNotEqual(a, n, v)  ==>
      0 <= IndexOfNotEqual(a, n, v, k);
```

Listing 7.50: The logic function `IndexOfNotEqual` (1)

The behavior of `IndexOfNotEqual` can be summarized as follows.

- Negative values of k correspond to invalid requests and yield -1.
- Out-of-range requests, that is, values of k greater than or equal to the number of elements of `a[0..n-1]` that are different from `v`, yield n, the first index past the end of the array.
- For intermediate values, the function is defined inductively: the index for rank k is obtained from the index for rank $k-1$ by advancing to the next array position whose value differs from `v`.

We recall that logic functions (and predicates) in ACSL must be total, that is, they must be defined for all possible argument values. The explicit treatment of invalid and out-of-range arguments in the definition of `IndexOfNotEqual` serves precisely this purpose.

```
  lemma IndexOfNotEqual_Core:
    \forall value_type *a, v, integer n, k;
      \let R = IndexOfNotEqual(a, n, v, k);
      0 < n                              ==>
      0 <= k < CountNotEqual(a, n, v)  ==>
      (R < n      &&
       a[R] != v  &&
       CountNotEqual(a, R, n, v) == CountNotEqual(a, 0, n, v) - k);

  lemma IndexOfNotEqual_Upper:
    \forall value_type *a, v, integer n, k;
      0 < n                              ==>
      0 <= k < CountNotEqual(a, n, v)  ==>
      IndexOfNotEqual(a, n, v, k) < n;

  lemma IndexOfNotEqual_NotEqual:
    \forall value_type *a, v, integer n, k;
      0 < n  ==>
      0 <= k < CountNotEqual(a, n, v)  ==>
      a[IndexOfNotEqual(a, n, v, k)] != v;

  lemma IndexOfNotEqual_Count:
    \forall value_type *a, v, integer n, k;
      0 < n                              ==>
      0 <= k < CountNotEqual(a, n, v)  ==>
      CountNotEqual(a, IndexOfNotEqual(a, n, v, k), n, v) ==
      CountNotEqual(a, 0, n, v) - k;

  lemma IndexOfNotEqual_StrictlyWeakIncreasing:
    \forall value_type *a, v, integer n, k;
      0 < n                             ==>
      0 < k < CountNotEqual(a, n, v)  ==>
      IndexOfNotEqual(a, n, v, k-1) < IndexOfNotEqual(a, n, v, k);

  lemma IndexOfNotEqual_Segment:
    \forall value_type *a, v, integer n, k;
      0 < n                              ==>
      0 <= k                             ==>
      k + 1 < CountNotEqual(a, n, v)   ==>
        AllEqual(a, IndexOfNotEqual(a, n, v, k) + 1,
                    IndexOfNotEqual(a, n, v, k+1), v);

  lemma IndexOfNotEqual_Unchanged{K,L}:
    \forall value_type *a, v, integer n, k;
      Unchanged{K,L}(a, n)  ==>
      IndexOfNotEqual{K}(a, n, v, k) == IndexOfNotEqual{L}(a, n, v, k);
*/
```

Listing 7.51: The logic function `IndexOfNotEqual` (2)

The lemmas associated with `IndexOfNotEqual` correspond directly to the properties identified in Section 7.16.1.

- Property (7.6) is captured by the lemmas `IndexOfNotEqual_Empty`, `IndexOfNotEqual_Left`, `IndexOfNotEqual_`, and `IndexOfNotEqual_StrictlyWeakIncreasing`, which together establish boundary behavior and monotonicity of the partitioning sequence.

- Properties (7.7) and (7.8) are formalized by the lemma `IndexOfNotEqual_Segment`, which characterizes the segments of consecutive occurrences of the value `v` in the input array.
- Property (7.9) is expressed by the lemma `IndexOfNotEqual_NotEqual`.
- Property (7.10) is formulated using the predicate `Remove` [7.52], which relates the partitioning sequence to the construction of the target array.

We single out the lemma `IndexOfNotEqual_Core`, which subsumes the statements of the following lemmas:

- `IndexOfNotEqual_Upper`,
- `IndexOfNotEqual_NotEqual`, and
- `IndexOfNotEqual_Count`.

Although these lemmas do not introduce additional information, they are retained because each corresponds directly to an individual property discussed in Section 7.16.1 and therefore provides a convenient and focused reference point.

The presence of the combined lemma `IndexOfNotEqual_Core` may nevertheless raise the question why such a formulation is needed at all. The reason is methodological. During verification, the individual properties turned out to be so tightly interdependent that they could not be established in isolation. Only their joint formulation admitted a successful proof, and `IndexOfNotEqual_Core` constitutes the formal embodiment of this observation.

7.16.4. The predicate Remove

The predicate `Remove` [7.52] is introduced to improve the readability and modularity of the specification of `remove_copy3` [7.53]. Rather than restating the correspondence between input and output elements directly in the contract, this relationship is encapsulated in a dedicated predicate.

```
/*@
  predicate Remove{K,L}(value_type* a, integer n, integer i, value_type* b,
      value_type v) =
    \forall integer k; 0 <= k < CountNotEqual{K}(a, i, v)  ==>
      \let j = IndexOfNotEqual{K}(a, n, v, k);
      \at(b[k],L) == \at(a[j],K);

  predicate Remove{K,L}(value_type* a, integer n, value_type* b, value_type v) =
    Remove{K,L}(a, n, n, b, v);

  predicate Remove{K,L}(value_type* a, integer n, integer i, value_type v) =
    \forall integer k; 0 <= k < CountNotEqual{K}(a, i, v)  ==>
      \let j = IndexOfNotEqual{K}(a, n, v, k);
      \at(a[k],L) == \at(a[j],K);

  predicate Remove{K,L}(value_type* a, integer n, value_type v) =
    Remove{K,L}(a, n, n, v);
*/
```

Listing 7.52: `Remove` (ACSL)

Conceptually, `Remove` formalizes Property (7.10) from Section 7.16.1, namely the fact that the i-th element of the output range originates from the input element at index p_i of the partitioning sequence. By isolating this relationship in a separate predicate, the specification becomes both shorter and easier to maintain.

As in earlier chapters, the predicate `Remove` is provided in multiple, overloaded variants. In addition to the version used here for `remove_copy3`, an alternative formulation is included that will be employed in the specification of the *in-place* variant `remove` [7.56].

7.16.5. Formal specification of `remove_copy3`

The following listing presents the formal specification of `remove_copy3` [7.53], which completes the refinement of the contracts introduced in `remove_copy` [7.42] and `remove_copy2` [7.46].

```
/*@
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                b[0..n-1];

  ensures    size:       \result == CountNotEqual{Old}(a, n, v);
  ensures    bound:      0 <= \result <= n;
  ensures    remove:     Remove{Old,Here}(a, n, b, v);
  ensures    discard:    NoneEqual(b, \result, v);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
  ensures    unchanged:  Unchanged{Old,Here}(b, \result, n);
*/
size_type remove_copy3(const value_type* a, size_type n, value_type* b,
                       value_type v);
```

Listing 7.53: Formal specification of `remove_copy3`

Compared to the specification of `remove_copy2` [7.46], the essential extension is the addition of the postcondition `remove`. This postcondition makes use of the predicate `Remove` to formalize the precise correspondence between elements of the input range `a[0..n-1]` and those of the output range `b[0..\result-1]`. In particular, it captures both the selection of elements that differ from `v` and the preservation of their relative order. All other postconditions are inherited unchanged. In particular, the postcondition `unchanged` again states that the source array `a[0..n-1]` is not modified by the execution of the algorithm.

7.16.6. Implementation of `remove_copy3`

We now highlight the aspects of the implementation of `remove_copy3` [7.54] that differ from the earlier versions of the algorithm. The algorithmic structure itself is unchanged; the refinements concern exclusively the loop annotations required to establish the final contract.

The central addition is the loop invariant `mapping`. This invariant relates the progress of the input index `i` to the current output index `k` by stating that `i` never exceeds `IndexOfNotEqual(a, n, v, k)`. In effect, `mapping` expresses that the algorithm has not yet skipped past the input position corresponding to the k-th element copied to the output range.

To enable the prover to re-establish this invariant after each loop iteration, the implementation includes an explicit assertion `mapping`. As in earlier implementations, such assertions serve as intermediate proof obligations and guide the automated reasoning.

```
size_type remove_copy3(const value_type* a, size_type n, value_type* b,
                       value_type v)
{
  size_type k = 0u;

  /*@
    loop invariant size:       k == CountNotEqual{Pre}(a,i,v);
    loop invariant bound:      0 <= k <= i <= n;
    loop invariant remove:     Remove{Pre,Here}(a, n, i, b, v);
    loop invariant discard:    NoneEqual(b, k, v);
    loop invariant interval:   IndexOfNotEqual{Pre}(a, n, v, k-1) <= i;
    loop invariant interval:   i <= IndexOfNotEqual{Pre}(a, n, v, k);
    loop invariant unchanged:  Unchanged{Pre,Here}(a, n);
    loop invariant unchanged:  Unchanged{Pre,Here}(b, k, n);
    loop assigns   k, i, b[0..n-1];
    loop variant   n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    if (a[i] != v) {
      b[k++] = a[i];
      //@ assert size:      k == CountNotEqual{Pre}(a, i+1, v);
      //@ assert update:    b[k-1] == At{Pre}(a, i);
      //@ assert interval:  i == IndexOfNotEqual{Pre}(a, n, v, k-1);
      //@ assert remove:    Remove{Pre,Here}(a, n, i, b, v);
      //@ assert remove:    Remove{Pre,Here}(a, n, i+1, b, v);
      //@ assert unchanged: Unchanged{Pre,Here}(a, n);
      //@ assert unchanged: Unchanged{Pre,Here}(b, k, n);
    }
    else {
      //@ assert unchanged: Unchanged{Pre,Here}(a, n);
    }

    //@ assert remove:    Remove{Pre,Here}(a, n, i, b, v);
    //@ assert unchanged: Unchanged{Pre,Here}(a, n);
  }

  return k;
}
```

Listing 7.54: Implementation of `remove_copy3`

In addition, the logic function `At` [7.3] is used in the annotations to refer to values of array elements at specific program points. This allows the correspondence between input and output elements, as expressed by the predicate `Remove` [7.52], to be stated concisely.

Finally, the implementation contains an explicit **`else`** branch whose body is empty apart from the assertion `unchanged`. Although this branch has no operational effect, its presence significantly reduces verification times by making the control-flow structure explicit and thereby simplifying the generation of proof obligations.

7.17. The remove algorithm

In addition to remove_copy, the C++ Standard Library provides the algorithm remove [18, p. 28.6.8], which performs the same logical operation as an *in-place* transformation. Rather than copying the selected elements to a separate output range, remove compacts the input range itself by overwriting those elements that are equal to the given value. Accordingly, the signature of remove is similar to that of remove_copy, but does not require a separate output array:

```
size_type remove(value_type* a, size_type n, value_type v);
```

The algorithm returns the length of the resulting prefix of the array that contains all elements different from v. Elements beyond this prefix are left in an unspecified state.

Figure 7.55 illustrates the effect of remove for a concrete example, showing how all occurrences of the value 4 are removed from the range by shifting the remaining elements towards the beginning of the array.

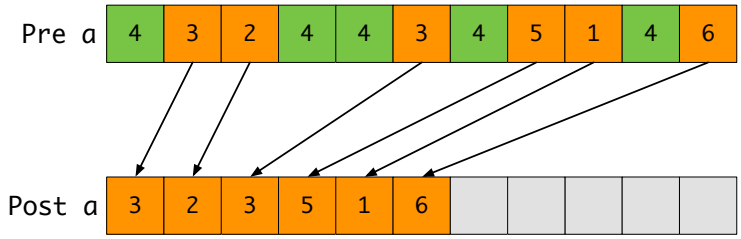


Figure 7.55.: Effects of remove

7.17.1. Formal specification of remove

The specification of remove [7.56] is closely related to that of remove_copy3 [7.53]. Both algorithms select exactly those elements of the input range that are different from v, preserve their relative order, and return the number of selected elements.

```
/*@
  requires   valid:      \valid(a + (0..n-1));

  terminates             \true;
  exits                  \false;
  assigns                a[0..n-1];

  ensures    size:       \result == CountNotEqual{Old}(a, n, v);
  ensures    bound:      0 <= \result <= n;
  ensures    remove:     Remove{Old,Here}(a, n, v);
  ensures    discard:    NoneEqual(a, \result, v);
  ensures    unchanged:  Unchanged{Old,Here}(a, \result, n);
*/
size_type remove(value_type* a, size_type n, value_type v);
```

Listing 7.56: Formal specification of remove

As in the specification of remove_copy3 [7.53], the predicate Remove [7.52] is used to formalize the correspondence between the elements of the original array and those of the resulting range. In the present case, this correspondence relates the initial contents of the array a to the resulting prefix a[0..\result -1]. The essential difference lies in the treatment of the output. While remove_copy3 writes the selected

elements to a separate output range, `remove` performs the transformation in place by compacting the array `a` itself.

7.17.2. Implementation of `remove`

The following listing shows the implementation of `remove` [7.57] together with its ACSL annotations. At the algorithmic level, the implementation closely follows that of `remove_copy3` [7.54]. The main difference is that the output range coincides with the input range, so that selected elements are written back into the array `a` itself.

```
size_type remove(value_type* a, size_type n, value_type v)
{
  size_type k = 0u;

  /*@
    loop invariant size:       k == CountNotEqual{Pre}(a,i,v);
    loop invariant bound:      0 <= k <= i <= n;
    loop invariant remove:     Remove{Pre,Here}(a, n, i, v);
    loop invariant discard:    NoneEqual(a, k, v);
    loop invariant interval:   IndexOfNotEqual{Pre}(a, n, v, k-1) <= i;
    loop invariant interval:   i <= IndexOfNotEqual{Pre}(a, n, v, k);
    loop invariant unchanged:  Unchanged{Pre,Here}(a, k, n);
    loop invariant unchanged:  a[k] == At{Pre}(a, k);
    loop assigns   k, i, a[0..n-1];
    loop variant   n-i;
  */
  for (size_type i = 0u; i < n; ++i ) {
    if (a[i] != v) {
      a[k++] = a[i];
      //@ assert size:      k == CountNotEqual{Pre}(a, 0, i+1, v);
      //@ assert update:    a[k-1] == At{Pre}(a, i);
      //@ assert interval:  i == IndexOfNotEqual{Pre}(a, n, v, k-1);
      //@ assert remove:    Remove{Pre,Here}(a, n, i,   v);
      //@ assert remove:    Remove{Pre,Here}(a, n, i+1, v);
    }
  }

  return k;
}
```

Listing 7.57: Implementation of `remove`

Accordingly, most loop invariants are reused with only minor adaptations. In particular, the invariants governing the result size, the mapping between input and output positions, and the exclusion of the value `v` from the resulting prefix remain unchanged in spirit.

Since the array is modified in place, the annotations must additionally refer to values of array elements in earlier program states. For this purpose, the logic function `At` [7.3] is used in the loop invariant `unchanged` and in the assertion `update`. As discussed earlier, this is a matter of convenience rather than necessity, and the same properties could equally be expressed using the built-in function `\at`.

7.18. The shuffle algorithm

The shuffle algorithm in the C++ Standard Library [18, §28.6.13] rearranges the elements of a given range in a randomized manner. For our purposes, the generic implementation has been adapted to operate on ranges of type value_type. The resulting signature reads:

```
void shuffle(value_type* a, size_type n, unsigned short* rand);
```

The argument rand represents the state of a simple pseudo-random number generator that is advanced during the execution of shuffle and controls the sequence of element exchanges performed by the algorithm.

Figure 7.58 illustrates a representative execution of shuffle. In the example shown, the values 1, 2, 3, and 4 occur twice, once, once, and three times, respectively, both before and after the call. This shows that shuffle preserves the multiset of elements while changing their relative order.

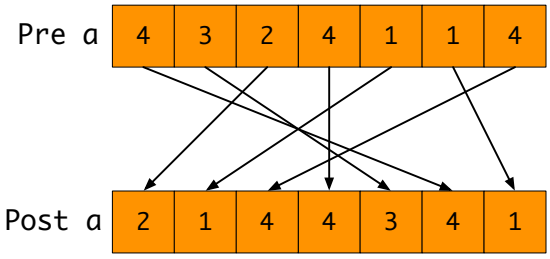


Figure 7.58.: Effects of shuffle

In the following sections, we will concentrate on formally capturing this rearrangement property. Rather than characterizing the result in terms of explicit permutations, we describe the effect of shuffle by relating the contents of the input range to those of the output range in a value-oriented manner.

7.18.1. The predicate MultisetReorder

The shuffle algorithm is the first algorithm in this document for which we must specify a genuine *rearrangement* of the elements of a given range. In contrast to algorithms that modify values or discard elements, shuffle is intended to change only the order of the elements.

To express this property precisely, we introduce the notion of a *multiset*. A multiset is a collection of values in which elements may occur with multiplicity, but in which no order is imposed. Two arrays represent the same multiset if and only if, for every possible value, they contain the same number of occurrences of that value.[12]

Accordingly, we say that an array has been reordered between two program states if the multiset of elements contained in the array remains unchanged. That is, a reordering preserves all values together with their multiplicities, while possibly changing their positions.

We use the predicate MultisetReorder [7.59] to formalize this notion. The predicate is provided in two overloaded variants and is defined in terms of the logic function Count [4.44], which counts the number of occurrences of a given value in a range.

In order to reason effectively about rearranging algorithms, it is not sufficient to introduce MultisetReorder alone. One also needs a collection of elementary properties that allow the predicate to be combined,

[12] See http://en.wikipedia.org/wiki/Multiset.

propagated through program fragments, and instantiated for subranges. For this purpose, we state a number of lemmas that capture basic structural properties of `MultisetReorder`, such as reflexivity, symmetry, and its behavior under range decomposition.

```
/*@
  predicate MultisetReorder{K,L}(value_type* a, integer m, integer n) =
    \forall value_type v;
      Count{K}(a, m, n, v) == Count{L}(a, m, n, v);

  predicate MultisetReorder{K,L}(value_type* a, integer n) =
    MultisetReorder{K,L}(a, 0, n);

  lemma Unchanged_MultisetReorder{K,L}:
    \forall value_type *a, integer k, n;
      Unchanged{K,L}(a, k, n)  ==>  MultisetReorder{K,L}(a, k, n);

  lemma MultisetReorder_DisjointUnion{K,L}:
    \forall value_type *a, integer i, k, n;
      0 <= i <= k <= n               ==>
      MultisetReorder{K,L}(a, i, k)  ==>
      MultisetReorder{K,L}(a, k, n)  ==>
      MultisetReorder{K,L}(a, i, n);

  lemma MultisetReorder_Symmetric{K,L}:
    \forall value_type *a, integer m, n;
      MultisetReorder{K,L}(a, m, n)  ==>  MultisetReorder{L,K}(a, m, n);

  lemma MultisetReorder_Transitive{K,L,M}:
    \forall value_type *a, integer m, n;
      MultisetReorder{K,L}(a, m, n)  ==>
      MultisetReorder{L,M}(a, m, n)  ==>
      MultisetReorder{K,M}(a, m, n);
*/
```

Listing 7.59: `MultisetReorder` (ACSL)

These lemmas are not specific to `shuffle`. Rather, they form a small reasoning toolkit that will be reused throughout the verification of algorithms whose effect consists solely in rearranging existing elements. They will play a central role in the verification of the heap-related algorithms in Chapter 9 and of the sorting algorithms in Chapter 10.

7.18.2. Formal specification of `shuffle`

The `shuffle` algorithm preserves the *multiset represented by the input array* (see Section 7.18.1), while modifying only the order of its elements. This essential property is formalized with the help of the predicate `MultisetReorder` [7.59], which expresses that the number of occurrences of each value in the range `a[0..n-1]` is unchanged upon termination.

Accordingly, the specification of `shuffle` requires that the range `a[0..n-1]` be valid for both reading and writing.

The array `rand` represents the mutable state of the pseudo-random number generator used by the implementation. By allowing assignments to `rand` in the `assigns`-clause, the specification reflects that successive calls to `shuffle` may produce different reorderings of the same input range.

Overall, the specification states exactly one semantic property of the resulting range: it is a rearrangement of the original range and nothing more. No assumption is made about which particular rearrangement is produced in any given call.

```
/*@
  requires   valid:     \valid(a + (0..n-1));
  requires   valid:     \valid(seed + (0..2));
  requires   sep:       \separated(a + (0..n-1), seed + (0..2));

  terminates            \true;
  exits                 \false;
  assigns               a[0..n-1];
  assigns               seed[0..2];

  ensures     reorder:  MultisetReorder{Old,Here}(a,n);
*/
void shuffle(value_type* a, size_type n, unsigned short* seed);
```

Listing 7.60: Formal specification of `shuffle`

Ideally, one would like to express that repeated calls to `shuffle` produce reorderings that are random in a well-defined sense. The informal specification of `shuffle` [18, §28.6.13] states that each possible ordering of the elements should occur with equal probability. Such a requirement, however, is inherently a property of sequences of executions rather than of a single function call. At present, ACSL does not provide mechanisms for specifying or verifying such temporal or probabilistic properties.

More generally, formalizing randomness in a mathematically precise and verification-friendly way is a nontrivial task. In practice, randomness is often assessed by means of statistical tests applied to sequences of generated values. From a theoretical perspective, one possible characterization is based on Kolmogorov complexity, which informally requires that the generated sequence admits no description substantially shorter than the sequence itself. Such notions, however, are undecidable in general and therefore lie outside the scope of deductive program verification.

7.18.3. Implementation of shuffle

The following listing shows our implementation of the function `shuffle` [7.61]. The algorithm proceeds incrementally and repeatedly calls `swap` [7.7] to exchange pairs of elements, thereby building up a reordering of the input range. The indices of the elements to be swapped are determined using a simple pseudo-random number generator; for details of this source of randomness we refer to the function `random_number` [7.64].

From a verification perspective, the central challenge is to relate the sequence of local swaps performed by the algorithm to the global postcondition stating that the entire range has been reordered. This is achieved by means of two loop invariants, labelled `reorder` and `unchanged`. At the beginning of the `i`th loop iteration, the prefix `a[0..i-1]` has already been rearranged, while the suffix `a[i..n-1]` is still equal to its initial contents.

```
void shuffle(value_type* a, size_type n, unsigned short* seed)
{
  if (0u < n) {
    /*@
      loop invariant bounds:    1 <= i <= n;
      loop invariant reorder:   MultisetReorder{Pre,Here}(a, 0, i);
      loop invariant unchanged: Unchanged{Pre,Here}(a, i, n);
      loop assigns   i, a[0..n-1], seed[0..2];
      loop variant   n - i;
    */
    for (size_type i = 1u; i < n; ++i) {
      size_type k = random_number(seed, i) + 1u;

      //@ assert less: 0 <= k <= i;
      if (k < i) {
        swap(&a[k], &a[i]);
        //@ assert swapped: ArraySwap{LoopCurrent,Here}(a, k, i, n);
        //@ assert reorder: MultisetReorder{LoopCurrent,Here}(a, i+1);
        //@ assert reorder: MultisetReorder{Pre,Here}(a, i+1);
      }
      else {
        //@ assert unchanged: Unchanged{LoopCurrent,Here}(a, i+1);
        //@ assert reorder:   MultisetReorder{Pre,Here}(a, i+1);
      }

      //@ assert reorder: MultisetReorder{Pre,Here}(a, i+1);
    }
  }
}
```

Listing 7.61: Implementation of `shuffle`

In order to make this reasoning accessible to the automatic provers, the implementation contains several auxiliary assertions labelled `reorder`. These assertions make explicit how the current iteration preserves the reordering property across loop steps. They are formulated using the predefined label `LoopCurrent` and serve solely to guide the verification of the loop invariant. In particular, the empty **`else`**-branch contains only such an assertion. Although operationally redundant, this assertion is crucial for establishing that the `reorder` invariant is maintained in all cases.

To keep the annotations concise, we do not reason directly with the postcondition of `swap` [7.7]. Instead, we introduce the auxiliary predicate `ArraySwap` [7.62], which abstracts the effect of exchanging two array elements in a value-oriented manner. Using this predicate leads to significantly more compact annotations and improves the performance of the automatic provers.

```
/*@
  predicate ArraySwap{K,L}(value_type* a, integer i, integer k, integer n) =
    0 <= i < k < n              &&
    At{K}(a, i) == At{L}(a, k)  &&
    At{K}(a, k) == At{L}(a, i)  &&
    Unchanged{K,L}(a, 0,   i)   &&
    Unchanged{K,L}(a, i+1, k)   &&
    Unchanged{K,L}(a, k+1, n);
*/
```

Listing 7.62: `ArraySwap` (ACSL)

On top of this abstraction, we rely on a small collection of lemmas that relate swapping operations to multiset preservation. In particular, the lemma `MultisetSwap_Middle` [7.63] states that swapping the elements `a[i]` and `a[k]` constitutes a specific instance of a multiset reordering on the range `a[i..k]`. This lemma forms the key link between the local effect of a single swap and the global `MultisetReorder` [7.59] property used in the contract of `shuffle`.

```
/*@
  lemma MultisetSwap_Middle{K,L}:
    \forall value_type* a, integer i, k, n;
      ArraySwap{K,L}(a, i, k, n)  ==>  MultisetReorder{K,L}(a, i, k+1);

  lemma MultisetSwap_FrontMiddle{K,L}:
    \forall value_type* a, integer i, k, n;
      ArraySwap{K,L}(a, i, k, n)  ==>  MultisetReorder{K,L}(a, 0, k+1);
*/
```

Listing 7.63: `MultisetSwap` (ACSL)

7.19. Verifying a random number generator

In this section, we describe the auxiliary function `random_number` [7.64], which implements a simple pseudo-random number generator used by `shuffle`. Unlike the algorithms studied elsewhere in this document, `random_number` is not part of the C++ Standard Library but serves solely as a supporting component. Accordingly, the structure of this section differs slightly from that of the surrounding algorithm chapters.

As in the case of `shuffle` itself, we do not attempt to formalize statistical properties of randomness. Instead, the specification of `random_number` merely requires that the returned value lies within the prescribed range `[0..n-1]`. The `assigns`-clause for the array `state` models the dependence of the generator on a mutable internal state and makes explicit that successive calls may yield different results.

In the same listing, we also provide the specification of the auxiliary function `random_init`, which initializes the generator state. Both specifications are shown below.

```
/*@
  requires   pos:     0 < n;
  requires   valid:   \valid(state + (0..2));

  terminates          \true;
  exits               \false;
  assigns             state[0..2];

  ensures    result:  0 <= \result < n;
*/
size_type random_number(unsigned short* state, size_type n);


/*@
  requires   valid:  \valid(state + (0..2));

  terminates         \true;
  exits              \false;
  assigns            state[0..2];
*/
void random_init(unsigned short* state);
```

Listing 7.64: Formal specification of `random_number`

The implementations of `random_number` and `random_init` are given in the following listing. Internally, the generator is based on a custom implementation of the POSIX.1 function `lrand48()`[13]. This generator is a linear congruential generator with a 48-bit state. Its iteration step is given by

$$x_{n+1} = ax_n + c \bmod 2^{48}, \tag{7.12}$$

where $a = 25214903917$ and $c = 11$ are relatively prime integers.

During the computation of Equation (7.12), unsigned arithmetic overflow may occur. This overflow is intentional and benign, since only the lowest 48 bits of the result are retained. However, one of the verification options used throughout this document, `-warn-unsigned-overflow`, instructs Frama-C/WP to assert the absence of such overflows. As a consequence, the verification of `random_number` would fail under the same option settings used for the other algorithms.

[13] See http://pubs.opengroup.org/onlinepubs/9699919799/functions/lrand48.html

As an exception, we therefore disable the `-warn-unsigned-overflow` option for this function. This is justified because unsigned overflow is well-defined in C and does not affect the correctness of the computation (cf. [15, §6.2.5, 9]).

```
// see IEEE 1003.1-2008, 2016 Edition for specification
/*@
  requires valid: \valid(seed + (0..2));
  assigns seed[0..2];
  ensures lower:  0 <= \result;
  ensures upper:  \result <= 0x7fffffff;
*/
static long my_lrand48(unsigned short* seed)
{
  unsigned long long state = (unsigned long long)seed[0] << 32
                           | (unsigned long long)seed[1] << 16
                           | (unsigned long long)seed[2];
  state = (0x5deece66dull * state + 0xbull) % (1ull << 48);
  //@ assert lower: state < (1ull << 48);
  long result = state / (1ull << 17);
  //@ assert lower: 0 <= result;
  seed[0u] = state >> 32 & 0xffff;
  seed[1u] = state >> 16 & 0xffff;
  seed[2u] = state >>  8 & 0xffff;
  return result;
}

size_type random_number(unsigned short* state, size_type n)
{
  return my_lrand48(state) % n;
}

void random_init(unsigned short* state)
{
  state[0] = 0x243f;
  state[1] = 0x6a88;
  state[2] = 0x85a3;
}
```

Listing 7.65: Implementation of `random_number`

Finally, we rely on the auxiliary ACSL lemma `RandomNumberModulo` [7.66], shown in the following listing, to support the verification of several assertions concerning modular arithmetic in the implementation.

```
/*@
  lemma RandomNumberModulo:
    \forall unsigned long long a;
      (a % (1ull << 48)) < (1ull << 48);
*/
```

Listing 7.66: `C_Bit` (ACSL)

8. Numeric algorithms

The algorithms considered so far operate primarily on the structure of sequences. They compare values, read elements, or copy data from one range to another, but they do not modify values by means of arithmetic operations.

In this chapter, we turn to the so-called *numeric* algorithms of the C++ Standard Library [18, §29.8]. These algorithms combine the elements of one or more sequences by applying arithmetic operations on `value_type` and thereby introduce additional semantic and verification challenges.

To reason about potential arithmetic overflows, we introduce the two constants shown in Listing 8.1, which denote the numeric limits of `value_type` (see also §2). These constants will be used systematically in the specifications to characterize admissible input ranges and to exclude undefined arithmetic behavior.

```
#define VALUE_TYPE_MAX  INT_MAX
#define VALUE_TYPE_MIN  INT_MIN
```

Listing 8.1: Limits of `value_type`

The following numeric algorithms are considered in this chapter.

- `iota`, which writes sequentially increasing values into a range (§8.1);
- `accumulate`, which computes the sum of the elements in a range (§8.2);
- `inner_product`, which computes the inner product of two ranges (§8.3);
- `partial_sum`, which computes the sequence of partial sums of a range (§8.4);
- `adjacent_difference`, which computes the differences of adjacent elements in a range (§8.5);
- finally, in §8.6, we show that, under appropriate preconditions, the algorithms `partial_sum` and `adjacent_difference` are inverse to each other.

The formal specification of these algorithms raises new issues that did not arise in the earlier chapters. In particular, the presence of arithmetic operations requires an explicit treatment of numeric bounds in order to exclude arithmetic overflow and to ensure that the computed results coincide with their mathematical definitions.

8.1. The iota algorithm

The iota algorithm of the C++ Standard Library [18, §29.8.12] initializes a range with sequentially increasing values, starting from a user-provided initial value. In contrast to algorithms that merely rearrange existing elements, iota actively constructs new values and assigns them to the range.

For our purposes, the generic interface of the standard library is specialized to ranges of type value_type. The resulting signature reads:

```
void iota(value_type* a, size_type n, value_type v);
```

Starting with the value v, the function assigns consecutive values v, v+1, up to v+n-1 to the elements of the array a. Since value_type is instantiated as an integer type, the specification must explicitly account for the possibility of arithmetic overflow when incrementing the initial value v. This aspect plays a central role in the formulation of the contract for iota.

8.1.1. Formal specification of iota

The specification of iota relies on the auxiliary predicate IotaGenerate [8.2], defined in the following listing.

```
/*@
  predicate IotaGenerate(value_type* a, integer n, value_type v) =
    \forall integer i; 0 <= i < n  ==>  a[i] == v+i;
*/
```

Listing 8.2: IotaGenerate (ACSL)

The predicate Iota(a,n,v) expresses the intended effect of iota as an arithmetic progression: for every index i with 0 <= i < n, the element a[i] equals v+i.

The complete specification of iota is shown in the following listing. Its postcondition increment is precisely the assertion IotaGenerate(a,n,v).

```
/*@
  requires   valid:      \valid(a + (0..n-1));
  requires   limit:      v + n <= VALUE_TYPE_MAX;

  terminates             \true;
  exits                  \false;
  assigns                a[0..n-1];

  ensures    increment:  IotaGenerate(a, n, v);
*/
void iota(value_type* a, size_type n, value_type v);
```

Listing 8.3: Formal specification of iota

The contract refers to VALUE_TYPE_MAX, the maximum representable value of value_type (see Listing 8.1). To rule out arithmetic overflow in the expression v+i occurring in IotaGenerate, the precondition limit requires v+n <= VALUE_TYPE_MAX. Since the quantification in IotaGenerate ranges over 0 <= i < n, we have v+i < v+n, and thus the bound v+n <= VALUE_TYPE_MAX implies v+i <= VALUE_TYPE_MAX for all indices covered by the postcondition. This formulation avoids a special treatment of the case n==0 and keeps the overflow guard uniform.

The specification of `iota` refers to `VALUE_TYPE_MAX` which is the maximum value of the underlying integer type (see Listing 8.1). In order to avoid integer overflows the sum `v+n` must not be greater than the constant `VALUE_TYPE_MAX`.

8.1.2. Implementation of iota

The following listing shows the implementation of the `iota` function together with its ACSL annotations.

```
void iota(value_type* a, size_type n, value_type v)
{
  /*@
    loop invariant bound:     0 <= i <= n;
    loop invariant limit:     v == \at(v, Pre) + i;
    loop invariant increment: IotaGenerate(a, i, \at(v, Pre));

    loop assigns i, v, a[0..n-1];
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    a[i] = v++;
  }
}
```

Listing 8.4: Implementation of `iota`

The implementation follows the straightforward iterative scheme suggested by the informal description of the algorithm. Starting from the initial value `v`, the loop assigns the current value to the next array element and then increments `v` before proceeding to the next index.

The central loop invariant is `increment`. It states that upon entry to iteration `i`, the current value of the local variable `v` equals the initial value supplied to the function, plus the number of iterations already completed. Formally, this relationship is expressed as `v == \at(v,Pre)+ i`, where `\at(v,Pre)` denotes the value of `v` in the pre-state of the function `iota`.

This invariant establishes the precise link between the evolving program state and the arithmetic progression described by the specification. Combined with the loop invariant that characterizes the contents of the already initialized prefix `a[0..i-1]`, it allows the verifier to derive the postcondition `IotaGenerate(a,n,\at(v,Pre))` upon loop termination. In particular, the invariant ensures that each assignment `a[i] = v` produces exactly the value required by the formal specification.

8.2. The accumulate algorithm

The accumulate algorithm in the C++ Standard Library [18, §29.8.2] computes the sum of an initial value and the elements of a given range. Conceptually, it performs a left fold over the range, starting from the supplied initial value and successively incorporating each element.

For the purposes of this discussion, we consider the following simplified version of the original interface, specialized to ranges of type value_type:

```
value_type
accumulate(const value_type* a, size_type n, value_type init);
```

The value returned by accumulate can be characterized mathematically as the sum of the initial value and all elements of the range. Formally, it equals

$$init + \sum_{i=0}^{n-1} a_i.$$

Here, a_i denotes the element stored at index i of the array a, and n is the length of the range. In particular, if the range is empty, that is, if n == 0, the algorithm returns the initial value init unchanged.

8.2.1. The logic function Accumulate

As in the case of count [4.48], we specify accumulate by first defining the *logic function* Accumulate [8.5], which formalizes summation over an array prefix.

The logic function Accumulate maps an array a, a length n, and an initial value init to the corresponding mathematical sum. For $n \geqslant 0$, the following equation characterizes its meaning:

$$\mathrm{Accumulate}(a, n, init) = init + \sum_{i=0}^{n-1} a_i. \tag{8.1}$$

Here, a_i denotes the element stored at index i of the array a. The right-hand side is a mathematical sum, and in particular it is not subject to the wrap-around behavior of machine arithmetic.

The specification also introduces the predicate AccumulateBounds [8.5]. This predicate is used to express overflow requirements in a compact form. It states that, for every i with $0 \leqslant i < n$, the partial sums

$$init + \sum_{k=0}^{i} a_k \tag{8.2}$$

are representable in the underlying integer type.

The role of AccumulateBounds is essential. If any intermediate sum overflowed, the value computed by the C implementation of accumulate could differ from the mathematical value defined by Accumulate, even when the final result itself lies within bounds.

```
/*@
    logic integer
    Accumulate{L}(value_type* a, integer n, integer init) =
      n <= 0 ? init : Accumulate(a, n-1, init) + a[n-1];

    predicate AccumulateBounds{L}(value_type* a, integer n, value_type init) =
      \forall integer i; 0 <= i <= n  ==>
        VALUE_TYPE_MIN <= Accumulate(a, i, init) <= VALUE_TYPE_MAX;

    lemma Accumulate_Init:
      \forall value_type *a, init, integer n;
         n <= 0  ==>  Accumulate(a, n, init) == init;

    lemma Accumulate_Next{L}:
      \forall value_type* a, init, integer n;
        0 <= n  ==>
        Accumulate(a, n+1, init) == Accumulate(a, n, init) + a[n];

    lemma Accumulate_Unchanged{K,L}:
      \forall value_type *a, init, integer n;
        Unchanged{K,L}(a, n)  ==>
        Accumulate{K}(a, n, init) == Accumulate{L}(a, n, init);

    lemma Accumulate_Unchanged_Shrink{K,L}:
      \forall value_type *a, init, integer m, n;
        0 <= m <= n            ==>
        Unchanged{K, L}(a, n)  ==>
        Accumulate{K}(a, m, init) == Accumulate{L}(a, m, init);

    lemma AccumulateBounds_Unchanged{K,L}:
      \forall value_type *a, init, integer n;
        Unchanged{K, L}(a, n)           ==>
        AccumulateBounds{K}(a, n, init) ==>
        AccumulateBounds{L}(a, n, init);
*/
```

Listing 8.5: `Accumulate` (ACSL)

8.2.2. `AccumulateDefault`— a variant of `Accumulate`

The following listing introduces the logic function `AccumulateDefault` [8.6], a second logic function related to `Accumulate` [8.5]. Unlike `Accumulate`, this variant does not take an explicit initial value as a parameter. Instead, the accumulation is defined to start from the constant value 0. Formally, `AccumulateDefault` is defined as a direct specialization of `Accumulate` with `init` fixed to 0. Consequently, for all $n \geqslant 0$, the following equation holds:

$$\text{AccumulateDefault}(a, n) = \sum_{i=0}^{n-1} a_i. \tag{8.3}$$

This formulation corresponds to the usual mathematical notion of a prefix sum and avoids the special treatment of the first array element. In particular, the case $n = 0$ is well defined and yields the neutral value 0.

We will use `AccumulateDefault` in the specification of the algorithm `partial_sum` [8.13], where such a canonical initialization is natural.

```
/*@
    logic integer
    AccumulateDefault{L}(value_type* a, integer n) =
      Accumulate(a, n, (value_type)(0));

    predicate AccumulateDefaultBounds{L}(value_type* a, integer n) =
      AccumulateBounds(a, n, (value_type)(0));

    lemma AccumulateDefault_Unchanged{K,L}:
      \forall value_type *a, integer n;
        0 <= n                ==>
        Unchanged{K,L}(a, n)  ==>
        AccumulateDefault{K}(a, n) == AccumulateDefault{L}(a, n);

    lemma AccumulateDefault_Zero{L}:
      \forall value_type* a; AccumulateDefault(a, 0) == 0;

    lemma AccumulateDefault_One{L}:
      \forall value_type* a; AccumulateDefault(a, 1) == a[0];

    lemma AccumulateDefault_Next{L}:
      \forall value_type* a, integer n;
        0 <= n  ==>
        AccumulateDefault(a, n+1) == AccumulateDefault(a, n) + a[n];

    lemma AccumulateDefaultBounds_Shrink{L}:
      \forall value_type* a, integer m, n;
        0 <= m <= n                    ==>
        AccumulateDefaultBounds(a, n)  ==>  AccumulateDefaultBounds(a, m);
*/
```

Listing 8.6: `AccumulateDefault` (ACSL)

The listing further contains lemmas that characterize the observable behavior of `AccumulateDefault`. In particular, the step lemma expresses how the accumulated value evolves when the range is extended by one element. Finally, the predicate `AccumulateDefaultBounds` specializes `AccumulateBounds` [8.5] to the case of zero initialization and provides corresponding numeric limits for `AccumulateDefault`.

8.2.3. Formal specification of accumulate

The formal specification of accumulate [8.7] is expressed in terms of the logic function Accumulate, which defines the mathematical meaning of the accumulation process. The predicate AccumulateBounds additionally constrains the range of admissible values and characterizes conditions under which numerical overflows cannot occur during the accumulation process. Together, these abstractions allow the contract of accumulate to be stated in a direct and compact manner, as shown in the following listing.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));
  requires   bounds:  AccumulateBounds(a, n, init);

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  \result == Accumulate(a, n, init);
*/
value_type accumulate(const value_type* a, size_type n, value_type init);
```

Listing 8.7: Formal specification of accumulate

8.2.4. Implementation of accumulate

The following listing shows an implementation of the accumulate function together with its loop annotations.

```
value_type accumulate(const value_type* a, size_type n, value_type init)
{
  /*@
    loop invariant index:    0 <= i <= n;
    loop invariant partial:  init == Accumulate(a, i, \at(init,Pre));
    loop assigns i, init;
    loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    //@ assert rte_help: init + a[i] == Accumulate(a, i+1, \at(init,Pre));
    init = init + a[i];
  }

  return init;
}
```

Listing 8.8: Implementation of accumulate

The central loop invariant partial expresses that upon entering the i-th iteration, the variable result represents the accumulated value defined by Equation (8.2) for the prefix a[0..i-1]. The preservation of this invariant relies on the precondition bounds of accumulate [8.7], which characterizes those input values for which updating the accumulator does not lead to numerical overflow.

8.3. The `inner_product` algorithm

The `inner_product` algorithm of the C++ Standard Library [18, §29.8.4] computes the *inner product*[14] of two ranges of equal length. Given two input sequences, the algorithm multiplies corresponding elements and accumulates the resulting products.

In our setting, the original signature is adapted to operate on `value_type` ranges and reads as follows:

```
value_type
inner_product(const value_type* a, const value_type* b, size_type n,
              value_type init);
```

The result returned by `inner_product` is defined as

$$\texttt{init} + \sum_{i=0}^{\texttt{n}-1} \texttt{a}[i] \cdot \texttt{b}[i].$$

In particular, for an empty range, the function returns the initial value `init`.

8.3.1. The logic function `InnerProduct`

As in the case of `accumulate` [8.7] we specify `inner_product` by defining in the following listing the logic function `InnerProduct` [8.9] that formally expresses the summation of the element-wise product of two arrays.

Predicate `ProductBounds` [8.9] expresses that for $0 \leqslant i < n$ the products

$$\texttt{a}[i] \cdot \texttt{b}[i] \tag{8.4}$$

do not overflow. Predicate `InnerProductBounds`, on the other hand, states that for $0 \leqslant i < n$ the following sums do not overflow.

$$\texttt{init} + \sum_{k=0}^{\texttt{i}} \texttt{a}[k] \cdot \texttt{b}[k] \tag{8.5}$$

These bounds are essential to ensure that any compliant implementation computes a result that coincides with the mathematical value defined by `InnerProduct`. Finally, Lemma `InnerProduct_Unchanged` states that the result of the `InnerProduct` only depends on the values of `a[0..n-1]` and `b[0..n-1]`.

[14] Also referred to as the *dot product*, see http://en.wikipedia.org/wiki/Dot_product.

```
/*@
  logic integer
  InnerProduct{L}(value_type* a, value_type* b, integer n,
                  value_type init) =
    n <= 0 ? init : InnerProduct(a, b, n-1, init) + (a[n-1] * b[n-1]);

  predicate ProductBounds(value_type* a, value_type* b, integer n) =
    \forall integer i; 0 <= i < n  ==>
      VALUE_TYPE_MIN <= a[i] * b[i] <= VALUE_TYPE_MAX;

  predicate InnerProductBounds(value_type* a, value_type* b, integer n,
                     value_type init) =
    \forall integer i; 0 <= i <= n  ==>
      VALUE_TYPE_MIN <= InnerProduct(a, b, i, init) <= VALUE_TYPE_MAX;

  lemma InnerProduct_Init:
    \forall value_type *a, *b, init, integer n;
       n <= 0  ==>  InnerProduct(a, b, n, init) == init;

  lemma InnerProduct_Next{L}:
    \forall value_type* a, *b, init, integer n;
      0 <= n  ==>
      InnerProduct(a, b, n+1, init) == InnerProduct(a, b, n, init) + a[n] * b[n];

  lemma InnerProduct_Unchanged{K,L}:
    \forall value_type *a, *b, init, integer n;
      Unchanged{K,L}(a, n)  ==>
      Unchanged{K,L}(b, n)  ==>
      InnerProduct{K}(a, b, n, init) == InnerProduct{L}(a, b, n, init);
*/
```

Listing 8.9: `InnerProduct` (ACSL)

8.3.2. Formal specification of `inner_product`

The logic function `InnerProduct` [8.9] captures the mathematical definition of the inner product of two ranges and forms the basis of the formal specification of `inner_product` [8.10]. Using this abstraction, the contract of `inner_product` can be stated in a direct and concise form, as shown in the following listing.

Note that the specification does not require the input ranges `a` and `b` to be separated. Since the algorithm only reads from both arrays and does not modify either of them, overlapping memory regions do not affect the correctness of the result.

```
/*@
  requires   valid:      \valid_read(a + (0..n-1));
  requires   valid:      \valid_read(b + (0..n-1));
  requires   bounds:     ProductBounds(a, b, n);
  requires   bounds:     InnerProductBounds(a, b, n, init);

  terminates             \true;
  exits                  \false;
  assigns                \nothing;

  ensures    result:     \result == InnerProduct(a, b, n, init);
  ensures    unchanged:  Unchanged{Old,Here}(a, n);
  ensures    unchanged:  Unchanged{Old,Here}(b, n);
*/
value_type inner_product(const value_type* a, const value_type* b, size_type n,
                         value_type init);
```

Listing 8.10: Formal specification of `inner_product`

8.3.3. Implementation of `inner_product`

The following listing shows an implementation of `inner_product` together with its loop annotations.

```
value_type inner_product(const value_type* a, const value_type* b, size_type n,
                         value_type init)
{
  /*@
     loop invariant index: 0 <= i <= n;
     loop invariant inner: init == InnerProduct(a, b, i, \at(init,Pre));
     loop assigns i, init;
     loop variant n-i;
  */
  for (size_type i = 0u; i < n; ++i) {
    /*@
      assert rte_help: init + a[i] * b[i] ==
                       InnerProduct(a, b, i+1, \at(init,Pre));
    */
    init = init + a[i] * b[i];
  }

  return init;
}
```

Listing 8.11: Implementation of `inner_product`

The central loop invariant `inner` expresses that upon entering the i-th iteration, the current value of `init` coincides with the partial inner product defined by Equation (8.5) for the prefix `a[0..i-1]` and `b[0..i-1]`. In each iteration step, this invariant is preserved by extending the sum with the product `a[i] * b[i]`.

The bounds properties stated in the contract of `inner_product` [8.10] underpin the validity of this reasoning. These properties describe conditions under which neither the individual products nor the successive updates of the accumulator `init` lead to arithmetic overflow, thereby ensuring that the computed result agrees with the mathematical definition of the inner product.

8.4. The partial_sum algorithm

The partial_sum algorithm of the C++ Standard Library [18, §29.8.6] computes prefix sums of a given input range. More precisely, it successively accumulates the elements of the input array and stores the intermediate results in an output array. In our setting, the original signature is adapted to operate on value_type ranges and reads as follows:

```
size_type
partial_sum(const value_type* a, size_type n, value_type* b);
```

After execution of partial_sum, the output array b[0..n-1] satisfies

$$\mathtt{b}[i] = \sum_{k=0}^{\mathtt{i}} \mathtt{a}[k] \tag{8.6}$$

for all $0 \leqslant i < n$. That is, each entry b[i] contains the sum of the prefix a[0..i].

8.4.1. The predicate PartialSum

The predicate PartialSum [8.12] formalizes the relationship between an input array a and an output array b, where each entry b[i] represents the partial sum of a up to index i, as defined in Equation (8.6). For each index i, it states that the value stored in b[i] corresponds to the sum of the prefix a[0..i].

```
/*@
  predicate PartialSum{L}(value_type* a, integer n, value_type* b) =
    \forall integer i; 0 <= i < n  ==>  b[i] == AccumulateDefault(a, i+1);

  lemma PartialSum_Section{K}:
    \forall value_type *a, *b, integer m, n;
    0 <= m <= n              ==>
    PartialSum{K}(a, n, b)  ==>  PartialSum{K}(a, m, b);

  lemma PartialSum_Step{L}:
    \forall value_type *a, *b, integer n;
      0 <= n                              ==>
      PartialSum(a, n, b)                 ==>
      b[n] == AccumulateDefault(a, n+1)  ==>  PartialSum(a, n+1, b);

  lemma PartialSum_Unchanged{K,L}:
    \forall value_type *a, *b, integer n;
      0 <= n  ==>
      PartialSum{K}(a, n, b)  ==>
      Unchanged{K, L}(a, n)   ==>
      Unchanged{K, L}(b, n)   ==>  PartialSum{L}(a, n, b);

  lemma PartialSum_One{L}:
    \forall value_type *a, *b, integer n;
      b[0] == AccumulateDefault(a, 1)  ==>  PartialSum(a, 1, b);
*/
```

Listing 8.12: PartialSum (ACSL)

Rather than introducing a new logic function, the definition of PartialSum is naturally expressed in terms of the logic function AccumulateDefault [8.6]. This reuse reflects the fact that partial_sum computes a sequence of accumulated values with a fixed initial value.

8.4.2. Formal specification of partial_sum

The specification of partial_sum [8.13] requires that the input array a[0..n-1] and the output array b[0..n-1] are separated, that is, their memory ranges do not overlap. This assumption simplifies the specification by ensuring that each value a[i] is read before any corresponding entry of the output array is written.

```
/*@
  requires   valid:       \valid_read(a + (0..n-1));
  requires   valid:       \valid(b + (0..n-1));
  requires   sep:         \separated(a + (0..n-1), b + (0..n-1));
  requires   bounds:      AccumulateDefaultBounds(a, n);

  terminates              \true;
  exits                   \false;
  assigns                 b[0..n-1];

  ensures    result:      \result == n;
  ensures    partialsum:  PartialSum(a, n, b);
  ensures    unchanged:   Unchanged{Old,Here}(a, n);
*/
size_type partial_sum(const value_type* a, size_type n, value_type* b);
```

Listing 8.13: Formal specification of partial_sum

This requirement is stricter than that of the original C++ version of partial_sum, which permits a and b to refer to the same array and thus allows the computation of partial sums *in place*. From a functional point of view, such in-place execution is well defined, but accommodating it at the specification level would require a more subtle treatment of aliasing and data dependencies.

For the purposes of the present exposition, the separation condition provides a clear and robust contract that avoids these additional complications while still capturing the essential behavior of the algorithm.

8.4.3. Implementation of partial_sum

The following listing shows an implementation of partial_sum together with its loop annotations.

```
size_type partial_sum(const value_type* a, size_type n, value_type* b)
{
  if (0u < n) {
    //@ assert limits:      AccumulateDefaultBounds(a, n);
    b[0u] = a[0u];
    //@ assert unchanged:   Unchanged{Pre,Here}(a, n);
    //@ assert limits:      AccumulateDefaultBounds(a, n);
    //@ assert accumulate:  b[0] == AccumulateDefault(a, 1);
    //@ assert partialsum:  PartialSum(a, 1, b);

    /*@
       loop invariant bound:      1 <= i <= n;
       loop invariant unchanged:  Unchanged{Pre,Here}(a, n);
       loop invariant accumulate: b[i-1] == AccumulateDefault(a, i);
       loop invariant limits:     AccumulateDefaultBounds(a, n);
       loop invariant partialsum: PartialSum(a, i, b);
       loop assigns i, b[1..n-1];
       loop variant n - i;
    */
    for (size_type i = 1u; i < n; ++i) {
      b[i] = b[i - 1u] + a[i];
      //@ assert unchanged:  Unchanged{LoopCurrent,Here}(b, i);
      //@ assert unchanged:  Unchanged{LoopCurrent,Here}(a, n);
      //@ assert partialsum: b[i] == AccumulateDefault(a, i+1);
      //@ assert limits:     AccumulateDefaultBounds(a, n);
    }
  }

  return n;
}
```

Listing 8.14: Implementation of partial_sum

The implementation computes the output array incrementally. In each iteration, the next partial sum is obtained by adding the current input element a[i] to the previously computed sum stored in b[i-1]. This iterative structure is reflected in the loop invariant partial, which states that upon entering the i-th iteration, the prefix b[0..i-1] already contains the correct partial sums as defined by Equation (8.6).

The correctness of the update step relies on the bounds conditions stated in the contract of partial_sum [8.13], which ensure that neither the individual additions nor the successive accumulation of values lead to arithmetic overflow. In addition, the separation requirement between the input array a and the output array b guarantees that each input value is read before any overlapping memory location is modified. This avoids aliasing effects and allows the invariant to be maintained in a straightforward manner throughout the loop.

8.5. The adjacent_difference algorithm

The adjacent_difference algorithm of the C++ Standard Library [18, §29.8.11] computes the differences between consecutive elements of a given input range. The result is written to an output range of the same length, where the first element is copied unchanged and each subsequent element records the difference to its predecessor.

In our setting, the original signature is adapted to operate on value_type ranges and reads as follows:

```
size_type
adjacent_difference(const value_type* a, size_type n, value_type* b);
```

After execution of adjacent_difference, the output array b[0..n-1] satisfies

$$\begin{aligned} \mathtt{b}[0] &= \mathtt{a}[0], \\ \mathtt{b}[i] &= \mathtt{a}[i] - \mathtt{a}[i-1] \qquad \text{for } 1 \leqslant i < n. \end{aligned} \tag{8.7}$$

8.5.1. The predicate AdjacentDifference

The predicate AdjacentDifference [8.16] forms the core of the specification of adjacent_difference and formalizes the relationship between the input array a and the output array b. As a preparatory step, we introduce the logic function Difference [8.15].

This function defines the difference between two adjacent elements of an array. It primarily serves as a convenient abstraction for expressing and structuring properties related to adjacent differences and is reused in several subsequent assertions.

```
/*@
  logic integer
  Difference{L}(value_type* a, integer n) =
    n <= 0 ? a[0] : a[n] - a[n-1];

  lemma Difference_Zero{L}:
    \forall value_type *a; Difference(a, 0) == a[0];

  lemma Difference_Next{L}:
    \forall value_type *a, integer n;
      0 < n  ==>  Difference(a, n) == a[n] - a[n-1];

  lemma Difference_Unchanged{K,L}:
    \forall value_type *a, integer n;
      0 <= n  ==>  Unchanged{K,L}(a, n+1)  ==>
      Difference{K}(a, n) == Difference{L}(a, n);
*/
```

Listing 8.15: Difference (ACSL)

Building on this logic function, we define the predicate AdjacentDifference, which states that the output array b contains the differences of adjacent elements of the input array a. To exclude arithmetic overflows when computing expressions of the form a[i] - a[i-1], the inputs are further restricted by the predicate AdjacentDifferenceBounds. The lemmas record fundamental properties of AdjacentDifference and will later be used to establish and maintain loop invariants.

```
/*@
    predicate AdjacentDifference{L}(value_type* a, integer n, value_type* b) =
      \forall integer i; 0 <= i < n  ==>  b[i] == Difference(a, i);

    predicate AdjacentDifferenceBounds(value_type* a, integer n) =
      \forall integer i; 1 <= i < n  ==>
        VALUE_TYPE_MIN <= Difference(a, i) <= VALUE_TYPE_MAX;

    lemma AdjacentDifference_Step{K,L}:
      \forall value_type *a, *b, integer n;
        AdjacentDifference{K}(a, n, b)      ==>
        Unchanged{K,L}(b, n)                ==>
        Unchanged{K,L}(a, n+1)              ==>
        \at(b[n],L) == Difference{L}(a, n)  ==>
        AdjacentDifference{L}(a, n+1, b);

    lemma AdjacentDifference_Section{K}:
      \forall value_type *a, *b, integer m, n;
        0 <= m <= n                     ==>
        AdjacentDifference{K}(a, n, b)  ==>
        AdjacentDifference{K}(a, m, b);
*/
```

Listing 8.16: `AdjacentDifference` (ACSL)

8.5.2. Formal specification of `adjacent_difference`

The predicates `AdjacentDifference` [8.16] and `AdjacentDifferenceBounds` form the basis of the formal specification of `adjacent_difference` [8.17]. Together, these predicates characterize the functional relationship between the input and output arrays and restrict the admissible inputs so as to exclude arithmetic overflows when computing adjacent differences. Using these abstractions, the contract of `adjacent_difference` can be stated in a compact and direct form.

```
/*@
   requires   valid:       \valid_read(a + (0..n-1));
   requires   valid:       \valid(b + (0..n-1));
   requires   sep:         \separated(a + (0..n-1), b + (0..n-1));
   requires   bounds:      AdjacentDifferenceBounds(a, n);

   terminates              \true;
   exits                   \false;
   assigns                 b[0..n-1];

   ensures    result:      \result == n;
   ensures    difference:  AdjacentDifference(a, n, b);
   ensures    unchanged:   Unchanged{Old,Here}(a, n);
*/
size_type adjacent_difference(const value_type* a, size_type n, value_type* b);
```

Listing 8.17: Formal specification of `adjacent_difference`

As in the specification of `partial_sum` [8.13], we require that the input array `a[0..n-1]` and the output array `b[0..n-1]` are separated. This assumption avoids aliasing between reads from `a` and writes to `b` and thereby simplifies both the specification and the subsequent verification.

8.5.3. Implementation of adjacent_difference

The following listing shows an implementation of adjacent_difference together with its loop annotations.

```
size_type adjacent_difference(const value_type* a, size_type n, value_type* b)
{
  if (0u < n) {
    b[0u] = a[0u];

    /*@
       loop invariant index:      1 <= i <= n;
       loop invariant unchanged:  Unchanged{Pre,Here}(a, n);
       loop invariant difference: AdjacentDifference(a, i, b);
       loop assigns i, b[1..n-1];
       loop variant n - i;
    */
    for (size_type i = 1u; i < n; ++i) {
      //@ assert bound: VALUE_TYPE_MIN <= Difference(a, i) <= VALUE_TYPE_MAX;
      b[i] = a[i] - a[i - 1u];
      //@ assert difference: AdjacentDifference(a, i+1, b);
    }
  }

  return n;
}
```

Listing 8.18: Implementation of adjacent_difference

The implementation proceeds sequentially through the input array and computes each adjacent difference independently. The central loop invariant difference expresses that upon entering the i-th iteration, the prefix b[0..i-1] already satisfies the predicate AdjacentDifference [8.16] with respect to the corresponding prefix of the input array a.

The verification of this invariant relies on several complementary ingredients. First, the assertions bound and difference record local properties of the current iteration state, in particular the absence of arithmetic overflow when computing differences. Second, the lemmas AdjacentDifference_Step [8.16] and AdjacentDifference_Section provide the logical means to extend the predicate AdjacentDifference from one iteration to the next and to reason about array sections. Finally, a statement contract with two postconditions labeled step is used to connect the effect of the assignment in the loop body with the corresponding logical step in the invariant.

Together, these elements allow the loop invariant difference to be established initially and preserved throughout the loop, thereby ensuring the correctness of the implementation with respect to the formal specification.

8.6. Inverting partial_sum and adjacent_difference

In this section, we show that, under suitable preconditions, `adjacent_difference` and `partial_sum` are inverse to each other. More precisely,

- applying `adjacent_difference` to the result of `partial_sum` recovers the original input sequence, and
- applying `partial_sum` to the result of `adjacent_difference` reconstructs the corresponding prefix sums.

The purpose of this section is therefore not to introduce new algorithms, but to relate the previously specified ones and to make their algebraic connection explicit. Accordingly, the presentation focuses on logical properties and their verification rather than on implementation details.

8.6.1. Inverting partial_sum

Let `a[0..n-1]` denote the input array and `b[0..n-1]` the output array produced by `partial_sum`. By definition of the algorithm, the entries of `b` are given by

$$\begin{aligned} \mathtt{b}[0] &= \mathtt{a}[0], \\ \mathtt{b}[1] &= \mathtt{a}[0] + \mathtt{a}[1], \\ &\vdots \\ \mathtt{b}[n-1] &= \mathtt{a}[0] + \mathtt{a}[1] + \cdots + \mathtt{a}[n-1]. \end{aligned}$$

If we now apply the algorithm `adjacent_difference` to the array `b[0..n-1]`, we obtain an output array `a'[0..n-1]` whose entries satisfy

$$\begin{aligned} \mathtt{a}'[0] &= \mathtt{b}[0] &&= \mathtt{a}[0], \\ \mathtt{a}'[1] &= \mathtt{b}[1] - \mathtt{b}[0] &&= \mathtt{a}[1], \\ &\vdots \\ \mathtt{a}'[n-1] &= \mathtt{b}[n-1] - \mathtt{b}[n-2] &&= \mathtt{a}[n-1]. \end{aligned}$$

Thus, under ideal arithmetic, the original input array `a` is recovered.

Before formalizing this argument in ACSL, we introduce the predicate `DefaultBounds` [8.19], which will be used to capture the absence of arithmetic overflow throughout both computations.

```
/*@
  predicate DefaultBounds{L}(value_type* a, integer n) =
    \forall integer i; 0 <= i < n  ==>
      VALUE_TYPE_MIN <= a[i] <= VALUE_TYPE_MAX;
*/
```

Listing 8.19: `DefaultBounds` (ACSL)

The lemma `PartialSum_Inverse`, shown in the following listing, formalizes the above reasoning. It states, at the logical level, that under the appropriate bounds assumptions, the algorithms `partial_sum` and `adjacent_difference` are inverse to each other.

```
/*@
  lemma PartialSum_Inverse:
    \forall value_type *a, *b, integer n;
      0 <= n                ==>
      PartialSum(a, n, b)   ==>
      AdjacentDifference(b, n, a);

  lemma AdjacentDifference_Inverse:
    \forall value_type *a, *b, integer n;
      0 <= n                        ==>
      AdjacentDifference(a, n, b)   ==>
      PartialSum(b, n, a);

  lemma AdjacentDifference_InverseBounds:
    \forall value_type *a, *b, integer n;
      0 <= n                        ==>
      DefaultBounds(a, n)           ==>
      AdjacentDifference(a, n, b)   ==>
      AccumulateDefaultBounds(b, n);
*/
```

Listing 8.20: `NumericInverse` (ACSL)

The following listing presents the C function `partial_sum_inv`, including both its contract and its implementation. This function first invokes `partial_sum` and subsequently applies `adjacent_difference` to the resulting array.

```
/*@
  requires   valid:      \valid(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));
  requires   bounds:     AccumulateDefaultBounds(a, n);
  requires   bounds:     DefaultBounds(a, n);

  terminates             \true;
  exits                  \false;
  assigns                a[0..n-1], b[0..n-1];

  ensures    unchanged:  Unchanged{Pre,Here}(a, n);
*/
void partial_sum_inv(value_type* a, size_type n, value_type* b)
{
  partial_sum(a, n, b);
  adjacent_difference(b, n, a);
}
```

Listing 8.21: Implementation of `partial_sum_inv`

The contract of `partial_sum_inv` formulates preconditions that ensure that neither arithmetic overflow (property `bounds`) nor unintended aliasing between arrays (property `sep`) can occur during the computation. Under these assumptions, Frama-C is able to verify that the final call to `adjacent_difference` restores exactly the original contents of the array `a[0..n-1]` supplied to the initial call to `partial_sum`.

8.6.2. Inverting adjacent_difference

Let a[0..n-1] denote the input array and b[0..n-1] the output array produced by adjacent_difference . By definition of the algorithm, the entries of b are given by

$$\begin{aligned} \mathtt{b}[0] &= \mathtt{a}[0], \\ \mathtt{b}[1] &= \mathtt{a}[1] - \mathtt{a}[0], \\ &\vdots \\ \mathtt{b}[n-1] &= \mathtt{a}[n-1] - \mathtt{a}[n-2]. \end{aligned}$$

If we now apply the algorithm partial_sum to the array b[0..n-1], we obtain an output array a'[0..n-1] satisfying

$$\begin{aligned} \mathtt{a}'[0] &= \mathtt{b}[0] &&= \mathtt{a}[0], \\ \mathtt{a}'[1] &= \mathtt{b}[0] + \mathtt{b}[1] &&= \mathtt{a}[1], \\ &\vdots \\ \mathtt{a}'[n-1] &= \mathtt{b}[0] + \mathtt{b}[1] + \cdots + \mathtt{b}[n-1] &&= \mathtt{a}[n-1]. \end{aligned}$$

Thus, under ideal arithmetic, applying partial_sum to the result of adjacent_difference reconstructs the original input array a. The lemma AdjacentDifference_Inverse [8.20] formally captures this property by stating that the two algorithms are inverse to each other under appropriate preconditions.

The following listing presents the C function adjacent_difference_inv. It invokes the algorithm adjacent_difference and then applies partial_sum to the resulting array. Its contract formulates preconditions that ensure that neither arithmetic overflow (property bounds) nor unintended aliasing between arrays (property sep) can occur during the computation.

```
/*@
  requires   size:       0 <= n;
  requires   valid:      \valid(a + (0..n-1));
  requires   valid:      \valid(b + (0..n-1));
  requires   sep:        \separated(a + (0..n-1), b + (0..n-1));
  requires   bounds:     DefaultBounds(a, n);
  requires   bounds:     AdjacentDifferenceBounds(a, n);

  terminates             \true;
  exits                  \false;
  assigns                a[0..n-1], b[0..n-1];

  ensures    unchanged:  Unchanged{Old,Here}(a, n);
*/
void adjacent_difference_inv(value_type* a, size_type n, value_type* b)
{
  adjacent_difference(a, n, b);
  partial_sum(b, n, a);
}
```

Listing 8.22: Implementation of adjacent_difference_inv

The lemma Unchanged_Transitive [7.2] is additionally used to facilitate automatic verification. Moreover, the lemma AdjacentDifference_InverseBounds [8.20] is used to establish the bounds precondition required for the subsequent call to partial_sum.

Part IV.

Heaps and Sorting Algorithms

Introduction

This part is devoted to algorithms that reorder the elements of a sequence according to a specified ordering relation. In contrast to the algorithmic patterns studied earlier, these procedures fundamentally transform the structure of the data by permuting elements, while preserving their values. As such, they combine mutation with global ordering properties.

Chapter 9 introduces heap-based algorithms. Heaps provide a structured representation of partially ordered data and serve both as an abstract data structure and as a foundation for efficient algorithms. From a verification perspective, heaps give rise to characteristic structural invariants that relate local ordering constraints to global correctness properties.

Building on these ideas, Chapter 10 studies sorting algorithms and well-known classical implementations. Sorting serves as a canonical example of algorithms that impose a total order on a sequence. The verification of sorting algorithms requires reasoning not only about mutation and ordering, but also about permutation properties and the preservation of multisets of values.

Together, these chapters address algorithms whose primary purpose is the reorganization of data. They bring together mutation, ordering, and global structural invariants, and they form a natural culmination of the algorithmic developments presented in the preceding parts.

9. Heap Algorithms

In this chapter we study the heap algorithms of the C++ Standard Library [18, p. 28.7.7] and discuss their formal verification in some detail. The exposition and verification approach adopted here are largely based on the bachelor thesis of one of the authors [20].

A heap is an array-based data structure that supports efficient access to its largest element. At any time, this element is stored at the beginning of the array and can be removed or replaced in logarithmic time. These properties make heaps a natural foundation for priority queues.

The heap algorithms provided by the C++ Standard Library can be summarized as follows:

- `make_heap()` rearranges an array into a heap.
- `push_heap()` and `pop_heap()` update an existing heap after inserting or removing an element.
- `sort_heap()` transforms a heap into an array that is ordered in increasing order.

These algorithms transform arrays; predicate-style algorithms such as `is_heap()` are also discussed.

Figure 9.1 provides an overview of the heap algorithms by means of a concrete example. For illustration purposes, the array is interpreted as a binary tree in the standard way; the underlying array-to-tree mapping is discussed in §9.1. Algorithms that stand in a typical caller–callee relationship in common implementations are shown in the same color.

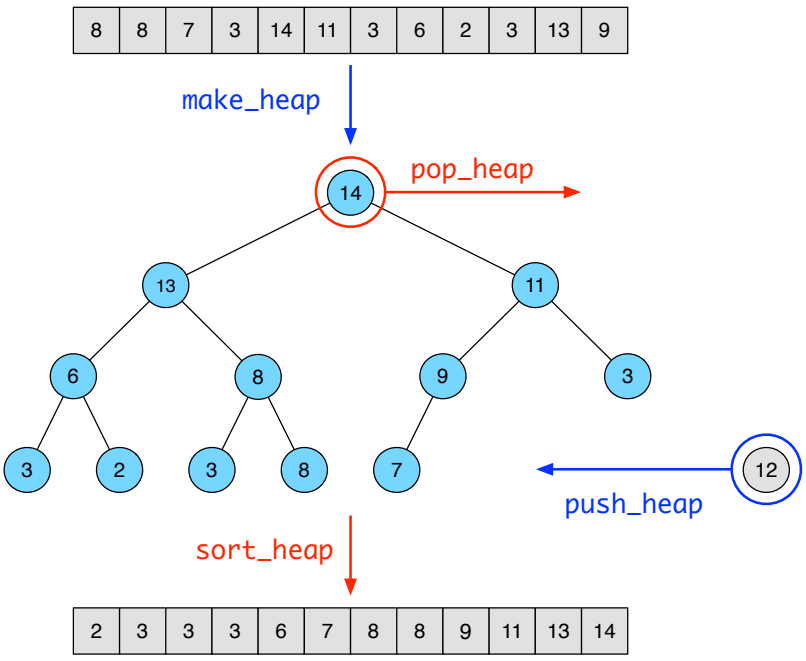


Figure 9.1.: Overview of heap algorithms

At a high level, the material covered in Figure 9.1 is organized as follows.

- In §9.1 we briefly recapitulate the basic concepts underlying heaps.
- In §9.2 we show how these concepts can be expressed in ACSL by means of logic functions and predicates.
- In §9.3 we verify two auxiliary functions that are used by several heap algorithms.
- The functions `is_heap_until` and `is_heap`, introduced in §9.4 and §9.5, allow one to check at run time whether a given array satisfies the heap property.
- In §9.6 we address the specific challenges that heap algorithms pose for formal reasoning. Although these algorithms preserve the multiset of elements globally, they may induce temporary local changes in multiplicity during intermediate steps. We therefore introduce predicates and lemmas that make such transient reordering behavior explicit while capturing multiset preservation as a global invariant.
- The algorithm `push_heap`, discussed in §9.7, inserts an element into a heap while preserving the heap property.
- Conversely, the algorithm `pop_heap` from §9.8 removes an element from a heap, again preserving the heap property.
- The algorithm `make_heap` from §9.9 rearranges an arbitrary array into a heap.
- Finally, the algorithm `sort_heap` from §9.10 transforms a heap into an array that is ordered in increasing order.

We begin in §9.1 with a precise definition of heaps. The ACSL logic functions and predicates that formalize the fundamental heap properties are then introduced in §9.2.

9.1. Basic heap concepts

The informal description of heaps given at the beginning of this chapter necessarily remains imprecise. It highlights the intended behavior of heap operations but does not yet identify the exact and verifiable properties that a range must satisfy in order to qualify as a heap.

A more explicit characterization can be found, for example, in the Apache C++ Standard Library User's Guide:[15]

> A heap is a binary tree in which every node is larger than the values associated with either child. A heap, and a binary tree in general, can be stored very efficiently in a vector by placing the children of node i at positions $2i + 1$ and $2i + 2$.

This description already suggests that the structure of a heap is determined entirely by simple arithmetic relations between array indices. More precisely, if a heap is represented as a contiguous range indexed from 0, the positions of parent and child nodes are given by the following functions.

$$\text{left child of index } i \qquad \text{child}_\text{l} : i \mapsto 2i + 1 \tag{9.1}$$

$$\text{right child of index } i \qquad \text{child}_\text{r} : i \mapsto 2i + 2 \tag{9.2}$$

and

$$\text{parent of index } i \qquad \text{parent} : i \mapsto \frac{i-1}{2} \tag{9.3}$$

[15] See http://stdcxx.apache.org/doc/stdlibug/14-7.html

These functions are not independent. For all integers i, they satisfy the following relations. Note that in ACSL integer division rounds towards zero (cf. [9, §2.2.4]).

$$\text{parent}(\text{child}_\text{l}(i)) = i \tag{9.4}$$

$$\text{parent}(\text{child}_\text{r}(i)) = i \tag{9.5}$$

To illustrate the usefulness of heaps, we consider on the following pages the following multiset of integers, using the notion of multisets introduced in Section 7.18.1.

$$X = \{2, 3, 3, 3, 6, 7, 8, 8, 9, 11, 13, 14\} \tag{9.6}$$

Figure 9.2 illustrates how the multiset from Equation (9.6) can be organized as a binary tree that respects the parent–child relations of a heap.

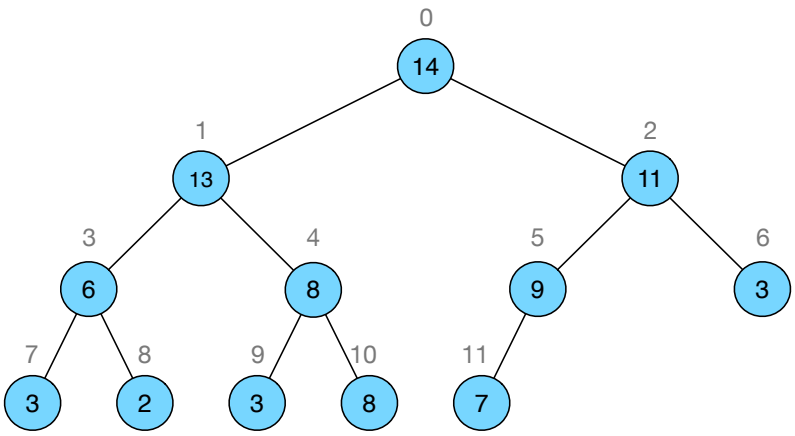


Figure 9.2.: Tree representation of the multiset X

The numbers shown outside the nodes in Figure 9.2 denote the array indices at which the corresponding values are stored. Using these indices, the tree representation can be flattened into a contiguous array, as shown in Figure 9.3.

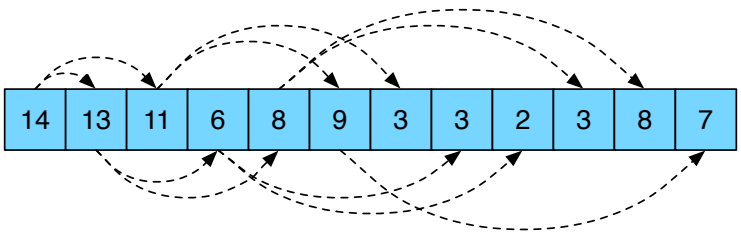


Figure 9.3.: Underlying array representation of the heap

At this point, it is worth emphasizing that the heap property constrains only local parent–child relations. It does not impose a unique global arrangement of the elements. In particular, a given multiset may admit several distinct heap realizations.

Consequently, a multiset does not determine a unique heap. Different arrangements of the same elements may satisfy the heap property, while inducing different tree shapes and array layouts. Figure 9.4 illustrates this by showing an alternative heap structure for the same multiset X.

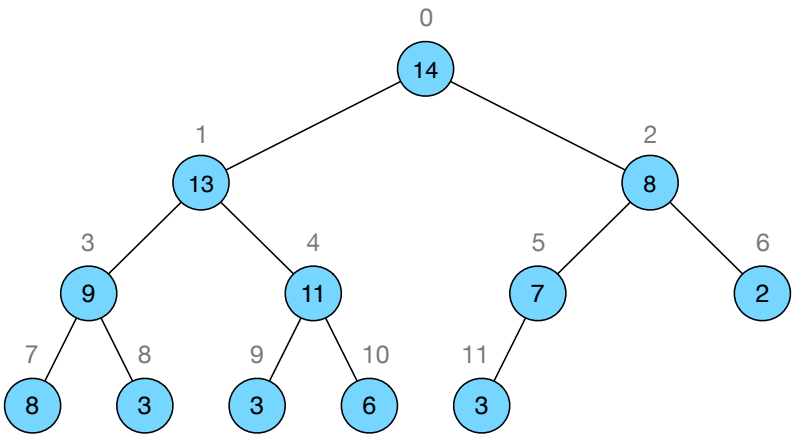


Figure 9.4.: An alternative heap representation of the multiset X

The corresponding array representation is given in Figure 9.5. Although the ordering of elements in the array differs from that in Figure 9.3, both arrays encode valid heaps and represent the same multiset.

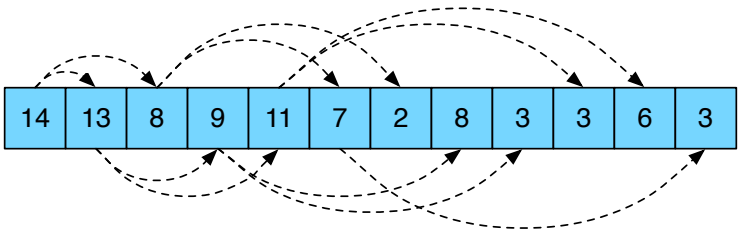


Figure 9.5.: Underlying array of the alternative heap representation

9.2. Representation of heap concepts in ACSL

We now formalize the index relations introduced in Section 9.1 using ACSL. The following listing defines three logic functions, `HeapLeft` [9.6], `HeapRight`, and `HeapParent`, which correspond directly to the mathematical definitions given in Equations (9.1), (9.2), and (9.3), respectively.

```
/*@
  logic integer HeapLeft(integer i) = 2*i + 1;

  logic integer HeapRight(integer i) = 2*i + 2;

  logic integer HeapParent(integer i) = (i-1) / 2;

  lemma HeapParent_Zero{L}: HeapParent(0) == 0;

  lemma Heap_ParentLeft:
    \forall integer p; 0 <= p  ==>  HeapParent(HeapLeft(p)) == p;

  lemma Heap_ParentRight:
    \forall integer p; 0 <= p  ==>  HeapParent(HeapRight(p)) == p;

  lemma Heap_ParentChild:
    \forall integer c, p;
      0 < c  ==>  HeapParent(c) == p  ==>
      (c == HeapLeft(p) || c == HeapRight(p));

  lemma Heap_Childs:
    \forall integer a, b;
      0 < a  ==>  0 < b                ==>
      HeapParent(a) == HeapParent(b)  ==>
      (a == b || a+1 == b || a == b+1);

  lemma Heap_ParentBounds:
    \forall integer c; 0 < c  ==>  0 <= HeapParent(c) < c;

  lemma Heap_ChildBounds:
    \forall integer p; 0 <= p  ==>  p < HeapLeft(p) < HeapRight(p);
*/
```

Listing 9.6: `HeapNodes` (ACSL)

In addition to these definitions, the listing contains several ACSL lemmas that capture essential algebraic properties of the index functions. In particular, the lemmas establish that

- the function `HeapParent` satisfies the identities (9.4) and (9.5), that is, applying `HeapParent` to either the left or right child of an index recovers the original index, and
- the function `HeapParent` acts as a *left inverse* of both `HeapLeft` and `HeapRight`, in the sense that composing `HeapParent` with either child function yields the identity on indices.[16]

These properties make explicit the close correspondence between the arithmetic index calculations used in heap implementations and their abstract tree-theoretic interpretation. They will be used repeatedly in subsequent specifications and proofs to relate array indices to parent–child relations in heaps.

Although the stated properties are mathematically elementary, it is important to record them explicitly as lemmas. Automated provers do not, in general, infer such relationships between user-defined logic functions and integer arithmetic without guidance. By making these properties available as named lemmas,

[16] See the section *Left and right inverses* at http://en.wikipedia.org/wiki/Inverse_function.

we ensure that subsequent proofs can rely on them uniformly when reasoning about parent–child relations and array indices in heap algorithms.

Building on these basic index definitions, we now introduce the predicate `Heap` [9.7], which formalizes the heap property for an array segment. Intuitively, this predicate expresses that for every valid index, the value stored at that index dominates the values stored at its children, whenever such children exist.

A fundamental consequence of this property is that the element stored at index 0 of a (maximum) heap is the largest element of the heap. This fact is stated explicitly by the lemma `Heap_Maximum` [9.7], which relates the predicate `Heap` [9.7] to the auxiliary predicate `MaxElement` [5.3].

```
/*@
  predicate Heap{L}(value_type* a, integer n) =
    \forall integer i; 0 < i < n  ==>  a[i] <= a[HeapParent(i)];

  lemma Heap_Maximum{L} :
    \forall value_type* a, integer n;
      0 < n  ==>  Heap(a, n)  ==>  MaxElement(a, n, 0);

 lemma Heap_Shrink{L}:
   \forall value_type *a, integer m, n;
     0 <= m <= n  ==>  Heap(a, n)   ==>  Heap(a, m);

 lemma Heap_Unchanged{K,L}:
   \forall value_type *a, integer n;
     0 <= n  ==>  Unchanged{K,L}(a, n)  ==>  Heap{K}(a, n)  ==>  Heap{L}(a, n);

 predicate
 HeapCompatible{L}(value_type* a, integer n, integer m, value_type v) =
   (0 <= m < n)                                    &&
   (0 <= HeapParent(m)  ==>  v <= a[HeapParent(m)])  &&
   (HeapLeft(m)  <  n   ==>  a[HeapLeft(m)]  <= v)   &&
   (HeapRight(m) <  n   ==>  a[HeapRight(m)] <= v);

  lemma HeapCompatible_Update{K,L}:
    \forall value_type *a, v, integer m, n;
       0 <= m < n                      ==>
       Heap{K}(a, n)                   ==>
       HeapCompatible{K}(a, n, m, v)   ==>
       ArrayUpdate{K,L}(a, n, m, v)    ==>  Heap{L}(a, n);

*/
```

Listing 9.7: `Heap` (ACSL)

The lemmas `Heap_Shrink` and `Heap_Unchanged` capture simple but frequently used reasoning principles. They allow the heap property to be transferred from a heap to a related (sub-)array, either by restricting the considered index range or by observing that modifications outside the relevant range do not affect the heap property.

Finally, the predicate `HeapCompatible` [9.7] characterizes the conditions under which updating a single element of a heap preserves the heap property. Such an update is modeled using the generic predicate `ArrayUpdate` [7.4], which expresses that exactly one array position is modified while all others remain unchanged. Together with the lemma `HeapCompatible_Update` [9.7], this yields a local reasoning principle for heap modifications. These results will play a central role in the verification of the algorithms `push_heap` [9.23] and `pop_heap` [9.30], where heap updates are realized by a sequence of local exchanges.

9.3. The auxiliary functions heap_parent and heap_child

This section introduces two auxiliary functions that support the implementation of heap algorithms. We begin with the function heap_parent [9.8], which serves as the C counterpart of the ACSL logic function HeapParent [9.6].

The correspondence is not exact. While HeapParent is defined for all integer indices, the executable function heap_parent deliberately avoids the boundary case of index 0, for which a parent node does not exist. This restriction reflects the intended use of heap_parent in heap algorithms, where it is applied only to valid child indices.

The interface of heap_parent is shown in the following listing. It specifies the expected behavior and the admissible range of inputs.

```
/*@
   terminates          \true;
   exits               \false;
   assigns             \nothing;

   ensures    parent:  \result == HeapParent(child);
 */
size_type heap_parent(size_type child);
```

Listing 9.8: Formal specification of heap_parent

The corresponding implementation is given next. It follows the specification closely and reduces the parent computation to a simple arithmetic expression.

```
size_type heap_parent(size_type child)
{
  return (0u < child) ?  (child - 1u) / 2u : 0u;
}
```

Listing 9.9: Implementation of heap_parent

We do not provide exact C counterparts for the logic functions HeapLeft [9.6] and HeapRight. Indeed, throughout the development we encounter only a single situation — in the implementation of pop_heap [9.30] — where such functions would appear to be useful.

Even in this case, however, direct access to the left and right child indices is not what is required. Instead, the algorithm needs to determine, for a given parent index p, a child index c at which the maximum of the two values a[HeapLeft(p)] and a[HeapRight(p)] is stored. This task is performed by the auxiliary function heap_child [9.10].

```
/*@
   requires   bounds:  0 <= p < n;
   requires   valid:   \valid(a + (0..n-1));

   terminates          \true;
   exits               \false;
   assigns             \nothing;

   ensures    bounds:  p < \result <= n;
   ensures    parent:  \result < n       ==>  p == HeapParent(\result);
   ensures    parent:  \result < n-1     ==>  HeapLeft(p)  < n-1;
   ensures    parent:  \result < n-1     ==>  HeapRight(p) < n;
   ensures    left:    HeapLeft(p)  < n  ==>  \result < n;
   ensures    right:   HeapRight(p) < n  ==>  \result < n;
   ensures    max:     HeapLeft(p)  < n  ==>  a[HeapLeft(p)]  <= a[\result];
   ensures    max:     HeapRight(p) < n  ==>  a[HeapRight(p)] <= a[\result];
   ensures    none:    \result == n      ==>  n <= HeapLeft(p);
   ensures    none:    \result == n      ==>  n <= HeapRight(p);
*/
size_type heap_child(const value_type* a, size_type n, size_type p);
```

Listing 9.10: Formal specification of heap_child

Note that the implementation of heap_child [9.11] explicitly accounts for the possibility that the computation of child indices may overflow. If such an overflow occurs, the function returns the value n, which signals that no valid child index is available.

```
size_type heap_child(const value_type* a, size_type n, size_type p)
{
  if (p + 1u <= n - p - 1u) {
    const size_type left  = 2u * p + 1u;
    const size_type right = left + 1u;

    if (right < n) {
      // case of two children: select child with maximum value
      return a[right] <= a[left] ? left : right;
    }
    else {
      // at most one child that comes before n-1 can exist
      return left;
    }
  }
  else {
    return n;
  }
}
```

Listing 9.11: Implementation of heap_child

9.4. The `is_heap_until` algorithm

The algorithm `is_heap_until` of the C++ Standard Library [18, §28.7.7.5] operates on generic sequences and determines the largest prefix that satisfies the heap property. For the purposes of this chapter, we adapt the generic formulation to arrays of type `value_type`. The resulting interface is given by the following signature.

```
size_type is_heap_until(const value_type* a, int n);
```

Given an array `a` of length `n`, the function `is_heap_until` returns the smallest index at which the heap property is violated. Equivalently, it returns the length of the largest prefix `a[0..k-1]` that forms a heap in the sense formalized in Section 9.1. If the entire array satisfies the heap property, the function returns `n`.

In particular, when applied to the array shown in Figure 9.3, the function `is_heap_until` returns the value `n`, indicating that the array represents a valid heap.

9.4.1. Formal specification of `is_heap_until`

The formal specification of `is_heap_until` is given in the following listing. The return value `\result` denotes the boundary of the maximal heap prefix: the array segment `a[0..\result-1]` satisfies the heap predicate `Heap` [9.7].

In addition, the postcondition `last` characterizes the returned index as the *first* position at which the heap property fails. More precisely, for every index `i` with `\result <= i < n`, the array prefix `a[0..i]` does not satisfy the predicate `Heap` [9.7]. This ensures that `\result` is maximal and not merely a valid heap boundary.

```
/*@
   requires   valid:  \valid_read(a + (0..n-1));

   terminates         \true;
   exits              \false;
   assigns            \nothing;

   ensures    bound:  0 <= \result <= n;
   ensures    heap:   Heap(a, \result);
   ensures    last:   \forall integer i; \result < i <= n ==> !Heap(a, i);
*/
size_type is_heap_until(const value_type* a, size_type n);
```

Listing 9.12: Formal specification of `is_heap_until`

9.4.2. Implementation of `is_heap_until`

The following listing presents an implementation of the function `is_heap_until`.

```
size_type is_heap_until(const value_type* a, size_type n)
{
  size_type parent = 0u;

  /*@
    loop invariant bound:    0 <= parent < child <= n+1;
    loop invariant parent:   parent == HeapParent(child);
    loop invariant heap:     Heap(a, child);
    loop invariant not_heap: a[parent] < a[child] ==>
                             \forall integer i; child < i <= n ==> !Heap(a, i);
    loop assigns child, parent;
    loop variant n - child;
  */
  for (size_type child = 1u; child < n; ++child) {
    if (a[parent] < a[child]) {
      return child;
    }

    if ((child % 2u) == 0u) {
      ++parent;
    }
  }

  return n;
}
```

Listing 9.13: Implementation of `is_heap_until`

The algorithm starts at index 1, which is the smallest index at which a child node may occur in the array-based representation of a heap. For each such index, the algorithm determines the corresponding parent index and checks whether the heap property is satisfied, that is, whether the value stored at the parent index is greater than or equal to the value stored at the child index.

If this condition is violated for some child index `i`, the algorithm terminates immediately and returns `i`, which marks the first position at which the heap property fails. If no such violation is encountered and the end of the array is reached, the algorithm returns the value `n`, indicating that the entire array satisfies the heap property.

9.5. The is_heap algorithm

The algorithm is_heap of the C++ Standard Library [18, §28.7.7.5] determines whether a given sequence satisfies the heap property. In the standard library, the algorithm is defined for generic iterator ranges. For the purposes of this chapter, we specialize the algorithm to operate on arrays of type value_type. The resulting interface is given by the following signature.

```
bool is_heap(const value_type* a, int n);
```

Given an array a of length n, the function is_heap returns **true** precisely if the array represents a heap in the sense defined in Section 9.1. That is, all parent–child relations induced by the array indices satisfy the heap property. In particular, when applied to the array shown in Figure 9.3, the function is_heap returns **true**.

9.5.1. Formal specification of is_heap

The formal specification of is_heap is shown in the following listing. The contract expresses a direct correspondence between the return value of the function and the abstract heap predicate. More precisely, the function returns **true** if and only if the input array satisfies the predicate Heap [9.7].

```
/*@
   requires   valid:  \valid_read(a +(0..n-1));

   terminates         \true;
   exits              \false;
   assigns            \nothing;

   ensures    heap:   \result <==> Heap(a, n);
*/
bool is_heap(const value_type* a, size_type n);
```

Listing 9.14: Formal specification of is_heap

9.5.2. Implementation of is_heap

The implementation of is_heap is shown in the following listing. Rather than rechecking the heap property directly, the function delegates the actual work to is_heap_until [9.12].

```
bool is_heap(const value_type* a, size_type n)
{
  return is_heap_until(a, n) == n;
}
```

Listing 9.15: Implementation of is_heap

The implementation simply tests whether the index returned by is_heap_until coincides with the array length n. This design emphasizes the close conceptual relationship between the two algorithms: is_heap acts as a Boolean wrapper around is_heap_until, trading detailed positional information for a simple yes-or-no answer.

9.6. Reorderings and temporary changes in multiplicity

Heap algorithms present a particular challenge for formal reasoning: while they do not alter the multiset of values as a whole, they may produce temporary changes in the multiplicity of values at specific positions in an array during intermediate steps. In other words, the count of a particular value at a given position can vary for a short period even though the overall collection of elements remains the same up to permutation.

To support rigorous specification and verification of such behavior we introduce a collection of predicates and supporting lemmas that make it possible to describe temporary local changes and the global invariant of multiset preservation.

9.6.1. Formalizing temporary changes in multiplicity

The predicate `MultisetAdd` [9.16] expresses that the number of occurrences of a specific value in an array has increased by one between two program points `K` and `L`.

```
/*@
  predicate MultisetAdd{K,L}(value_type* a, integer n, value_type v) =
    Count{L}(a, 0, n, v) == Count{K}(a, 0, n, v) + 1;

  predicate MultisetMinus{K,L}(value_type* a, integer n, value_type v) =
    MultisetAdd{L,K}(a, n, v);

  predicate MultisetRetain{K,L}(value_type* a, integer n, value_type v) =
    Count{K}(a, 0, n, v) == Count{L}(a, 0, n, v);

  lemma MultisetAdd_Distinct{K,L}:
    \forall value_type *a, v, integer m, n;
      0 <= m < n                        ==>
      At{K}(a, m) != v                  ==>  At{L}(a, m) == v  ==>
      MultisetReorder{K,L}(a, 0, m)     ==>
      MultisetReorder{K,L}(a, m+1, n)   ==>  MultisetAdd{K,L}(a, n, v);

  lemma MultisetMinus_Distinct{K,L}:
    \forall value_type *a, v, integer m, n;
      0 <= m < n                        ==>
      At{K}(a, m) == v                  ==>  At{L}(a, m) != v  ==>
      MultisetReorder{K,L}(a, 0, m)     ==>
      MultisetReorder{K,L}(a, m+1, n)   ==>  MultisetMinus{K,L}(a, n, v);

  lemma MultisetRetain_Distinct{K,L}:
    \forall value_type *a, v, integer m, n;
      0 <= m < n                        ==>
      At{K}(a, m) != v                  ==>  At{L}(a, m) != v  ==>
      MultisetReorder{K,L}(a, 0, m)     ==>
      MultisetReorder{K,L}(a, m+1, n)   ==>  MultisetRetain{K,L}(a, n, v);
*/
```

Listing 9.16: `MultisetOperations` (ACSL)

The predicate `MultisetMinus`, in contrast, expresses that the number of occurrences of a specific value in an array has decreased by one between the program points `K` and `L`. For convenience, `MultisetMinus` is defined by invoking `MultisetAdd` with the labels `K` and `L` exchanged.

Finally, the predicate `MultisetRetain` expresses that the number of occurrences of a given value remains unchanged between two program points. Together, these predicates make it possible to describe local and temporary changes in multiplicity while reasoning about the evolution of an array during heap operations.

To support automated reasoning, we also provide a number of lemmas that characterize conditions under which the respective predicates hold. These lemmas are formulated using the predicate `MultisetReorder` [7.59] and the logic function `At` [7.3].

9.6.2. Simple properties of temporary changes in multiplicity

The predicate `MultisetRetainRest` [9.17] builds on the predicate `MultisetRetain` [9.16] to express that, between two program points `K` and `L`, all values of an array occur with the same multiplicity, except possibly for two distinguished values `u` and `v`. In other words, only the occurrences of `u` and `v` are allowed to change, while the multiplicities of all other values are preserved.

```
/*@
  predicate MultisetRetainRest{K,L}(value_type* a, integer n, value_type  v,
      value_type w) =
    \forall value_type x;
      x != v  ==>  x != w  ==>  MultisetRetain{K,L}(a, n, x);

  lemma Multiset_AddMinusRetain{K,L,M}:
    \forall value_type *a, u, integer n;
      MultisetAdd{K,L}(a, n, u)    ==>
      MultisetMinus{L,M}(a, n, u)  ==>  MultisetRetain{K,M}(a, n, u);

  lemma Multiset_MinusAddRetain{K,L,M}:
    \forall value_type *a, u, integer n;
      MultisetMinus{K,L}(a, n, u) ==>
      MultisetAdd{L,M}(a, n, u)   ==>  MultisetRetain{K,M}(a, n, u);

  lemma Multiset_AddMinusRetainReorder{K,L,M}:
    \forall value_type *a, u, v, integer n;
      u != v                              ==>
      MultisetAdd{K,L}(a, n, u)           ==>
      MultisetMinus{K,L}(a, n, v)         ==>
      MultisetRetainRest{K,L}(a, n, u, v) ==>
      MultisetAdd{L,M}(a, n, v)           ==>
      MultisetMinus{L,M}(a, n, u)         ==>
      MultisetRetainRest{L,M}(a, n, v, u) ==>  MultisetReorder{K,M}(a, n);
*/
```

Listing 9.17: `MultisetRetainRest` (ACSL)

The accompanying lemmas capture simple cancellation properties of temporary changes in multiplicity. They express conditions under which an increase in the number of occurrences of a value at one program point, described by `MultisetAdd` [9.16], is compensated by a corresponding decrease at a later program point, described by `MultisetMinus`. Taken together, these lemmas allow local changes in multiplicity between intermediate program points to be reconciled with the preservation of the overall multiset.

9.6.3. Combining temporary changes in multiplicity

Temporary changes in multiplicity occur so frequently in the central heap algorithms `push_heap` [9.23] and `pop_heap` [9.30] that it is useful to capture them by a dedicated predicate. For this purpose, we introduce the predicate `MultisetParity` [9.18].

The predicate `MultisetParity` [9.18] describes a very specific pattern of local change between two program points. Given two distinct values, it expresses that

- the number of occurrences of the first value increases by one,
- the number of occurrences of the second value decreases by one, and
- the multiplicities of all other values remain unchanged.

The term *parity* reflects the fact that the total number of elements is preserved, and that the two changes compensate each other. Such paired changes arise naturally in heap algorithms, for example when a value is moved upward or downward in the array and temporarily replaces another value.

```
/*@
  predicate MultisetParity{K,L}(value_type* a, integer n, value_type u, value_type
       v) =
      MultisetAdd{K,L}(a, n, u)    &&
      MultisetMinus{K,L}(a, n, v)  &&
      MultisetRetainRest{K,L}(a, n, u, v);


  lemma MultisetParity_UnchangedFirst{K,L,M}:
    \forall value_type *a, u, v, integer n;
      u != v                          ==>
      Unchanged{K,L}(a, n)            ==>
      MultisetParity{L,M}(a, n, u, v)  ==>  MultisetParity{K,M}(a, n, u, v);

  lemma MultisetParity_UnchangedSecond{K,L,M}:
    \forall value_type *a, u, v, integer n;
      u != v                          ==>
      MultisetParity{K,L}(a, n, u, v)  ==>
      Unchanged{L,M}(a, n)            ==>  MultisetParity{K,M}(a, n, u, v);

  lemma MultisetParity_MultisetReorder{K,L,M}:
    \forall value_type *a, u, v, integer n;
      u != v                          ==>
      MultisetParity{K,L}(a, n, u, v)  ==>
      MultisetParity{L,M}(a, n, v, u)  ==>  MultisetReorder{K,M}(a, n);

  lemma MultisetParity_Combined{K,L,M}:
    \forall value_type *a, u, v, w, integer n;
      u != v                          ==>
      u != w                          ==>
      v != w                          ==>
      MultisetParity{K,L}(a, n, u, v)  ==>
      MultisetParity{L,M}(a, n, w, u)  ==>  MultisetParity{K,M}(a, n, w, v);
*/
```

Listing 9.18: `MultisetParity` (ACSL)

Using the predicate `MultisetParity` [9.18], we can formulate a number of lemmas that describe how temporary changes in multiplicity interact with reorderings. For example, the situation in which two consecutive parity changes cancel each other is captured by the lemma `MultisetParity_MultisetReorder`. As a result, the net effect between the initial and final program points is a pure reordering of the array, that is, preservation of the overall multiset.

9.6.4. How do temporary changes in multiplicity arise?

The simplest way to create a temporary change in multiplicity is to update a single array element with a different value. Such an update removes one occurrence of the original value and introduces one occurrence of the new value, thereby producing a compensating pair of changes and hence a *parity change* in the multiset of array elements.

Accordingly, and in analogy to the predicate `ArrayUpdate` [7.4], we introduce in the following Listing the predicate `MultisetUpdate`. This predicate characterizes array updates that give rise to a parity change in the sense formalized by `MultisetParity` [9.18].

```
/*@
  predicate MultisetUpdate{K,L}(value_type* a, integer n, integer i, value_type v)
      =
    \let u = At{K}(a, i);
    u != v                          &&
    0 <= i < n                      &&
    MultisetReorder{K,L}(a, 0, i)   &&
    MultisetReorder{K,L}(a, i+1, n) &&
    MultisetParity{K,L}(a, n, v, u);

  lemma ArrayUpdate_MultisetUpdate{K,L}:
    \forall value_type *a, v, integer n, i;
      ArrayUpdate{K,L}(a, n, i, v)  ==>  MultisetUpdate{K,L}(a, n, i, v);
*/
```

Listing 9.19: `MultisetUpdate` (ACSL)

Lemma `ArrayUpdate_MultisetUpdate` [9.19] makes this relationship precise. It formalizes the fact that updating an array element with a different value induces a temporary change in multiplicity, while preserving the total number of elements.

9.7. The push_heap algorithm

The push_heap algorithm extends an existing heap by one additional element. More precisely, it assumes that the first $n-1$ elements of an array of length n already form a heap, and it incorporates the element a[n-1] into this structure.

In the C++ Standard Library [18, §28.7.7.1], push_heap operates on a range of random access iterators. In contrast, our variant is specialized to arrays of value_type. Accordingly, we use the following signature:

```
void push_heap(value_type* a, size_type n);
```

The algorithm requires that n is greater than or equal to 1 and that the subarray a[0..n-2] forms a heap. It then rearranges the elements of a[0..n-1] such that the entire array satisfies the heap property. In this sense, the algorithm *pushes* the element a[n-1] onto the given heap.

9.7.1. Formal specification of push_heap

The following listing presents our formal specification of push_heap. A central role is played by the postcondition reorder, which states that push_heap is not allowed to change the multiplicity of the array elements.

Without this postcondition, the specification would be far too weak. For instance, an implementation that assigns the value 0 to every array element would trivially satisfy the postcondition heap, yet would clearly violate the intended semantics of the algorithm.

```
/*@
   requires   nonempty:  0 < n;
   requires   valid:     \valid(a + (0..n-1));
   requires   heap:      Heap(a, n-1);

   terminates           \true;
   exits                \false;
   assigns              a[0..n-1];

   ensures    heap:     Heap(a, n);
   ensures    reorder:  MultisetReorder{Old,Here}(a, n);
*/
void push_heap(value_type* a, size_type n);
```

Listing 9.20: Formal specification of push_heap

Pushing an element onto a heap typically requires the rearrangement of several array elements, as illustrated in Figures 9.21 and 9.22. It is therefore essential to express that push_heap only permutes the elements of the array, without introducing or removing values. To formalize this requirement, we reuse the predicate MultisetReorder [7.59], which precisely captures the notion that the result array is a reordering of the original one.

9.7.2. Implementation of push_heap

The following two figures illustrate the effect of `push_heap` on an array. The array is represented as a tree whose nodes are colored blue or grey, indicating heap and non-heap nodes, respectively. Figure 9.21 shows the heap from Figure 9.2 together with the additional element `12` that is to be pushed onto the heap. At this stage, the element `12` is the last element of the array and is not yet part of the heap.

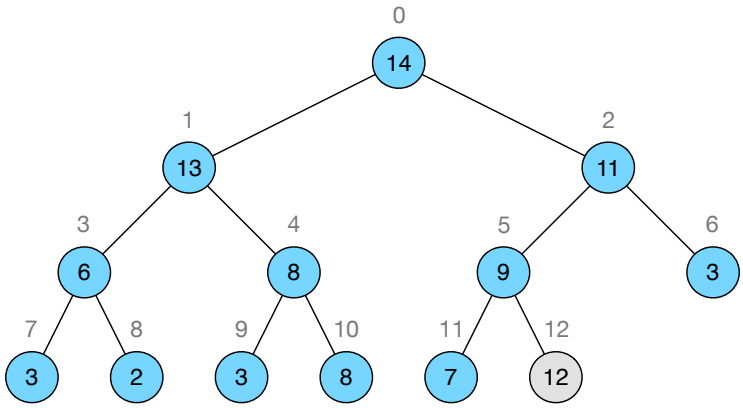


Figure 9.21.: Example heap before the call of `push_heap`

Figure 9.22 shows the array after the call of `push_heap`. All nodes are now colored blue, indicating that they belong to the heap. The dashed nodes mark those heap elements whose positions have changed during the execution of the algorithm. The element to be pushed has reached its correct position in the heap. The arrows indicate the *cyclic reordering* of array elements that leads to this final configuration.

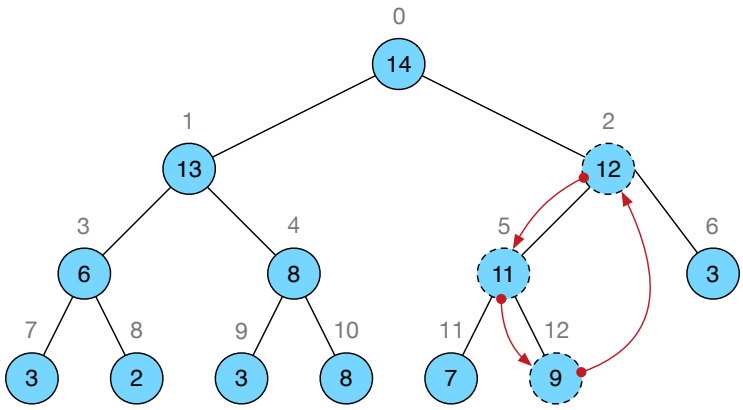


Figure 9.22.: Example heap after the call of `push_heap`

Verifying our implementation of `push_heap` [9.23] is a non-trivial task. To structure the discussion, we divide the algorithm into three conceptual parts. We refer to the central loop as the *main act*, and to the code preceding and following it as the *prologue* and the *epilogue*, respectively.

The `heap` property required by the specification of `push_heap` [9.20] can already be established in the prologue. In contrast, the `reorder` property holds only at the function boundaries and is temporarily violated while `push_heap` manipulates the array. More precisely, the `reorder` property is lost in the prologue and replaced by a slightly more general invariant that is maintained throughout the main act. From this invariant, the original `reorder` property is recovered in the epilogue.

The effect of each stage on the underlying array will be illustrated by figures showing the array in tree form. All figures are based on the `push_heap` example introduced in Figure 9.21.

```
void push_heap(value_type* a, size_type n)
{
  if (1u < n) { // otherwise nothings needs to be done
    size_type c = n - 1u;
    size_type p = heap_parent(c);
    //@ assert parent: p == HeapParent(c);

    if (a[p] < a[c]) {
      const value_type v  = a[c];
      a[c] = a[p];
      //@ assert update:   ArrayUpdate{Pre,Here}(a, n, c, a[p]);
      //@ assert heap:     Heap(a, n);
      //@ assert reorder:  MultisetParity{Pre,Here}(a, n, a[p], v);

      /*@
        loop invariant bound:      0 <= c < n-1;
        loop invariant heap:       Heap(a, n);
        loop invariant less:       a[c] < v;
        loop invariant parent:     p == HeapParent(c);
        loop invariant reorder:    MultisetParity{Pre,Here}(a, n, a[c], v);
        loop invariant unchanged:  Unchanged{Pre,Here}(a, c);
        loop assigns               c, p, a[0..n-1];
        loop variant               c;
      */
      for (c = p, p = heap_parent(c); 0u < c && a[p] < v;
           c = p, p = heap_parent(c)) {
        //@ ghost value_type ac =  a[c];
        if (a[c] < a[p]) {
          a[c] = a[p];
          //@ assert update:     ArrayUpdate{LoopCurrent,Here}(a, n, c, a[p]);
          //@ assert update:     MultisetUpdate{LoopCurrent,Here}(a, n, c, a[p]);
          //@ assert bound:      0 <= c < n;
          //@ assert less:       ac < a[c] < v;
          //@ assert reorder:    MultisetParity{Pre,Here}(a, n, a[c], v);
        }
      }

      //@ ghost Epilogue: ;
      //@ assert heap: 0 == c || v <= a[HeapParent(c)];
      //@ ghost  value_type  ac = a[c];
      //@ assert update:     ac == At{Epilogue}(a, c) < v;
      //@ assert reorder:    MultisetParity{Pre,Here}(a, n, ac, v);
      a[c] = v;
      //@ assert update:   ArrayUpdate{Epilogue,Here}(a, n, c, v);
      //@ assert heap:     HeapCompatible(a, n, c, v);
      //@ assert heap:     Heap(a, n);
      //@ assert update:   MultisetUpdate{Epilogue,Here}(a, n, c, v);
      //@ assert reorder:  MultisetParity{Epilogue,Here}(a, n, v, ac);
      //@ assert reorder:  MultisetReorder{Pre,Here}(a, n);
    }
  }
}
```

Listing 9.23: Implementation of push_heap

9.7.2.1. Prologue

In the prologue, the algorithm first checks that the initial heap is nonempty and initializes the required auxiliary variables. It then compares the value `a[c]`, which is the element to be pushed onto the heap and the last element of the array, with its parent node. If `a[c]` already satisfies the heap condition with respect to its parent, no further action is required at this stage.

Otherwise, the value `a[c]` is temporarily stored in the variable `v`, and the parent value `a[p]` is assigned to `a[c]`. This assignment is performed only if the two values differ. As a consequence, its effect can be described precisely using the predicate `ArrayUpdate` [7.4].

Figure 9.24 illustrates the main effect of the prologue. Here, and in all subsequent figures, the currently active node is highlighted.

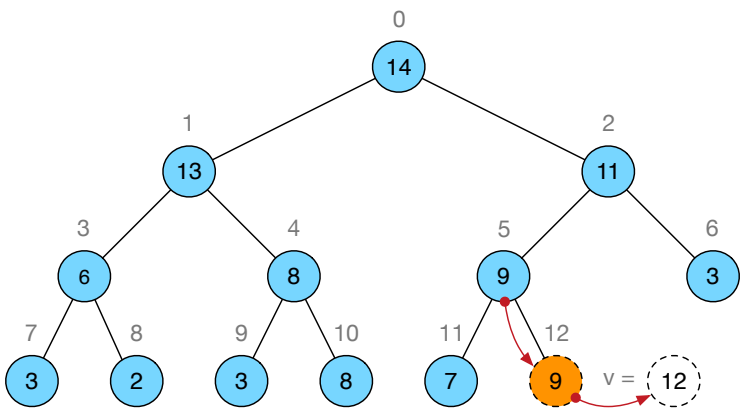


Figure 9.24.: Heap after the prologue of `push_heap`

At this point, two essential properties have been established.

1. The array `a[0..n-1]` satisfies the heap property.
2. With respect to the multiplicities of values in `a[0..n-1]`, the following changes have occurred:
    - the original value `a[c]` occurs one time less,
    - the original value `a[p]` occurs one time more,
    - all other values occur with the same multiplicity as before.

The first property is captured by the assertion `heap`. The temporary imbalance in element multiplicities described by the second property is expressed using the predicate `MultisetParity` [9.18] in the assertion `reorder`.

9.7.2.2. Main act

Figure 9.25 shows the effect of the main act of `push_heap` on the example heap. In this example, only a single iteration of the loop is required until a node is reached whose parent value `a[p]` is greater than or equal to `v`. Assignments performed during the iteration are indicated by dashed arrows.

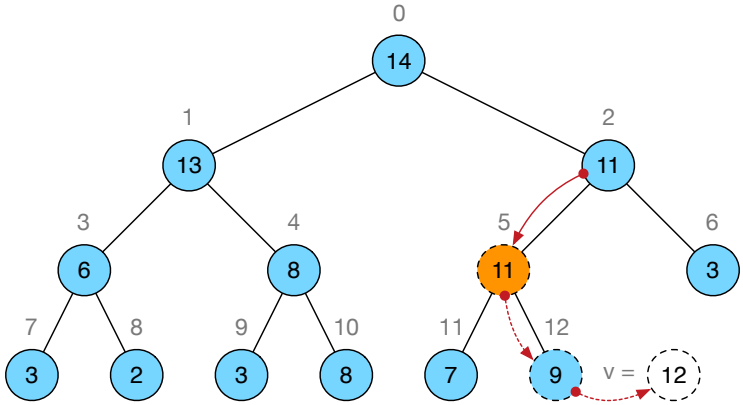


Figure 9.25.: Heap after the main act of `push_heap`

In the main act, the algorithm starts at the parent position reached in the prologue. This index is stored in the variable `c`, referring to the current *child* position. Compared to the pre-state of `push_heap`, the array `a[0..n-1]` now satisfies the following multiplicity relation:

- the value `v` occurs one time less,
- the value `a[c]` occurs one time more,
- all other values occur with the same multiplicity as in the pre-state.

As long as the index `c` is not the root of the heap and its (necessarily existing) parent value `a[p]` is strictly less than `v`, no admissible position for `v` has yet been found without violating the heap property.

Within the loop body, the following cases are distinguished.

- If `a[c]` is strictly less than `a[p]`, the parent value `a[p]` is copied to `a[c]`. This assignment preserves the heap property of the array. With respect to multiplicities, the value `a[p]` now occurs one time more than in the pre-state, the overwritten value `a[c]` occurs as often as in the pre-state, and the value `v` continues to occur one time less. The next iteration proceeds with `c` updated to `p`.

  At the level of specification, these effects can be decomposed into elementary updates captured by the predicates `ArrayUpdate` [7.4] and `MultisetUpdate` [9.19]. The lemma `MultisetParity_Combined` [9.18] indicates how these local changes combine into the maintained global relation. This discussion is intended to convey the structure of the invariants, not to present a detailed proof.

- Otherwise, since `c` is a child of `p`, we necessarily have `a[c]` equal to `a[p]`. In this case no assignment is required, and the next iteration proceeds after updating `c` to `p`.

At the beginning of each iteration, the same multiplicity relation as above holds. Compared to the pre-state of `push_heap`, the array `a[0..n-1]`

- contains the value `v` one time less,
- contains the value `a[c]` one time more,
- while all other values are unchanged in their multiplicity.

9.7.2.3. Epilogue

Figure 9.26 shows the final assignment performed by `push_heap` and the resulting configuration of the heap. The figure highlights the completion of the cycle initiated in Figure 9.24. It also makes clear that the value `v` behaves like an additional element temporarily circulating through the array during this cycle.

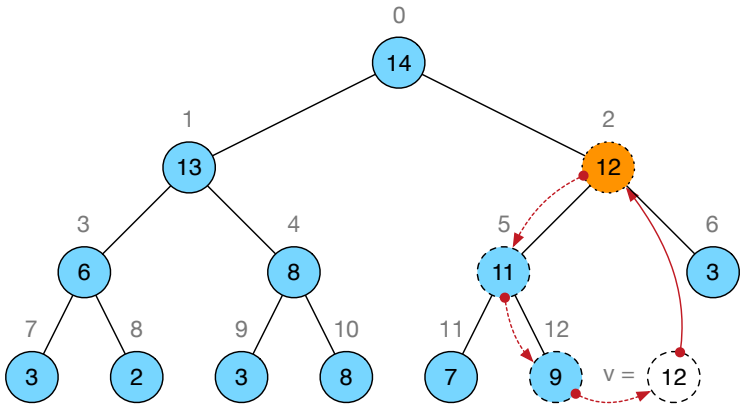


Figure 9.26.: Heap after the epilogue of `push_heap`

At this point, the algorithm has reached an index `c` at which assigning the value `v` preserves the heap property. This condition is expressed formally by the predicate `HeapCompatible` [9.7].

At the same time, this final assignment corrects the temporary imbalance in value multiplicities that was introduced earlier. The value `v`, which up to now occurred one time less than in the pre-state, is reinserted into the array, while the value currently stored at `a[c]` loses its extra occurrence. As a result, the desired `reorder` property of `push_heap` is restored.

The formal justification that this correction indeed yields a proper reordering of the array elements relies on the lemma `MultisetParity_MultisetReorder` [9.18].

9.8. The pop_heap algorithm

The pop_heap algorithm removes the largest element from a heap. More precisely, it moves the first element of the array, which contains the maximum value of the heap, to the end of the underlying sequence.

In the C++ Standard Library [18, §28.7.7.2], pop_heap operates on a range of random access iterators. In contrast, our variant is specialized to arrays of value_type. Accordingly, we use the following signature:

```
void pop_heap(value_type* a, size_type n);
```

The pop_heap algorithm requires that n is greater than or equal to 1 and that the array a[0..n-1] forms a heap. It then rearranges the elements of a[0..n-1] such that the following properties hold:

- a[n-1] = \old(a[0]), that is, the largest element of the original heap is moved to the end of the array,
- the subarray a[0..n-2] satisfies the heap property.

In this sense, the algorithm *pops* the largest element from the heap.

9.8.1. Formal specification of pop_heap

On the basis of the preceding semi-formal description, we now present the formal specification of pop_heap.

```
/*@
   requires   bounds:   0 < n;
   requires   valid:    \valid(a + (0..n-1));
   requires   heap:     Heap(a, n);

   terminates           \true;
   exits                \false;
   assigns              a[0..n-1];

   ensures    heap:     Heap(a, n-1);
   ensures    result:   a[n-1] == \old(a[0]);
   ensures    max:      MaxElement(a, n, n-1);
   ensures    reorder:  MultisetReorder{Old,Here}(a, n);
*/
void pop_heap(value_type* a, size_type n);
```

Listing 9.27: Formal specification of pop_heap

The contract expresses three essential aspects of the algorithm. First, it states that the last element of the array a[n-1] contains the value that was stored at the root of the heap in the pre-state. Second, it requires that the prefix a[0..n-2] forms a heap after the function call. Third, it enforces that the function only reorders existing elements of the array and does not introduce or remove values.

As in the case of push_heap, the last requirement is crucial. Without an explicit reordering condition, implementations that destroy or duplicate values could satisfy the heap-related postconditions while violating the intended semantics of the algorithm.

9.8.2. Implementation of pop_heap

In an abstract sense, the algorithm pop_heap is closely related to push_heap, but proceeds in the opposite direction. In the case of push_heap, we start at the last array element and climb upwards through the tree until a suitable position for the new value is found. At each step, the value of the current parent node is moved downwards to the position from which we just ascended.

For pop_heap, this process is reversed. We start at the root of the tree and descend from there by repeatedly selecting an appropriate child node. At each step, the value of the selected child is lifted to the current node. This continues until a node is reached at which the value of the last array element can be inserted without violating the heap property. Once this insertion has been completed, the original root value — the maximum element of the heap — can safely be placed at the last position of the array.

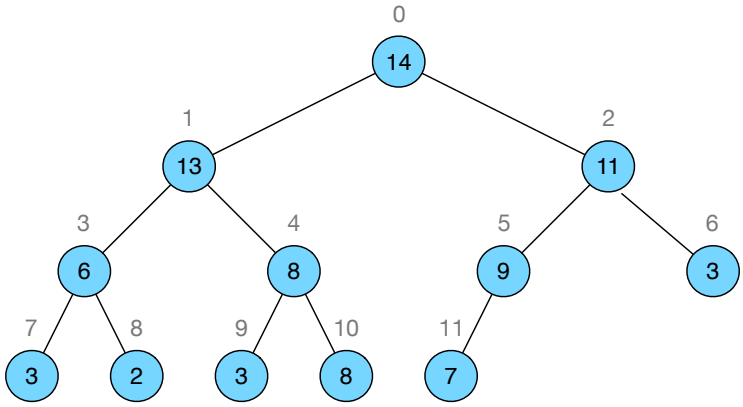


Figure 9.28.: Heap before the call of pop_heap

Figure 9.29 depicts the state of the array after the execution of pop_heap. The maximum element, originally stored at the root, has been moved to the last array position and is no longer part of the heap. Conversely, the last array element has been inserted into the heap by descending along a suitable path. The dashed nodes indicate those heap positions whose values have changed during the operation. The arrows visualize the *cyclic reordering* of array elements that implements this transformation.

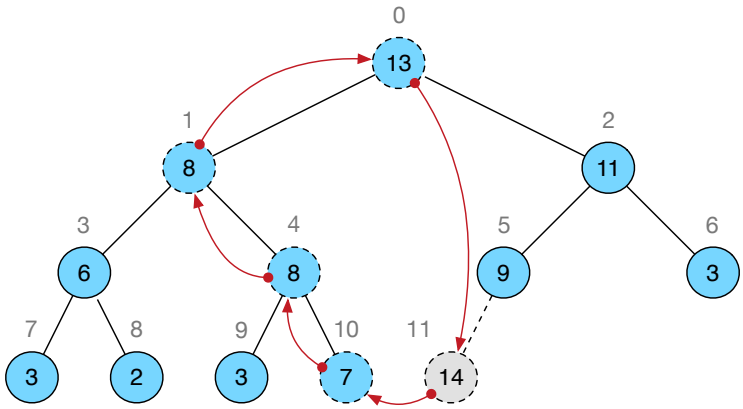


Figure 9.29.: Heap after the call of pop_heap

As in the case of push_heap [9.23], we subdivide the discussion of the implementation of pop_heap [9.30] into three parts: a prologue, the main act, and an epilogue.

```
void pop_heap(value_type* a, size_type n)
{
  if (1u < n) {
    //@ assert max:  MaxElement(a, n, 0);
    if (a[n - 1u] < a[0u]) { // otherwise a[0] == a[n-1] and nothing to be done
      size_type p = 0u;
      const value_type v = a[n - 1u];
      a[n - 1u] = a[p];
      //@ assert max:       a[n-1] == a[p];
      //@ assert update:    ArrayUpdate{Pre,Here}(a, n, n-1, a[p]);
      //@ assert heap:      Heap(a, n-1);
      //@ assert reorder:   MultisetParity{Pre,Here}(a, n, a[p], v);

      size_type c = heap_child(a, n - 1u, p);

      /*@
        loop invariant bounds:     0 <= p < c <= n-1;
        loop invariant parent:     c <  n-1 ==> p == HeapParent(c);
        loop invariant child:      c <  n-1 ==> HeapLeft(p) < n-1;
        loop invariant child:      c == n-1 ==> n-1 <= HeapLeft(p);
        loop invariant child:      HeapLeft(p)  < n-1 ==> a[HeapLeft(p)]  <= a[c];
        loop invariant child:      HeapRight(p) < n-1 ==> a[HeapRight(p)] <= a[c];
        loop invariant unchanged:  Unchanged{LoopEntry,Here}(a, p, n);
        loop invariant update:     a[p] == a[HeapParent(p)];
        loop invariant max:        UpperBound(a, n, a[n-1]);
        loop invariant reorder:    MultisetParity{Pre,Here}(a, n, a[p], v);
        loop invariant heap:       v < a[p];
        loop invariant heap:       Heap(a, n-1);
        loop assigns               p, c, a[0..n-2];
        loop variant               n - p;
       */
      for (; c < n - 1u && v < a[c];  p = c, c = heap_child(a, n - 1u, p)) {
        //@ assert max:          a[p] <= a[n-1];
        //@ assert heap:         a[c] <= a[p];
        if (a[c] < a[p]) {
          a[p] = a[c];
          //@ assert update:     ArrayUpdate{LoopCurrent,Here}(a, n, p, a[c]);
          //@ assert update:     ArrayUpdate{LoopCurrent,Here}(a, n-1, p, a[c]);
          //@ assert update:     MultisetUpdate{LoopCurrent,Here}(a, n, p, a[c]);
          //@ assert update:     a[c] == At{LoopCurrent}(a, c);
          //@ assert unchanged:  Unchanged{LoopEntry,Here}(a, p+1, n);
          //@ assert compatible: HeapCompatible(a, n-1, p, a[p]);
          //@ assert reorder:    MultisetParity{Pre,Here}(a, n, a[c], v);
        }
      }

      //@ ghost Epilogue: ;
      //@ assert child:      c == n-1  ||  a[c] <= v;
      //@ assert parent:     p <  n-1  &&  v < a[p];
      //@ assert compatible: HeapCompatible(a, n-1, p, v);
      a[p] = v;
      //@ assert update:     ArrayUpdate{Epilogue,Here}(a, n, p, v);
      //@ assert update:     MultisetUpdate{Epilogue,Here}(a, n, p, v);
      //@ assert reorder:    MultisetReorder{Pre,Here}(a, n);
      //@ assert heap:       Heap(a, n-1);
      //@ assert max:        UpperBound(a, n, a[n-1]);
    }
  }
}
```

Listing 9.30: Implementation of pop_heap

9.8.2.1. Prologue

The prologue of `pop_heap` [9.30] performs a number of preliminary tasks. First, it checks whether the heap contains at least two elements. It then initializes the required local variables and tests whether the last array element is equal to the maximum element of the heap, which resides at index `p == 0`. If this is not the case, the last array element is set aside for later use and stored in the variable `v`. Finally, the value `a[p]` is copied to its final position at the end of the array. This assignment is performed only if the respective values differ, which allows its effect to be formally described, as in the case of `push_heap` [9.23], by means of the predicate `ArrayUpdate` [7.4].

Figure 9.31 illustrates the effect of the prologue using our running example. The root of the heap is highlighted to indicate the currently active node.

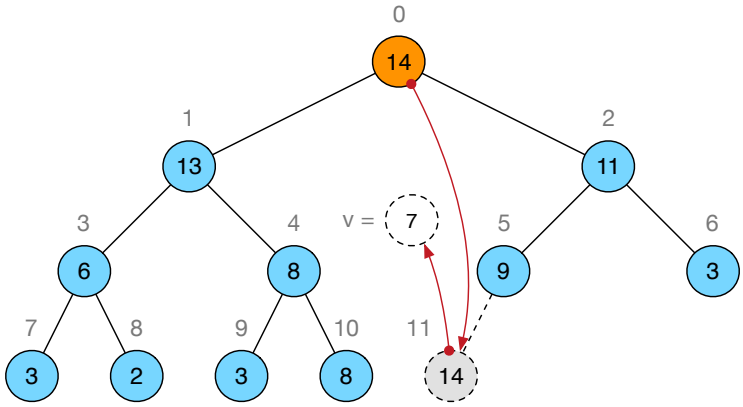


Figure 9.31.: Heap after the prologue of `pop_heap`

9.8.2.2. Main act

In the main act, the algorithm starts at a child node `c` of the prologue index `p`. Compared to the pre-state of `pop_heap` at the beginning of the main act, the array `a[0..n-1]`

- contains the value `v` one time less,
- contains the value `a[p]` one time more,
- while the multiplicities of all other values remain unchanged.

Moreover, the maximum element of the original heap is now stored at the last position of the array. At this point, we can only guarantee the heap property for the prefix `a[0..n-2]`. These observations form the basis for the loop invariants used in the main act.

When referring to a *child node* in the context of `pop_heap`, we implicitly mean the child — among the at most two children of a node — that carries the larger value. Copying this larger value to the parent node preserves the heap property. The index of this maximum child is computed using the function `heap_child` [9.10].

As long as the index `c` does not denote the last element of the heap and the value `a[c]` is strictly less than `v`, no valid position for inserting `v` has yet been found. In this situation, the algorithm continues to descend along the heap.

In each iteration of the loop, the following cases are distinguished.

- If `a[c]` is strictly greater than `a[p]`, the value `a[c]` is copied to position `p`. Since `c` is the index of the maximum child, this update preserves the heap property of the affected prefix. This fact is expressed formally using the predicate `HeapCompatible` [9.7].

  From a multiset perspective, the value `a[c]` now occurs one time more than in the pre-state, while the overwritten value `a[p]` occurs exactly as often as before. The value `v` continues to occur one time less than in the pre-state of `pop_heap`. The verification of these multiplicity changes relies on the predicates `ArrayUpdate` [7.4] and `MultisetUpdate` [9.19], as well as on the lemma `MultisetParity_Combined` [9.18].

  After the update, the algorithm proceeds by setting `p` to `c` and recomputing the next maximum child index.

- Otherwise, since the array prefix already satisfies the heap property, we necessarily have `a[c]` equal to `a[p]`. In this case, no update is required. The algorithm nevertheless descends by setting `p` to `c` and computing the corresponding maximum child.

The effect of the successive loop iterations is illustrated in the following figures, which depict the evolution of the array for our running example. Each of Figures 9.32, 9.33, and 9.34 represents a characteristic iteration of the main loop body and highlights the child index `c` selected during that iteration.

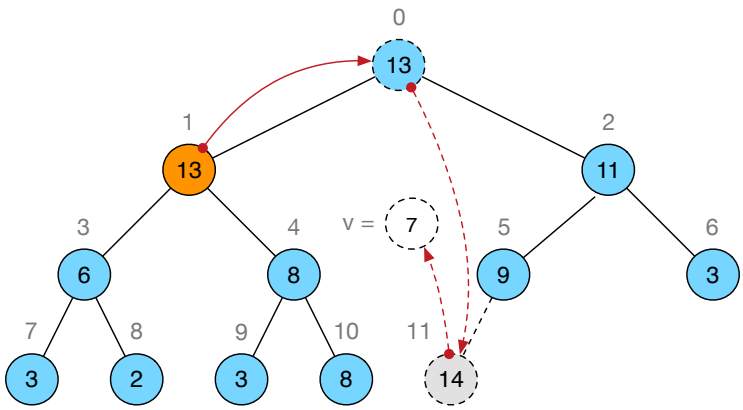


Figure 9.32.: Heap after a representative iteration of `pop_heap`

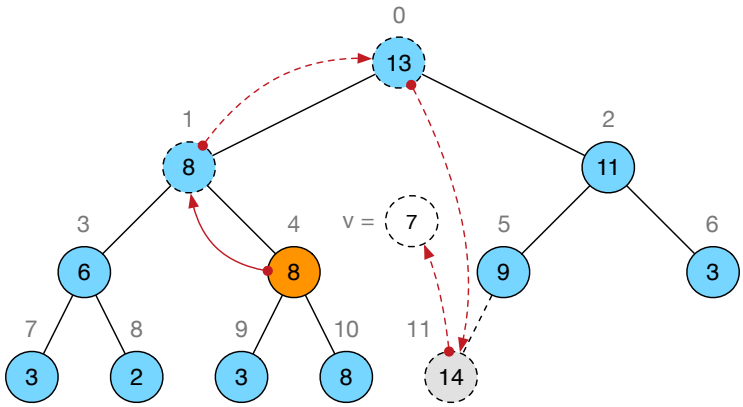


Figure 9.33.: Heap after a further representative iteration of `pop_heap`

In the last iteration shown, no value is actually copied, as the involved nodes already contain identical values.

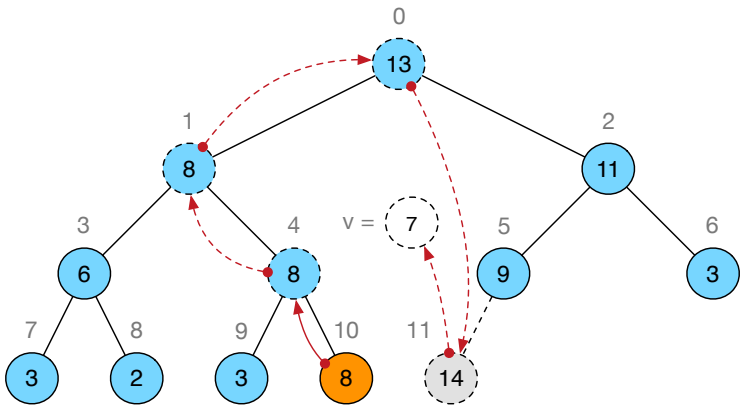


Figure 9.34.: Heap after a final representative iteration of `pop_heap`

Finally, we note that throughout the main act the last array element is never modified. Consequently, the maximum element of the original heap remains safely stored at the end of the array.

9.8.2.3. Epilogue

After leaving the loop, we know that the value `v` can be inserted into the array at index `p` without violating the heap property of the first $n - 1$ elements. Moreover, compared to the pre-state of `pop_heap`, the array `a[0..n-1]` still

- contains the value `v` one time less,
- contains the value `a[p]` one time more,
- while the multiplicities of all other values remain unchanged.

Assigning the value `v` to `a[p]` cancels this imbalance and establishes that `pop_heap` merely reorders the array elements.

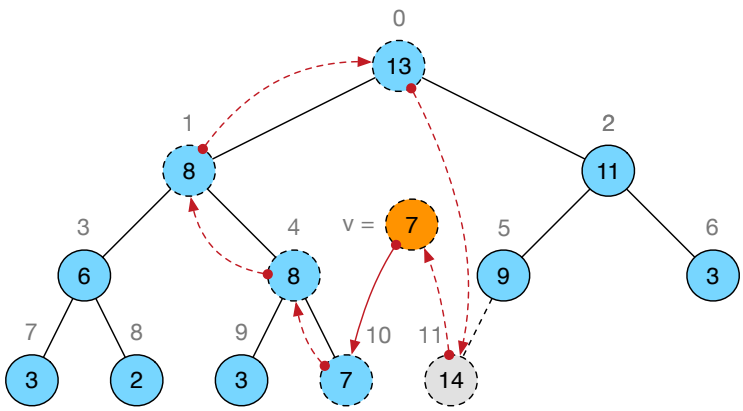


Figure 9.35.: Heap after the epilogue of `pop_heap`

In Figure 9.35, the value `v` is marked as the currently active node, although it is not yet stored in the array.

9.9. The make_heap algorithm

Whereas in the C++ Standard Library [18, §28.7.7.3] make_heap operates on a pair of generic random access iterators, our version operates on an array of value_type. Accordingly, the signature of make_heap reads

```
void make_heap(value_type* a, size_type n);
```

The function make_heap rearranges the elements of the array a[0..n-1] such that they satisfy the heap property.

As an example, consider the array shown in Figure 9.36. When interpreted as a binary tree via the usual array-to-heap mapping, the ordering of its elements violates the heap property. Executing make_heap on this array rearranges the elements so that the heap property holds.

8	8	7	3	14	11	3	6	2	3	13	9

Figure 9.36.: Array before the call of make_heap

The result can be viewed as an array in heap form, as shown in Figure 9.3. As already noted, this arrangement is not unique: the same multiset of elements may admit several different array representations that all satisfy the heap property.

9.9.1. Formal specification of make_heap

The following listing presents the formal specification of make_heap.

```
/*@
   requires   valid:    \valid(a + (0..n-1));

   terminates           \true;
   exits                \false;
   assigns              a[0..n-1];

   ensures    heap:     Heap(a, n);
   ensures    reorder:  MultisetReorder{Old,Here}(a, n);
*/
void make_heap(value_type* a, size_type n);
```

Listing 9.37: Formal specification of make_heap

As in the case of push_heap, the specification of make_heap must establish two essential properties. First, the resulting array a[0..n-1] must satisfy the heap property, that is, it must represent a heap of size n. Second, the operation must preserve the multiset of array elements with respect to the pre-state.

These requirements are captured by the postconditions heap and reorder, respectively. The reorder postcondition relies on the predicate MultisetReorder [7.59] to formalize that make_heap only permutes the elements of the array and does not create or remove values.

9.9.2. Implementation of make_heap

The implementation of make_heap, shown in the following listing, follows a simple incremental strategy. Starting from a trivial heap, the elements of the array are inserted one by one into a growing heap prefix.

More precisely, the loop iterates from low to high indices and applies push_heap [9.20] to successively larger prefixes of the array. We use the loop condition i < n, rather than the more tempting i <= n, in order to also admit the boundary case n == SIZE_TYPE_MAX. As a consequence, push_heap [9.20] is invoked with the argument i+1. The iteration starts with i+1 == 2, since an array of length one already satisfies the heap property.

```
void make_heap(value_type* a, size_type n)
{
  if (0u < n) {
    /*@
       loop invariant bounds:     1 <= i <= n;
       loop invariant heap:       Heap(a, i);
       loop invariant reorder:    MultisetReorder{Pre,Here}(a, n);
       loop invariant unchanged:  Unchanged{Pre,Here}(a, i+1, n);
       loop assigns   i, a[0..n-1];
       loop   variant n - i;
    */
    for (size_type i = 1u; i < n; ++i) {
      push_heap(a, i + 1u);
      //@ assert reorder:    MultisetReorder{LoopCurrent,Here}(a, i+1);
      //@ assert unchanged:  Unchanged{LoopCurrent,Here}(a, i+1, n);
      //@ assert reorder:    MultisetReorder{LoopCurrent,Here}(a, n);
    }

    //@ assert reorder:    MultisetReorder{Pre,Here}(a, n);
  }

  //@ assert  heap: Heap(a, n);
}
```

Listing 9.38: Implementation of make_heap

Since the loop body consists solely of a call to push_heap [9.20], the loop invariants heap and reorder can be obtained directly by lifting the corresponding postconditions from the contract of push_heap.

The postcondition of push_heap specifies preservation of the multiset of elements only for the prefix a[0..i]. It is therefore necessary to additionally state that the elements in the suffix a[i+1..n-1] are merely reordered. This property follows from the unchanged clause of push_heap, which guarantees that elements outside the affected prefix remain unchanged.

9.10. The sort_heap algorithm

The algorithm sort_heap completes the heap-based sorting process by transforming a heap into a fully ordered sequence. Whereas in the C++ Standard Library [18, §28.7.7.4] sort_heap operates on a range of random access iterators, our version works directly on an array of value_type. Accordingly, we use the following signature for sort_heap.

```
void sort_heap(value_type* a, size_type n);
```

The function sort_heap expects that the array segment a[0..n-1] satisfies the heap property. It then rearranges the elements of this heap into increasing order, thereby producing a sorted array. Thus, applying sort_heap to the heap shown in Figure 9.3 yields the increasing sequence displayed in Figure 9.39.

2	3	3	3	6	7	8	8	9	11	13	14

Figure 9.39.: Array after the call of sort_heap

9.10.1. Formal specification of sort_heap

The following listing presents our formal specification of sort_heap. The specification must ensure that, upon termination, the array a[0..n-1] is sorted in increasing order. Moreover, the algorithm must not introduce or remove elements; it may only permute them. These requirements are expressed by the postconditions increasing and reorder, respectively.

```
/*@
   requires   valid:       \valid(a + (0..n-1));
   requires   heap:        Heap(a, n);

   terminates              \true;
   exits                   \false;
   assigns                 a[0..n-1];

   ensures    reorder:     MultisetReorder{Old,Here}(a, n);
   ensures    increasing:  Increasing(a, n);
*/
void sort_heap(value_type* a, size_type n);
```

Listing 9.40: Formal specification of sort_heap

The specification effort is relatively modest. Since sort_heap merely rearranges the elements of the input heap, we can directly reuse the established predicates for sortedness and multiset preservation without introducing additional auxiliary logic.

9.10.2. Implementation of sort_heap

The implementation of sort_heap is comparatively simple. It relies on pop_heap [9.27] to perform the essential work of removing the maximum element from the heap and relocating it within the array. The algorithm repeatedly calls pop_heap in order to remove the maximum element from the current heap and place it at the end of the array. With each iteration, the heap stored in the prefix of the array shrinks, while the suffix grows into an increasing sequence.

The loop invariants describe this decomposition of the array into two parts. The first i elements form a heap and are characterized by the invariant heap. The remaining elements a[i..n-1] are already arranged in increasing order and are captured by the invariant increasing.

```
void sort_heap(value_type* a, size_type n)
{
  /*@
     loop invariant bound:       0 <= i <= n;
     loop invariant heap:        Heap(a, i);
     loop invariant lower:       LowerBound(a, i, n, a[0]);
     loop invariant reorder:     MultisetReorder{Pre,Here}(a, 0, n);
     loop invariant increasing:  WeaklyIncreasing(a, i, n);
     loop assigns i, a[0..n-1];
     loop variant i;
  */
  for (size_type i = n; i > 1u; --i) {
    //@ ghost Before: ;
    pop_heap(a, i);
    //@ assert lower:    LowerBound(a, i, n, a[i-1]);
    //@ assert reorder:  MultisetReorder{Before,Here}(a, 0, i);
    //@ assert reorder:  Unchanged{Before,Here}(a, i, n);
  }

  //@ assert increasing: Increasing(a, n);
}
```

Listing 9.41: Implementation of sort_heap

As already noted in the introduction of Chapter 6, the loop annotation increasing is expressed using the predicate WeaklyIncreasing [6.2]. Consequently, upon loop termination we only obtain the property WeaklyIncreasing(a, n). The final postcondition increasing, which relies on the predicate Increasing [6.1], is then derived by an application of the lemma WeaklyIncreasing_Increasing [6.3].

The preservation of the multiset property reorder follows from the specification of pop_heap. After each call to pop_heap, we introduce assertions relating the current state to the ghost label Before. These assertions establish

- MultisetReorder{Before,Here}(a, 0, i), and
- Unchanged{Before,Here}(a, i, n).

Together, they imply the loop invariant MultisetReorder{Pre,Here}(a, 0, n), which suffices to prove the final postcondition reorder.

10. Sorting Algorithms

Sorting algorithms provide a classical and well-studied domain in which many fundamental issues of computer science can be illustrated; see, for example, the seminal textbook by Knuth [21]. Accordingly, we consider a selection of representative sorting algorithms. In this chapter, we focus on algorithms from the C++ Standard Library [18, §28.7.1] that operate on linear arrays.

Following [22], we employ (possibly C-rephrased) interfaces of C++ Standard Library algorithms wherever feasible in order to realize the different algorithmic paradigms in a uniform manner.

- `is_sorted` in §10.2 checks whether a given array is already arranged in non-decreasing order.
- `partial_sort` in §10.3 rearranges an array into two contiguous parts. All elements of the first part are less than or equal to those of the second part. Moreover, the first part is sorted, whereas the order of the remaining elements is left unspecified.
- `bubble_sort` in §10.4 presents the simple and well-known *bubble sort* algorithm.
- `selection_sort` in §10.5 discusses the classical *selection sort* algorithm.
- `insertion_sort` in §10.6 describes the equally well-known *insertion sort* algorithm.
- `heap_sort` in §10.7 introduces the comparatively efficient *heap sort* algorithm, which builds upon the heap algorithms studied in Chapter 9.
- `merge` in §10.8 covers the *merge* operation that forms the core of *merge sort*.

With respect to asymptotic complexity, `heap_sort` achieves an upper bound of $\mathcal{O}(n \cdot \log(n))$, owing to the efficiency of the underlying heap data structure. In contrast, both `selection_sort` and `insertion_sort` require $\mathcal{O}(n^2)$ time in the average case as well as in the worst case.

The `sort` algorithm of the C++ Standard Library is not treated here, since it typically relies on *introspection sort*, a sophisticated hybrid that combines several classical algorithms.[17] We plan to address this and further sorting-related algorithms in future revisions.

[17] See https://en.wikipedia.org/wiki/Introsort

10.1. Common contract of sorting algorithms

The sorting algorithms discussed in this chapter essentially share the same formal contract. They differ not in their specification, but in the algorithmic strategy used to establish it. Consequently, Listing 10.1 states the common specification in the form of a generic sorting routine.

```
/*@
  requires valid: \valid(a + (0..n-1));

  assigns a[0..n-1];

  ensures increasing: Increasing(a, n);
  ensures reorder:    MultisetReorder{Old, Here}(a, n);
*/
void generic_sort(value_type* a, size_type n);
```

Listing 10.1: Common specification of sorting algorithms

The contract expresses two fundamental properties. First, the array segment `a[0..n-1]` is arranged in increasing order upon termination. Second, the algorithm performs a pure reordering of elements: no values are created or lost.

Throughout this chapter we adopt the convention established in Section 6.2. Global ordering requirements are stated in contracts using `Increasing`, whereas loop invariants and intermediate assertions are phrased in terms of `WeaklyIncreasing`. Whenever it is necessary to pass from `WeaklyIncreasing` to `Increasing`, we appeal to Lemma `WeaklyIncreasing_Increasing`.

10.2. The `is_sorted` algorithm

Our version of the `is_sorted` algorithm differs slightly from its counterpart in the C++ Standard Library [18, §28.7.1.5]. In particular, we consider a variant that operates directly on arrays of `value_type` and is therefore given the following signature.

```
bool is_sorted(const value_type* a, size_type n);
```

The function returns **`true`** if and only if the array segment `a[0..n-1]` is in increasing order. Otherwise, it returns **`false`**.

10.2.1. Formal specification of `is_sorted`

The following listing presents the ACSL specification of `is_sorted`. The contract characterizes sortedness by means of the predicate `Increasing` [6.1], which expresses the required global ordering property of the array segment.

```
/*@
  requires   valid:   \valid_read(a + (0..n-1));

  terminates          \true;
  exits               \false;
  assigns             \nothing;

  ensures    result:  \result <==> Increasing(a, n);
*/
bool is_sorted(const value_type* a, size_type n);
```

Listing 10.2: Formal specification of `is_sorted`

10.2.2. Implementation of `is_sorted`

The implementation of `is_sorted` is shown in the following listing. The algorithm proceeds sequentially and checks only adjacent pairs. If at any position an element is found to be greater than its successor, the array cannot be sorted and the algorithm terminates immediately. This linear strategy is sufficient, since any violation of the global ordering property implies the existence of an adjacent pair `a[i] > a[i+1]`.

```
bool is_sorted(const value_type* a, size_type n)
{
  if (0u < n) {
    /*@
        loop invariant increasing: WeaklyIncreasing(a, i+1);
        loop assigns i;
        loop variant n - i;
    */
    for (size_type i = 0u; i < n - 1u; ++i) {
      if (a[i] > a[i + 1u]) {
        return false;
      }
    }
  }

  return true;
}
```

Listing 10.3: Implementation of `is_sorted`

Following the convention established earlier, the predicate `WeaklyIncreasing` [6.2] is used as the loop invariant `increasing`. The loop invariant is preserved by each iteration and falsified as soon as a counterexample is encountered. After loop termination, the desired postcondition `increasing` is obtained from the corresponding loop invariant and Lemma `WeaklyIncreasing_Increasing` [6.3].

Maintaining the stronger predicate `Increasing` as a loop invariant is possible, but leads to heavier annotations without additional benefit.

10.3. The partial_sort algorithm

Compared to its counterpart in the C++ Standard Library [18, §28.7.1.3], our version of the `partial_sort` algorithm has the following signature.

```
void partial_sort(value_type* a, size_type m, size_type n);
```

The algorithm *reorders* the array `a` so that it is partitioned at position `m`. Every element of the left part `a[0..m-1]` is less than or equal to every element of the right part `a[m..n-1]`. In addition, the left part is *sorted* in increasing order. No ordering guarantees are provided for the elements of the right part.

In contrast to a full sorting algorithm, `partial_sort` does not establish a global ordering of the entire array. It guarantees only that the first `m` elements are the `m` smallest elements of the array and that they appear in increasing order. This makes `partial_sort` particularly suitable when only a prefix of the sorted sequence is required. Figure 10.4 illustrates the effect of a call `partial_sort(a, m, n)` using a bar chart. In the post-state, the left and right parts of the array are highlighted in green and orange, respectively.

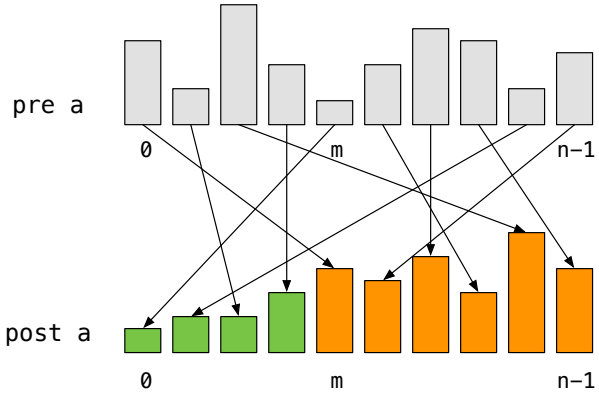


Figure 10.4.: Effects of `partial_sort`

10.3.1. The predicate Partition

We begin by introducing the predicate `Partition` [10.5], which captures the essence of a partition at position `m` within the prefix `a[0..n-1]`. It states a *cross-ordering* property: every element in the left part `a[0..m-1]` is less than or equal to every element in the right part `a[m..n-1]`. No ordering is required *within* either part.

```
/*@
  predicate Partition{L}(value_type* a, integer m, integer n) =
    \forall integer i, k; 0 <= i < m <= k < n  ==>  a[i] <= a[k];
*/
```

Listing 10.5: `Partition` (ACSL)

10.3.2. Auxiliary lemmas for `Partition`

The lemmas collected in the following listing support the reasoning about partitioning properties under reordering operations. They are stated independently of any particular algorithmic context and are later instantiated in the verification of `partial_sort`.

```
/*@
  lemma MultisetReorder_SomeEqual{K,L}:
    \forall value_type *a, integer n, i;
      0 < n                        ==>
      0 <= i < n                   ==>
      MultisetReorder{K,L}(a, n)   ==>
      SomeEqual{K}(a, n, \at(a[i],L));

  lemma MultisetReorder_LowerBound{K,L}:
    \forall value_type* a, integer n, value_type v;
      0 <= n                       ==>
      MultisetReorder{K,L}(a, n)   ==>
      LowerBound{K}(a, n, v)       ==>
      LowerBound{L}(a, n, v);

  lemma MultisetReorder_UpperBound{K,L}:
    \forall value_type* a, integer n, value_type v;
      0 <= n                        ==>
      MultisetReorder{K,L}(a, n)    ==>
      UpperBound{K}(a, n, v)        ==>
      UpperBound{L}(a, n, v);

  lemma MultisetReorder_PartitionLowerBound{K,L}:
    \forall value_type* a, integer m, n;
      0 < m <= n                     ==>
      MultisetReorder{K,L}(a, 0, m)  ==>
      Partition{K}(a, m, n)          ==>
      Unchanged{K,L}(a, m, n)        ==>
      LowerBound{L}(a, m, n, \at(a[0],L));
*/
```

Listing 10.6: `PartitionLemmas` (ACSL)

- Lemma `MultisetReorder_SomeEqual` formalizes the fact that reordering a range does not introduce new values. Any value `a[i]` occurring in the range `a[0..n-1]` after reordering must already have occurred in that range beforehand. This lemma serves as a technical foundation for the subsequent results.
- Lemma `MultisetReorder_LowerBound` states that a lower bound `v` of a range `a[0..n-1]` remains a lower bound after the range is reordered.
- Dually, lemma `MultisetReorder_UpperBound` asserts that reordering preserves upper bounds.
- Lemma `MultisetReorder_PartitionLowerBound` addresses a more specific situation. If the array is partitioned at position `m` and the left part `a[0..m-1]` is reordered while the right part `a[m..n-1]` remains unchanged, then the first element of the left part remains a lower bound of the right part. This lemma will later be used to maintain partitioning properties across reordering steps.

10.3.3. Formal specification of partial_sort

The formal specification of the function partial_sort is given in the following listing. It reflects directly the informal description of the algorithm. The predicate Partition [10.5] expresses the partitioning property at position m, while the predicate Increasing [6.1] captures the required ordering of the left part a[0..m-1]. As usual, the predicate MultisetReorder [7.59] is used to state that partial_sort merely reorders the elements of the array without changing their multiplicities.

```
/*@
  requires   valid:      \valid(a + (0..n-1));
  requires   split:      0 <= m <= n;

  terminates              \true;
  exits                   \false;
  assigns                 a[0..n-1];

  ensures    reorder:     MultisetReorder{Old,Here}(a, n);
  ensures    partition:   Partition(a, m, n);
  ensures    increasing:  Increasing(a, m);
*/
void partial_sort(value_type* a, size_type m, size_type n);
```

Listing 10.7: Formal specification of partial_sort

10.3.4. Implementation of partial_sort

The implementation of partial_sort is shown for better readability in two listings. Listing 10.8 shows the overall structure of the algorithm together with the scan loop. Listing 10.10 contains the loop body and the finalization step in full detail, including the assertions used to support the verification.

10.3.4.1. Prologue: building the initial heap

The algorithm begins by calling make_heap [9.37] to rearrange the left part a[0..m-1] into a heap. This representation makes the maximal element of the left part readily available at position a[0]. It will serve as the separating element during the subsequent scan of the right part.

10.3.4.2. Main act: scanning and maintaining the invariants

The right part a[m..n-1] is scanned from left to right. Whenever the current element a[i] is not smaller than the heap maximum a[0], the scan proceeds without modifying the array. Otherwise, execution enters the then branch of the loop, where the current element is exchanged with the heap maximum by a suitable combination of pop_heap [9.27] and push_heap [9.20]. Throughout the scan loop, the following invariants are maintained:

- the left part a[0..m-1] forms a heap (invariant heap);
- the heap maximum a[0] separates the left part from the already scanned right part a[m..i-1] (invariants upper and lower);
- the suffix a[i..n-1] has not yet been modified (invariant unchanged); and
- all modifications of a[0..i-1] are permutations (invariant reorder).

```
void partial_sort(value_type* a, size_type m, size_type n)
{
  if (m > 0u) {
    make_heap(a, m);

    //@ assert reorder: Unchanged{Pre,Here}(a, m, n);
    /*@
      loop invariant bound:     m <= i <= n;
      loop invariant heap:      Heap(a, m);
      loop invariant upper:     UpperBound(a, 0, m, a[0]);
      loop invariant lower:     LowerBound(a, m, i, a[0]);
      loop invariant reorder:   MultisetReorder{Pre,Here}(a, i);
      loop invariant unchanged: Unchanged{Pre,Here}(a, i, n);
      loop assigns              i, a[0..n-1];
      loop variant              n-i;
    */
    for (size_type i = m; i < n; ++i) {
      if (a[i] < a[0u]) {
        //@ ghost BeforePop: ;
        pop_heap(a, m);
        //@ assert lower:      a[0] <= a[m-1];
        //@ assert lower:      a[i] <  a[m-1];
        //@ assert lower:      LowerBound(a, m, i, a[m-1]);
        //@ assert upper:      UpperBound(a, 0, m-1, a[0]);
        //@ assert upper:      UpperBound(a, 0, m,   a[m-1]);
        //@ assert partition:  Partition(a, m, i);
        //@ assert unchanged:  Unchanged{BeforePop,Here}(a, m, i);
        //@ assert reorder:    MultisetReorder{BeforePop,Here}(a, m);
        //@ assert reorder:    MultisetReorder{Pre,Here}(a, i);
        //@ assert unchanged:  Unchanged{Pre,Here}(a, i, n);

        //@ ghost BeforeSwap: ;
        //@ assert reorder:    MultisetReorder{Pre,Here}(a, i+1);
        swap(a + m - 1u, a + i);
        //@ assert swapped:    ArraySwap{BeforeSwap,Here}(a, m-1, i, n);
        //@ assert unchanged:  Unchanged{BeforeSwap,Here}(a, m-1);
        //@ assert unchanged:  Unchanged{Pre,Here}(a, i+1, n);
        //@ assert reorder:    MultisetReorder{BeforeSwap,Here}(a, m-1, i+1);
        //@ assert reorder:    MultisetReorder{BeforeSwap,Here}(a, i+1);
        //@ assert reorder:    MultisetReorder{Pre,Here}(a, i+1);
        //@ assert lower:      a[m-1] < a[i];
        //@ assert partition:  Partition(a, m, i+1);
        //@ assert upper:      UpperBound(a, 0, m-1, a[0]);

        //@ ghost BeforePush: ;
        push_heap(a, m);
        //@ assert heap:       Heap(a, m);
        //@ assert unchanged:  Unchanged{BeforePush,Here}(a, m, i+1);
        //@ assert unchanged:  Unchanged{BeforePush,Here}(a, i+1, n);
        //@ assert reorder:    MultisetReorder{Pre,Here}(a, i+1);
        //@ assert upper:      UpperBound(a, 0, m,   a[0]);
        //@ assert lower:      LowerBound(a, m, i+1, a[0]);
      }
    }
```

Listing 10.8: Implementation of `partial_sort` (1): overall structure and scan loop

The intermediate states arising in the non-trivial case of the loop are illustrated in Figure 10.9.

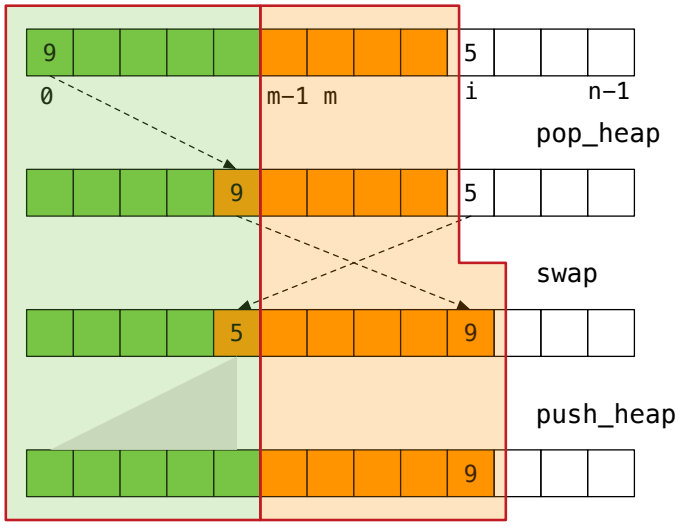


Figure 10.9.: An iteration of `partial_sort`

The elements considered so far are shown using the same coloring convention as in Figure 10.4. In particular, the heap is highlighted in green. The transparent red overlay indicates the ranges to which the predicate `Partition` applies in the respective states. For concreteness, the figure assumes initial values 9 and 5 at positions `a[0]` and `a[i]`. In the general reasoning below, these values are denoted by p and q, respectively.

After the calls to `pop_heap` and `swap`, the value p resides at index `i`, while q is placed at index `m-1`. At this point, the following relations hold:

1. $q < p \leqslant a[k]$ for all $m \leqslant k < i$;

2. $q < p = a[i]$;

3. $a[j] \leqslant p \leqslant a[k]$ for all $0 \leqslant j < m - 1$ and all $m \leqslant k < i$; and

4. $a[j] \leqslant p = a[i]$ for all $0 \leqslant j < m - 1$.

Combining these inequalities, we obtain

$$a[j] \leqslant p \leqslant a[k] \quad \text{for all } 0 \leqslant j < m \text{ and all } m \leqslant k < i + 1.$$

Hence, `Partition(a,m,i+1)` holds, even though no distinguished separating element can be identified at this stage.

The subsequent call to `push_heap` only reorders elements within the left part and therefore preserves this partitioning property. At the same time, the heap property is restored, making `a[0]` again an upper bound of the left part and thus a separating element between `a[0..m-1]` and `a[m..i]`. Consequently, the invariants `upper` and `lower` are re-established. They are later used to prove the postcondition `partition`.

10.3.4.3. Epilogue: sorting the heap prefix

After termination of the scan loop (see Listing 10.10), the invariant `partition` guarantees that the array `a[0..n-1]` is partitioned at position `m`. Consequently, the left part `a[0..m-1]` contains exactly the `m` smallest elements of the array. At this stage, however, these elements are still arranged as a heap rather than in increasing order.

```
    //@ assert partition: Partition(a, m, n);
    //@ ghost BeforeSort: ;
    sort_heap(a, m);
    //@ assert unchanged:  Unchanged{BeforeSort,Here}(a, m, n);
    //@ assert reorder:    MultisetReorder{BeforeSort,Here}(a, m);
    //@ assert reorder:    MultisetReorder{Pre,Here}(a, n);
    //@ assert partition:  Partition(a, m, n);
  }
}
```

Listing 10.10: Implementation of `partial_sort` (2)

The final call to `sort_heap` transforms this heap into an increasingly ordered range. To reason about this call, a ghost label `BeforeSort` marks the program state immediately preceding it. This allows the post-state of `sort_heap` to be related both to the pre-state of the call and to the initial pre-state of `partial_sort`.

The predicate `Unchanged` [7.1] records that the right part `a[m..n-1]` is not affected by the call to `sort_heap` . Moreover, the predicate `MultisetReorder` [7.59] is instantiated twice: first to express that `sort_heap` only reorders the left part, and second to re-establish that the entire array is a permutation of its initial contents. Since `sort_heap` preserves the multiset of the left part and leaves the right part unchanged, the partitioning property `Partition(a,m,n)` remains valid.

10.3.5. Remarks on specification style and complexity

Relative to its size, the `partial_sort` algorithm performs a considerable number of procedure calls. In this respect, it is closer to real-world software than many of the algorithms presented in this tutorial. For most procedure calls, the callee's contract is instantiated with the actual parameters and recorded as assertions, which may refer to the pre-state of the call by means of *ghost labels*. By contrast, explicit `assert` clauses are used to document local reasoning steps that are required for subsequent verification arguments.

The worst-case time complexity of the implementation is $\mathcal{O}((n + m) \cdot \log m)$. An implementation that ignores the parameter `m` and simply sorts the entire range `a[0..n-1]` also satisfies the contract of `partial_sort` [10.7], but has worst-case complexity $\mathcal{O}(n \cdot \log n)$. Elementary arithmetic shows that `partial_sort` is asymptotically more efficient if and only if

$$\log m < \frac{n}{m} \log\left(\frac{n}{m}\right),$$

that is, whenever n is sufficiently larger than m.

10.4. The bubble_sort algorithm

The bubble_sort algorithm is the first sorting algorithm whose implementation we verify in this chapter.[18] From an algorithmic point of view, bubble_sort is simple and not particularly efficient. Its importance in our context lies elsewhere: its control structure is elementary, and the underlying idea can be expressed using straightforward loop invariants. This makes bubble_sort a suitable introductory example for the verification of sorting algorithms.

Algorithmically, bubble_sort repeatedly scans the array a[0..n-1] from left to right. During these scans it maintains a right-aligned suffix a[n-i..n-1] that is already in increasing order and whose length grows monotonically. This suffix is established by comparing adjacent elements and swapping them whenever they appear in the wrong order, thereby moving larger values step by step towards the end of the array.

10.4.1. Formal specification of bubble_sort

The following listing instantiates the generic sorting contract from Listing 10.1 for the bubble_sort algorithm. As expected, the specification does not reflect any algorithm-specific details, but merely states the common correctness properties of sorting.

```
/*@
  requires   valid:       \valid(a + (0..n-1));

  terminates              \true;
  exits                   \false;
  assigns                 a[0..n-1];

  ensures    reorder:     MultisetReorder{Old,Here}(a, n);
  ensures    increasing:  Increasing(a, n);
 */
void bubble_sort(value_type* a, size_type n);
```

Listing 10.11: Formal specification of bubble_sort

10.4.2. Implementation of bubble_sort

The implementation of bubble_sort is shown in the following listing. As is typical for this algorithm, it consists of two nested loops.

We first address the verification of the postcondition increasing. To this end, the *outer loop* is annotated with the invariant increasing, which states that the suffix a[n-i+1..n-1] is already in increasing order. A crucial ingredient in establishing this property is the observation that the first element a[n-i+1] of the sorted suffix is an upper bound for *all* elements to its left. This fact is captured by the outer-loop invariant upper. To justify this claim, we exploit a property of the *inner loop*: at iteration index j, the element a[j] is the maximum of the subrange a[0..j]. This property is formalized by the loop invariant max of the inner loop.

The invariants increasing and upper are also stated for the inner loop. This ensures that the outer loop may rely on these properties being preserved throughout the execution of the inner loop.

[18] See https://en.wikipedia.org/wiki/Bubble_sort.

```
void bubble_sort(value_type* a, size_type n)
{
  if (0 < n) {
    /*@
      loop invariant bound:          1 <= i <= n;
      loop invariant increasing:     WeaklyIncreasing(a, n-i+1, n);
      loop invariant upper:          1 < i ==> UpperBound(a, n-i+1, a[n-i+1]);
      loop invariant reorder:        MultisetReorder{Pre,Here}(a, n);
      loop assigns i, a[0..n-1];
      loop variant n-i;
    */
    for (size_type i = 1u; i < n; ++i) {
      /*@
        loop invariant bound:        0 <= j <= n-i;
        loop invariant increasing:   WeaklyIncreasing(a, n-i+1, n);
        loop invariant upper:        1 < i ==> UpperBound(a, n-i+1, a[n-i+1]);
        loop invariant max:          MaxElement(a, j+1, j);
        loop invariant reorder:      MultisetReorder{LoopEntry,Here}(a, j+1);
        loop invariant reorder:      Unchanged{LoopEntry,Here}(a, j+1, n);
        loop assigns                 j, a[0..n-1];
        loop variant n-j;
      */
      for (size_type j = 0u; j < n - i; ++j) {
        if (a[j] > a[j + 1u]) {
          //@ assert max:       MaxElement(a, j+1, j);
          //@ assert reorder:   MultisetReorder{LoopEntry,Here}(a, 0, j+1);
          //@ assert reorder:   MultisetReorder{LoopEntry,Here}(a, 0, j+2);
          swap(&a[j], &a[j + 1u]);
          //@ assert max:       MaxElement(a, j+2, j+1);
          //@ assert swap:      ArraySwap{LoopCurrent,Here}(a, j, j+1, n);
          //@ assert unchanged: Unchanged{LoopCurrent,Here}(a, j);
          //@ assert reorder:   a[j+1] == At{LoopCurrent}(a, j);
          //@ assert reorder:   a[j]   == At{LoopCurrent}(a, j+1);
          //@ assert reorder:   MultisetReorder{LoopCurrent,Here}(a, j, j+2);
        }
      }
    }
  }

  //@ assert increasing: Increasing(a, n);
}
```

Listing 10.12: Implementation of `bubble_sort`

Here are a few remarks about the verification of the postcondition `reorder`.

- Within each iteration of the outer loop, elements of the unsorted prefix `a[0..n-1]` are swapped with their immediate neighbours.
- More precisely, at iteration `j` of the inner loop, the subrange `a[0..j]` may be rearranged, while the subrange `a[j+1..n-1]` remains unchanged.

Together, these facts suffice to establish the loop invariant `reorder` for the outer loop, and hence the desired postcondition.

10.5. The `selection_sort` algorithm

Next we consider the sorting algorithm `selection_sort`.[19] The algorithm proceeds by growing an increasingly ordered initial segment from left to right. In each iteration, it selects the minimum element of the remaining suffix and swaps it with the first element of that suffix.

As a consequence of this strategy, after each iteration the array is partitioned into two contiguous segments. The initial segment is in increasing order, and every element in this segment is less than or equal to every element in the remaining segment. This invariant captures the essence of `selection_sort` and will later be reflected directly in the loop annotations.

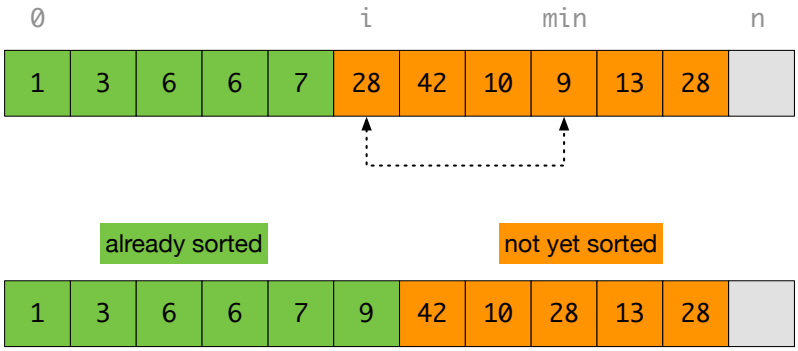


Figure 10.13.: An iteration of `selection_sort`

Figure 10.13 illustrates a typical iteration. The minimum element `9` of the unsorted suffix, located at position `min`, is swapped with the element `28` at position `i`. This operation extends the increasingly ordered initial segment by one array cell.

10.5.1. Formal specification of `selection_sort`

The following listing instantiates the generic sorting contract from Listing 10.1 for the `selection_sort` algorithm. As expected, the specification does not reflect any algorithm-specific details, but merely states the common correctness properties of sorting.

```
/*@
  requires   valid:       \valid(a + (0..n-1));

  terminates              \true;
  exits                   \false;
  assigns                 a[0..n-1];

  ensures    reorder:     MultisetReorder{Old,Here}(a, n);
  ensures    increasing:  Increasing(a, n);
*/
void selection_sort(value_type* a, size_type n);
```

Listing 10.14: Formal specification of `selection_sort`

[19] See https://en.wikipedia.org/wiki/Selection_sort.

10.5.2. Implementation of `selection_sort`

The implementation of `selection_sort` is shown in the following listing. The algorithm iterates over the array from left to right and, in each iteration, selects the minimum element of the remaining suffix using `min_element` [5.15]. This element is then swapped into its final position at the boundary between the already sorted prefix and the unsorted remainder.

```
void selection_sort(value_type* a, size_type n)
{
  /*@
    loop invariant bound:       0 <= i <= n;
    loop invariant reorder:     MultisetReorder{Pre,Here}(a, n);
    loop invariant increasing:  WeaklyIncreasing(a, i);
    loop invariant increasing:  0 < i  ==> LowerBound(a, i, n, a[i-1]);
    loop assigns   i, a[0..n-1];
    loop variant   n - i;
  */
  for (size_type i = 0u; i < n; ++i) {
    const size_type sel = i + min_element(a + i, n - i);

    if (i < sel) {
      //@ ghost Before: ;
      swap(a + sel, a + i);
      //@ assert swapped: ArraySwap{Before,Here}(a, i, sel, n);
    }

    //@ assert reorder: MultisetReorder{LoopCurrent,Here}(a, n);
    //@ assert reorder: MultisetReorder{Pre,Here}(a, n);
  }

  //@ assert increasing: Increasing(a, n);
}
```

Listing 10.15: Implementation of `selection_sort`

The loop invariants `increasing` and `lower` formalize the central ordering argument. They establish that the prefix `a[0..i-1]` is in increasing order and that its last element `a[i-1]` is a lower bound for all elements in the remaining segment `a[i..n-1]`. Together, these properties ensure that the prefix grows monotonically and never needs to be modified again.

Because the call to `min_element` operates on a shifted address, its use requires the application of the *shift lemmas* collected in [6.14], which allow us to relate properties of the suffix back to the original array.

The loop invariant `reorder` expresses that `selection_sort` only rearranges the elements of the array. Although this is intuitively clear—since `swap` [7.7] is the only operation that modifies `a`—its formal verification requires an explicit account of the effect of swapping.

In the current code, this effect is expressed using ghost code rather than a statement contract. A ghost label `Before` is placed immediately before the call to `swap`, and the predicate `swapped` is asserted afterwards. Together, these annotations relate the states at `Before` and `Here` via `ArraySwap{Before,Here}(a, i, sel, n)`, which suffices to apply Lemma `MultisetSwap_Middle` [7.63] and thus establish the required `MultisetReorder` [7.59] property.

10.6. The insertion_sort algorithm

As another simple comparison-based sorting algorithm studied in this chapter, we consider the algorithm `insertion_sort`. [20] Like `selection_sort`, the algorithm `insertion_sort` traverses the given array `a[0..n-1]` from left to right and maintains a left-adjusted prefix `a[0..i-1]` that is already in increasing order.

The defining idea of `insertion_sort` is how this sorted prefix is extended. In the `i`th iteration, the next element `a[i]` is inserted into its appropriate position within the sorted prefix, thereby increasing the sorted range by one element. Conceptually, this insertion is achieved by shifting the elements of the prefix to make room for `a[i]` at its correct position. Figure 10.16 illustrates a single iteration of this process.

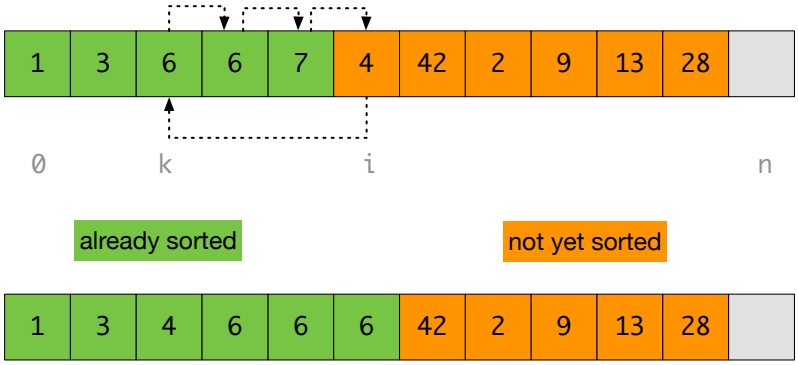


Figure 10.16.: An iteration of `insertion_sort`

10.6.1. Formal specification of insertion_sort

The next listing gives the formal specification of the `insertion_sort` algorithm. It is obtained by instantiating the generic sorting contract from Listing 10.1.

```
/*@
  requires   valid:       \valid(a + (0..n-1));

  terminates              \true;
  exits                   \false;
  assigns                 a[0..n-1];

  ensures    reorder:     MultisetReorder{Old,Here}(a, n);
  ensures    increasing:  Increasing(a, n);
*/
void insertion_sort(value_type* a, size_type n);
```

Listing 10.17: Formal specification of `insertion_sort`

As for the other sorting algorithms considered in this chapter, the contract is deliberately free of algorithm-specific details and captures only the essential correctness properties that characterize sorting.

[20] See https://en.wikipedia.org/wiki/Insertion_sort.

10.6.2. Implementation of insertion_sort

The implementation of insertion_sort is shown in the next listing. In each iteration, the appropriate insertion position for the current element a[i] is determined by a call to upper_bound [6.8], which returns the rightmost index at which a[i] can be inserted without violating the ordering. The insertion itself is then realized by a call to rotate [7.32], which performs a circular shift of the affected subrange. This decomposition separates the identification of the insertion position from the actual rearrangement of elements, resulting in a modular implementation. As we will see shortly, this modular structure also simplifies the verification effort, since it allows us to rely on the contracts of these auxiliary operations.

```
void insertion_sort(value_type* a, size_type n)
{
  /*@
    loop invariant bound:         0 <= i <= n;
    loop invariant reorder:       MultisetReorder{Pre,Here}(a, 0, i);
    loop invariant unchanged:     Unchanged{Pre,Here}(a, i, n);
    loop invariant increasing:    WeaklyIncreasing(a, i);
    loop assigns   i, a[0..n-1];
    loop variant   n - i;
  */
  for (size_type i = 0u; i < n; ++i) {
    const size_type k = upper_bound(a, i, a[i]);
    //@ assert bound:   0 <= k <= i;
    //@ ghost Before: ;
    //@ assert lower:       StrictLowerBound(a, k, i, a[i]);
    //@ assert upper:       UpperBound(a, k, a[i]);
    rotate(a + k, i - k, i + 1u - k);
    //@ assert rotate:      Rotate{Before,Here}(a, k, i, i+1);
    //@ assert upper:       UpperBound(a, k, a[k]);
    //@ assert increasing:  WeaklyIncreasing(a, k+1);
    //@ assert increasing:  WeaklyIncreasing(a, k+1, i+1);
    //@ assert increasing:  WeaklyIncreasing(a, i+1);
    //@ assert reorder:     MultisetReorder{LoopCurrent,Here}(a, 0, k);
    //@ assert reorder:     MultisetReorder{LoopCurrent,Here}(a, k, i+1);
    //@ assert reorder:     MultisetReorder{LoopCurrent,Here}(a, 0, i+1);
    //@ assert reorder:     MultisetReorder{Pre,Here}(a, i+1);
  }

  //@ assert increasing: Increasing(a, n);
}
```

Listing 10.18: Implementation of insertion_sort

The effects of the call to rotate are formally captured by a version of the predicate Rotate [7.28], which was introduced in Equation (7.3). Using the labels Before and Here to denote the states before and after the call, respectively, and abbreviating them by B and H, the corresponding assertion states the following: The former prefix $a\big[k \mathinner{..} i\big)$ becomes the suffix of the rotated slice, that is,

$$a^H\big[k+1 \mathinner{..} i+1\big) = a^B\big[k \mathinner{..} i\big),$$

while the former suffix $a[i] = a\big[i \mathinner{..} i+1\big)$ becomes the new prefix,

$$a^H\big[k \mathinner{..} k+1\big) = a^B\big[i \mathinner{..} i+1\big),$$

or, equivalently,

$$a^H[k] = a^B[i].$$

For the parts of the array preceding and following the rotated slice, no changes occur:

$$a^H\big[0\,..\,k\big) = a^B\big[0\,..\,k\big),$$
$$a^H\big[i+1\,..\,n\big) = a^B\big[i+1\,..\,n\big).$$

When `rotate` was originally implemented and verified, its later use within `insertion_sort` was not yet anticipated. As a consequence, certain properties required in the present context — in particular the fact that a rotation constitutes a *reordering* of the affected elements — are not stated explicitly in its contract. One conceivable remedy would be to strengthen the contract of `rotate` [7.32] by adding the properties needed for `insertion_sort` and to repeat its verification. Such an approach, however, is inappropriate if `rotate` is regarded as part of a pre-verified library: in that case, its implementation may no longer be available for re-verification.

We therefore adopt a different strategy. Rather than modifying the existing specification, we show that the original contract of `rotate` already *implies* all properties required for `insertion_sort`. To make this implication explicit, we introduce and prove a small number of auxiliary lemmas that derive the needed properties from the contract of `rotate`. This argument is a direct application of the implication rule of the Hoare calculus (see §3.4).

10.6.2.1. Preservation of Order Under Elementwise Equality

Lemma `Increasing_Equal` [6.3] assumes that a range $a[m\,..\,n)$ is ordered and states that any elementwise equal range $a\big[m+p\,..\,n+p\big)$ is ordered as well. This lemma is used to show that the call to `rotate` [7.32] preserves the order of those elements that are shifted upwards (see Figure 10.16).

10.6.2.2. Auxiliary Lemmas for Reasoning About Rotations

In addition, we rely on three further lemmas, listed below, which make the required logical implications explicit and assist the providers in establishing the desired properties of rotations.

```
/*@
  lemma Rotate_Shift{K,L}:
    \forall value_type *a, integer k, m, p, n;
      Rotate{K,L}(a+k, m, p, n)  ==>  Rotate{K,L}(a, m+k, p+k, n+k);

  lemma RotateOne_StrictLowerBound{K,L}:
    \forall value_type* a, integer m, n;
      StrictLowerBound{K}(a, m,   n,   \at(a[n],K))  ==>
      Rotate{K,L}(a, m, n, n+1)                      ==>
      StrictLowerBound{L}(a, m+1, n+1, \at(a[m],L));

  lemma RotateOne_MultisetReorder{K,L}:
    \forall value_type* a, integer m, n;
      0 <= m <= n                ==>
      Rotate{K,L}(a, m, n, n+1)  ==>
      MultisetReorder{K,L}(a, m, n+1);
*/
```

Listing 10.19: `RotateLemmas` (ACSL)

Lemma `Rotate_Shift` [10.19] relates rotations applied to array segments accessed through pointer offsets to corresponding rotations on the underlying base array. It allows us to transfer properties established for a rotated slice $a\big[m+k\,..\,n+k\big)$ back to the corresponding slice $a[m\,..\,n)$.

The remaining two lemmas deal with rotations by exactly one element.

- To show that a rotation by one position preserves the strict lower-bound relationship required by the loop invariant, we rely on Lemma `RotateOne_StrictLowerBound`. More precisely, if the range $a[k \mathinner{..} i)$ has $a[i]$ as a strict lower bound in the state before the call to `rotate`, then after the call the same range has $a[k]$ as a strict lower bound. This statement captures precisely the effect of using `rotate` as a *circular shift by one position* at the call site.
- Lemma `RotateOne_MultisetReorder` establishes that a rotation by one element merely reorders the range to which it is applied. Together with the assertions relating the states of the array before and after the call to `rotate`, this lemma allows us to conclude that the multiset of array elements is preserved, as required by the loop invariant of `insertion_sort`.

  Unlike the preceding lemma, this result could, in principle, be generalized to rotations by an arbitrary number of positions; however, the specialized formulation suffices for the present use case.

10.7. The heap_sort algorithm

The heap_sort algorithm[21] is entirely built on top of the heap primitives introduced in Chapter 9. In particular, it combines a call to make_heap, repeated applications of pop_heap, and a final call to sort_heap to transform an array into increasing order. This process is already suggested by Figure 9.1, which highlights the interaction between the individual heap algorithms and their effect on the array state.

10.7.1. Formal specification of heap_sort

The following listing presents the formal specification of the heap_sort algorithm. It is derived by instantiating the generic sorting contract from Listing 10.1. As in the preceding sorting algorithms, the specification abstracts from algorithm-specific mechanisms and records only the fundamental properties that any correct sorting procedure must satisfy.

```
/*@
  requires   valid:       \valid(a + (0..n-1));

  terminates              \true;
  exits                   \false;
  assigns                 a[0..n-1];

  ensures    reorder:     MultisetReorder{Old,Here}(a, n);
  ensures    increasing:  Increasing(a, n);
*/
void heap_sort(value_type* a, size_type n);
```

Listing 10.20: Formal specification of heap_sort

10.7.2. Implementation of heap_sort

The implementation of heap_sort is shown in the next listing. Conceptually, the algorithm proceeds in two phases. First, the input array a[0..n-1] is rearranged into a heap by a call to make_heap [9.37], establishing the heap structure required for efficient access to the maximum element. Second, a call to sort_heap [9.40] repeatedly removes this maximum element and places it at the end of the array, thereby transforming the heap into an increasingly ordered array segment.

```
void heap_sort(value_type* a, size_type n)
{
  make_heap(a, n);
  sort_heap(a, n);
}
```

Listing 10.21: Implementation of heap_sort

From a verification perspective, heap_sort is particularly instructive, as it combines non-trivial structural invariants of heaps with the global ordering and multiset preservation properties common to all sorting algorithms.

[21] See https://en.wikipedia.org/wiki/Heapsort.

10.8. The merge algorithm

The algorithm merge is taken from the C++ standard library [18, p. 28.7.5]. Unlike the sorting algorithms studied in the surrounding sections, merge is not a sorting algorithm in its own right. Instead, it combines two already ordered sequences into a single ordered sequence and thus serves as a fundamental building block for sorting algorithms such as merge_sort.[22]

Our version of merge has the following signature.

```
void merge(const value_type* a, size_type n,
           const value_type* b, size_type m, value_type* result);
```

The function merge takes two input arrays a[0..n-1] and b[0..m-1], both of which are assumed to be ordered with respect to the ordering on value_type. It produces an output array that contains exactly the elements of both inputs, arranged in ordered form.

The array starting at result must designate a memory region that is disjoint from the input arrays and large enough to store exactly $n + m$ elements. Upon termination, the range result[0..n+m-1] is ordered and contains each element of a and b exactly once.

If two elements from different input arrays compare equal, the element originating from a precedes the element originating from b. Moreover, the relative order of equal elements within each individual input array is preserved. In this sense, merge is a *stable* combining operation.

10.8.1. Formal specification of merge

Listing 10.22 shows the formal specification of merge. It requires the input arrays a[0..n-1] and b[0..m-1] to be valid, non-overlapping, and ordered in increasing order, and the output array result to be valid for exactly $n + m$ elements and disjoint from the inputs.

```
/*@
  requires   bound:        m + n <= SIZE_TYPE_MAX;
  requires   valid:        \valid_read(a + (0..m-1));
  requires   valid:        \valid_read(b + (0..n-1));
  requires   valid:        \valid(c + (0..m+n-1));
  requires   sep:          \separated(a + (0..m-1), c + (0..m+n-1));
  requires   sep:          \separated(b + (0..n-1), c + (0..m+n-1));
  requires   increasing:   Increasing(a, m);
  requires   increasing:   Increasing(b, n);

  terminates               \true;
  exits                    \false;
  assigns                  c[0 .. m+n-1];

  ensures    increasing:   Increasing(c, m + n);
  ensures    unchanged:    Unchanged{Old,Here}(a, m);
  ensures    unchanged:    Unchanged{Old,Here}(b, n);
 */
void merge(const value_type* a, size_type m,
           const value_type* b, size_type n, value_type* c);
```

Listing 10.22: Formal specification of merge

[22] See https://en.wikipedia.org/wiki/Merge_sort.

The postconditions of the contract characterize the output array only at a coarse level. In particular, they ensure that the range `result[0..n+m-1]` is ordered and that its length matches the combined lengths of the two input arrays. These properties are sufficient to establish that `merge` produces an ordered array of the expected size.

However, the specification deliberately abstracts from detailed relationships between the input and output arrays. Unlike the specifications of the sorting algorithms studied earlier, the contract does not formalize how individual elements of the input arrays contribute to the output. In particular, it neither states that the output is a permutation of the inputs nor captures any notion of stability.

As a result, several properties that belong to the intended behavior of `merge` are not yet expressed at the logical level. This gap is made explicit in the following open issue, which outlines possible extensions of the postcondition.

Open Issue 10.8.1 (Completeness of the postcondition of merge).
The current formal specification characterizes only the size and the ordering of the output array. The following additional properties, which are stated informally, are not yet captured by the formal postconditions:

- *all elements of the input arrays appear exactly once in the output;*
- *if two elements from different input arrays are equal, the element originating from a precedes the element originating from b;*
- *the relative order of equal elements within each input array is preserved.*

These aspects will be addressed in subsequent versions of this document.

10.8.2. More Lemmas on `WeaklyIncreasing`

The following listing introduces several auxiliary lemmas about `WeaklyIncreasing` [6.2] that are repeatedly used in the formal verification of `merge`. Each lemma captures a simple but frequently needed reasoning step about weakly increasing array segments.

- Lemma `WeaklyIncreasing_Shrink` [10.23] expresses stability under restriction: any sub-range of a weakly increasing array is itself weakly increasing.
- Lemma `WeaklyIncreasing_AddElement` characterizes extension by one element. It states that appending an element preserves the weakly increasing property, provided the new element is greater than or equal to the last element of the existing range.
- Lemma `WeaklyIncreasing_Shift` captures invariance under pointer arithmetic. It allows rewriting `WeaklyIncreasing` when array segments are accessed through shifted pointers.
- Lemma `WeaklyIncreasing_Unchanged` states that the weakly increasing property is preserved between two program points whenever the corresponding array segment remains unchanged.
- Lemma `WeaklyIncreasing_Equal` formalizes preservation under element-wise equality. If two array segments are pointwise equal, then `WeaklyIncreasing` holds for one segment if and only if it holds for the other.
- Lemma `WeaklyIncreasing_Join` provides sufficient conditions for concatenating two consecutive weakly increasing ranges. The combined range is weakly increasing if both sub-ranges are weakly increasing and the last element of the first does not exceed the first element of the second.

```
/*@
  lemma WeaklyIncreasing_Shrink{L}:
    \forall value_type *a, integer m, n, p, q;
      m <= p <= q <= n                ==>
      WeaklyIncreasing(a, m, n)       ==>  WeaklyIncreasing(a, p, q);

  lemma WeaklyIncreasing_AddElement{L}:
    \forall value_type *a, integer n;
      1 < n                           ==>
      a[n-2] <= a[n-1]                ==>
      WeaklyIncreasing(a, n-1)        ==>  WeaklyIncreasing(a, n);

  lemma WeaklyIncreasing_Shift{L}:
    \forall value_type *a, integer m, n;
      WeaklyIncreasing(a + m, 0, n)  <==>  WeaklyIncreasing(a, m, n + m);

  lemma WeaklyIncreasing_Equal{K,L}:
    \forall value_type *a, *b, integer m, n;
      Equal{K,L}(a, m, n, b)          ==>
      WeaklyIncreasing{K}(a, m, n)    ==>  WeaklyIncreasing{L}(b, m, n);

  lemma WeaklyIncreasing_Unchanged{K,L}:
    \forall value_type *a, integer m, n;
      Unchanged{K,L}(a, m, n)         ==>
      WeaklyIncreasing{K}(a, m, n)    ==>  WeaklyIncreasing{L}(a, m, n);

  lemma WeaklyIncreasing_Join{L}:
    \forall value_type *a, integer m, n;
      0 < m < n                       ==>
      WeaklyIncreasing(a, m)          ==>
      WeaklyIncreasing(a, m, n)       ==>
      a[m-1] <= a[m]                  ==>  WeaklyIncreasing(a, n);
*/
```

Listing 10.23: `WeaklyIncreasingLemmas` (ACSL)

10.8.3. Implementation of `merge`

The implementation of `merge`, shown in Listings 10.24 and 10.25, follows the standard merge procedure. Two read positions traverse the input arrays `a` and `b`, while a write position advances through the output array `result`. At each step, the smaller of the two current input elements is written to the next position of the result array. In this way, an increasing prefix of the output array is constructed incrementally.

The main **`while`** loop is responsible for this incremental construction. Its loop invariants express that the already written prefix of `result` is weakly increasing and that all elements written so far are taken from the corresponding prefixes of the input arrays. Several assertions appear in the loop body. These assertions do not affect the algorithmic behavior but serve to make intermediate reasoning steps explicit for the automatic provers.

```
void merge(const value_type* a, size_type m,
           const value_type* b, size_type n, value_type* c)
{
  //@ assert increasing:  WeaklyIncreasing(a, 0, m);
  size_type i = 0;
  size_type j = 0;
  size_type x = 0;

  if (0 < m || 0 < n) {
    /*@ loop invariant index:       0 <= i <= m;
        loop invariant index:       0 <= j <= n;
        loop invariant index:       x == i+j;
        loop invariant index:       0 <= x <= m+n-1;
        loop invariant upper:       i < m  ==>  UpperBound(c, 0, x, a[i]);
        loop invariant upper:       j < n  ==>  UpperBound(c, 0, x, b[j]);
        loop invariant increasing:  WeaklyIncreasing(c, x);
        loop assigns i, j, x, c[0 .. m+n-1];
        loop variant (m+n) - (i+j);
     */
    while (i < m && j < n) {
      if (a[i] < b[j]) {
        c[x++] = a[i++];
        //@ assert upper:       i < m  ==>  UpperBound(c, 0, x, a[i]);
        //@ assert increasing:  WeaklyIncreasing(c, 0, x);
      }
      else {
        c[x++] = b[j++];
        //@ assert upper:       j < n  ==>  UpperBound(c, 0, x, b[j]);
        //@ assert increasing:  WeaklyIncreasing(c, 0, x);
      }

      //@ assert increasing:  WeaklyIncreasing(c, 0, x);
    }
```

Listing 10.24: Implementation of `merge` (1)

After termination of the **`while`** loop, at least one of the input arrays has been completely processed. At this point, the algorithm enters its epilogue, starting at the ghost label `Epilogue` (see Listing 10.25). The remaining elements of the other input array are copied unchanged to the result array.

Since these elements are already ordered and all previously written elements form a weakly increasing prefix, the entire output array is weakly increasing. The postcondition `increasing` then follows by Lemma `WeaklyIncreasing_Increasing` [6.3].

```
    //@ ghost  Epilogue: ;
    //@ assert index:       x == i+j;
    //@ assert index:       i == m  ^^  j == n;
    //@ assert index:       i <  m  ^^  j <  n;
    //@ assert increasing:  WeaklyIncreasing(c, 0, x);
    //@ assert unchanged:   Unchanged{Pre,Here}(a, 0, m);
    //@ assert increasing:  WeaklyIncreasing(a, 0, m);

    if (i < m) {
      //@ assert upper:       0 < x  ==>  c[x-1] <= a[i];
      //@ assert increasing:  WeaklyIncreasing(a, i, m);
      //@ assert increasing:  WeaklyIncreasing(a+i, 0, m-i);

      copy(a + i, m - i, c + x);
      //@ assert equal:       c[x] == At{Epilogue}(a, i);
      //@ assert equal:       a[i] == At{Epilogue}(a, i);
      //@ assert equal:       c[x] == a[i];
      //@ assert equal:       Equal{Epilogue,Here}(a+i, m-i, c+x);
      //@ assert index:       m-i+x == m+n;
      //@ assert unchanged:   Unchanged{Pre,Here}(a, 0, m);
      //@ assert unchanged:   Unchanged{Pre,Here}(b, 0, n);
      //@ assert increasing:  WeaklyIncreasing(c+x, 0, m-i);
      //@ assert increasing:  WeaklyIncreasing(c, x, m+n);
    }
    else {
      //@ assert upper:       0 < x  ==>  c[x-1] <= b[j];
      //@ assert unchanged:   Unchanged{Pre,Here}(a, 0, m);
      //@ assert unchanged:   Unchanged{Pre,Here}(b, 0, n);
      //@ assert increasing:  WeaklyIncreasing(b, 0, n);
      //@ assert increasing:  WeaklyIncreasing(b+j, 0, n-j);

      copy(b + j, n - j, c + x);
      //@ assert equal:       c[x] == At{Epilogue}(b, j);
      //@ assert equal:       b[j] == At{Epilogue}(b, j);
      //@ assert equal:       c[x] == b[j];
      //@ assert equal:       Equal{Epilogue,Here}(b+j, n-j, c+x);
      //@ assert index:       n-j+x == m+n;
      //@ assert unchanged:   Unchanged{Epilogue,Here}(c, 0, x);
      //@ assert increasing:  WeaklyIncreasing(c+x, 0, n-j);
      //@ assert increasing:  WeaklyIncreasing(c, x, m+n);
    }

    //@ assert unchanged:   Unchanged{Epilogue,Here}(c, 0, x);
    //@ assert increasing:  WeaklyIncreasing(c, 0, x);
    //@ assert increasing:  WeaklyIncreasing(c, x, m+n);
    //@ assert increasing:  0 < x  ==>  c[x-1] <= c[x];
    //@ assert increasing:  0 < x  ==>  WeaklyIncreasing(c, x-1, m+n);
    //@ assert increasing:  WeaklyIncreasing(c, 0, m+n);
  }
}
```

Listing 10.25: Implementation of merge (2)

Part V.

Verification of data structures

Introduction

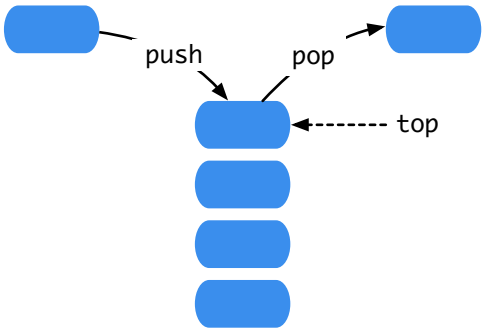
This part initiates the study of data structure verification. As a first representative example, we focus on a stack abstraction, which allows us to introduce the relevant specification patterns and proof techniques in a controlled setting. Further data structures will be incorporated in subsequent extensions of this document.

11. The Stack data type

So far, we have used the ACSL specification language primarily to specify and verify individual C functions in isolation. In practical software development, however, one is often concerned with the implementation of a whole family of functions centered around a non-trivial data structure. Such functions are not independent. Rather, they are required to interact in a disciplined way and to satisfy consistency properties that relate their behaviors.

In this setting, it is no longer sufficient to reason about each function in isolation. Instead, we must also formulate and verify properties that describe how several functions operate together on shared data and how their effects are meant to compose.

As a first example of such a data structure, we consider the *stack* abstraction. A stack is a deliberately simple abstraction, yet it already exhibits non-trivial interaction between its operations. Its defining characteristic is a strict *last-in, first-out* discipline, which constrains the order in which stored elements may be accessed and removed. Despite its simplicity, the stack therefore provides a suitable setting for studying consistency properties that relate multiple operations.



A *stack* is a data type for storing objects with the property that, if an object a is *pushed* onto the stack *before* an object b, then a can only be removed (*popped*) after b. In other words, a stack is a *first-in, last-out* data type. The *top* operation returns the element that was most recently pushed onto the stack and has not yet been removed.

Figure 11.1.: Basic operations of a stack

The development presented here builds on the results of a previous diploma thesis by one of the authors [23], which established the overall approach to the formal specification and verification of the `Stack` data type.

The C++ Standard Library provides the generic container adaptor `std::stack` [18, §26.6.6], which models a stack whose maximal size is not fixed a priori and may grow to an arbitrary number of elements, subject only to the availability of memory. Its interface and observable behavior serve as a reference that we follow as closely as the constraints of our C implementation allow.

In contrast, we consider only stacks with a finite *capacity*. Such a stack can store at most a fixed number c of elements, which is determined at the moment the stack is initialized. This bound remains constant throughout the lifetime of the stack and allows us to define the data type without relying on dynamic memory allocation.

When a stack is created or initialized, it contains no elements, that is, its *size* is 0. The operations *push* and *pop* increase and decrease the size of the stack by at most one, respectively.

11.1. Methodology overview

In this section we present a methodology for this task using the data type of stacks as a running example. The stack abstraction serves as a representative case study through which the general principles of our approach can be made precise. Our methodology distinguishes three systematically related levels, illustrated in Figure 11.2.

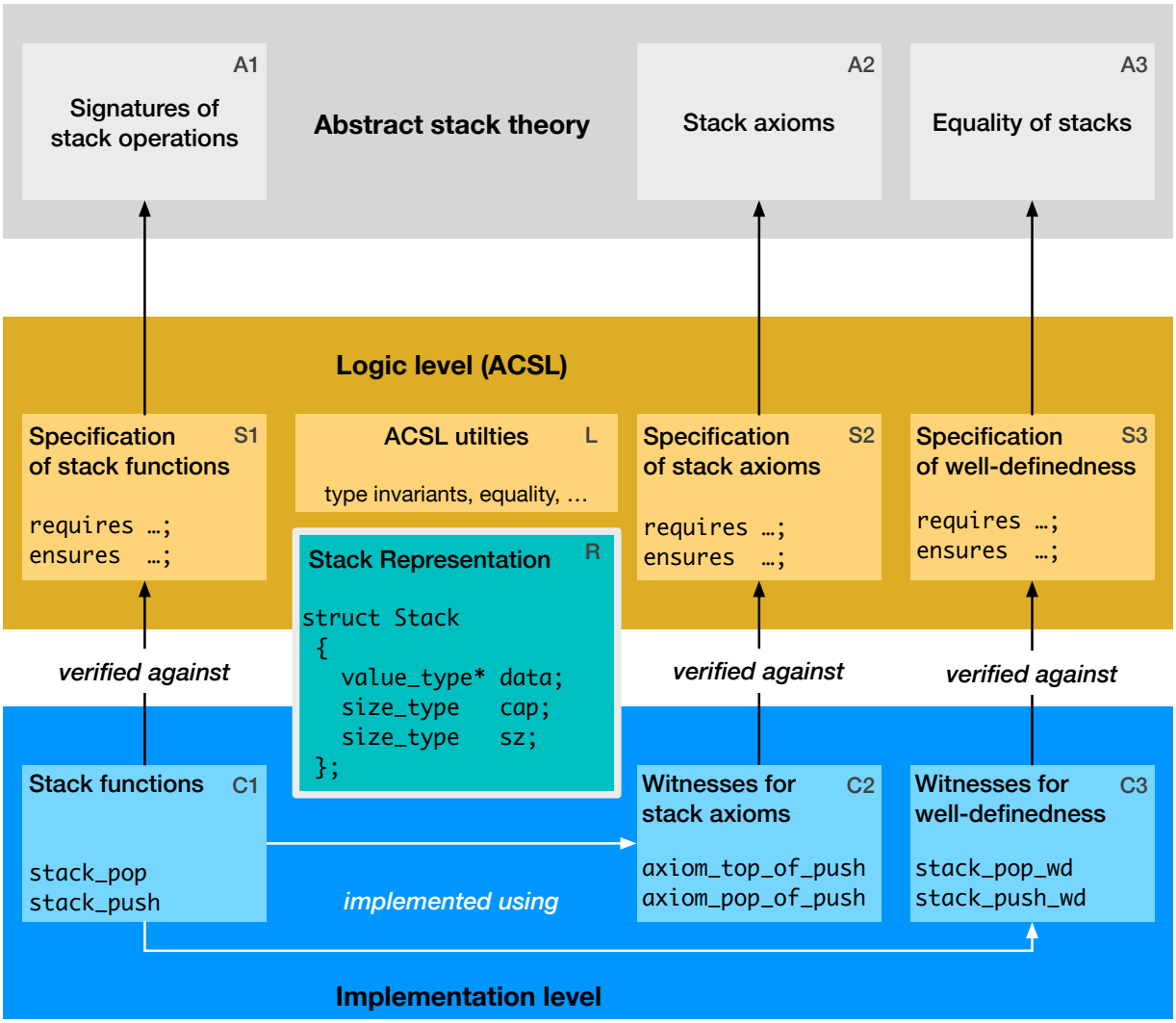


Figure 11.2.: Methodology for verifying the stack data type.

1. At the *abstract level*, the stack is specified algebraically by a collection of abstract operations (A1), whose interaction is governed by well-known axioms (A2). These axioms induce a representation as finite sequences together with a corresponding notion of equality (A3). This yields a purely mathematical model, independent of any concrete programming-language representation.

2. At the *logic specification level*, reusable ACSL definitions capture fundamental properties of the C representation type `Stack` (R), such as invariants and observational equality. They form the basis for the contracts of the stack functions (S1) and for the specification of *witnesses* establishing the stack axioms (S2) and well-definedness (S3).

3. At the *implementation level*, C functions (C1) implement the abstract operations. Other C functions act as witnesses that establish essential properties, in particular the axioms and well-definedness results (C2, C3).

The following sections elaborate these three levels in turn, starting with the abstract axiomatization (§11.2), followed by a discussion of the concrete C representation of stacks (§11.3), then the logic specifications, comprising invariants and observational equality (§11.4 and §11.5), and culminating in the verified implementation (§11.6).

Verifying operations and axioms

Verifying the contracts of individual stack operations ensures that each function behaves according to its specification. However, the axioms of the abstract stack theory typically describe relationships between several operations and therefore concern their interaction rather than the behavior of a single function.

To verify such properties, we introduce small auxiliary C functions that combine several stack operations in the manner described by a given axiom. Together with an ACSL contract expressing the equality stated by the axiom, such a function forms what we call an *axiom witness*, which is introduced and studied in detail in §11.7.

Verifying such a witness function establishes that the implementation satisfies the corresponding algebraic property.

Observational equality and well-definedness

The stack axioms implicitly assume a notion of equality between stacks. At the level of C implementations this notion must be defined explicitly. For our finite-capacity stacks, equality is defined observationally. Two stacks are considered equal if they exhibit the same observable behavior, even if their internal capacities differ (§11.5). This notion of equality forms the basis for both the formulation of axiom witnesses and the well-definedness results discussed below.

Once equality is defined in this way, it is no longer immediate that all stack operations respect this equivalence. In particular, operations whose behavior depends on the internal capacity of a stack might violate observational equality.

This raises the question whether the stack operations are *well-defined* with respect to observational equality. Informally, this means that equal stacks must be mapped to equal stacks by every operation. To answer this question, we introduce additional auxiliary functions, called *well-definedness witnesses*. Their contracts express the required preservation of observational equality (§11.8).

11.2. Axioms and invariants of finite-capacity stacks

To specify the interaction of the stack operations, we introduce a small collection of axioms. These axioms characterize stacks implicitly through the properties of their operations, rather than through a concrete representation.

Let V be a value type in the usual programming sense, that is, a type whose objects can be copied and compared for equality. We denote by S the type of stacks whose elements are of type V. The stack operations considered in this chapter have the following signatures:

Constructor: $$\text{init} : \mathbb{N} \to S \tag{11.1}$$

Observers: $$\text{capacity} : S \to \mathbb{N} \qquad \text{size} : S \to \mathbb{N} \qquad \text{top} : S \to V \tag{11.2}$$

Modifiers: $$\text{push} : S \times V \to S \qquad \text{pop} : S \to S \tag{11.3}$$

Every stack $s \in S$ satisfies the following invariant, capturing the idea of a stack with finite capacity:

$$0 \leqslant \text{size}(s) \leqslant \text{capacity}(s). \tag{11.4}$$

Closely related to these invariants are the two auxiliary predicates $\text{empty} : S \to \mathbb{B}$ and $\text{full} : S \to \mathbb{B}$. They are defined in terms of the primitive observer functions size and capacity and therefore impose no additional constraints on stacks.

$$\text{empty}(s) \iff \text{size}(s) = 0, \tag{11.5}$$
$$\text{full}(s) \iff \text{size}(s) = \text{capacity}(s). \tag{11.6}$$

To qualify as a stack, these operations are required to satisfy a number of basic consistency properties. We collectively refer to these properties as the *stack axioms*.

Initialization of a stack:

$$\text{size}(\text{init}(c)) = 0, \tag{11.7}$$
$$\text{capacity}(\text{init}(c)) = c. \tag{11.8}$$

Adding an element to a stack:

$$\text{size}(\text{push}(s, v)) = \text{size}(s) + 1, \qquad \text{if} \quad \neg\,\text{full}(s), \tag{11.9}$$
$$\text{capacity}(\text{push}(s, v)) = \text{capacity}(s), \qquad \text{if} \quad \neg\,\text{full}(s), \tag{11.10}$$
$$\text{top}(\text{push}(s, v)) = v, \qquad \text{if} \quad \neg\,\text{full}(s). \tag{11.11}$$

Removing an element from a stack:

$$\text{size}(\text{pop}(s)) = \text{size}(s) - 1, \qquad \text{if} \quad \neg\,\text{empty}(s), \tag{11.12}$$
$$\text{capacity}(\text{pop}(s)) = \text{capacity}(s), \qquad \text{if} \quad \neg\,\text{empty}(s), \tag{11.13}$$
$$\text{pop}(\text{push}(s, v)) = s, \qquad \text{if} \quad \neg\,\text{full}(s), \tag{11.14}$$
$$\text{push}(\text{pop}(s), \text{top}(s)) = s, \qquad \text{if} \quad \neg\,\text{empty}(s). \tag{11.15}$$

A note on exception handling We deliberately refrain from specifying the behavior of `push(s, v)` when `s` is full, as well as the behavior of `pop(s)` and `top(s)` when `s` is empty. Specifying stack operations

in such *exceptional* situations is a substantial problem in its own right, and a wide range of approaches can be found in the literature.

Rather than pursuing this topic further, we restrict ourselves to a single illustrative example that demonstrates how seemingly innocuous exception specifications may lead to unintended and even inconsistent consequences.

Suppose that the element type V is extended by an additional error value `err`, and that the `top` operation is required to satisfy

$$\mathrm{top}(s) = \mathrm{err}$$

whenever `s` is empty. This immediately raises the question of how the operation `push(s, err)` should behave. At first glance, it might appear reasonable to simply ignore such a value and to postulate

$$\mathrm{push}(s, \mathrm{err}) = s. \tag{11.16}$$

However, this assumption leads to a contradiction. Indeed, for any stack `s` that is neither full nor empty, we obtain

$$\begin{aligned}
\mathrm{size}(s) &= \mathrm{size}(\mathrm{pop}(\mathrm{push}(s, \mathrm{err}))) && \text{by (11.14)},\\
&= \mathrm{size}(\mathrm{pop}(s)) && \text{by (11.16)},\\
&= \mathrm{size}(s) - 1 && \text{by (11.12)}.
\end{aligned}$$

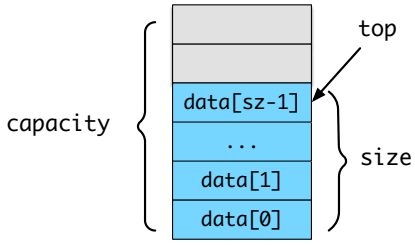
This equality is impossible. Consequently, no such stack `s` can exist, and the axioms collapse into an inconsistent theory. This example illustrates that exception handling cannot be added naïvely at the axiomatic level without a careful reconsideration of the overall specification.

11.3. The structure Stack and its associated functions

We now introduce a concrete C implementation that realizes the stack axioms presented in §11.2. Listing 11.3 defines the concrete representation of type `Stack`. The schematic view on the right merely illustrates the source code.

```
struct Stack {
  value_type*  data;
  size_type    cap;
  size_type    sz;
};

typedef struct Stack Stack;
```



Listing 11.3: Definition of type `Stack` and schematic interpretation

Starting at address `data`, `Stack` contains an array of fixed length `cap`, called the *capacity*, which represents the maximum number of elements that an instance of `Stack` can hold. The field `sz` records the *size* of a `Stack` object, that is, the number of elements currently stored in the stack.

Based on the abstract operations introduced in §11.2, we now declare the concrete interface of the `Stack` data type. Table 11.4 relates these function declarations to the corresponding methods of the C++ interface `std::stack`, where such a correspondence exists.

C function	`std::stack` counterpart	Remark
`stack_init(s, a, n)`	`std::stack<T,Cont> st(c);`	Construction from container.
`stack_equal(s, t)`	`operator==`	Same purpose.
`stack_size(s)`	`size()`	Same purpose.
`stack_capacity(s)`	—	No fixed capacity concept.
`stack_empty(s)`	`empty()`	Same purpose.
`stack_full(s)`	—	No fixed capacity concept.
`stack_top(s)`	`top()`	Same purpose (undefined if empty).
`stack_push(s, v)`	`push(v)`	Defined only for non-full stacks.
`stack_pop(s)`	`pop()`	Same purpose (undefined if empty).

Table 11.4.: C `Stack` interface versus `std::stack`.

Most of these functions closely correspond to the operations provided by the C++ `std::stack` template class [18, §26.6.6.1]. In particular, `stack_equal` corresponds to the equality operator `==`. The function `stack_init` is used to bring a stack into a well-defined initial state. By contrast, the function `stack_full` has no direct counterpart in `std::stack`. Similarly, `stack_push` is only defined for `Stack` objects that are not full. This difference reflects a deliberate design choice: our implementation operates on a fixed-capacity data structure and therefore avoids dynamic memory allocation, whereas `std::stack` relies on it.

11.4. Logical interface and invariants of Stack

In this section, we introduce a collection of logic functions and predicates that will be used throughout this chapter. Together, these definitions constitute a *logical interface* to stacks and establish an abstraction barrier between the concrete representation and its formal specification. They allow us to reason about stacks at an abstract level, without reference to implementation details. In addition, we introduce the predicate `StackInvariant` [11.7], which characterizes exactly those instances of `Stack` that represent valid stacks.

11.4.1. Logical interface: functions and predicates

We begin by introducing in Listing 11.5 three auxiliary logic functions: `StackCapacity`, `StackSize`, and `StackData`. These functions provide logical access to the fields `cap`, `sz`, and `data` of a `Stack` instance, respectively. They can be used conveniently in specifications without exposing the underlying concrete representation.

```
/*@
  logic value_type* StackData{S}(Stack* s)     = s->data;

  logic integer     StackCapacity{S}(Stack* s) = s->cap;

  logic integer     StackSize{S}(Stack* s)     = s->sz;
*/
```

Listing 11.5: `Stack` (ACSL)

Listing 11.6 defines the logic function `StackTop` and the predicates `StackEmpty` and `StackFull`, which characterize empty and full stacks (see Equations (11.5) and (11.6)).

```
/*@
  logic value_type StackTop{S}(Stack* s)  =  StackData(s)[StackSize(s)-1];

  predicate StackEmpty{S}(Stack* s)       =  StackSize(s) == 0;

  predicate StackFull{S}(Stack* s)        =  StackSize(s) == StackCapacity(s);
*/
```

Listing 11.6: `StackObservers` (ACSL)

Finally, we note that the numeric results of all logic functions introduced here are using the ACSL type `integer`. This choice is intentional. Using bounded logic types in combination with arithmetic operations in axioms can easily lead to inconsistencies, whereas unbounded integers provide a sound foundation for reasoning about sizes, indices, and cardinalities.

11.4.2. Invariants of Stack

Not every instance of type `Stack` represents a valid stack. For example, with the definition of `Stack` given in Listing 11.3, the initializer `Stack s = {{0,0,0,0}, 4, 5}` is invalid, since it violates the size–capacity inequalities stated in (11.4): a stack of capacity 4 cannot contain 5 elements.

In ACSL, structural consistency conditions are naturally expressed as *type invariants*. More precisely, ACSL distinguishes between *strong* and *weak* type invariants [24, § 2.12]. A strong invariant must hold at every program point during execution. A weak invariant, by contrast, is required to hold only at function

boundaries — upon entry and upon exit — and may be temporarily violated within a function body. At the time of writing, however, support for type invariants is not yet implemented in Frama-C. For this reason, and in order to distinguish formally between well-formed and ill-formed instances of `Stack`, we introduce in the following listing the predicate `StackInvariant`.

```
/*@
  predicate StackInvariant{L}(Stack* s) =
    0 < StackCapacity(s)                          &&
    0 <= StackSize(s) <= StackCapacity(s)         &&
    \valid(StackData(s) + (0..StackCapacity(s)-1)) &&
    \separated(s, StackData(s) + (0..StackCapacity(s)-1));

  predicate StackValid{L}(Stack* s) = \valid(s) && StackInvariant(s);
*/
```

Listing 11.7: `StackInvariant` (ACSL)

It is worth emphasizing that `StackInvariant` does not merely restate the mathematical properties of the abstract data type *stack*. In particular, additional conditions on memory validity and separation are required to guarantee that stack operations are well-defined at the level of concrete program execution. More precisely, the invariant ensures that:

- The capacity is strictly positive.
- The size is nonnegative and does not exceed the capacity, as expressed by (11.4).
- The underlying data array is valid over the entire memory region determined by the capacity.
- The stack object itself is disjoint from the memory region used to store its elements.

Listing 11.7 also defines the predicate `StackValid`, which combines two complementary aspects of correctness. First, the condition `\valid(s)` ensures that the pointer s itself denotes a valid memory location, so that accessing the structure fields is well-defined in the sense of the C memory model. Second, the predicate `StackInvariant` encapsulates the type invariants of the stack object.

11.5. Equality of stacks

Defining equality for instances of non-trivial data types is a subtle problem, particularly in the presence of mutable state and object identity. This issue is well known in object-oriented programming: for example, the book *Programming in Scala* [25] devotes an entire chapter to the definition and correct use of equality. In a formal verification setting the problem is even more delicate, since an imprecise notion of equality can invalidate otherwise correct reasoning.

Conceptually, one distinguishes between *structural* equality and *observational* equality. Structural equality compares the complete internal representation of objects. Observational equality, by contrast, identifies objects that exhibit the same externally visible behavior, even if their internal representations differ.

For stacks we adopt an observational notion of equality. Two stacks are considered equal if they have the same size and represent the same sequence of elements, irrespective of their capacity. Equality is therefore defined in terms of observable contents, not in terms of member-wise comparison of all fields. Thus the two stacks shown in Figure 11.8 are considered equal even though their internal arrays differ outside the range of stored elements. In particular, equal stacks may have different capacities, and one may be full while the other is not. This mirrors the behavior of the C++ standard container `vector`, whose equality is likewise determined by its stored elements rather than its reserved capacity.

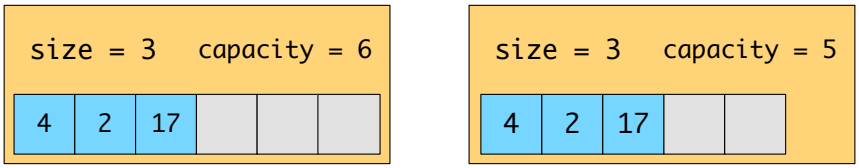


Figure 11.8.: Example of two equal stacks

The predicate `StackEqual` in Listing 11.9 formalizes equality of stacks in ACSL. Its definition is factored through the auxiliary predicate `StackPrefix`. This predicate uses `Equal` [4.28] to express that the contents of a stack `s` coincide with an initial segment of the contents of `t`.

```
/*@
  predicate StackPrefix{S,T}(Stack* s, Stack* t) =
    StackSize{S}(s) <= StackSize{T}(t)  &&
    Equal{S,T}(StackData{S}(s), StackSize{S}(s), StackData{T}(t));

  predicate StackEqual{S,T}(Stack* s, Stack* t) =
    StackSize{S}(s) == StackSize{T}(t) &&
    StackPrefix{S,T}(s, t);

  lemma StackEqual_Reflexive{S} :
    \forall Stack* s; StackEqual{S,S}(s, s);

  lemma StackEqual_Symmetric{S,T} :
    \forall Stack *s, *t;
      StackEqual{S,T}(s, t) ==>  StackEqual{T,S}(t, s);

  lemma StackEqual_Transitive{S,T,U}:
    \forall Stack *s, *t, *u;
      StackEqual{S,T}(s, t) ==> StackEqual{T,U}(t, u) ==> StackEqual{S,U}(s, u);
*/
```

Listing 11.9: `StackEqual` (ACSL)

Based on this notion, `StackEqual` requires that both stacks have the same size and that `s` is a prefix of `t`. Equal sizes exclude proper prefixes and therefore ensure that both stacks store exactly the same sequence

of elements. The capacity of a stack is intentionally disregarded, since equality is defined solely in terms of observable contents.

`StackEqual` as an equivalence relation Calling a relation "equality" is not merely a matter of terminology. At the very least, it must satisfy the defining properties of an equivalence relation. Accordingly, the listing establishes the following properties:

$$\textit{Reflexivity}: \quad \forall s \in S: \quad s = s, \tag{11.17a}$$

$$\textit{Symmetry}: \quad \forall s, t \in S: \quad s = t \implies t = s, \tag{11.17b}$$

$$\textit{Transitivity}: \quad \forall s, t, u \in S: \quad s = t \implies t = u \implies s = u. \tag{11.17c}$$

As already mention in Equations (5.1), we adopt the usual convention that the implication symbol $\implies$ associates to the right.

In Listing 11.9, these properties are formulated explicitly as lemmas about `StackEqual`. They are therefore not postulated but must be proved within the verification framework. In fact, Frama-C/WP is able to discharge the corresponding verification conditions automatically with the help of SMT provers. This provides formal confirmation that `StackEqual` indeed behaves as a well-defined notion of equality.

Well-definedness with respect to stack equality Any relation satisfying the properties (11.17) is called an *equivalence relation*. The set of all instances that are equal to a given instance `s` is called the equivalence class of `s`.

The two stacks shown in Figure 11.8 illustrate that an equivalence class of `StackEqual` may contain more than one element. This is a familiar situation in mathematics: for example, the rational number $\frac{1}{2}$ represents the same equivalence class as $\frac{2}{4}$ or $\frac{7}{14}$. Accordingly, the stacks `s` and `t` in the figure can be regarded as two *representatives* of the same equivalence class.

In such a setting, an important question arises: can functions defined on stacks be specified in a way that is independent of the chosen representative? In other words, are these functions *well-defined* with respect to the equivalence relation `StackEqual`? We will answer this question in Section 11.8.

11.6. Specification and implementation of stack functions

In previous chapters, the verification of individual functions was usually carried out directly at the beginning of the chapter. For abstract data types, however, this approach is no longer sufficient. Before individual operations can be verified in isolation, a number of conceptual issues must be addressed, including data type invariants, user-defined equality, and the notion of well-definedness with respect to equivalence classes.

Having settled these foundations, we now turn to the verification of the individual stack operations. In §11.7, the verification results are lifted to a more abstract level. We show there that the combined behavior of the verified operations satisfies the stack axioms introduced in §11.2.

11.6.1. Verification of `stack_equal`

The function `stack_equal` decides at execution time whether two stacks are equal in the sense of `StackEqual` [11.9]. The specification of `stack_equal` in Listing 11.10 requires that both arguments satisfy the type invariant `StackInvariant` [11.7].

```
/*@
  requires   valid:      StackValid(s) && StackValid(t);
  terminates             \true;
  exits                  \false;
  assigns                \nothing;
  ensures    equal:      \result == true   <==>  StackEqual{Here,Here}(s, t);
  ensures    not_equal:  \result == false  <==> !StackEqual{Here,Here}(s, t);
*/
bool stack_equal(const Stack* s, const Stack* t);
```

Listing 11.10: Formal specification of `stack_equal`

The two postconditions `equal` and `not_equal` are not merely stylistic. In our setting **`bool`** is a type alias for **`int`**. The return value of `stack_equal` is therefore not restricted by the type system to the values **`true`** (1) and **`false`** (0). The first equivalence ensures that the value **`true`** corresponds exactly to the logical predicate `StackEqual`. However, it does not exclude other integer values in the non-equal case. The second equivalence closes this gap by enforcing that logical false is represented precisely by **`false`**. Together, the two clauses establish a bijective correspondence between the two-valued predicate `StackEqual` and its concrete integer representation.

The implementation of `stack_equal`, shown in Listing 11.11, first compares the sizes of the two stacks. If the sizes coincide, it invokes the function `equal` [4.29] to compare the first `sz` elements of the underlying data arrays. Thus, equality is determined solely by the number and sequence of stored elements. The stack capacity does not enter the comparison and therefore has no influence on the result.

```
bool stack_equal(const Stack* s, const Stack* t)
{
  return (s->sz == t->sz) && equal(s->data, s->sz, t->data);
}
```

Listing 11.11: Implementation of `stack_equal`

11.6.2. Verification of stack_init

The contract of stack_init is shown in Listing 11.12. Its preconditions require a valid stack point, a strictly positive capacity, and a contiguous storage area of sufficient size that is separated from the stack descriptor itself.

The assigns clauses make the data dependencies explicit. The data pointer and the capacity field of the Stack object are set from the respective function arguments, while the size field is set to 0, independent of any input state. The postconditions state that the stack is properly initialized and the stack invariants are satisfied.

```
/*@
  requires   valid:      \valid(s);
  requires   capacity:   0 < cap;
  requires   storage:    \valid(data + (0..cap-1));
  requires   sep:        \separated(s, data + (0..cap-1));

  terminates             \true;
  exits                  \false;
  assigns                s->data \from data;
  assigns                s->cap  \from cap;
  assigns                s->sz   \from \nothing;

  ensures    valid:      StackValid(s);
  ensures    storage:    StackData(s)     == data;
  ensures    capacity:   StackCapacity(s) == cap;
  ensures    empty:      StackSize(s)     == 0;
*/
void stack_init(Stack* s, value_type* data, size_type cap);
```

Listing 11.12: Formal specification of stack_init

The implementation of stack_init is shown in the next listing. It performs exactly the initialization prescribed by the contract.

```
void stack_init(Stack* s, value_type* data, size_type cap)
{
  s->data  = data;
  s->cap   = cap;
  s->sz    = 0u;
}
```

Listing 11.13: Implementation of stack_init

11.6.3. Verification of stack_size

The function stack_size returns the current number of elements stored in a stack. Its specification, shown in Listing 11.14, requires that the argument is a valid stack. The contract further states that the function has no side effects (assigns \nothing) and always terminates without exceptional control flow.

The central postcondition expresses the functional behaviour of the operation. It states that the returned value is exactly the logical size of the stack as given by the logic function StackSize.

```
/*@
  requires   valid: StackValid(s);
  terminates        \true;
  exits             \false;
  assigns           \nothing;
  ensures    size:  \result == StackSize(s);
*/
size_type stack_size(const Stack* s);
```

Listing 11.14: Formal specification of `stack_size`

The implementation of `stack_size`, shown in the next listing, directly returns the value stored in the structure field `sz`. Since `StackSize` is defined to correspond to this field for valid stacks, the implementation can easily be verified.

```
size_type stack_size(const Stack* s)
{
  return s->sz;
}
```

Listing 11.15: Implementation of `stack_size`

11.6.4. Verification of `stack_capacity`

The function `stack_capacity` returns the maximum number of elements that can be stored in a stack. Its specification, shown in Listing 11.16, requires that the argument is a valid stack. The contract further states that the function has no side effects (`assigns \nothing`) and always terminates without exceptional control flow.

The functional behaviour is expressed by the postcondition stating that the return value is exactly the logical capacity of the stack as given by the logic function `StackCapacity`.

```
/*@
  requires   valid:    StackValid(s);
  terminates           \true;
  exits                \false;
  assigns              \nothing;
  ensures    capacity: \result == StackCapacity(s);
*/
size_type stack_capacity(const Stack* s);
```

Listing 11.16: Formal specification of `stack_capacity`

The implementation of `stack_capacity`, shown in the following listing, simply returns the value stored in the structure field `cap`.

```
size_type stack_capacity(const Stack* s)
{
  return s->cap;
}
```

Listing 11.17: Implementation of `stack_capacity`

11.6.5. Verification of stack_empty

The function stack_empty provides a runtime check for the logical predicate StackEmpty [11.6]. Its specification is shown in Listing 11.18. The contract requires that the argument is a valid stack. Furthermore, the function has no side effects and always terminates normally.

The functional behaviour is specified by two equivalent characterizations. The first postcondition states that the function returns **true** exactly when the logical predicate StackEmpty holds for the given stack. The second postcondition states the complementary condition: the function returns **false** exactly when the stack is not empty. Together, these clauses establish a precise correspondence between the runtime result and the logical notion of stack emptiness.

```
/*@
  requires   valid:      StackValid(s);
  terminates             \true;
  exits                  \false;
  assigns                \nothing;
  ensures    empty:      \result == true   <==>  StackEmpty(s);
  ensures    not_empty:  \result == false  <==> !StackEmpty(s);
*/
bool stack_empty(const Stack* s);
```

Listing 11.18: Formal specification of stack_empty

The implementation of stack_empty, shown in the following listing, checks whether the size of the stack is zero.

```
bool stack_empty(const Stack* s)
{
  return stack_size(s) == 0u;
}
```

Listing 11.19: Implementation of stack_empty

11.6.6. Verification of stack_full

The function stack_full provides a runtime check for the logical predicate StackFull [11.6]. Its specification is shown in Listing 11.20. The contract requires that the argument is a valid stack. Furthermore, the function has no side effects and always terminates normally.

The functional behaviour is expressed by postconditions that relate the return value of the function to the logical predicate StackFull. In particular, the result is **true** exactly when the stack is full, and **false** otherwise.

```
/*@
  requires   valid:     StackValid(s);
  terminates            \true;
  exits                 \false;
  assigns               \nothing;
  ensures    full:      \result == \true   <==>   StackFull(s);
  ensures    not_full:  \result == \false  <==>  !StackFull(s);
*/
bool stack_full(const Stack* s);
```

Listing 11.20: Formal specification of stack_full

The implementation of `stack_full`, shown in the following listing, checks whether the current size of the stack is equal to its capacity.

```
bool stack_full(const Stack* s)
{
  return stack_size(s) == s->cap;
}
```

Listing 11.21: Implementation of `stack_full`

11.6.7. Verification of stack_top

The function `stack_top` returns the element currently stored at the top of a stack. Its specification is given in Listing 11.22. The contract requires that the argument is a valid stack. Furthermore, the function has no side effects and always terminates normally.

The functional behaviour is specified conditionally. If the stack is not empty, the returned value must coincide with the logical value given by `StackTop`. No requirement is imposed on the result when the stack is empty.

```
/*@
  requires   valid:  StackValid(s);
  terminates         \true;
  exits              \false;
  assigns            \nothing;
  ensures    top:    !StackEmpty(s) ==> \result == StackTop(s);
*/
value_type stack_top(const Stack* s);
```

Listing 11.22: Formal specification of `stack_top`

The implementation of `stack_top` is shown in Listing 11.23. It returns the element stored at index `sz-1` of the internal array, which corresponds to the top element of the stack. If the stack is empty, the expression evaluates to the first array element. Since the specification does not constrain the result in this case, this behaviour is admissible.

```
value_type stack_top(const Stack* s)
{
  if (!stack_empty(s)) {
    return s->data[s->sz - 1u];
  }
  else {
    return s->data[0u];
  }
}
```

Listing 11.23: Implementation of `stack_top`

11.6.8. Verification of stack_push

The predicate $\mathrm{StackPush}(s, v, t)$ in Listing 11.24 characterizes the situation in which the stack t results from pushing the element v onto the stack s. It states that t has the same capacity as s (Axiom (11.10)), that its size is larger by one (Axiom (11.9)), that v is the new top element of t (Axiom (11.11)), and that the remaining elements of t form the same sequence as the elements of s.

```
/*@
  predicate StackPush{S,T}(Stack* s, value_type v, Stack* t) =
    StackTop{T}(t)      == v                    &&
    StackCapacity{S}(s) == StackCapacity{T}(t)  &&
    StackSize{S}(s) + 1 == StackSize{T}(t)      &&
    StackPrefix{S,T}(s, t);
*/
```

Listing 11.24: StackPush (ACSL)

The specification of the function stack_push, shown in Listing 11.25, uses the predicate StackPush [11.24] to describe the effect of this operation when the stack is not full. In this case, the predicate captures that the resulting stack is obtained from the original one by adding the element to the top. The contract furthermore guarantees that the resulting stack is not empty. If the stack is already full, then stack_push leaves the stack unchanged.

```
/*@
  requires  valid:      StackValid(s);
  terminates            \true;
  exits                 \false;
  assigns               s->sz, s->data[s->sz];
  ensures   valid:      StackValid(s);

  behavior  full:
    assumes             StackFull(s);
    assigns             \nothing;
    ensures unchanged:  StackEqual{Old,Here}(s,s);

  behavior  not_full:
    assumes             !StackFull(s);
    assigns             s->sz, s->data[s->sz];
    ensures push:       StackPush{Old,Here}(s, v, s);
    ensures not_empty:  !StackEmpty(s);

  complete behaviors;
  disjoint behaviors;
*/
void stack_push(Stack* s, value_type v);
```

Listing 11.25: Formal specification of stack_push

The implementation of stack_push is given in Listing 11.26. It first checks whether the stack is not full. If this condition holds, the value to be pushed is written to the array position data[sz], and only afterwards is the field sz incremented.

```
void stack_push(Stack* s, value_type v)
{
  if (!stack_full(s)) {
    s->data[s->sz++] = v;
  }
}
```

Listing 11.26: Implementation of stack_push

11.6.9. Verification of stack_pop

The predicate StackPop in Listing 11.27 characterizes the situation in which the stack t results from removing the top element of the stack s. It states that both stacks have the same capacity (Axiom (11.13)), that the size of t is smaller by one (Axiom (11.12)), and that the sequence of elements stored in t coincides with the prefix of s obtained by removing its top element.

```
/*@
  predicate StackPop{S,T}(Stack* s, Stack* t) =
    StackCapacity{S}(s) == StackCapacity{T}(t)  &&
    StackSize{S}(s) - 1 == StackSize{T}(t)      &&
    StackPrefix{T,S}(t, s);
*/
```

Listing 11.27: StackPop (ACSL)

The specification of the function stack_pop is shown in Listing 11.28. The contract uses the predicate StackPop [11.27] to describe the effect of this operation when the stack is not empty.

```
/*@
  requires  valid:      StackValid(s);
  terminates            \true;
  exits                 \false;
  assigns               s->sz;
  ensures   valid:      StackValid(s);

  behavior  empty:
    assumes             StackEmpty(s);
    assigns             \nothing;
    ensures empty:      StackEmpty(s);
    ensures unchanged:  StackEqual{Old,Here}(s,s);

  behavior  not_empty:
    assumes             !StackEmpty(s);
    assigns             s->sz;
    ensures pop:        StackPop{Old,Here}(s, s);
    ensures not_full:   !StackFull(s);

  complete behaviors;
  disjoint behaviors;
*/
void stack_pop(Stack* s);
```

Listing 11.28: Formal specification of stack_pop

The implementation of stack_pop is given in the next listing. It checks whether the stack is non-empty and, if so, decreases the field size by one. No memory is modified when the stack is empty, in accordance with the specification.

```
void stack_pop(Stack* s)
{
  if (!stack_empty(s)) {
    --s->sz;
  }
}
```

Listing 11.29: Implementation of stack_pop

11.7. Witnesses of stack axioms

In the preceding section, we verified each stack operation individually against its specification. These verification results ensure that every function behaves correctly with respect to its contract when considered in isolation. They do not, however, guarantee that the operations interact in the intended way when combined.

The axioms of the abstract stack data type formulated in §11.2 describe algebraic relationships between several stack operations. Establishing that the implementation satisfies these axioms therefore requires reasoning about combinations of operations rather than about individual functions alone.

To this end, we introduce small auxiliary C functions, called *axiom witnesses*. Each witness combines stack operations in the manner prescribed by a given axiom and is equipped with an ACSL contract that captures the corresponding property. The `requires` clauses encode the preconditions of the axiom, while the postcondition expresses the required relationship between the resulting stack states or observable values.

Proving the contract of such a witness establishes that the concrete implementation of the involved operations satisfies the corresponding algebraic property. In this way, we show that the verified implementations introduced in §11.6 jointly realize the axiomatic specification of stacks.

Each witness function follows the same general scheme. It executes the operations occurring in the axiom and then returns either an observed value or the modified stack state. The essential property is expressed by a distinguished postcondition, typically labeled `size`, `capacity`, `top`, or `equal`. Whenever the axiom refers to the initial state of the stack, this is captured by the pre-state label `Old` or the ACSL operator `\old`.

Omission of assigns and termination clauses For better readability, we omit assigns and termination clauses in the witness specifications. These functions are not reused in other proofs and are not part of a modular verification chain. Their sole purpose is to establish the logical properties expressed by the stack axioms, rather than to serve as callable, reusable components.

11.7.1. Witnesses for the initialization of stacks

The operation `stack_init` creates a new stack with a given capacity. In the abstract stack theory, this corresponds to the constructor that produces an initial stack state. The associated axioms characterize fundamental properties of such a stack, in particular its size and capacity.

In this subsection, we verify these properties using axiom witnesses. Each witness initializes a stack and then observes the relevant property via the corresponding stack operation. The contract expresses the equality stated by the respective axiom.

11.7.1.1. A witness for Axiom (11.7)

Axiom (11.7) states that initializing a stack yields a stack whose size is zero. Listing 11.30 shows how this property is formalized and verified using an *axiom witness*.

It initializes the stack using `stack_init` and returns the result of `stack_size`. The central postcondition of the contract is the clause labeled `size`, which states that the returned value is zero, thereby establishing the axiom.

```
/*@
  requires   valid:  \valid(s);
  requires   valid:  \valid(a + (0..capacity-1));
  requires   sep:    \separated(s, a + (0..capacity-1));
  requires   pos:    0 < capacity;

  ensures    size:   \result == 0;
  ensures    valid:  StackValid(s);
*/
size_type axiom_size_of_init(Stack* s, value_type* a, size_type capacity)
{
  stack_init(s, a, capacity);
  return stack_size(s);
}
```

Listing 11.30: Implementation of `axiom_size_of_init`

11.7.1.2. A witness for Axiom (11.8)

Axiom (11.8) states that initializing a stack yields a stack whose capacity equals the specified bound. Listing 11.31 shows how this property is formalized and verified using an *axiom witness*.

```
/*@
  requires   valid:    \valid(s);
  requires   valid:    \valid(a + (0..capacity-1));
  requires   sep:      \separated(s, a + (0..capacity-1));
  requires   pos:      0 < capacity;

  ensures    capacity: \result == capacity;
  ensures    valid:    StackValid(s);
*/
size_type axiom_capacity(Stack* s, value_type* a, size_type capacity)
{
  stack_init(s, a, capacity);
  return stack_capacity(s);
}
```

Listing 11.31: Implementation of `axiom_capacity_of_init`

It initializes the stack using `stack_init` with a given capacity and returns the result of `stack_capacity`. The central postcondition of the contract is the clause labeled `capacity`, which states that the returned value equals the initialization parameter, thereby establishing the axiom.

11.7.2. Witnesses for adding an element to a stack

We now consider properties introduced by adding elements.

Elements are added to a stack by calling the function `stack_push`. In the abstract stack theory, this operation corresponds to the constructor push (11.3). The associated axioms describe how fundamental stack properties, such as size and top element, change when a new element is pushed onto a stack.

In this subsection, we present axiom witnesses for these properties. Each witness combines `stack_push` with other stack operations as prescribed by the corresponding axiom, while the contract expresses the required relationship between the resulting states.

11.7.2.1. A witness for Axiom (11.9)

Axiom (11.9) specifies how the size of a stack changes when an element is pushed onto it. If the stack is not full, pushing a new element increases its size by one. Listing 11.32 shows how this property is formalized and verified using an *axiom witness*.

```
/*@
  requires   valid:    StackValid(s);
  requires   not_full: !StackFull(s);

  ensures    size:     \result == \old(StackSize(s)) + 1;
  ensures    valid:    StackValid(s);
*/
size_type axiom_size_of_push(Stack* s, value_type v)
{
  stack_push(s, v);
  return stack_size(s);
}
```

Listing 11.32: Implementation of `axiom_size_of_push`

It invokes `stack_push` on the given stack and returns the result of `stack_size`. The central postcondition of the contract is the clause labeled `size`, which states that the returned value equals the previous size of the stack plus one.

11.7.2.2. A witness for Axiom (11.10)

Axiom (11.10) states that pushing an element onto a stack does not change its capacity. Listing 11.33 shows how this property is formalized and verified using an *axiom witness*.

```
/*@
  requires   valid:     StackValid(s);
  requires   not_full:  !StackFull(s);

  ensures    capacity:  \result == \old(StackCapacity(s));
  ensures    valid:     StackValid(s);
*/
size_type axiom_capacity_of_push(Stack* s, value_type v)
{
  stack_push(s, v);
  return stack_capacity(s);
}
```

Listing 11.33: Implementation of `axiom_capacity_of_push`

It invokes `stack_push` on the given stack and returns the result of `stack_capacity`. The central postcondition of the contract is the clause labeled `capacity`, which states that the returned value equals the capacity of the stack in the pre-state.

11.7.2.3. A witness for Axiom (11.11)

Axiom (11.11) states that, for a stack that is not full, the element pushed onto the stack becomes the top element of the resulting stack. Listing 11.34 shows how this property is formalized and verified using an *axiom witness*.

```
/*@
  requires   valid:    StackValid(s);
  requires   not_full: !StackFull(s);

  ensures    top:      \result == v;
  ensures    valid:    StackValid(s);
*/
value_type axiom_top_of_push(Stack* s, value_type v)
{
  stack_push(s, v);
  return stack_top(s);
}
```

Listing 11.34: Implementation of `axiom_top_of_push`

It invokes `stack_push` to add the value `v` to the stack and returns the result of `stack_top`. The central postcondition of the contract is the clause labeled `top`, which states that the returned value equals `v`.

11.7.3. Witnesses for removing an element from a stack

We now turn to the axioms describing the behavior of a stack when an element is removed. In the abstract stack theory, this operation corresponds to the function `stack_pop`.

These axioms characterize both the direct effect of removing the top element and the interaction between `stack_push` and `stack_pop`. In particular, they describe how the size and top element change after a pop operation and how pushing and subsequently popping an element restores the previous stack state.

In this subsection, we verify these properties using axiom witnesses, combining `stack_pop` with other stack operations as prescribed by the corresponding axioms.

11.7.3.1. A witness for Axiom (11.12)

Axiom (11.12) specifies how the size of a stack changes when an element is removed. For a non-empty stack, applying `stack_pop` decreases its size by one. Listing 11.35 shows how this property is formalized and verified using an *axiom witness*.

```
/*@
  requires   valid:  StackValid(s);
  requires   empty:  !StackEmpty(s);

  ensures    size:   \result == StackSize{Old}(s) - 1;
  ensures    valid:  StackValid(s);
*/
size_type axiom_size_of_pop(Stack* s)
{
  stack_pop(s);
  return stack_size(s);
}
```

Listing 11.35: Implementation of `axiom_size_of_pop`

It invokes `stack_pop` on the stack and returns the result of `stack_size`. The central postcondition of the contract is the clause labeled `size`, which states that the returned value equals the pre-state size of the stack minus one.

11.7.3.2. A witness for Axiom (11.13)

Axiom (11.13) states that removing an element from a stack does not change its capacity. Listing 11.36 shows how this property is formalized and verified using an *axiom witness*.

```
/*@
  requires   valid:    StackValid(s);
  requires   empty:    !StackEmpty(s);

  ensures    capacity: \result == StackCapacity{Old}(s);
  ensures    valid:    StackValid(s);
*/
size_type axiom_capacity_of_pop(Stack* s)
{
  stack_pop(s);
  return stack_capacity(s);
}
```

Listing 11.36: Implementation of `axiom_capacity_of_pop`

It invokes `stack_pop` on the stack and returns the result of `stack_capacity`. The central postcondition of the contract is the clause labeled `capacity`, which states that the returned value equals the pre-state capacity of the stack.

11.7.3.3. A witness for Axiom (11.14)

Axiom (11.14) describes the interaction between the operations `stack_push` and `stack_pop`. It states that if an element is pushed onto a non-full stack and then immediately removed again, the resulting stack is equal to the original stack. Thus, on non-full stacks, `stack_pop` acts as a left inverse of `stack_push`. Listing 11.37 shows how this property is formalized and verified using an *axiom witness*.

```
/*@
  requires   valid:     StackValid(s);
  requires   not_full:  !StackFull(s);

  ensures    equal:     StackEqual{Old,Here}(s, s);
  ensures    valid:     StackValid(s);
*/
void axiom_pop_of_push(Stack* s, value_type v)
{
  stack_push(s, v);
  stack_pop(s);
}
```

Listing 11.37: Implementation of `axiom_pop_of_push`

It invokes `stack_push` to insert the value `v` and subsequently applies `stack_pop`. The central postcondition of the contract is the clause labeled `equal`, which states that the resulting stack is equal to the pre-state stack.

11.7.3.4. A witness for Axiom (11.15)

Axiom (11.15) describes the complementary interaction between `stack_pop` and `stack_push`. It states that if an element is removed from a non-empty stack and the former top element is subsequently pushed back, the original stack is recovered. In this sense, `stack_push` acts as a right inverse of `stack_pop`,

provided that the value being pushed is the top element of the original stack. Listing 11.38 shows how this property is formalized and verified using an *axiom witness*.

```
/*@
  requires   valid:      StackValid(s);
  requires   not_empty:  !StackEmpty(s);

  ensures    equal:      StackEqual{Old,Here}(s, s);
  ensures    valid:      StackValid(s);
*/
void axiom_push_of_pop_top(Stack* s)
{
  const value_type v = stack_top(s);
  stack_pop(s);
  stack_push(s, v);
}
```

Listing 11.38: Implementation of `axiom_push_of_pop_top`

It first obtains the top element using `stack_top`, then invokes `stack_pop`, and finally applies `stack_push` to reinsert the value. The central postcondition of the contract is the clause labeled `equal`, which states that the resulting stack is equal to the pre-state stack.

Summary. Taken together, the witnesses presented in this section establish that the implementations of the stack operations introduced in §11.6 satisfy the algebraic properties formulated in the stack axioms of §11.2.

11.8. Witnesses for well-definedness

As discussed in §11.5, the predicate `StackEqual` [11.9] induces an equivalence relation on stacks. Since different stack instances may represent the same abstract stack, it is important that stack operations behave independently of the chosen representative. Functions with this property are said to be *well-defined* with respect to `StackEqual`.

Not every stack function satisfies this requirement. Some operations may depend on aspects of the concrete representation that are intentionally ignored by `StackEqual`. In the following, we therefore establish the well-definedness of the relevant stack operations with respect to this equivalence relation.

Functions not subject to well-definedness The notion of well-definedness is relevant only for operations whose result should depend solely on the abstract stack represented by a concrete instance. Two functions fall outside this consideration.

- The function `stack_equal` implements the equality test corresponding to the predicate `StackEqual`, whose relationship has already been discussed in §11.6.1. Since `StackEqual` provides the reference relation with respect to which well-definedness is formulated, the notion does not meaningfully apply to `stack_equal`, which merely implements the corresponding equality test.
- The function `stack_init` (§11.6.2), on the other hand, constructs a new stack and does not operate on an existing abstract stack. Consequently, the choice of representative does not arise for these two functions, and the notion of well-definedness is therefore not applicable.

Witnesses for well-definedness In analogy to the witnesses introduced for the stack axioms in Section 11.7, the well-definedness of a function is established by exhibiting a corresponding witness. Such a witness certifies that equivalent inputs, in the sense of `StackEqual`, are mapped to equivalent outputs. As in the case of axiom witnesses, these functions are not reused in other proofs and are not part of a modular verification chain. For readability, we therefore omit assigns and termination clauses in their specifications.

11.8.1. Well-definedness of `stack_size`

Since `stack_size` does not modify its `Stack` argument, well-definedness reduces to the requirement that equal stacks yield equal results. Listing 11.39 shows how this property is formalized and verified using a *well-definedness witness*.

```
/*@
  requires   valid:  StackValid(s) && StackValid(t);
  requires   equal:  StackEqual{Here,Here}(s, t);

  ensures    equal:  \result == true;
*/
bool stack_size_wd(const Stack* s, const Stack* t)
{
  return stack_size(s) == stack_size(t);
}
```

Listing 11.39: Implementation of `stack_size_wd`

It invokes `stack_size` on two equal stacks. The central postcondition of the contract is the clause labeled `equal`, which states that the returned values are equal.

11.8.2. Well-definedness of stack_empty

The function stack_empty is well-defined with respect to the equivalence relation StackEqual if equal stacks yield equal results. Listing 11.40 shows how this property is formalized and verified using a *well-definedness witness*.

```
/*@
  requires   valid:  StackValid(s) && StackValid(t);
  requires   equal:  StackEqual{Here,Here}(s, t);

  ensures    equal:  \result == true;
*/
bool stack_empty_wd(const Stack* s, const Stack* t)
{
  return stack_empty(s) == stack_empty(t);
}
```

Listing 11.40: Implementation of stack_empty_wd

It invokes stack_empty on two equal stacks and compares the results. The central postcondition of the contract is the clause labeled equal, which states that the returned values are equal.

11.8.3. No Well-definedness of stack_full

With our definition of StackEqual [11.9], two equal stacks may have different capacities. Listing 11.41 shows how this fact is used to demonstrate that stack_full is *not* well-defined with respect to StackEqual.

```
/*@
  requires   valid:  StackValid(s) && StackValid(t);
  requires   equal:  StackEqual{Here,Here}(s, t);
  requires   size:   StackSize(s) == StackCapacity(s);
  requires   cap:    StackCapacity(s) < StackCapacity(t);

  ensures    equal:  \result == false;
*/
bool stack_full_not_wd(const Stack* s, const Stack* t)
{
  //@ assert  StackFull(s);
  //@ assert !StackFull(t);
  return stack_full(s) == stack_full(t);
}
```

Listing 11.41: Implementation of stack_full_not_wd

It considers two equal stacks whose sizes coincide with their common value, but whose capacities differ. In particular, one stack is full while the other is not. The central postcondition of the contract is the clause labeled equal, which states that the returned value is **false**.

11.8.4. Well-definedness of stack_top

The function stack_top is well-defined with respect to the equivalence relation StackEqual if equal stacks yield equal results. Listing 11.42 shows how this property is formalized and verified using a *well-definedness witness*.

```
/*@
  requires   valid:      StackValid(s) &&  StackValid(t);
  requires   not_empty: !StackEmpty(s) && !StackEmpty(t);
  requires   equal:      StackEqual{Here,Here}(s, t);

  ensures    equal:     \result == true;
*/
bool stack_top_wd(const Stack* s, const Stack* t)
{
  return stack_top(s) == stack_top(t);
}
```

Listing 11.42: Implementation of `stack_top_wd`

It invokes `stack_top` on two equal, non-empty stacks. The central postcondition of the contract is the clause labeled `equal`, which states that the returned values are equal.

11.8.5. Well-definedness of `stack_push`

Since `stack_push` does not return a value but modifies its stack argument, well-definedness must be expressed relationally. In particular, the operation must map equal stacks to equal stacks.

To make this reasoning sound, aliasing between the two stacks must be excluded. If both arguments referred to overlapping memory regions, modifying one stack could implicitly affect the other, invalidating the relational argument. To capture this requirement, we introduce the predicate `StackSeparated` in Listing 11.43, which asserts that the memory regions of two stacks are disjoint.

```
/*@
  predicate StackSeparated(Stack* s, Stack* t) =
    \separated(s, s->data + (0..s->cap-1), t, t->data + (0..t->cap-1));
*/
```

Listing 11.43: `StackSeparated` (ACSL)

Listing 11.44 shows how the well-definedness of `stack_push` is formalized and verified using a *well-definedness witness*.

```
/*@
  requires   valid:     StackValid(s) && StackValid(t);
  requires   equal:     StackEqual{Here,Here}(s, t);
  requires   not_full:  !StackFull(s) && !StackFull(t);
  requires   sep:       StackSeparated(s, t);

  ensures    valid:     StackValid(s) && StackValid(t);
  ensures    equal:     StackEqual{Here,Here}(s, t);
*/
void stack_push_wd(Stack* s, Stack* t, value_type v)
{
  stack_push(s, v);
  stack_push(t, v);
}
```

Listing 11.44: Implementation of `stack_push_wd`

It invokes `stack_push` on two equal and separated stacks. The central postcondition of the contract is the clause labeled `equal`, which states that the resulting stacks are equal.

11.8.6. Well-definedness of stack_pop

Since stack_pop modifies its stack argument, well-definedness is expressed relationally: equal stacks must be mapped to equal stacks. As in the case of stack_push_wd [11.44], aliasing between the two stacks is excluded using the predicate StackSeparated [11.43].

Listing 11.45 shows how this property is formalized and verified using a *well-definedness witness*.

```
/*@
  requires   valid:  StackValid(s) && StackValid(t);
  requires   equal:  StackEqual{Here,Here}(s, t);
  requires   sep:    StackSeparated(s, t);

  ensures    valid:  StackValid(s) && StackValid(t);
  ensures    equal:  StackEqual{Here,Here}(s, t);
*/
void stack_pop_wd(Stack* s, Stack* t)
{
  stack_pop(s);
  stack_pop(t);
}
```

Listing 11.45: Implementation of stack_pop_wd

It invokes stack_pop on two equal and separated stacks. The central postcondition of the contract is the clause labeled equal, which states that the resulting stacks are equal.

Part VI.

Appendices

Introduction

This part collects supplementary material that supports the main exposition but is not required for its linear reading. It provides additional reference information and background details that may be consulted independently, depending on the reader's interests and the intended use of the document. In particular, the appendices include technical material related to the verification setup (Appendix A), as well as a consolidated record of changes in previous releases of this tutorial (Appendix B).

A. Results of Formal Verification with Frama-C

This chapter introduces the formal verification tools used throughout this tutorial. We then present the extent to which the examples from Chapters 4–11 can be deductively verified.

Within Frama-C, the Frama-C/WP plug-in [1] enables deductive verification of C programs annotated using the ANSI/ISO-C Specification Language (ACSL) [2]. The Frama-C/WP plug-in relies on weakest-precondition calculations to generate proof obligations. These obligations can subsequently be discharged by external automatic theorem provers or by interactive proof assistants.

For the exact configuration of Frama-C/WP used in this release, we refer to Section 1.3.

A.1. Verification settings

This section summarizes the most important Frama-C options that were used for almost all functions.[23]

```
-pp-annot
-no-unicode
-wp
-wp-rte
-warn-unsigned-overflow
-warn-unsigned-downcast
-wp-model Typed
-wp-session ../..
-wp-interactive=batch
-wp-timeout 2
```

We deliberately use a relatively small timeout value for the provers. Depending on the employed tool versions, individual examples may require a larger timeout to obtain successful proofs. For the precise versions of the employed provers we refer to Table 1.1 on Page 11.

[23] For the function `my_lrand48()` in `shuffle`, the option `-warn-unsigned-overflow` is disabled, as explained in §7.19.

A.2. Verification results

In our verification scenario, each proof obligation is processed by a fixed sequence of automatic and interactive theorem provers arranged as a *pipeline*.[24] Each prover attempts to discharge the verification conditions it receives and forwards only the remaining obligations to the next prover. The resulting *verification pipeline* is illustrated in Figure A.1.

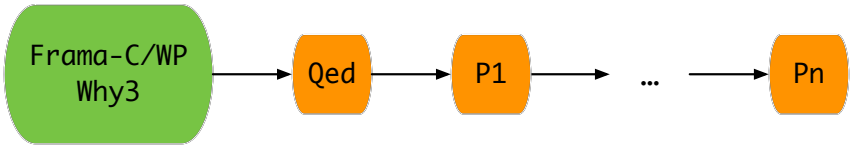


Figure A.1.: Verification pipeline of automatic and interactive theorem provers

For each algorithm, the following tables list the total number of generated verification conditions, the percentage of verified conditions, and the number of verification conditions discharged by each prover. Table cells marked with a · indicate a value of zero. In all cases, all generated verification conditions could be verified.

The distribution of verified conditions across provers should not be interpreted as a measure of their relative strength or quality. Due to the sequential pipeline structure, each verification condition is passed through the prover sequence until it is discharged (see Figure A.1).

Legend of verification result tables In the verification result tables, the first two columns report outcomes classified internally by the Frama-C/WP plug-in:

- WP^{T}: verification conditions classified as *terminating* by WP, corresponding to termination-related conditions that WP proves syntactically, for instance trivial termination inferred from the program structure, such as the presence of annotated loops and the absence of function calls;
- WP^{U}: verification conditions classified as *unreachable* by WP, corresponding to conditions that WP considers valid because the corresponding program points are syntactically unreachable, so that associated annotations (assertions, requires, ensures, etc.) cannot be triggered.

In other words, these two categories reflect WP-internal classifications of verification conditions prior to prover dispatch. All remaining columns give the number of verification conditions discharged by the respective external provers (e.g. Qed, Alt-Ergo, Z3, CVC5, Coq).

[24] Sequential processing is enforced by passing the option `-wp-par 1` to Frama-C/WP.

Algorithm		Verification Conditions		Individual Provers						
				WP^T	WP^U	QD	AE	Z3	C5	CQ
find	§4.1	32 / 32	(100%)	1	1	23	7	·	·	·
find2	§4.2	34 / 34	(100%)	1	1	21	11	·	·	·
find3	§4.3	34 / 34	(100%)	1	1	12	17	·	·	3
find4	§4.3.4	34 / 34	(100%)	·	·	13	18	·	·	3
find5	§4.3.4	23 / 23	(100%)	·	·	7	13	·	·	3
find_if_not	§4.4	40 / 40	(100%)	1	1	12	21	·	·	5
find_first_of	§4.5	48 / 48	(100%)	·	·	38	10	·	·	·
adjacent_find	§4.6	43 / 43	(100%)	1	1	29	12	·	·	·
mismatch	§4.7	33 / 33	(100%)	1	1	23	8	·	·	·
equal	§4.7	9 / 9	(100%)	·	·	8	1	·	·	·
search	§4.8	60 / 60	(100%)	·	·	48	12	·	·	·
search_n	§4.9	81 / 81	(100%)	1	1	55	24	·	·	·
find_end	§4.10	54 / 54	(100%)	·	·	33	21	·	·	·
count	§4.11	42 / 42	(100%)	1	1	15	18	·	·	7

Table A.2.: Results for Nonmutating

Algorithm		Verification Conditions		Individual Provers						
				WP^T	WP^U	QD	AE	Z3	C5	CQ
clamp	§5.4	27 / 27	(100%)	1	1	21	4	·	·	·
make_pair	§5.3	5 / 5	(100%)	1	1	3	·	·	·	·
max_element	§5.5	44 / 44	(100%)	1	1	32	10	·	·	·
max_element2	§5.6	50 / 50	(100%)	1	1	37	11	·	·	·
max_seq	§5.7	16 / 16	(100%)	·	·	12	4	·	·	·
min_element	§5.8	50 / 50	(100%)	1	1	37	11	·	·	·
minmax_element	§5.9	107 / 107	(100%)	·	·	79	28	·	·	·

Table A.3.: Results for MinMax

Algorithm		Verification Conditions		Individual Provers						
				WP^T	WP^U	QD	AE	Z3	C5	CQ
lower_bound	§6.3	32 / 32	(100%)	1	1	14	16	·	·	·
upper_bound	§6.4	32 / 32	(100%)	1	1	16	14	·	·	·
equal_range	§6.5	26 / 26	(100%)	·	·	20	6	·	·	·
equal_range2	§6.5	124 / 124	(100%)	·	·	74	44	·	·	6
binary_search	§6.6	22 / 22	(100%)	·	·	15	7	·	·	·

Table A.4.: Results for BinarySearch

Algorithm		Verification Conditions		Individual Provers						
				WP^T	WP^U	QD	AE	Z3	C5	CQ
`fill`	§7.3	17 / 17	(100%)	1	1	9	6	·	·	·
`swap`	§7.4	10 / 10	(100%)	1	1	5	3	·	·	·
`swap_ranges`	§7.5	25 / 25	(100%)	·	·	14	11	·	·	·
`copy`	§7.6	18 / 18	(100%)	1	1	9	7	·	·	·
`copy_backward`	§7.7	19 / 19	(100%)	1	1	8	9	·	·	·
`reverse_copy`	§7.8	20 / 20	(100%)	1	1	9	9	·	·	·
`reverse`	§7.9	26 / 26	(100%)	·	·	9	17	·	·	·
`rotate_copy`	§7.10	18 / 18	(100%)	·	·	7	11	·	·	·
`rotate`	§7.11	30 / 30	(100%)	·	·	19	10	1	·	·
`replace_copy`	§7.12	27 / 27	(100%)	1	1	15	10	·	·	·
`replace`	§7.13	25 / 25	(100%)	1	1	14	9	·	·	·
`remove_copy`	§7.14	33 / 33	(100%)	1	1	18	13	·	·	·
`remove_copy2`	§7.15	84 / 84	(100%)	1	1	19	46	1	·	16
`remove_copy3`	§7.16	124 / 124	(100%)	1	1	30	71	1	·	20
`remove`	§7.17	116 / 116	(100%)	1	1	22	71	1	·	20
`shuffle`	§7.18	69 / 69	(100%)	·	·	28	32	·	·	9
`random_number`	§7.19	40 / 40	(100%)	2	2	21	15	·	·	·

Table A.5.: Results for Mutating

Algorithm		Verification Conditions		Individual Provers						
				WP^T	WP^U	QD	AE	Z3	C5	CQ
`iota`	§8.1	19 / 19	(100%)	1	1	10	7	·	·	·
`accumulate`	§8.2	26 / 26	(100%)	1	1	9	13	·	·	2
`inner_product`	§8.3	29 / 29	(100%)	1	1	9	16	·	·	2
`partial_sum`	§8.4	62 / 62	(100%)	1	1	19	39	·	·	2
`adjacent_difference`	§8.5	37 / 37	(100%)	1	1	13	22	·	·	·
`partial_sum_inv`	§8.6	42 / 42	(100%)	·	·	10	29	·	·	3
`adjacent_difference_inv`	§8.6	42 / 42	(100%)	·	·	10	29	·	·	3

Table A.6.: Results for Numeric

Algorithm		Verification Conditions		Individual Provers						
				WP^T	WP^U	QD	AE	Z3	C5	CQ
`heap_parent`	§9.3	14 / 14	(100%)	1	1	3	9	·	·	·
`heap_child`	§9.3	77 / 77	(100%)	1	1	34	40	·	·	1
`is_heap_until`	§9.4	52 / 52	(100%)	1	1	23	26	·	·	1
`is_heap`	§9.5	27 / 27	(100%)	·	·	12	14	·	·	1
`push_heap`	§9.7	152 / 152	(100%)	·	·	50	89	·	2	11
`pop_heap`	§9.8	185 / 185	(100%)	·	·	70	100	·	4	11
`make_heap`	§9.9	75 / 75	(100%)	·	·	27	39	·	·	9
`sort_heap`	§9.10	76 / 76	(100%)	·	·	22	43	·	·	11

Table A.7.: Results for Heap

Algorithm		Verification Conditions		Individual Provers						
				WP^{T}	WP^{U}	QD	AE	Z3	C5	CQ
`is_sorted`	§10.2	29 / 29	(100%)	1	1	13	12	·	·	2
`partial_sort`	§10.3	158 / 158	(100%)	·	·	51	89	·	·	18
`bubble_sort`	§10.4	106 / 106	(100%)	·	·	41	53	·	1	11
`selection_sort`	§10.5	86 / 86	(100%)	·	·	33	38	·	·	15
`insertion_sort`	§10.6	83 / 83	(100%)	·	·	27	42	3	·	11
`heap_sort`	§10.7	53 / 53	(100%)	·	·	15	29	·	·	9
`merge`	§10.8	389 / 389	(100%)	·	·	268	109	6	2	4

Table A.8.: Results for Sorting

Algorithm		Verification Conditions		Individual Provers						
				WP^{T}	WP^{U}	QD	AE	Z3	C5	CQ
`stack_equal`	§11.6.1	24 / 24	(100%)	·	·	9	15	·	·	·
`stack_init`	§11.6.2	15 / 15	(100%)	1	1	3	10	·	·	·
`stack_size`	§11.6.3	8 / 8	(100%)	1	1	1	5	·	·	·
`stack_capacity`	§11.6.4	8 / 8	(100%)	1	1	1	5	·	·	·
`stack_empty`	§11.6.5	12 / 12	(100%)	·	·	7	5	·	·	·
`stack_full`	§11.6.6	13 / 13	(100%)	·	·	7	6	·	·	·
`stack_top`	§11.6.7	19 / 19	(100%)	·	·	9	10	·	·	·
`stack_push`	§11.6.8	40 / 40	(100%)	·	·	28	12	·	·	·
`stack_pop`	§11.6.9	33 / 33	(100%)	·	·	23	10	·	·	·

Table A.9.: Results for Stack

Algorithm		Verification Conditions		Individual Provers						
				WP^{T}	WP^{U}	QD	AE	Z3	C5	CQ
`axiom_size_of_init`	§11.7.1.1	12 / 12	(100%)	·	·	9	3	·	·	·
`axiom_capacity_of_init`	§11.7.1.2	12 / 12	(100%)	·	·	9	3	·	·	·
`axiom_size_of_push`	§11.7.2.1	9 / 9	(100%)	·	·	5	4	·	·	·
`axiom_capacity_of_push`	§11.7.2.2	9 / 9	(100%)	·	·	5	4	·	·	·
`axiom_top_of_push`	§11.7.2.3	9 / 9	(100%)	·	·	5	4	·	·	·
`axiom_size_of_pop`	§11.7.3.1	9 / 9	(100%)	·	·	5	4	·	·	·
`axiom_capacity_of_pop`	§11.7.3.2	9 / 9	(100%)	·	·	5	4	·	·	·
`axiom_pop_of_push`	§11.7.3.3	9 / 9	(100%)	·	·	5	4	·	·	·
`axiom_push_of_pop_top`	§11.7.3.4	10 / 10	(100%)	·	·	6	4	·	·	·

Table A.10.: Results for Stack / StackAxioms

Algorithm		Verification Conditions		Individual Provers						
				WP^T	WP^U	QD	AE	Z3	C5	CQ
`stack_size_wd`	§11.8.1	8 / 8	(100%)	·	·	4	4	·	·	·
`stack_empty_wd`	§11.8.2	8 / 8	(100%)	·	·	4	4	·	·	·
`stack_full_not_wd`	§11.8.3	10 / 10	(100%)	·	·	5	5	·	·	·
`stack_top_wd`	§11.8.4	8 / 8	(100%)	·	·	4	4	·	·	·
`stack_push_wd`	§11.8.5	9 / 9	(100%)	·	·	3	6	·	·	·
`stack_pop_wd`	§11.8.6	9 / 9	(100%)	·	·	3	6	·	·	·

Table A.11.: Results for Stack / StackWellDefinedness

B. Changes in previous releases

This chapter documents the changes introduced in earlier versions of this document. For a description of the most recent changes we refer to Section 1.3.

The version numbers of this document are tied to the versioning scheme of Frama-C [3]. The releases of Frama-C are named consecutively after chemical elements of the periodic table. Accordingly, the version numbering of this tutorial follows the scheme **X.Y.Z**, defined as follows:

- **X** denotes the major version number of the tutorial. It coincides with the atomic number of the chemical element after which the corresponding Frama-C release is named.
- **Y** denotes the subrelease number of Frama-C.
- **Z** denotes the subrelease number of this tutorial.

The remainder of this chapter consists of a chronological record of changes for previous releases. Each release is documented separately in order to preserve a stable and traceable history.

B.1. New in Version 32.0.2 (Germanium, January 2026)

This release is intended for Frama-C [3, p. v32.0] issued in December 2025. We are also using for this release the Why3 platform[10, p. v1.8.2] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.6.2	[11]
CVC5	automatic	1.3.2	[12]
Z3	automatic	4.15.2	[13]
Coq	interactive	8.20.1	[14]

Table B.1.: Information on automatic and interactive theorem provers

New examples

None.

Improvements

- introduced the predicate `Rotate` and employed it systematically in the formal specification and verification of `rotate_copy`, `rotate`, and `insertion_sort`:
  - added the lemma `Rotate_Shift`,
  - renamed `CircularShift_StrictLowerBound` to `RotateOne_StrictLowerBound`,
  - renamed `CircularShift_MultisetReorder` to `RotateOne_MultisetReorder`.
- improved and expanded the descriptions of the algorithms in Parts 3–5.
- updated and revised numerous figures.

- introduced the slice notation $a[m \mathinner{..} n)$ in place of the previous notation `a[m..n-1]`.
- refined the presentation of the predicate `Unchanged` and clarified its relationship to the `assigns` clause.
- revised the overall structure of the document, strengthening the role of parts as the primary organizational units.
- updated the font packages used for the report.

B.2. New in Version 32.0.1 (Germanium, January 2026)

This release is intended for Frama-C [3, p. v32.0] issued in December 2025. We are also using for this release the Why3 platform[10, p. v1.8.2] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.6.2	[11]
CVC5	automatic	1.3.2	[12]
Z3	automatic	4.15.2	[13]
Coq	interactive	8.20.1	[14]

Information on automatic and interactive theorem provers

New examples

None.

Improvements

- Added `terminates` and `exits` clauses to all function contracts.
- Completed the `assigns` clauses of `stack_init` and `axiom_size_of_init` with explicit `\from` dependencies, clarifying the origin of assigned values and aligning the contracts with Frama-C/WP 's pedantic assigns checks.
- Replaced statement contracts in `pop_heap`, `partial_sort`, `selection_sort`, `insertion_sort`, and `merge` by assertions referring to an explicit ghost label, thereby making the proof obligations local to the respective loop bodies.
- Improved the description of nonmutating algorithms; removed `count2` and the inductively defined logic function `CountInd`.
- Improved the description of minimum and maximum algorithms.
- Improved the description of binary search algorithms and removed `binary_search2`.

B.3. New in Version 32.0.0 (Germanium, December 2025)

This release is intended for Frama-C [3, p. v32.0] issued in December 2025. We are also using for this release the Why3 platform[10, p. v1.8.2] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.6.2	[11]
CVC5	automatic	1.3.2	[12]
Z3	automatic	4.15.2	[13]
Coq	interactive	8.20.1	[14]

Information on automatic and interactive theorem provers

The main changes in this release concern the rewriting of Coq proofs to reflect changes in Frama-C/WP.

New examples

None.

Improvements

- Logic and proofs: rename `RemovePartition` to `IndexOfNotEqual` and adjust `remove` and `remove_copy3` implementations and tutorial text accordingly.
- Add new lemmas to improve verification rate: `Accumulate_Next`, `InnerProduct_Init` and `InnerProduct_Next`.
- Cleanup: remove references to `count2` from tutorial.
- Build system: fix and clean up Makefile targets.

Open issues

These following warnings are mainly due to changes in the behavior and diagnostics of Frama-C/WP and will be addressed in a later revision of ACSL by Example.

- **Missing explicit `terminates` / `exits` clauses**
  Function contracts are present, but lack explicit `terminates` and/or `exits` clauses for functions without visible bodies; WP therefore generates default clauses.
- **Pedantic assigns warning**
  An `assigns` clause is missing or too weak compared to the implementation, reported under `wp:pedantic-assigns`.
- **Unsupported statement-level specification**
  A statement contract is present but currently ignored by WP due to lack of support.

B.4. New in Version 22.0.0 (Titanium, November 2020)

This release is intended for Frama-C [3, p. v22.1] issued in November 2020. We are also using for this release the Why3 platform[10, p. v1.3.3] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.3	[11]
CVC4	automatic	1.7	[26]
Z3	automatic	4.8.6	[13]
Coq	interactive	8.12.1	[14]

Information on automatic and interactive theorem provers

Note that all automatic provers use the Why3 interface. However, the interactive prover Coq still relies on the native interface provided by Frama-C/WP.

New examples

None.

Improvements

Updated to Coq 8.12.1.

Open issues

- The contract of algorithm `merge` does not handle the reordering of the involved arrays.

B.5. New in Version 21.1.1 (Scandium, September 2020)

This release is intended for Frama-C [3, p. v21.1] issued in June 2020. We are also using for this release the Why3 platform[10, p. v1.3.3] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.3	[11]
CVC4	automatic	1.7	[26]
Z3	automatic	4.8.6	[13]
Coq	interactive	8.9.1	[14]

Information on automatic and interactive theorem provers

Note that all automatic provers use the Why3 interface. However, the interactive prover Coq still relies on the native interface provided by Frama-C/WP.

New examples

None.

Improvements

- general changes
  - disable CVC3 and switch back to Z3 `4.8.6`
  - refactor Coq proofs
  - reduce general timeout to 1s (Coq timeout 5s)
- improve loop invariants of `remove_copy3` and `remove` to reduce timeout
- add lemma `AdjacentDifference_InverseBounds` to reduce timeout of `adjacent_difference_inv` robust
- add lemmas `Count_Single`, `Count_Single_Bounds`, `Count_Single_Shift` and `Count_Cut`
- heap algorithms
  - rework `push_heap` and add more assertions to reduce timeout
  - rework `pop_heap` and finally verify property `reorder`
  - add predicates `ArrayUpdate`, `MultisetParity`, `MultisetUpdate` and supporting lemmas

  - remove predicate `PushHeapAdjust` and accompanying lemmas
  - rename `heap_child_max` to `heap_child` and improve both contract and implementation
  - add lemma `HeapParent_Zero`
- sorting and reordering
  - rework contract and annotations of merge
  - no need more for option `-wp-split`
  - add lemma `WeaklyIncreasing_Shrink`
  - add lemma `WeaklyIncreasing_Unchanged`
  - add more annotations to `bubble_sort` to reduce timeout
  - add lemma `MultisetSwap_FrontMiddle`

Open issues

- The contract of algorithm `merge` does not handle the reordering of the involved arrays.

Renaming of ACSL definitions

We sometimes rename predicates, logic functions or lemmas in order to make them more precise or make the naming more consistent.

- rename suffix `_Read` to `_Unchanged` in names of lemmas

Old name	New name
`EqualRanges`	`Equal`
`Equal_Increasing`	`Increasing_Equal`
`Equal_Count`	`Count_Equal`
`MultisetUnchanged`	`MultisetReorder`
`MultisetUnchanged_Union`	`MultisetReorder_DisjointUnion`
`Reorder_Match`	`MultisetReorder_SomeEqual`
`Reorder_LowerBound`	`MultisetReorder_LowerBound`
`Reorder_LowerBounds`	`MultisetReorder_PartitionLowerBound`
`Reorder_UpperBound`	`MultisetReorder_UpperBound`
`SwappedInside`	`ArraySwap`
`SwappedInside_Reorder`	`MultisetSwap_Middle`

B.6. New in Version 21.1.0 (Scandium, July 2020)

This release is intended for Frama-C [3, p. v21.1] issued in June 2020. We are also using for this release the Why3 platform[10, p. v1.3.1] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.2	[11]
CVC4	automatic	1.7	[26]
CVC3	automatic	2.4.1	[27]
Z3	automatic	4.8.8	[13]
Coq	interactive	8.9.1	[14]

Information on automatic and interactive theorem provers

Note that all automatic provers use the Why3 interface. However, the interactive prover Coq still relies on the native interface provided by Frama-C/WP.

New examples

None.

Improvements

- Improve many code annotations in order to maintain the verification rate and to reduce timeout values.
- Use predicate `WeaklyIncreasing` instead of `Increasing` in the assertions and invariants of sorting algorithms. This allows the removal of lemma `IncreasingUpperBound`.
- Add Coq to parallel verification
- Replace an ACSL lemma on integer division by a Coq lemma in driver.
- `remove_copy` and `remove`
  - Remove logic helper function `NextNotEqual` for `RemovePartition` which served as a workaround in Frama-C 20.
  - Remove lemma `RemovePartition_StrictlyIncreasing`.
  - Make the definition of predicate `Remove` more flexible.
  - Remove lemma `Remove_Update`.
- heap algorithms
  - Simplify definition of predicates `MultisetRetainRest` and `MultisetMinus`.
  - Add predicate `PushHeapAdjust`.
  - Add lemmas `PushHeapAdjust_Init` and `PushHeapAdjust_Finish`.
  - Rename lemma `MultisetPushHeapRetain` to `PushHeapAdjust_Retain`.
  - Add predicate `HeapCompatible` and lemmas `Heap_Shrink`, `Heap_Unchanged` and `Heap_Update` .
  - Improve annotations of `push_heap` and `pop_heap`.
  - Remove predicate `HeapChildMax` and simplify the contract of `heap_child_max`.
- Remove lemma `SwappedInside_Preserve` which was of limited usefulness.
- Add lemmas `Accumulate_Init`, `AccumulateBounds_Read` and `Accumulate_Read_Shrink`, which were suggested by Allan Blanchard, in to simplify the verification of `partial_sum` and `adjacent_difference_inv`.

- Add lemmas `Unchanged_Symmetric` and `MultisetUnchanged_Symmetric`.

Open issues The following algorithms and/or lemmas are not completely verified

- `pop_heap`
- `merge`

B.7. New in Version 20.0.2 (Calcium, April 2020)

This release is intended for Frama-C [3, p. v20.0] issued in December 2019. We are also using for this release the Why3 platform[10, p. v1.2.1] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.2	[11]
CVC4	automatic	1.6	[26]
CVC3	automatic	2.4.1	[27]
Z3	automatic	4.8.6	[13]
Coq	interactive	8.9.1	[14]

Information on automatic and interactive theorem provers

Note that all automatic provers use the Why3 interface. However, the interactive prover Coq still relies on the native interface provided by Frama-C/WP.

New examples

- Add examples `find4` and `find5` that verify the equivalence of the contracts of `find2` and `find3`.
- Add example `find_if_not`.

Improvements

- Add indices for examples and logic definitions.
- Re-add results of running all provers in parallel. Thanks to Allan Blanchard for explaining how Frama-C/WP's *session* mechanism can be used in the implementation.
- Fix a ghost label in `partial_sort`. Thanks to Virgile Prevosto for pointing out stricter checks in upcoming releases of Frama-C.
- Reduce very long verification times of several examples.
  - Add assertion `unchanged` to empty else branch of `remove_copy3`.
  - Add assertion `reorder` to empty else branch of `shuffle`.
  - Rewrite assertion `update` of `remove`.
  - Add another assertion `heap` to `push_heap`.
- Remove chapter on `unique_copy` because on its reliance on axioms. Moreover, the main ideas are already extensively discussed in the sections on `remove_copy` and `remove`.
- Verify properties of operator `<` within example `clamp`.
- Improve admitted Coq proof of `Reorder_Match`.

- Fix misplaced arrow in figure of `equal_range` algorithm

Open issues The following algorithms and/or lemmas are not completely verified

- `pop_heap` (property `reorder`)
- `merge` (property `reorder`)
- `Reorder_Match`

B.8. New in Version 20.0.1 (Calcium, March 2020)

This release is intended for Frama-C [3, p. v20.0] issued in December 2019. We are also using for this release the Why3 platform[10, p. v1.2.1] and the provers listed in the following table.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.1	[11]
CVC4	automatic	1.6	[26]
CVC3	automatic	2.4.1	[27]
Z3	automatic	4.8.6	[13]
Coq	interactive	8.9.1	[14]

Information on automatic and interactive theorem provers

Note that all automatic provers use the Why3 interface. However, the interactive prover Coq still relies on the native interface provided by Frama-C/WP.

New examples

- add a third version of `find` that is specified using the new logic function `Find`

Improvements

- improve text in many places
- improve specification of `remove_copy` and `remove`
  - provide an explicit definition of `RemovePartition` that allows to replace axioms by lemmas
  - rename predicate `ConstantRange` to `AllEqual` and add its negation `SomeNotEqual`
  - add logic functions `CountNotEqual` and `FindNotEqual`
- place all logic definitions in `axiomatic` blocks to better control generated names
- make names of ACSL predicates, functions and lemmas more uniform and place them together in files where appropriate
- among the renamed ACSL entities are
  - rename predicate `HasValue` to `SomeEqual` and add its negation `NoneEqual`
  - rename lemma `HasValueImpliesPositiveCount` to `SomeEqualCount`
  - rename lemma `PositiveCountImpliesHasValue` to `Count_SomeEqual`
  - rename `RotatePreservesStrictLowerBound` to `CircularShift_StrictLowerBound`
  - rename `RotateImpliesMultisetUnchanged` to `CircularShift_MultisetReorder`

Open issues

The following algorithms and/or lemmas are not completely verified

- `pop_heap`
- `Reorder_Match`

B.9. New in Version 20.0.0 (Calcium, December 2019)

Aside from the above-mentioned version of Frama-C we are using for this release the Why3 platform[10, p. v1.2.1] and the provers listed in the following table. Note that all automatic provers are use the Why3 interface. In other words, we do not use anymore the native interface for Alt-Ergo.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.0	[11]
CVC4	automatic	1.6	[26]
CVC3	automatic	2.4.1	[27]
Z3	automatic	4.8.6	[13]
Coq	interactive	8.9.1	[14]

Information on automatic and interactive theorem provers

New examples

- add `bubble_sort`

Improvements

- remove Why3 and Alt-Ergo lemmas from driver
- switch from memory model 'Typed+Ref' to 'Typed'
- the E theorem prover is not yet supported by this version of Frama-C
- no results on parallel verification are reported in this release
- rewrite `random_shuffle` to `shuffle`
  - adapt signature of `random_number`
  - add auxiliary function `random_init`
- replace, where applicable, ghost labels by loop labels or statement labels
- remove lemma `SwapImpliesMultisetUnchanged` by using predicate `SwappedInside` and its related lemmas
- improve specification and verification rate of numeric algorithms
  - resolve overloaded version of `Accumulate` into `AccumulateDefault`
  - resolve overloaded version of `AccumulateBounds` into `AccumulateDefaultBounds`
  - improve definition of predicate `PartialSum`
  - add lemmas `Difference_Zero` and `Difference_Next`
  - add predicate `DefaultBounds`

- add assigns in behaviors of maxmin and nonmutating algorithms
  - `find`, `find2`, `find_first_of`, `adjacent_find`, `mismatch`, `search`, `find_end`
  - `max_element`, `max_element2`, `min_element`, `minmax_element`
- rename predicate `Sorted` to `Increasing`; also rename related logic names
  - rename `EqualRangesPreservesSorted` ↦ `EqualRangesPreservesIncreasing`
  - rename `SortedUpperBound` ↦ `IncreasingUpperBound`
  - rename `WeaklySortedAddElement` ↦ `WeaklyIncreasingAddElement`
  - rename `WeaklySortedShift` ↦ `WeaklyIncreasingShift`
  - rename `EqualRangesWeaklySorted` ↦ `EqualRangesWeaklyIncreasing`
  - rename `WeaklySortedJoin` ↦ `WeaklyIncreasingJoin`
  - rename `WeaklySortedLemmas` ↦ `WeaklyIncreasingLemmas`
  - rename `SortedIFFWeaklySorted` ↦ `IncreasingIFFWeaklyIncreasing`
  - rename `SortedImpliesWeaklySorted` ↦ `IncreasingImpliesWeaklyIncreasing`
  - rename `WeaklySortedImpliesSorted` ↦ `WeaklyIncreasingImpliesIncreasing`
  - rename `WeaklySorted` ↦ `WeaklyIncreasing`
  - rename `SortedShift` ↦ `Increasing_Shift`
- remove lemma `SortedDownIsHeap`

Open issues

The following algorithms and/or lemmas are not completely verified

- `adjacent_difference_inv`
- `pop_heap`
- `random_number`
- `ReorderImpliesMatch`

B.10. New in Version 19.1.0 (Potassium, October 2019)

This release is intended for Frama-C 19.1 (*Potassium*), issued in September 2019.[3]

Aside from the above-mentioned version of Frama-C we are using for this release the Why3 platform[10, p. v1.2.0] and the provers listed in the following table. Note that all automatic provers are use the Why3 interface. In other words, we do not use anymore the native interface for Alt-Ergo.

Prover	Type	Version	Reference
Alt-Ergo	automatic	2.3.0	[11]
CVC4	automatic	1.6	[26]
CVC3	automatic	2.4.1	[27]
Z3	automatic	4.8.6	[13]
E	automatic	2.3	[28]
Coq	interactive	8.9.1	[14]

Information on automatic and interactive theorem provers

Improvements

- Rename arguments of `search` and `find_end` and improve also the description of these algorithms.
- Rename and reorder arguments of `search_n`, make the verification more robust and improve its description.
- Make verification of property `size` of `remove_copy2` more robust.
- Explain role of lemma `RemoveImpliesNotHasValue` in `remove_copy3` and `remove`.
- Simplify definition of `RemoveSize` and `RemovePartition`.
- Make verification of property `reorder` of `partial_sort` more robust.
- Strengthen precondition of `replace_copy`.
- Rename lemma `random_number_modulo` into `RandomNumberModulo`.
- Differentiate between properties `unique` and `solitary` for `unique_copy` examples.
- Simplify the implementation of `is_heap` by calling the new function `is_heap_until`.
- Replace remaining instances of label `Pre` in contracts by `Old`.
- Unify use of `Unchanged` predicate for mutating algorithms.

New examples

- Add the algorithm `clamp` which "clips" a value between a pair of boundary values.
- Add the algorithm `minmax_element` and improve description of other algorithms related to finding minimum and maximum values.
- Add new example `is_heap_until` that generalizes `is_heap`.
- The following examples are not new since they were implicitly used as helper functions for other examples. They are now explicitly listed as examples.
  - `make_pair`
  - `random_number`
  - `heap_parent`
  - `heap_child_max` (formerly known as `heap_maximum_child`)

Open issues

The following algorithms and/or lemmas are not completely verified

- `adjacent_difference_inv`
- `partial_sum_inv`
- `pop_heap`
- `ReorderImpliesMatch`

B.11. New in Version 19.0.0 (Potassium, June 2019)

- Structure of document
  - The document is now structured into several parts.
  - The chapter on classic sorting algorithms has been merged into the chapter on sorting.
  - The various variants of `unique_copy` are now grouped into a separate chapter.
- Fix various inconsistencies
  - Change the return types of the logic functions `Accumulate`, `Difference`, `Capacity`, `Size`, `Top` from bounded one (e.g., `value_type`, `size_type`) to integer. A combination of bounded type for a logic function with an arithmetic operations in the logical definitions may lead to inconsistency. This fixes the inconsistencies in the `accumulate`, `stack` and `stack_wd` examples.
  - Fix an inconsistency in `DifferenceRead` axiom: restriction on the array size added to premises.
- Various improvements
  - An important change is the rewriting of the implicit, *axiomatic* definitions of `Accumulate`, `Count`, `Difference`, `InnerProduct` and `UniqueSize` logic functions to explicit, *recursive* ones. Accordingly, all axioms in the respective examples have been rewritten as lemmas.
  - Generalize `CountSectionMonotonic`, `UnchangedSection` lemmas: remove restriction on lower bound for the range.
  - Fix typo in postcondition of `find`.
  - Rewrite specifications of `remove_copy` and `remove` examples.
  - Rename predicate `RemoveCount` to `RemoveSize`.
  - Gather all versions of `MultisetRetainRest` in section on `push_heap`.
  - Add another figure to highlight simple contract for `unique_copy`.
  - Adapt Coq proofs to the fact that the `Z` scope is not available by default.
- New examples
  - Add `count2` example with an inductive predicate instead of a logic function in `count`.
  - Add `merge` example.
- Infrastructure
  - Travis-CI configuration for the GitHub repository added as an illustrative example of how the verification results could be reproduced.
  - Add support for Frama-C/AstraVer plugin.

B.12. New in Version 18.0.0 (Argon, December 2018)

- Replace the links to the (now abandoned) original site of *Standard Template Library* (STL) by references to the C++ standard.
- Add new algorithm `unique_copy` (two versions).
- Add another assertion `half` for `reverse`.
- Add two overloaded versions of predicate `ConstantRange` and use them for the algorithms `fill` and `unique_copy`, respectively.

B.13. New in Version 17.1.0 (Chlorine, July 2018)

The exact version number of Frama-C originally was Chlorine-20180502. This version number was changed in October 2018 to 17.1

- Slightly change the definition of predicate `HasEqualNeighbors` and its use in the specification of `adjacent_find`.
- Remove the algorithm `remove` and the more elaborate version of `remove_copy`. We are currently working on new specifications of these algorithms.
- Adapt some Coq proofs related to the logic function `Count` in order to reflect changes in output of Frama-C/WP.
- Remove table on ACSL lemmas that had to be proved by Coq.

B.14. New in Version 16.1.1 (Sulfur, March 2018)

- fix several errors reported by Aaron Rocha, including,
  - fix an error in figure for `upper_bound` algorithms
- fix merging of contracts in second version of `binary_search`
- improve and justify the `retain` annotations of in the implementation of `remove`
- Alt-Ergo is now directly called in the parallel setting (instead of going through Why3) to be compatible with the sequential setting
- add a third assertion `reorder` in the `random_shuffle` body to keep verification rate at 100% after prover upgrade

B.15. New in Version 16.1.0 (Sulfur, December 2017)

- special thanks to Aaron Rocha who provided various improvements for Chapters 4, 5, and 6
- improve some mutating algorithms
  - add more assertions to `reverse` to reduce reliance on CVC3
  - improve structure and ACSL annotations of `remove_copy` and `remove`

        - add overloaded version of predicate `MultisetRetainRest`
        - add lemma `HasValueImpliesPositiveCount`
        - add lemma `PositiveCountImpliesHasValue`
        - remove lemma `HasValueShiftInversion`
        - remove lemma `HasValueCountInversion`
    - add custom lemma `random_number_modulo` for `random_shuffle`
- add new Chapter 10 with more algorithms related to sorting
    - add algorithm `is_sorted` including predicate `WeaklyIncreasing`
        - add lemma `IncreasingImpliesWeaklyIncreasing`
        - add lemma `WeaklyIncreasingImpliesIncreasing`
    - add algorithm `partial_sort` including predicate `Partition`
        - add lemma `ReorderImpliesMatch`
        - add lemma `ReorderPreservesUpperBound`
        - add lemma `ReorderPreservesLowerBound`
        - add lemma `PartialReorderPreservesLowerBounds`
        - add lemma `SwappedInside`
        - add lemma `SwappedInsideMultisetUnchanged`
        - add lemma `SwappedInsidePreservesMultisetUnchanged`
- improve various lemmas
    - rename lemma `SortedUp` to `IncreasingUpperBound`
    - generalize lemma `UnchangedSection`
    - refactor lemma `HeapBounds` into `C_Division_Two`

B.16. New in Version 15.1.2 (Phosphorus, October 2017)

- fix several typos reported by `seniorlackey@github` (thanks a lot!)
- add a new chapter on classic sorting algorithms which comprises
    - `selection_sort` including lemma `SwapImpliesMultisetUnchanged`
    - `insertion_sort` including lemmas
        - `RotatePreservesStrictLowerBound`
        - `RotateImpliesMultisetUnchanged`
        - `EqualRangesPreservesIncreasing`
        - `EqualRangesPreservesCount`
    - `heap_sort`

- heap algorithms
  - remove length requirements in `pop_heap`, `sort_heap`, `make_heap`, and `heap_sort`
    * introduce `SIZE_TYPE_MAX` to catch border cases in ACSL and C
  - improve description of `pop_heap`
    * add predicate `HeapChildMax`
    * provide the auxiliary function `heap_child_max`
    * the postcondition `reorder` is still not verified
  - improve description of `push_heap`
  - other, minor improvements
    * add auxiliary function `heap_parent`
    * add predicate `SortedDown` and lemma `SortedDownIsHeap`
    * add lemmas `HeapParentChild` and `HeapChilds`
    * add lemmas `HeapParentBounds` and `HeapChildBounds`

B.17. New in Version 15.1.1 (Phosphorus, September 2017)

- add ensures clause to default behavior of the following algorithms
  - `find`, `find_first_of`, `adjacent_find`, `mismatch`, `search`, `search_n`, `find_end`
  - `max_element`, `min_element`
- rewrite axiomatic definitions to ensure disjoint guards which is better suited for E-ACSL
  - concerns the axiomatic definitions of `Count`, `Accumulate`, `InnerProduct` and `Difference`
  - some Coq proofs related to `Count` had to be adapted as well
- shorten names of some auxiliary algorithms
  - `adjacent_difference_inverse` $\mapsto$ `adjacent_difference_inv`
  - `partial_sum_inverse` $\mapsto$ `partial_sum_inv`
- heap algorithms
  - fix a typo in Figure 9.3
  - fix a typo in Figure 9.39
  - explain that there can be multiple representations of an array as a heap
  - add a version of `pop_heap` that is, however, not completely verified

B.18. New in Version 15.1.0 (Phosphorus, June 2017)

- The verification results are now part of the appendix.
- Fix an error in the specification of the well-definition of `stack_size`.
- This release of Frama-C/WP could not discharge some of our assertions of `push_heap`. We therefore have completely rewritten the annotations and also tweaked the implementation of `push_heap`. We also added some new predicates and lemmas to maintain a concise specification that can easily be verified by automatic provers.
  - add predicate `MultisetAdd` and lemma `MultisetAdd_Distinct`
  - add predicate `MultisetMinus` and lemma `MultisetMinus_Distinct`
  - add predicate `MultisetRetain` and lemma `MultisetPushHeapRetain`
  - provide an additional version of predicate `MultisetRetainRest`
  - and lemma `MultisetPushHeapClosure`

B.19. New in Version 14.1.1 (Silicon, April 2017)

- changes in verification infrastructure
  - add verification results for the case where each proof obligation is submitted to all automatic theorem provers
- changes in algorithms
  - simplify loop invariants of `search_n` and improve description
  - rename predicate `CountOneHit` to `CountHit`
  - rename predicate `CountOneMiss` to `CountMiss`
  - rewrite predicates `EqualRanges` and `Reverse` in order to simplify the task for automatic theorem provers
  - remove lemmas on `Reverse` that were necessary for `rotate` but are not needed anymore
  - rename predicate `Valid(Stack*)` to `Invariant(Stack*)` and remove `\valid` from `Invariant(Stack*)`
  - add a simple random number generator to `random_shuffle` and verify it
- fix an inconsistency in the axioms for `Count` (thanks to Denis Efremov for reporting this issue)
  - add more guards to axioms `CountSectionHit` and `CountSectionMiss`
  - add corresponding guards to lemmas
    - `CountSectionOne`, `CountHit`, `CountMiss` and `CountOne`
    - `RemoveCountHit` and `RemoveCountMiss`
  - add lemma `Unchanged_Shift` and add more assertions to `remove` in order to simplify the task for automatic theorem provers

B.20. New in Version 14.1.0 (Silicon, January 2017)

- use label `Old` instead of `Pre` in function contracts
- add algorithm `rotate`
- rewrite definition of predicates `EqualRanges` and `Reverse` and provide more overloaded versions
- add figures for algorithms `rotate` and `replace_copy`
- update figure for predicate `Reverse`
- update Coq proofs and add a table with more information on the ACSL lemmas that had to be verified with Coq

B.21. New in Version 13.1.1 (Aluminium, November 2016)

- improve layout of tables of verification results
- use two additional automatic theorem provers (CVC3 and E)
- nonmutating algorithms
  - add algorithm `find_end`
  - add definition of predicate `HasSubRange` on subranges
  - add definition of predicate `EqualRanges` on subranges
  - rename lemma `HasSubRange_fit_size` to `HasSubRangeSize`
  - rename lemma `HasConstantSubRange_fit_size` to `HasSubRangeSize`
  - rename logic function `CountSection` to `Count` (using overloading in ACSL)
  - add lemma `HasValueCountInversion`
  - add lemma `HasValueShiftInversion`
  - add lemma `Count_Shift`
- mutating algorithms
  - add algorithm `copy_backward`
  - relax precondition on separation of `copy`, `replace_copy` and `remove_copy`
  - provide a more sophisticated implementation of `remove`
  - re-introduce a second version of `remove_copy` that also specifies the *stability* of the algorithm
  - add algorithm `random_shuffle`

B.22. New in Version 13.1.0 (Aluminium, August 2016)

The most notable changes of this version are the re-introduction of heap algorithms in Chapter 9. This new description of heap algorithms is based to a large extend on the bachelor thesis of one of the authors [20].

- provide names ("labels") for more ACSL annotations

- nonmutating algorithms
  - reorder and improve description in chapter on nonmutating algorithms
  - add more figures to describe algorithms
  - add nonmutating algorithm `search_n`
  - rewrite logic function `Count` with new logic function `CountSection`
  - move lemmas `Count_Bounds` and `CountMonotonic` to separate files
  - use `integer` instead of `size_type` in `HasSubRange`
  - change index computation in `HasEqualNeighbors`
- maximum and minimum algorithms
  - isolate predicate `ConstantRange` from predicates on lower and upper bounds
  - fix typo in precondition of first version of `max_element`
- binary search algorithms
  - add version `Sorted` for subranges
  - add second (more efficient) version of `equal_range`
    - add lemmas `SortedShift`, `LowerBound_Shift`, `StrictLowerBound_Shift`, `UpperBound_Shift` and `StrictUpperBound_Shift` to support the automatic verification of this version of `equal_range`
  - add figures to binary search algorithms and improve description
- mutating algorithms
  - greatly reduce the number of assertions needed to verify the first version `remove_copy`
  - temporarily remove the second version of `remove_copy` which also specified the *stability* of the algorithm
  - add `remove`, an in-place variant of `remove_copy`
  - rename predicate `RetainAllButOne` to `MultisetRetainRest`
- re-introduce chapter on heap algorithms
  - includes the heap algorithms `is_heap`, `push_heap`, `make_heap` and `sort_heap`
  - for `pop_heap` only a function contract is provided in this version
  - add lemma `SortedUp` to support verification of `sort_heap`
  - add several lemmas to combine the predicates `Unchanged` and `MultisetUnchanged`

B.23. New in Version 12.1.0 (Magnesium, February 2016)

A main goal of this release is to reduce the number of proof obligations that cannot be verified automatically and therefore must be tackled by an interactive theorem prover such as Coq. To this end, we analyzed the proof obligations (often using Coq) and devised additional assertions or ACSL lemmas to guide the automatic provers. Often we succeeded in enabling automatic provers to discharge the concerned obligations. Specifically, whereas the previous version 11.1.1 of *ACSL by Example* listed *nine* proof obligations that

could only be discharged with Coq, the document at hand (version 12.1.0) only counts *five* such obligations. Moreover, all these remaining proof obligations are associated to ACSL lemmas, which are usually easier to tackle with Coq than proof obligations directly related to the C code. The reason for this is that ACSL lemmas usually have a much smaller set of hypotheses.

Adding assertions and lemmas also helps to alleviate a problem in Frama-C/WP Magnesium and Sodium where prover processes are not properly terminated.[25] Left-over "zombie processes" lead to a deterioration of machine performance which sometimes results in unpredictable verification results.

- mutating algorithms
  - simplify annotations of `replace_copy` and add new algorithm `replace`
    - add predicate `Replace` to write more compact post conditions and loops invariants
  - add several lemmas for predicate `Unchanged` and use predicate `Unchanged` in postconditions of mutating and numeric algorithms
  - simplify annotations of `reverse`
    - rename `Reversed` to `Reverse` (again) and provide another overloaded version
    - add figure to support description of the `Reverse` predicate
  - changes regarding `remove_copy`
    - rename `PreserveCount` to `RetainAllButOne`
    - rename `StableRemove` to `RemoveMapping`
    - add statement contracts for both versions of `remove_copy` such that only ACSL lemmas require Coq proofs
- numeric algorithms
  - define limits `VALUE_TYPE_MIN` and `VALUE_TYPE_MAX`
  - simplify specification of `iota` by using new logic function `Iota`
  - simplify implementation of `accumulate`
    - add overloaded predicates `AccumulateBounds`
    - add lemmas `AccumulateDefault0`, `AccumulateDefault1`, `AccumulateDefaultNext`, and `AccumulateDefault_Read`
  - simplify implementation of `inner_product`
    - add predicates `ProductBounds` and `InnerProductBounds`
  - enable automatic verification of `partial_sum`
    - add lemmas `PartialSumSection`, `PartialSumUnchanged`, `PartialSum_Step`, and `PartialSumStep2` to automatically discharge loop invariants
  - enable automatic verification of `adjacent_difference`
    - add logic function `Difference` and predicate `AdjacentDifference`
    - add predicate `AdjacentDifferenceBounds`

[25] See https://bts.frama-c.com/view.php?id=2154

    - add lemmas `AdjacentDifference_Step` and `AdjacentDifference_Section` to automatically discharge proof obligation
  - add two auxiliary functions `partial_sum_inverse` and `adjacent_difference_inverse` in order to verify that `partial_sum` and `adjacent_difference` are inverse to each other
    - add lemmas `PartialSumInverse` and `AdjacentDifferenceInverse` to support the automatic verification of the auxiliary functions
- stack functions
  - add lemma `StackPushEqual` to enable the automatic verification of the well-definition of `stack_push`

B.24. New in Version 11.1.1 (Sodium, June 2015)

- add Chapter on numeric algorithms
  - move `iota` algorithm to numeric algorithms (§8.1)
  - add `accumulate` algorithm (§8.2)
  - add `inner_product` algorithm (§8.3)
  - add `partial_sum` algorithm (§8.4)
  - add `adjacent_difference` algorithm (§8.5)

B.25. New in Version 11.1.0 (Sodium, March 2015)

- Use built-in predicates `\valid` and `\valid_read` instead of `valid_range`.
- Simplify loop invariants of `find_first_of`.
- Replace two loop invariants of `remove_copy` by ACSL lemmas.
- Rename several predicates
  - `IsEqual` ↦ `EqualRanges`.
  - `IsMaximum` ↦ `MaxElement`.
  - `IsMinimum` ↦ `MinElement`.
  - `Reverse` ↦ `Reversed`.
  - `IsSorted` ↦ `Sorted`.
- Several changes for `Stack`:
  - Rename `Stack` functions from *`foo`*`_stack` to `stack_`*`foo`*.
  - Equality of stacks now ignores the `capacity` field. This is similar to how equality for objects of type `std::vector<T>` is defined. As a consequence `stack_full` is not well-defined any more. Other stack functions are not effected.
  - Remove all assertions from stack functions (including in axioms).

  - Describe predicate `Separated` in text.

B.26. New in Version 10.1.1 (Neon, January 2015)

- use option `-wp-split` to create simpler (but more) proof obligations
- simplify definition of predicate `Count`
- add new predicates for lower and upper bounds of ranges and use it in
  - `max_element`
  - `min_element`
  - `lower_bound`
  - `upper_bound`
  - `equal_range`
  - `fill`
- use a new auxiliary assertion in `equal_range` to enable the complete *automatic* verification of this algorithm
- add predicate `Unchanged` and use it to simplify the specification of several algorithms
  - `swap_ranges`
  - `reverse`
  - `remove_copy`
  - `stack_push` and `stack_push_wd`
  - `stack_pop` and `stack_pop_wd`
- add predicate `Reverse` and use it for more concise specifications of
  - `reverse_copy`
  - `reverse`
- several changes in the two versions of `remove_copy`
  - use predicate `HasValue` instead of logic function `Count`
  - add predicate `PreserveCount`
  - reformulate logic function `RemoveCount`
  - add predicate `StableRemove`
  - add predicate `RemoveCountMonotonic`
  - add predicate `RemoveCountJump`
- use overloading in ACSL to create shorter logic names for `Stack`
- remove unnecessary labels in several `Stack` functions

B.27. New in Version 10.1.0 (Neon, September 2014)

- remove additional labels in the `assumes` clauses of some stack function that were necessary due to an error in Oxygen
- provide a second version of `remove_copy` in order to explain the specification of the *stability* of the algorithms
- coarsen loop assigns of mutating algorithms
- temporarily remove the `unique_copy` algorithm

B.28. New in Version 9.3.1 (Fluorine, not published)

- specify bounds of the return value of `count` and fix reads clause of `Count` predicate
- use an auxiliary function `make_pair` in the implementation of `equal_range`
- provide more precise loop assigns clauses for the mutating algorithms
  - simplify implementation of `fill`
  - removed the `ensures \valid(p)` clause in specification of `swap`
  - simplify implementation of `swap_ranges`
  - simplify implementation of `copy`
  - fix implementation of `reverse_copy` after discovering an undefined behavior
  - new implementation of `reverse` that uses a simple **`for`**-loop
  - simplify implementation of `replace_copy`
  - refactor specification and simplify implementation of `remove_copy`
- remove work-around with `Pre`-label in `assumes` clauses of `stack_push` and `stack_pop`

B.29. New in Version 9.3.0 (Fluorine, December 2013)

- adjustments for *Fluorine* release of Frama-C
- `swap` now ensures that its pointer arguments are valid after the function has been called
- change definition of `size_type` to **`unsigned int`**
- change implementation of the `iota` algorithm . The content of the field `a` is calculated by increasing the value `val` instead of sum `val+i`.
- change implementation of `fill`.
- The specification/implementation of `Stack` has been revised by Kim Völlinger [23] and now has a much better verification rate.

B.30. New in Version 8.1.0 (Oxygen, not published)

- simplified specification and loop annotations of `replace_copy`
- add binary search variant `equal_range`
- greatly simplified specification of `remove_copy` by using the logic function `Count`
- remove chapter on heap operations

B.31. New in Version 7.1.1 (Nitrogen, August 2012)

- improvements with respect to several suggestions and comments of Yannick Moy, e.g., specification refinements of `remove_copy`, `reverse_copy` and `iota`
- restricted verification of algorithms to Frama-C/WP with Alt-Ergo
- replaced deprecated `\valid_range` by `\valid`
- fixed inconsistencies in the description of the `Stack` data type
- binary search algorithms can now be proven without additional axioms for integer division
- changed axioms into lemmas to document that provability is expected, even if not currently granted
- adopted new Fraunhofer logo and contact email

B.32. New in Version 7.1.0 (Nitrogen, December 2011)

- changed to Frama-C Nitrogen
- changed to Why 2.30
- discussed both plug-ins Frama-C/WP and Jessie
- removed `swap_values` algorithm

B.33. New in Version 6.1.0 (Carbon, not published)

- changed definition of `Stack`
- renamed `reset_stack` to `init_stack`

B.34. New in Version 5.1.1 (Boron, February 2011)

- prepared algorithms for checking by the new Frama-C/WP plug-in of Frama-C
- changed to Alt-Ergo Version 0.92, Z3 Version 2.11 and Why 2.27
- added List of user-defined predicates and logic functions
- added remarks on the relation of logical values in C and ACSL

- rewrote section on `equal` and `mismatch`
- used a simpler logical function to count elements in an array
- added `search` algorithm
- added chapter to unite the maximum/minimum algorithms
- added chapter for the new `lower_bound`, `upper_bound` and `binary_search` algorithms
- added `swap_values` algorithm
- used `IsEqual` predicate for `swap_ranges` and `copy`
- added `reverse_copy` and `reverse` algorithms
- added `rotate_copy` algorithm
- added `unique_copy` algorithm
- added chapter on specification of the data type `Stack`

B.35. New in Version 5.1.0 (Boron, May 2010)

- adaption to Frama-C Boron and Why 2.26 releases
- changed from the `-jessie-no-regions` command-line option to using the pragma `SeparationPolicy(`*`value`*`)`

B.36. New in Version 4.2.2 (Beryllium, May 2010)

- changed to latest version of CVC3 2.2
- added additional remarks to our implementation of `find_first_of`
- changed `size_type` (**`int`**) to `integer` in all specifications
- removed casts in `fill` and `iota`
- renamed `is_valid_range` as `IsValidRange`
- renamed `has_value` as `HasValue`
- renamed predicate `all_equal` as `IsEqual`
- extended timeout to 30 sec.

B.37. New in Version 4.2.1 (Beryllium, April 2010)

- added alternative specification of `remove_copy` algorithm that uses `ghost` variables
- added Chapter on heap operations
- added `mismatch` algorithm
- moved algorithms `adjacent_find` and `min_element` from the appendix to chapter on nonmutating algorithms

- added typedefs `size_type` and `value_type` and used them in all algorithms
- renamed `is_valid_int_range` as `is_valid_range`

B.38. New in Version 4.2.0 (Beryllium, January 2010)

- complete rewrite of pre-release
- adaption to Frama-C Beryllium 2 release

Bibliography

[1] *WP Plug-in*. http://frama-c.com/wp.html.

[2] *ANSI/ISO C Specification Language*. http://frama-c.com/acsl.html. 2025.

[3] CEA List, INRIA, and Université Paris-Saclay. *Frama-C. A Framework for the Analysis of C Programs*. 2026. URL: https://frama-c.com/ (visited on 02/06/2026).

[4] *CEA LIST, Laboratory of Applied Research on Software-Intensive Technologies*. http://www-list.cea.fr/gb/index_gb.htm.

[5] *INRIA-Saclay, French National Institute for Research in Computer Science and Control*. http://www.inria.fr/saclay/.

[6] *LRI, Laboratory for Computer Science at Université Paris-Sud*. http://www.lri.fr/.

[7] *Fraunhofer-Institut für Offene Kommunikationssysteme (FOKUS)*. http://www.fokus.fraunhofer.de.

[8] Virgile Prevosto. *ACSL Mini-Tutorial*. http://frama-c.com/download/acsl-tutorial.pdf.

[9] Patrick Baudin et al. *ACSL 1.13 Implementation in Argon 18.0*. https://frama-c.com/download/acsl-implementation-18.0-Argon.pdf. 2018.

[10] *Why – Where Programs Meet Provers*. https://www.why3.org. 2025.

[11] Sylvain Conchon, Evelyne Contejean, and Johannes Kanig. *The Alt-Ergo SMT Solver*. http://alt-ergo.lri.fr. 2018.

[12] Clark Barrett and Cesare Tinelli. *Homepage of CVC5*. https://cvc5.github.io. 2025.

[13] Microsoft Research. *The Z3 Theorem Prover*. https://github.com/Z3Prover/z3. 2025.

[14] The ROCQ Consortium. *https://rocq-prover.org*. https://rocq-prover.org. 2025.

[15] *Programming languages – C, Committee Draft*. http://www.open-std.org/JTC1/SC22/WG14/www/docs/n1362.pdf. 2009.

[16] C.A.R. Hoare. "An Axiomatic Basis for Computer Programming". In: *Communications of the ACM* 12 (1969), pp. 576–583.

[17] Robert W. Floyd. "Assigning Meanings to Programs". In: *Proc. Symposium on Applied Mathematics*. Ed. by J. T. Schwartz. Vol. 19. Mathematical Aspects of Computer Science. Providence, RI: American Mathematical Society, 1967, pp. 19–32.

[18] Richard Smith. *Working Draft, Standard for Programming Language C++*. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4659.pdf. publicly available draft of C++ 17 standard. 2017.

[19] Donald E. Knuth, James H. Morris, and Vaughan R. Pratt. "Fast Pattern Matching in Strings". In: *SIAM J Comput* 6.2 (June 1977), pp. 323–350.

[20] Timon Lapawczyk. "Formale Verifikation von Heap-Algorithmen mit Frama-C". bachelor thesis. Humboldt-Universität zu Berlin, July 2016.

[21] D.E. Knuth. *Sorting and Searching*. Vol. 3. The Art of Computer Programming. Addison-Wesley, 1973.

[22] Sellibitze. *How to Implement Classic Sorting Algorithms in Modern C++*. https://stackoverflow.com/questions/24650626/how-to-implement-classic-sorting-algorithms-in-modern-c. Blog post. Aug. 2014.

[23] Kim Völlinger. "Einsatz des Beweisassistenten Coq zur deduktiven Programmverifikation". https://www2.informatik.hu-berlin.de/top/_media/www/mitarbeiter/diplomarbeit-kim-voellinger.pdf. Diplomarbeit. Humboldt-Universität zu Berlin, Aug. 2013.

[24] *ANSI/ISO C Specification Language (ACSL)*. Reference manual for the specification language used by Frama-C to describe functional properties of C programs. CEA List, INRIA, and Université Paris-Saclay. 2026. URL: https://frama-c.com/html/acsl.html (visited on 02/06/2026).

[25] Martin Odersky, Lex Spoon, and Bill Venners. *Programming in Scala*. Artima, 2008. URL: http://www.artima.com/shop/programming_in_scala.

[26] Clark Barrett and Cesare Tinelli. *Homepage of CVC4*. http://cvc4.cs.stanford.edu/web/. 2018.

[27] Clark Barrett and Cesare Tinelli. *Homepage of CVC3*. http://www.cs.nyu.edu/acsys/cvc3/. 2010.

[28] Stephan Schulz. *The E Theorem Prover*. https://wwwlehre.dhbw-stuttgart.de/~sschulz/E/E.html. 2018.

Index of ACSL definitions

Caveat: This index has been automatically generated from the ACSL sources. For the time being it mentions only the page where an ACSL definition is first included in the text. Moreover, if a listing had to be split, then the page number refers to the first part of the listing even if the specific ACSL definition appears in the second part.

A

Accumulate 162, 163
Accumulate_Init 162
Accumulate_Next 162
Accumulate_Unchanged 162
Accumulate_Unchanged_Shrink 162
AccumulateBounds 162, 164
AccumulateBounds_Unchanged 162
AccumulateDefault 163, 169
AccumulateDefault_Next 163
AccumulateDefault_One 163
AccumulateDefault_Unchanged 163
AccumulateDefault_Zero 163
AccumulateDefaultBounds 163
AccumulateDefaultBounds_Shrink 163
AdjacentDifference 172–174
AdjacentDifference_Inverse 175, 177
AdjacentDifference_InverseBounds 175, 177
AdjacentDifference_Section 172
AdjacentDifference_Step 172, 174
AdjacentDifferenceBounds 172
AllEqual 45, 62, 97, 112, 113
AllSomeNot 45
ArrayBounds 75
ArrayBoundsShift 100, 226
ArrayExtrema 75
ArraySwap 156
ArrayUpdate 111, 188, 197, 201, 202, 207, 208
ArrayUpdate_MultisetUpdate 197
ArrayUpdate_Shrink 111
ArrayUpdate_UpperBound 111
At 111, 149, 151, 195

C

C_Bit 158
Count 68, 71, 140, 152
Count_Bounds 68
Count_Cut 68

Count_Empty 68, 69
Count_Equal 68
Count_Hit 68, 69
Count_Increasing 68
Count_Miss 68
Count_One 68, 69
Count_Shift 68
Count_Single 68
Count_Single_Bounds 68
Count_Single_Shift 68
Count_SomeEqual 70
Count_Unchanged 68
Count_Union 68
CountFind 70
CountFindNotEqual 144
CountNotEqual 140, 141, 144
CountNotEqual_AllEqual 144
CountNotEqual_Count 140
CountNotEqual_Decrement 144
CountNotEqual_Empty 140
CountNotEqual_FindNotEqual 144
CountNotEqual_Hit 140
CountNotEqual_Increasing 140
CountNotEqual_Lower 140
CountNotEqual_Miss 140
CountNotEqual_SomeNotEqual 144
CountNotEqual_Unchanged 140
CountNotEqual_Union 140
CountNotEqual_Upper 140
CountNotEqual_WeaklyIncreasing 140
CountNotEqual_Zero 144

D

DefaultBounds 175
Difference 172
Difference_Next 172
Difference_Unchanged 172
Difference_Zero 172

E

Equal 55, 57–59, 115, 116, 118, 120, 126, 249

F

Find 40–42, 46, 48
Find_Empty 40
Find_Hit 40
Find_Increasing 40
Find_Limit 40
Find_Lower 40
Find_MissHit 40
Find_MissMiss 40
Find_NoneEqual 40

Find_ResultEqual 40
Find_ResultNoneEqual 40
Find_SomeEqual 40
Find_Upper 40
Find_WeaklyIncreasing 40
FindNotEqual 45, 46, 48, 144
FindNotEqual_AllEqual 46
FindNotEqual_Empty 46
FindNotEqual_Hit 46
FindNotEqual_Increasing 46
FindNotEqual_Limit 46
FindNotEqual_Lower 46
FindNotEqual_MissHit 46
FindNotEqual_MissMiss 46
FindNotEqual_ResultAllEqual 46
FindNotEqual_ResultNotEqual 46
FindNotEqual_SomeNotEqual 46
FindNotEqual_Unchanged 46
FindNotEqual_Upper 46
FindNotEqual_WeaklyIncreasing 46

G

Greater_Less 74
GreaterOrEqual_Less 74

H

HasConstantSubRange 62, 63
HasConstantSubRange_Sizes 62
HasEqualNeighbors 52–54
HasSubRange 59, 60, 62, 66
HasSubRange_Sizes 59
HasValueOf 49–51
Heap 188, 191, 193
Heap_ChildBounds 187
Heap_Childs 187
Heap_Maximum 188
Heap_ParentBounds 187
Heap_ParentChild 187
Heap_ParentLeft 187
Heap_ParentRight 187
Heap_Shrink 188
Heap_Unchanged 188
HeapCompatible 188, 203, 208
HeapCompatible_Update 188
HeapLeft 187, 189
HeapNodes 187
HeapParent 187, 189
HeapParent_Zero 187
HeapRight 187

I

Increasing 89–91, 213, 216, 219

Increasing_Equal 90, 229
Increasing_Shift 90
Increasing_WeaklyIncreasing 90
IncreasingLemmas 90
IndexOfNotEqual 145
IndexOfNotEqual_Core 145
IndexOfNotEqual_Count 145
IndexOfNotEqual_Empty 145
IndexOfNotEqual_Left 145
IndexOfNotEqual_Lower 145
IndexOfNotEqual_Next 145
IndexOfNotEqual_NotEqual 145
IndexOfNotEqual_Right 145
IndexOfNotEqual_Segment 145
IndexOfNotEqual_StrictlyWeakIncreasing 145
IndexOfNotEqual_Unchanged 145
IndexOfNotEqual_Upper 145
InnerProduct 166, 167
InnerProduct_Init 166
InnerProduct_Next 166
InnerProduct_Unchanged 166
InnerProductBounds 166
IotaGenerate 160

L

Less_Antisymmetry 74
Less_Irreflexivity 74
Less_Transitivity 74
LessOrEqual_Less 74
LessThanComparable 74
LowerBound 75, 84, 85, 92
LowerBound_Shift 100

M

MaxElement 75, 81, 87, 188
MinElement 75, 84, 87
Multiset_AddMinusRetain 195
Multiset_AddMinusRetainReorder 195
Multiset_MinusAddRetain 195
MultisetAdd 194, 195
MultisetAdd_Distinct 194
MultisetMinus 194
MultisetMinus_Distinct 194
MultisetOperations 194
MultisetParity 195–197, 201
MultisetParity_Combined 196, 202, 208
MultisetParity_MultisetReorder 196, 203
MultisetParity_UnchangedFirst 196
MultisetParity_UnchangedSecond 196
MultisetReorder 152–154, 156, 195, 198, 210, 219, 222, 226
MultisetReorder_DisjointUnion 153

MultisetReorder_LowerBound 218
MultisetReorder_PartitionLowerBound 218
MultisetReorder_SomeEqual 218
MultisetReorder_Symmetric 153
MultisetReorder_Transitive 153
MultisetReorder_UpperBound 218
MultisetRetain 194, 195
MultisetRetain_Distinct 194
MultisetRetainRest 195
MultisetSwap 156
MultisetSwap_FrontMiddle 156
MultisetSwap_Middle 156, 226
MultisetUpdate 197, 202, 208

N

NoneEqual 37, 38, 40, 138
NoneEqual_NotSomeEqual 37
NotAllEqual_SomeNotEqual 45
NotSomeEqual_NoneEqual 37
NumericInverse 175

P

PartialSum 169
PartialSum_Inverse 175
PartialSum_One 169
PartialSum_Section 169
PartialSum_Step 169
PartialSum_Unchanged 169
Partition 217, 219
PartitionLemmas 218
ProductBounds 166

R

RandomNumberModulo 158
Remove 147, 149, 150
Replace 133–136
Reverse 121–125
Reverse_Shift 122, 132
ReverseLemmas 122
Rotate 126–130, 132, 228
Rotate_Shift 229
RotateLemmas 229
RotateOne_MultisetReorder 229
RotateOne_StrictLowerBound 229

S

SomeEqual 37, 38, 40, 45, 49, 50, 70, 102
SomeNone 37
SomeNotEqual 45
SomeNotEqual_NotAllEqual 45
Stack 247
StackCapacity 247

StackData 247
StackEmpty 247, 254
StackEqual 249, 251, 264, 265
StackEqual_Reflexive 249
StackEqual_Symmetric 249
StackEqual_Transitive 249
StackFull 247, 254
StackInvariant 247, 248, 251
StackObservers 247
StackPop 257
StackPrefix 249
StackPush 255, 256
StackSeparated 266, 267
StackSize 247
StackTop 247
StackValid 248
StrictLowerBound 75, 84, 85, 87, 94
StrictLowerBound_Shift 100
StrictUpperBound 75, 81, 87, 92, 97
StrictUpperBound_Shift 100

U

Unchanged 47, 108–111, 125, 222
Unchanged_Extend 110
Unchanged_MultisetReorder 153
Unchanged_Shift 110
Unchanged_Shrink 110
Unchanged_Symmetric 110
Unchanged_Transitive 110, 177
UnchangedLemmas 110
UpperBound 75, 81, 94
UpperBound_Shift 100

W

WeaklyIncreasing 90, 91, 213, 216, 233
WeaklyIncreasing_AddElement 233
WeaklyIncreasing_Equal 233
WeaklyIncreasing_Increasing 90, 91, 213, 216, 235
WeaklyIncreasing_Join 233
WeaklyIncreasing_Shift 233
WeaklyIncreasing_Shrink 233
WeaklyIncreasing_Unchanged 233
WeaklyIncreasingLemmas 233

Index of examples

Caveat. This index has been generated automatically from the ACSL sources and currently records only the page on which a given ACSL definition is first introduced. Moreover, when a code listing is split across multiple pages, the page number shown refers to the first part of the listing, even if the ACSL definition itself appears later.

A

accumulate 165, 166
adjacent_difference 173, 174
adjacent_difference_inv 177
adjacent_find 53, 54
axiom_capacity_of_init 259
axiom_capacity_of_pop 262
axiom_capacity_of_push 260
axiom_pop_of_push 262
axiom_push_of_pop_top 263
axiom_size_of_init 258
axiom_size_of_pop 261
axiom_size_of_push 260
axiom_top_of_push 260

B

binary_search 101, 102
bubble_sort 223

C

clamp 77, 78
copy 117–119
copy_backward 119, 120
count 71, 162

E

equal 57, 58, 251
equal_range 96–98, 100
equal_range2 ..... 96, 98

F

fill 112, 113
find 34, 36–38, 60
find2 ..... 38, 39, 41–44, 50, 51, 58, 63, 66, 102
find3 ..... 41, 42, 44, 48
find4 43
find5 ..... 43, 44
find_end 66, 67
find_first_of 50, 51

find_if_not 48

H
heap_child 189, 190, 207
heap_parent 189
heap_sort 231

I
inner_product 167, 168
insertion_sort 129, 227, 228
iota 160, 161
is_heap 193
is_heap_until 191–193
is_sorted 216

L
lower_bound 92–94, 96, 98, 102

M
make_heap 210, 211, 219, 231
make_pair 76, 87
max_element 79, 80, 82, 85
max_element2 ..... 81–83, 86
max_seq 83
merge 232, 234, 235
min_element 84–86, 226
minmax_element 73, 86, 87
mismatch 57, 58

P
partial_sort 219, 222
partial_sum 163, 170, 171, 173
partial_sum_inv 176
pop_heap 188, 189, 195, 204, 205, 207, 213, 219
push_heap 188, 195, 198, 199, 205, 207, 211, 219

R
random_number 155, 157, 158
remove 148, 150, 151
remove_copy 138–141, 143, 148
remove_copy2 ..... 138, 141–143, 148
remove_copy3 ..... 45, 138, 142, 147, 148, 150, 151
replace 134–136
replace_copy 134, 135
reverse 124, 125, 131, 132
reverse_copy 122, 123
rotate 130, 131, 228, 229
rotate_copy 127–129

S
search 60, 61, 63, 67
search_n 63
selection_sort 225, 226

shuffle 154, 155
size_pair_type 76
sort_heap 90, 212, 213, 231
stack_capacity 253
stack_empty 254
stack_empty_wd 265
stack_equal 251
stack_full 254, 255
stack_full_not_wd 265
stack_init 252
stack_pop 257
stack_pop_wd 267
stack_push 256
stack_push_wd 266, 267
stack_size 252, 253
stack_size_wd 264
stack_top 255
stack_top_wd 265
swap 114, 116, 125, 155, 156, 226
swap_ranges 115, 116

U

upper_bound 94–96, 98, 228